MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)



FIRST STATE PRIVATE UNIVERSITY IN RAJASTHAN
A+ NAAC
3.28 SCORE

B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026

COURSE NAME: DIGITAL SYSTEM

COURSE CODE : ECE 1001

LECTURE SERIES NO : 01(ONE)

CREDITS : 3

MODE OF DELIVERY : ONLINE (POWER POINT PRESENTATION)

FACULTY :

EMAIL-ID :

PROPOSED DATE OF DELIVERY:

MANIPAL UNIVERSITY
JAIPUR

VISION
Global Leadership in Higher Education and Human Development

MISSION
- Be the most preferred University for innovative and interdisciplinary learning
- Foster academic, research and professional excellence in all domains
- Transform young minds into competent professionals with good human values

VALUES
Integrity, Transparency, Quality,
Team Work, Execution with Passion, Humane Touch

SESSION OUTCOME

"APPLY BOOLEAN ALGEBRA TO SIMPLIFY LOGICAL EXPRESSIONS AND IMPLEMENT LOGIC FUNCTIONS USING BASIC AND UNIVERSAL GATES "

ASSIGNMENT

QUIZ

MID TERM EXAMINATION –II

END TERM EXAMINATION

ASSESSMENT CRITERIA'S



PROGRAM OUTCOMES MAPPING WITH CO1

[PO1]

APPLY BOOLEAN ALGEBRA TO SIMPLIFY LOGICAL EXPRESSIONS AND IMPLEMENT LOGIC FUNCTIONS USING BASIC AND UNIVERSAL GATES.

BOOLEAN ALGEBRA

Boolean algebra is the mathematical framework on which logic design based.

It is used in synthesis & analysis of binary logical function.

George Boole in 1854 invented a new kind of algebra known as Boolean algebra.

It is sometimes called switching algebra.

- (A+B)(B+C)=AB+B.B+AC+BC
- A+A’B=(A+A’)(A+B)=A+B
- (A
- AND
- OR
- NOT (single input and single output)
- A+BC = (A+B)(A+C)
- A+A’B=(A+A’)(A+B)

BASIC LAWS OF BOOLEAN ALGEBRA

- **Laws of complementation**: The term complement means invert. i.e. to change 0's to 1's and 1' to 0's.
- The following are the laws of complement. Here A is Bolean variables which can either value '0' or '1'.
- **" OR" laws (logical 'OR' operation)**

0+0=0; 0+1=1; 1+0=1; 1+1=1

1+A=1; A+ A' =1; A+A=A; 1+ A' =1

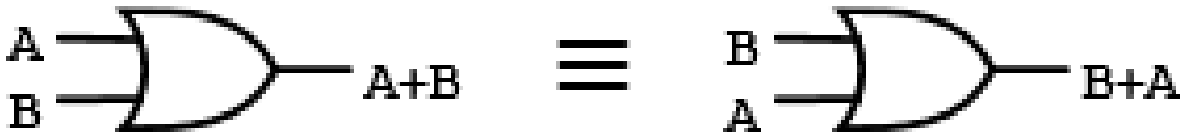
- **" AND'' laws**

0.0=0; 0.1=0; 1.0=0; 1.1=1; A. A' =0; A.A=A

COMMUTATIVE LAW:

- **Property 1:** This states that the order in which the variables OR makes no difference in output. i.e. A+B=B+A

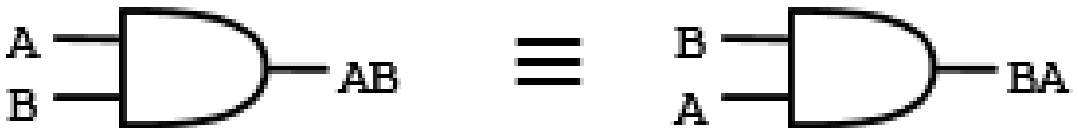
A	B	A+B		B	A	B+A
0	0	0		0	0	0
0	1	1	=	1	0	1
1	0	1		0	1	1
1	1	1		1	1	1



COMMUTATIVE LAW:

Property 2: This property of multiplication states that the order in which the variables are AND makes no difference in the output. i.e. **A.B=B.A**

A	B	A.B		B	A	B.A
0	0	0		0	0	0
0	1	0	=	1	0	0
1	0	0		0	1	0
1	1	1		1	1	1



ASSOCIATIVE PROPERTY

- **Property1:** This property states that in the OR'ing of the several variables, the result is same regardless of grouping of variables. For three variables i.e.(A OR'ed with B)or'ed with C is same as A OR'ed with (B OR'ed with C)
- i.e. **(A+B)+C = A+(B+C)**

A	B	C	A+B	B+C	(A+B)+C		A+(B+C)
0	0	0	0	0	0		0
0	0	1	0	1	1		1
0	1	0	1	1	1		1
0	1	1	1	1	1	=	1
1	0	0	1	0	1		1
1	0	1	1	1	1		1
1	1	0	1	1	1		1
1	1	1	1	1	1		1

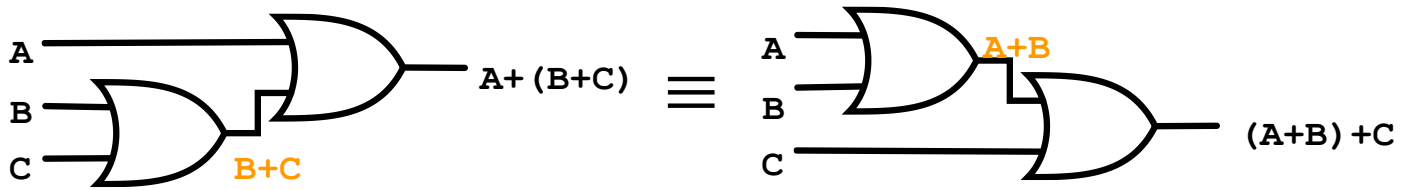
ASSOCIATIVE PROPERTY

- **Property2:** The associative property of multiplication states that, it makes no difference in what order the variables are grouped when AND'ing several variables. For three variables(A AND'ed B)AND'ed C is same as A AND'ed (B AND'ed C)
- i.e. **(A.B)C = A(B.C)**

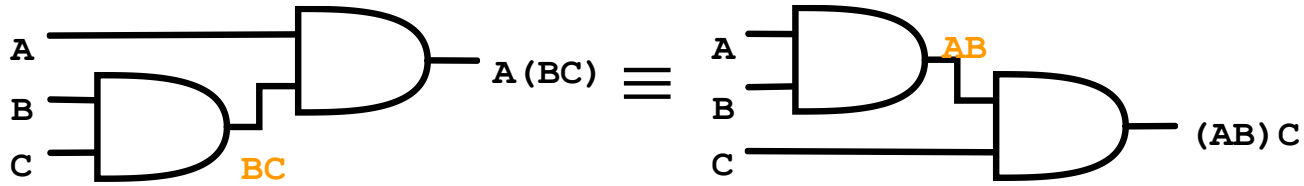
A	B	C	A.B	B.C	(A.B)C		A(B.C)
0	0	0	0	0	0		0
0	0	1	0	0	0		0
0	1	0	0	0	0		0
0	1	1	0	1	0	=	0
1	0	0	0	0	0		0
1	0	1	0	0	0		0
1	1	0	1	0	0		0
1	1	1	1	1	1		1

ASSOCIATIVE LAWS

- The *associative law of addition* for 3 variables is written as: *A+(B+C) = (A+B)+C*



- The *associative law of multiplication* for 3 variables is written as: *A(BC) = (AB)C*



DISTRIBUTIVE PROPERTY

Property 1: A(B+C) = A.B + A.C

1	2	3	4	5	6	7	8
A	B	C	B+C	A(B+C)	A.B	A.C	A.B+A.C
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

Column number 5= Column number 8, hence the proof.

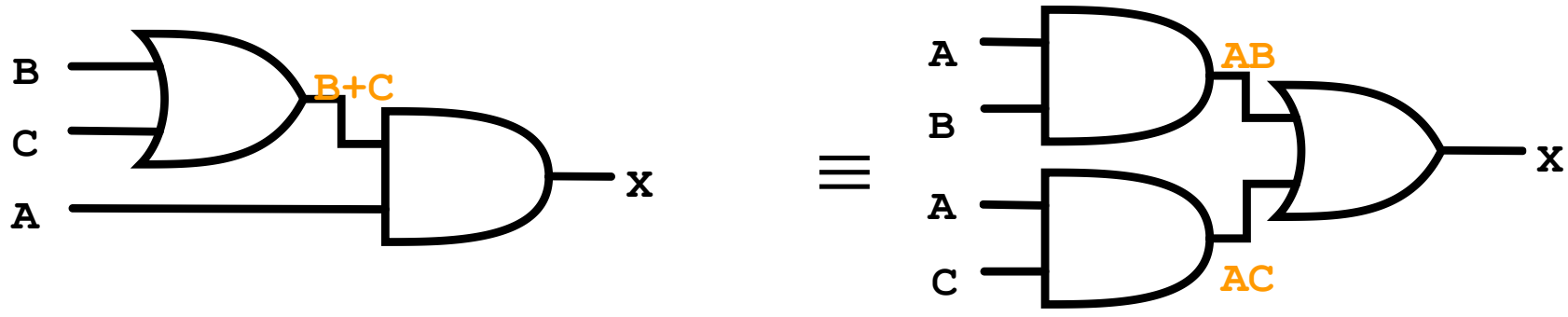
DISTRIBUTIVE PROPERTY

- **Property 2:** A + A' B = A+B

A	B	A'	A'B	A'B+A		A+B
0	0	1	0	0		0
0	1	1	1	1	=	1
1	0	0	0	1		1
1	1	0	0	1		1

DISTRIBUTIVE LAWS

- The *distributive law* is written for 3 variables as follows: $A(B+C) = AB + AC$



X=A (B+C) **X=AB+AC**

RULES OF BOOLEAN ALGEBRA

$$1.\ A + 0 = A$$
$$2.\ A + 1 = 1$$
$$3.\ A \bullet 0 = 0$$
$$4.\ A \bullet 1 = A$$
$$5.\ A + A = A$$
$$6.\ A + \overline{A} = 1$$
$$7.\ A \bullet A = A$$
$$8.\ A \bullet \overline{A} = 0$$
$$9.\ \overline{\overline{A}} = A$$
$$10.\ A + AB = A$$
$$11.\ A + \overline{A}B = A + B$$
$$12.\ (A + B)(A + C) = A + BC$$

A, B, and C can represent a single variable or a combination of variables.

DUALITY

- The important property to Boolean algebra is called Duality principle. The Dual of any expression can be obtained easily by the following rules.
- 1. Change all 0's to 1's
- 2. Change all 1's to 0's
- 3. .'s (dot's) are replaced by +'s (plus)
- 4. +'s (plus) are replaced by .'s (dot's)

Examples:

X +0=X ≡ X .1=X
X+Y=Y+X ≡ X.Y=Y.X
X+1=1 ≡ X.0=0

Example Transformations

- **Simple expression:**
  - Original: `A + 0 = A`
  - Dual: `A · 1 = A`

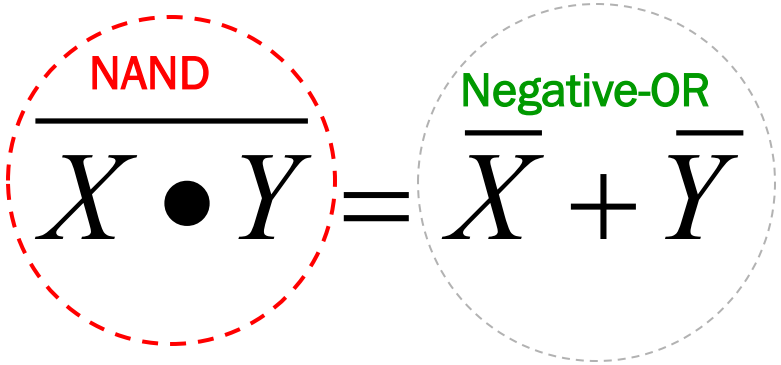
    (OR becomes AND; `0` becomes `1`)
- **Algebraic identity duality:**
  - Original: `B + 1 = 1`
  - Dual: `B · 0 = 0`

Sanfoundry

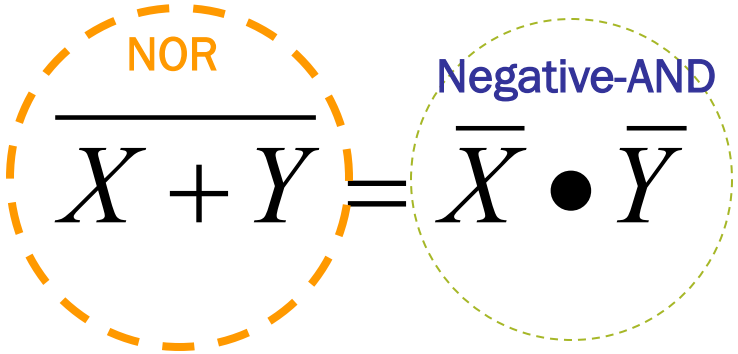
- **Expression example:**
  - Given `A(B + C) = AB + AC`
  - Dual: `A + (B · C) = (A + B)(A + C)`

DEMORGAN'S THEOREMS

- The complement of two or more ANDed variables is equivalent to the OR of the complements of the individual variables.



- The complement of two or more ORed variables is equivalent to the AND of the complements of the individual variables.



DE MORGON'S FIRST THEOREM

- It states that " the complements of product of two variables equal to sum of the complements of individual variable". i.e. (AB)' = A' +B'

A	B	A'	B'	A.B	(AB)'		A'+B'
0	0	1	1	0	1		1
0	1	1	0	0	1	≡	1
1	0	0	1	0	1		1
1	1	0	0	1	0		0

DE MORGON'S SECOND THEOREM

- It states that complement of sum of two variables is equal to product of complement of two individual variables.
- (A+B)' = A' . B'

A	B	A'	B'	A+B	(A+B)'		A' *B'
0	0	1	1	0	1		1
0	1	1	0	1	0	≡	0
1	0	0	1	1	0		0
1	1	0	0	1	0		0

DEMORGAN'S THEOREMS (EXERCISES)

- Apply DeMorgan's theorems to the expressions:

$$\overline{X \bullet Y \bullet Z}$$

$$\overline{X + Y + Z}$$

$$\overline{\overline{X} + \overline{Y} + \overline{Z}}$$

$$\overline{\overline{W} \bullet \overline{X} \bullet \overline{Y} \bullet \overline{Z}}$$

FUNCTION MINIMIZATION USING BOOLEAN ALGEBRA

- *Examples*:

(a) $a + ab = a(1+b)=a$

(b) $a(a + b) = a.a + ab=a+ab=a(1+b)=a.$

(c) $a + a'b = (a + a')(a + b)=1(a + b) =a+b$

(d) $a(a' + b) = a.\ a' + ab=0+ab=ab$

THE OTHER TYPE OF QUESTION

Show that;

1- $ab + ab' = a$

2- $(a + b)(a + b') = a$

1- $ab + ab' = a(b+b') = a.1=a$

2- $(a + b)(a + b') = a.a + a.b' + a.b + b.b'$

$= a + a.b' + a.b + 0$

$= a + a.(b' + b) + 0$

$= a + a.1 + 0$

$= a + a = a$

MORE EXAMPLES

- Show that;

  (a) $ab + ab'c = ab + ac$

  (b) $(a + b)(a + b' + c) = a + bc$

  (a) $ab + ab'c = a(b + b'c)$

  $= a((b+b').(b+c))=a(b+c)=ab+ac$

  (b) $(a + b)(a + b' + c)$

  $= (a.a + a.b' + a.c + ab + b.b' + bc)$

  = …

$\overline{X}$

SIMPLIFY THE BOOLEAN EXPRESSION

- 1. X'Y' Z+ X' YZ

  = ZX'[Y' +Y]

  = Z X'[1]= ZX'

- 2. f = B(A+C)+C

  =BA+BC+C

  =BA+C(1+B)

  =BA+C

BOOLEAN ALGEBRA PROPERTIES

Let X: Boolean variable, 0,1: constants

1. $X + 0 = X$ -- Zero Axiom
2. $X \bullet 1 = X$ -- Unit Axiom
3. $X + 1 = 1$ -- Unit Property
4. $X \bullet 0 = 0$ -- Zero Property
5. $X + X = X$ -- Idepotence
6. $X \bullet X = X$ -- Idepotence
7. $X + X' = 1$ -- Complement
8. $X \bullet X' = 0$ -- Complement
9. $(X')' = X$ -- Involution

5. $X + Y = Y + X$
6. $X \bullet Y = Y \bullet X$ -- Commutative
7. $X + (Y+Z) = (X+Y) + Z$
8. $X \bullet (Y \bullet Z) = (X \bullet Y) \bullet Z$ -- Associative
9. $X \bullet (Y+Z) = X \bullet Y + X \bullet Z$
10. $X+(Y \bullet Z) = (X+Y) \bullet (X+Z)$ -- Distributive

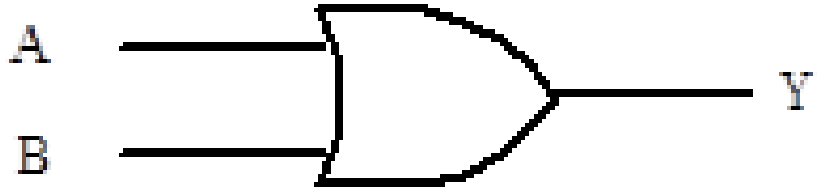
16. $(X + Y)' = X' \bullet Y'$
17. $(X \bullet Y)' = X' + Y'$ -- DeMorgan's

LOGIC GATES

- It is an electronic circuit, which makes logic decisions.
- A digital circuit is referred to as logic gate for simple reason i.e. it can be analysed based on Boolean algebra.
- To make logical decisions, three gates are used.
- They are OR, AND & NOT gate. These logic gates are building blocks, which are available in the form of IC.

OR GATE

- The OR gate performs logical additions commonly known as OR function.
- Two or more inputs and only one output.
- The operation of OR gate is such that a HIGH(1) on the output is produced when any of the input is HIGH.
- The output is LOW(0) only when all the inputs are LOW.

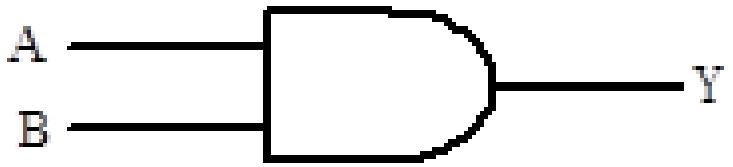


Input		Output
A	B	Y= A+B
0	0	0
0	1	1
1	0	1
1	1	1

Truth table for two input OR gate

AND GATE

- The AND gate performs logical multiplication.
- Two or more inputs and a single output.
- The output of an AND gate is HIGH only when all the inputs are HIGH.
- Even if any one of the input is LOW, the output will be LOW.



Input		Output
A	B	Y=A.B
0	0	0
0	1	0
1	0	0
1	1	1

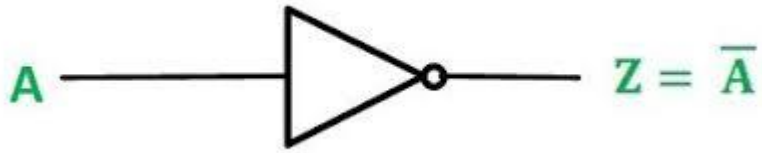
Truth table for two input AND gate:

NOT GATE (INVERTER)

- The NOT gate performs the basic logical function called inversion or complementation.

- The purpose of this gate is to convert one logic level into the opposite logic level.

- It has one input and one output.

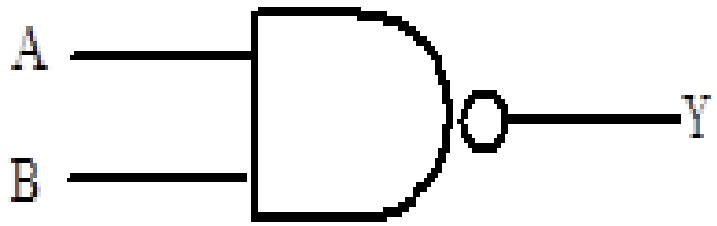


Input	output
A	Z= A’
0	1
1	0

Truth Table

NAND GATE

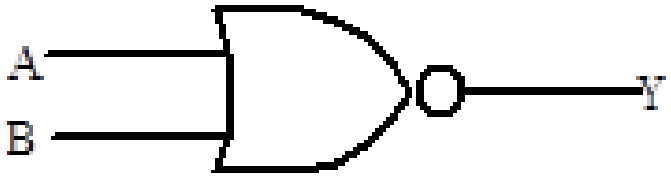
- The output of a NAND gate is LOW only when all inputs are HIGH and output of the NAND is HIGH if one or more inputs are LOW.



Input		Output
A	B	Y = (AB)’
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE

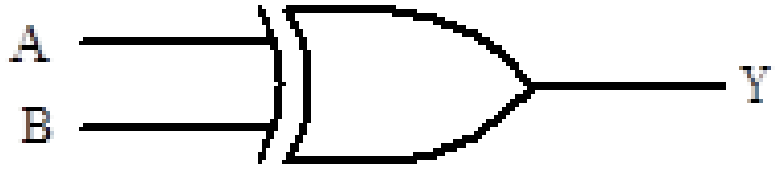
The output of the NOR gate is HIGH only when all the inputs are LOW.



Input		Output
A	B	Y =
0	0	1
0	1	0
1	0	0
1	1	0

XOR GATE OR EXCLUSIVE OR GATE

- In this gate output is HIGH only when any one of the input is HIGH.
- The circuit is also called as inequality comparator, because it produces output when two inputs are different.

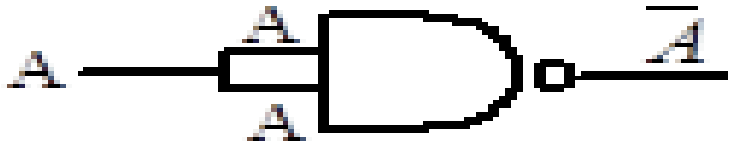


$$Y = A \oplus B = A\overline{B} + \overline{A}B$$

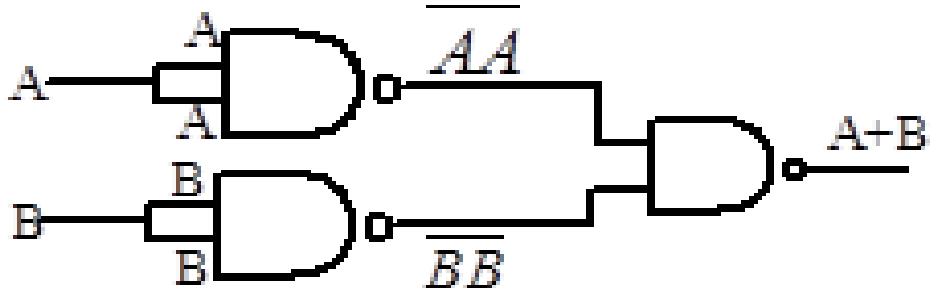
Input		Output
A	B	Y =
0	0	0
0	1	1
1	0	1
1	1	0

UNIVERSAL LOGIC GATE

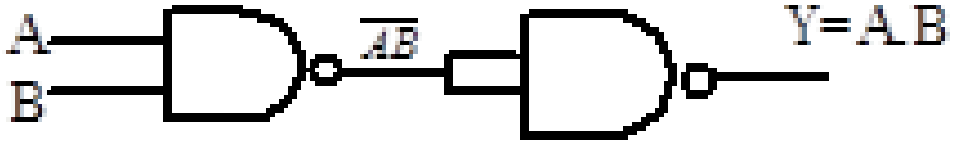
- NAND and NOR gates are called Universal gates or Universal building blocks, because both can be used to implement any gate like AND,OR an NOT gates or any combination of these basic gates.



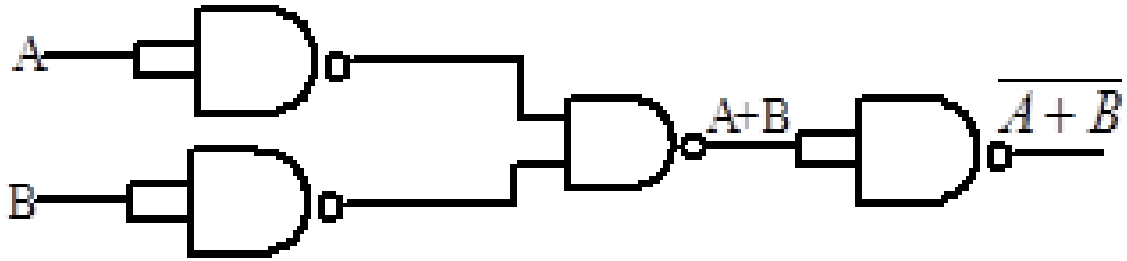
NOT operation:



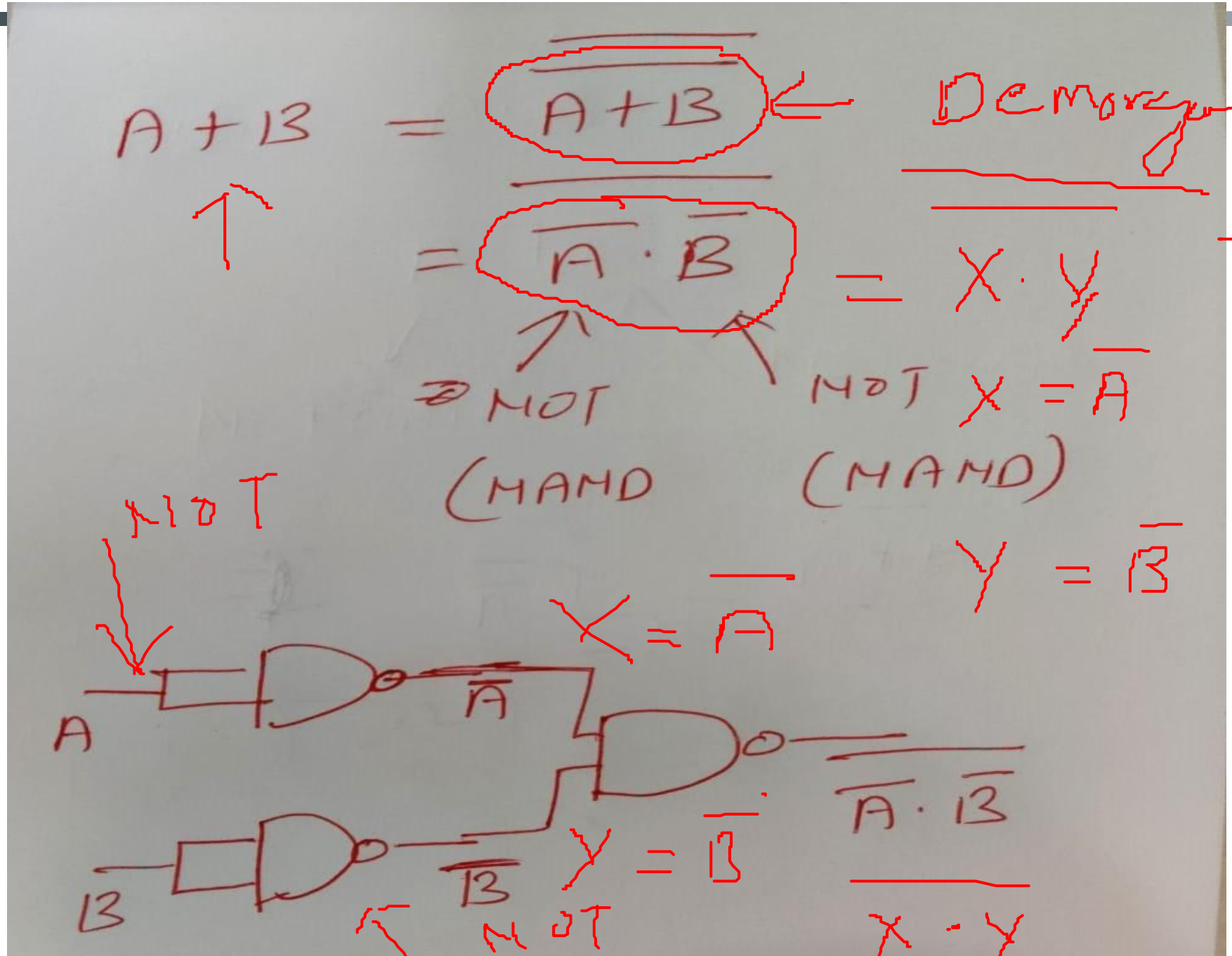
OR operation:

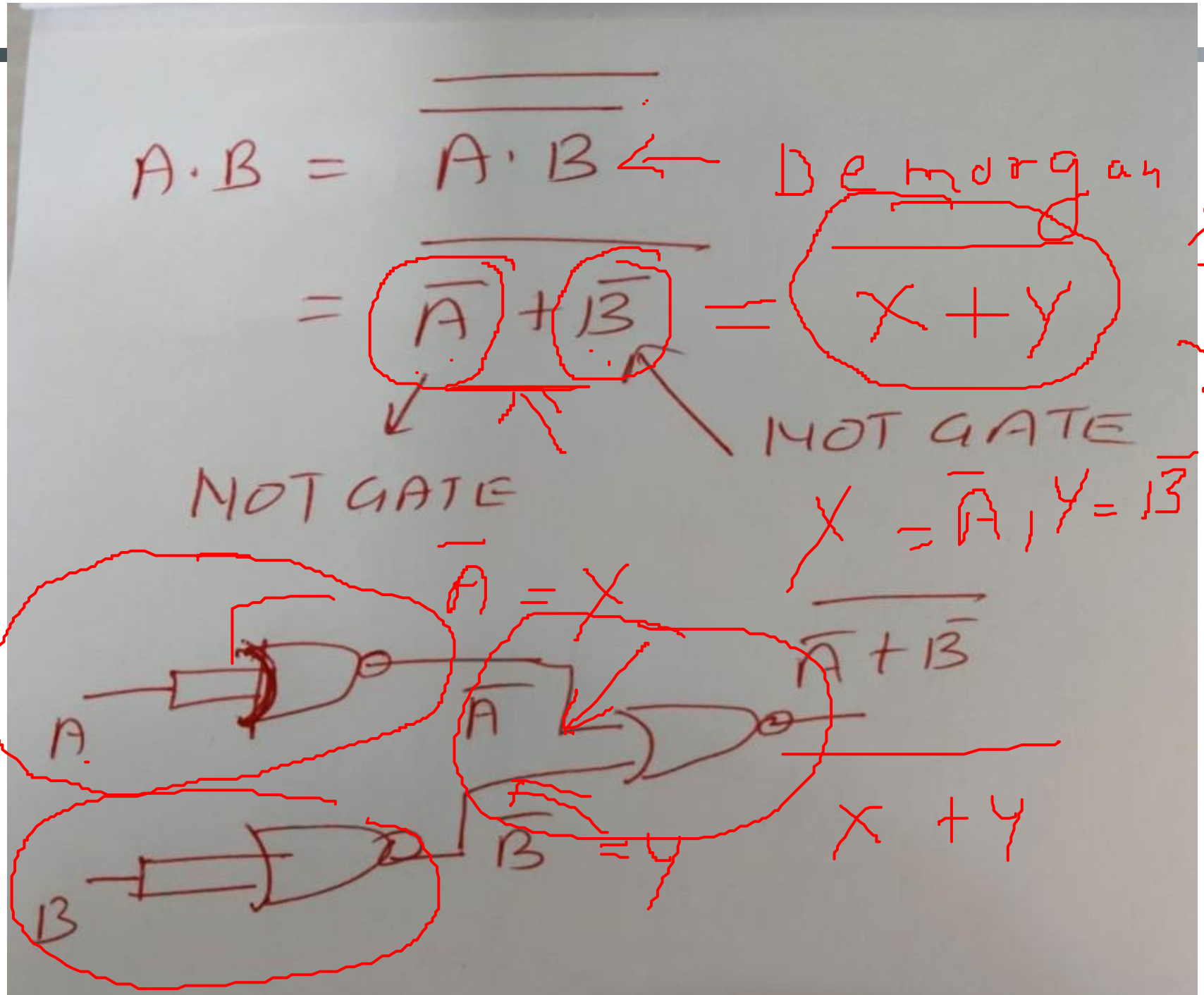


AND operation:



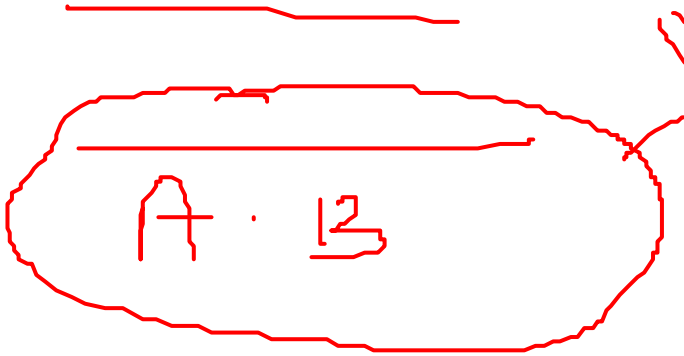
NOR operation:


A + B = $\overline{\overline{A+B}}$
Demorgan
$= \overline{\overline{A} \cdot \overline{B}}$
$= X \cdot Y$
X
Y
D
NOT
NOT $X = \overline{A}$
(NAND
(NAND)
NOT
$Y = \overline{B}$
$X = \overline{A}$
A
$\overline{A}$
B
$\overline{B}$
$Y = \overline{B}$
$\overline{A \cdot B}$
NOT
$X \cdot Y$


$\overline{A \cdot B} = \overline{A} + \overline{B}$
De morgan
$= \overline{X + Y}$
NOT GATE
NOT GATE
$X = \overline{A}, Y = \overline{B}$
$A = X$
$\overline{A} + \overline{B}$
A
$\overline{A}$
B
$\overline{B} = Y$
$X + Y$

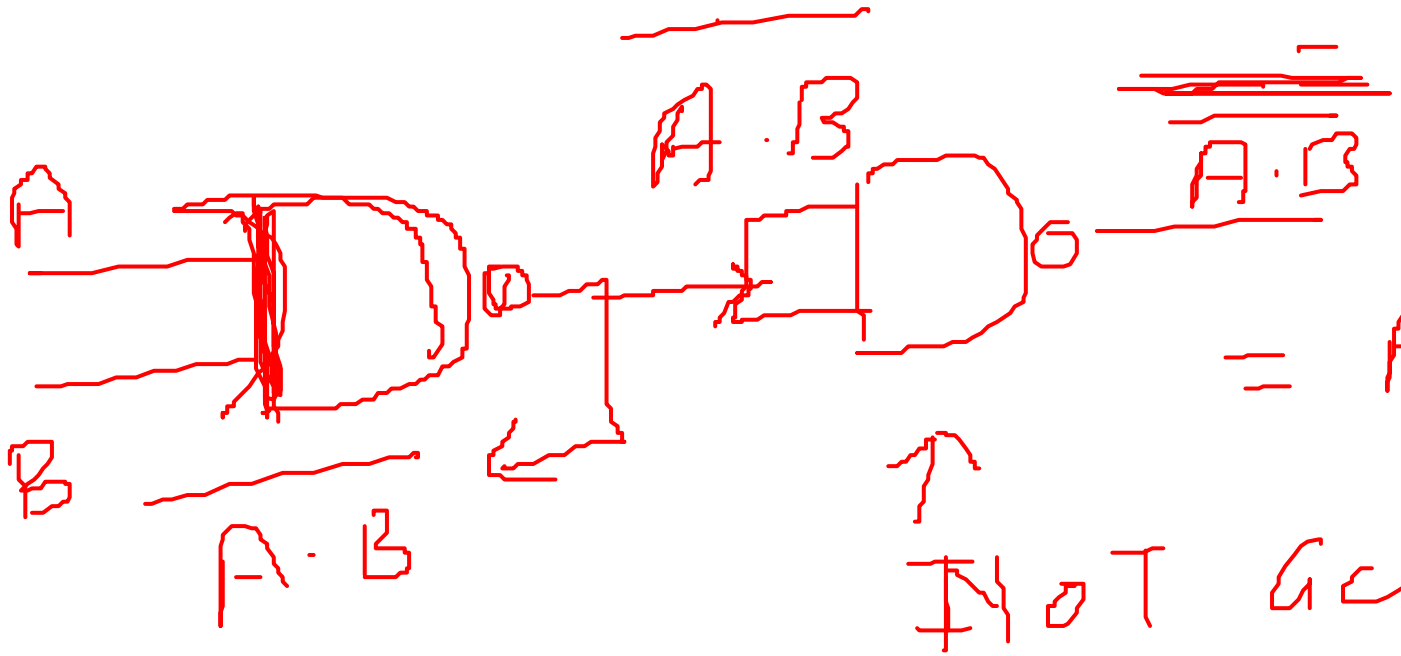
AND by NAND

$= A \cdot B = \overline{\overline{A \cdot B}} =$



$X = \overline{A \cdot B}$

X

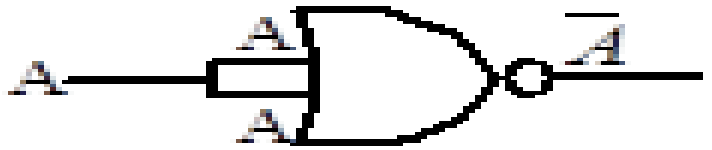


$= A \cdot B$

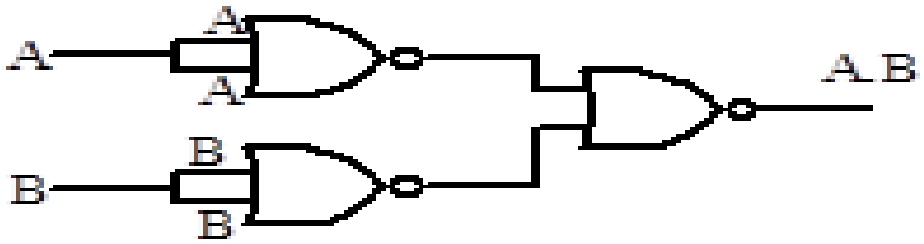
NOT gate

NOR GATE AS UNIVERSAL GATE:

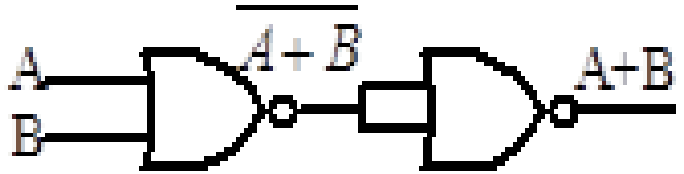
NOT operation:



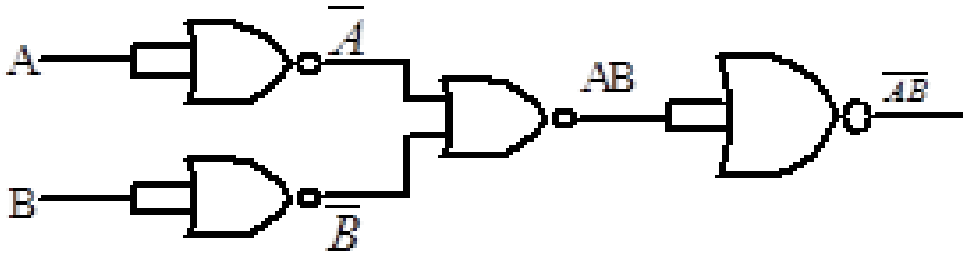
AND operation:



OR operation:

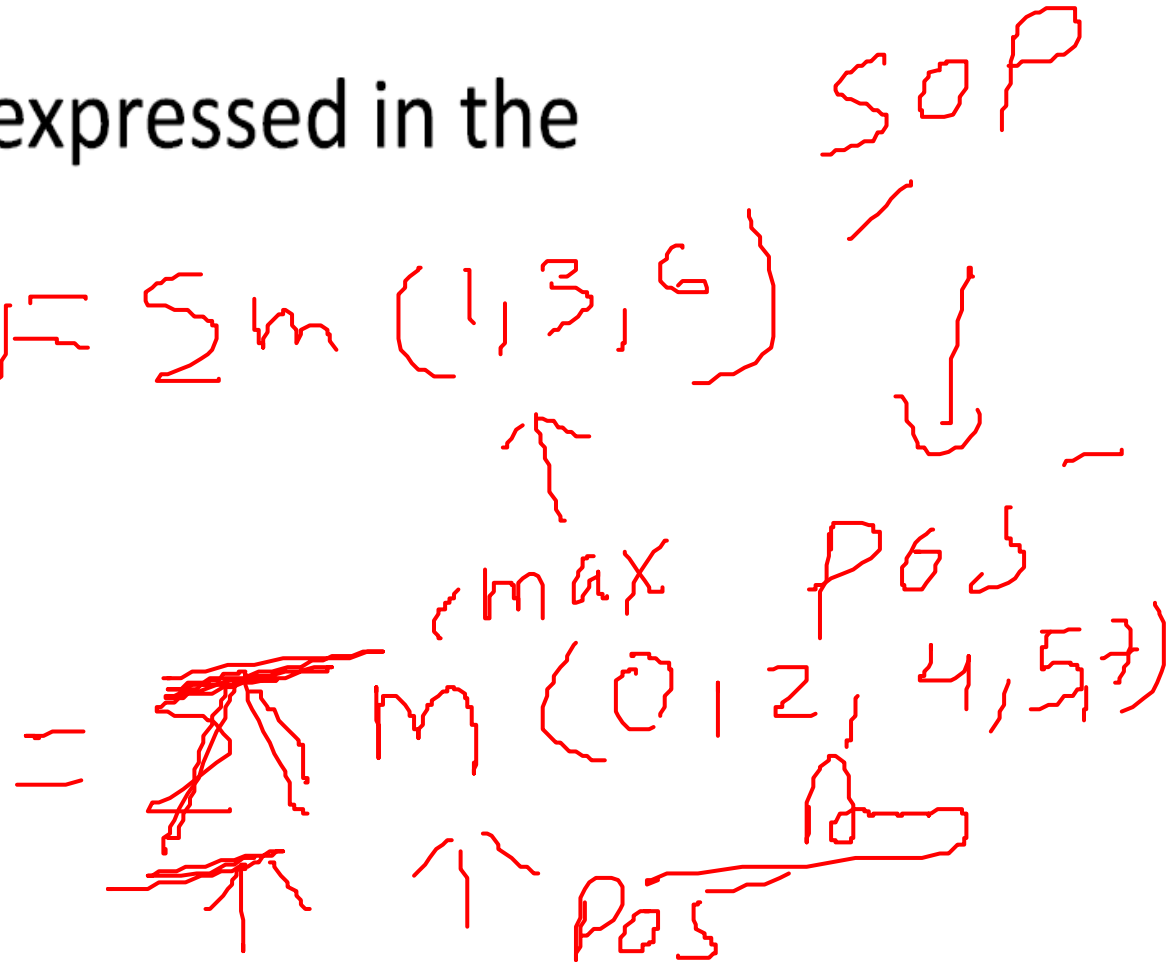


NAND operation:



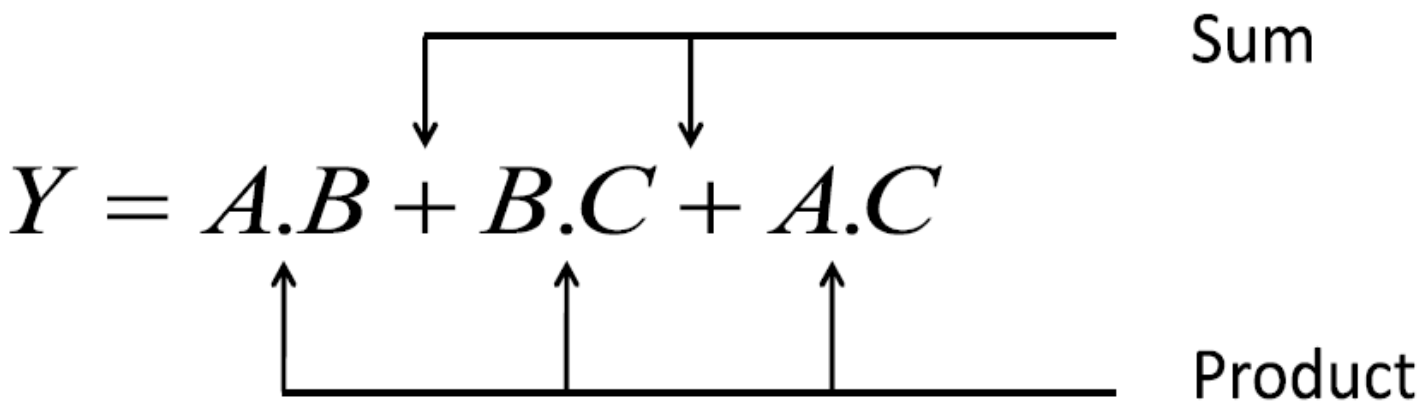
Standard Representation

- Any logical expression can be expressed in the following two forms:
  - ✓Sum of Product (SOP) Form
  - ✓Product of Sum (POS) Form



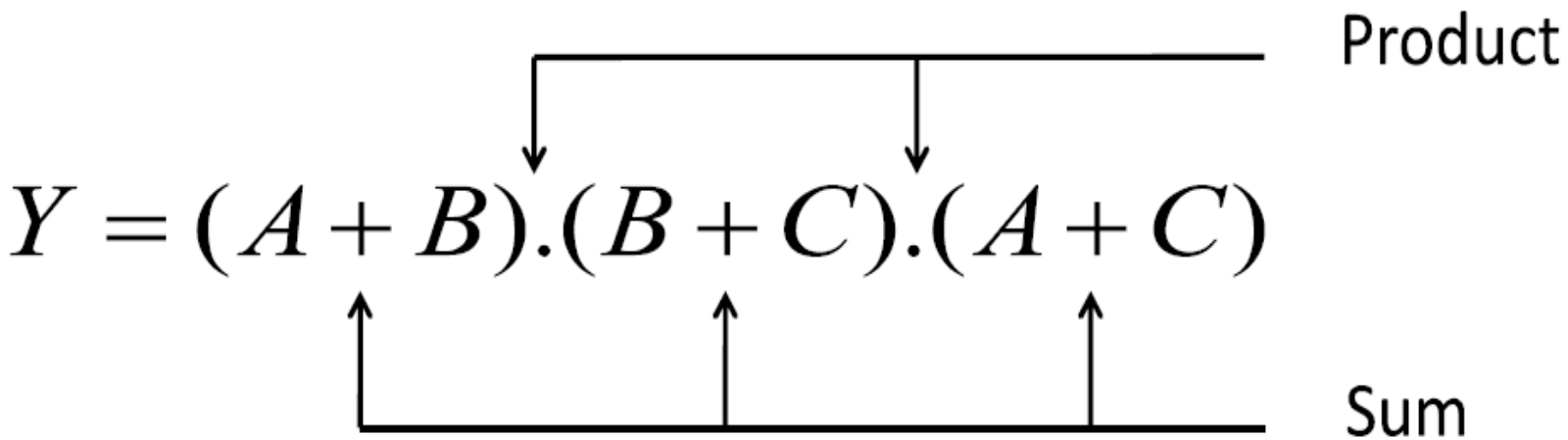
SOP Form

For Example, logical expression given is;



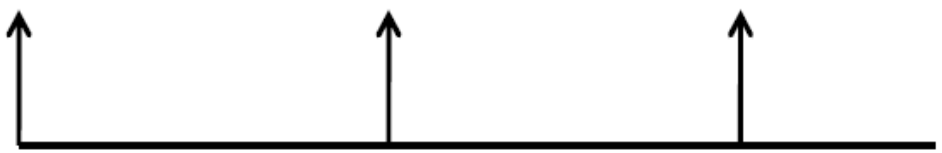
POS Form

For Example, logical expression given is;



Standard SOP

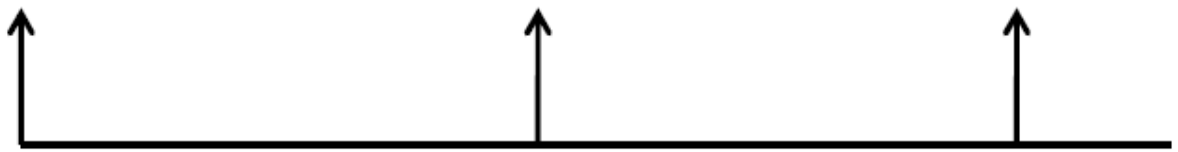
$$Y = ABC + A\overline{B}\,\overline{C} + \overline{A}BC$$



Each product term consists all the literals

Standard POS

$$Y = (A + B + C).(A + \overline{B} + \overline{C}).(\overline{A} + B + C)$$



Each sum term consists all the literals

Sr. No.	Expression	Type
1	$Y = AB + AB\overline{C} + \overline{A}BC$	Non Standard SOP
2	$Y = AB + A\overline{B} + \overline{A}\overline{B}$	Standard SOP
3	$Y = (\overline{A} + B).(A + \overline{B}).(\overline{A} + \overline{B})$	Standard POS
4	$Y = (\overline{A} + B).(A + \overline{B} + C)$	Non Standard POS

Conversion of SOP form to Standard SOP

Procedure:

1. Write down all the terms.
2. If one or more variables are missing in any product term, expand the term by multiplying it with the sum of each one of the missing variable and its complement .
3. Drop out the redundant terms

Convert given expression into its standard SOP form $Y = AB + A\overline{C} + BC$

$$Y = AB + A\overline{C} + BC$$

Missing literal is A

Missing literal is B

Missing literal is C

$$Y = AB.(C + \overline{C}) + A\overline{C}.(B + \overline{B}) + BC.(A + \overline{A})$$

Term formed by ORing of missing literal & its complement

Example 1

Continue….

$$Y = AB.(C + \overline{C}) + A\overline{C}.(B + \overline{B}) + BC.(A + \overline{A})$$

$$Y = ABC + AB\overline{C} + AB\overline{C} + A\overline{B}\overline{C} + ABC + \overline{A}BC$$

$$Y = \underline{ABC} + \underline{AB\overline{C}} + \underline{AB\overline{C}} + A\overline{B}\overline{C} + \underline{ABC} + \overline{A}BC$$

$$Y = ABC + AB\overline{C} + A\overline{B}\overline{C} + \overline{A}BC$$



Standard SOP form
Each product term consists all the literals

Conversion of POS form to Standard POS

Procedure:

1. Write down all the terms.
2. If one or more variables are missing in any sum term, expand the term by adding the products of each one of the missing variable and its complement .
3. Drop out the redundant terms

Convert given expression into its standard SOP form $Y = (A+B).(A+C)+(B+\overline{C})$

$$Y = (A+B).(A+C)+(B+\overline{C})$$

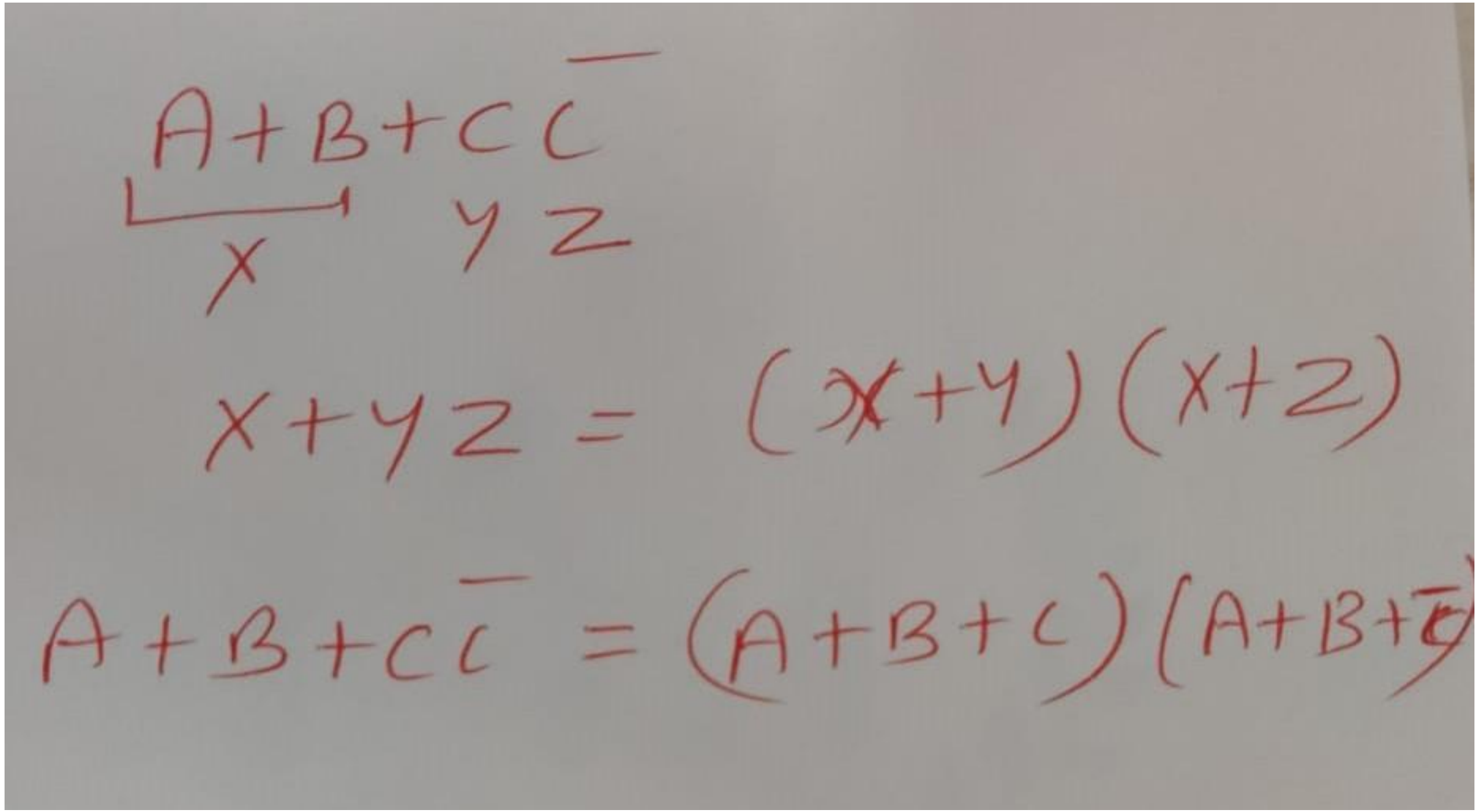
Missing literal is A

Missing literal is B

Missing literal is C

$$Y = (A+B+C\overline{C}).(A+C+B\overline{B}).(B+\overline{C}+A\overline{A})$$

Term formed by ANDing of missing literal & its complement


A+B+C C
X Y Z
X+YZ = (X+Y)(X+Z)
A+B+C C = (A+B+C)(A+B+C)

$$Y = (A + B + C\overline{C}).(A + C + B\overline{B}).(B + \overline{C} + A\overline{A})$$

$$Y = (A+B+C)(A+B+\overline{C}).(A+B+C)(A+\overline{B}+C).(A+B+\overline{C})(\overline{A}+B+\overline{C})$$

$$Y = (A+B+C)(A+B+\overline{C})(A+\overline{B}+C)(\overline{A}+B+\overline{C})$$

$$Y = (A+B+C)(A+B+\overline{C})(A+\overline{B}+C)(\overline{A}+B+\overline{C})$$

Standard POS form

Each sum term consists all the literals

Concept of Minterm and Maxterm

- Minterm: Each individual term in the standard SOP form is called as “Minterm”.

- Maxterm: Each individual term in the standard POS form is called as “Maxterm”.

$$Y = \bar{A}B\bar{C} + \bar{A}\bar{B}C + AB\bar{C} + A\bar{B}\bar{C}$$

$\bar{A}\,B\,C$

$$Y = AB\bar{C} + \bar{A}\bar{B}C + \bar{A}BC + \bar{A}B\bar{C}$$

1 1 0 — 6

0 0 0 — 0

0 1 0 — 2

0 1 1 — 3

$$= \sum (m_0, m_2, m_3, m_6)$$ minterm

$$= \sum m(0, 2, 3, 6)$$

min term , max term

$$Y = AB + \overline{A}\,\overline{B}$$

	A	B	Y
0	0	0	$m_0 = \overline{A}\,\overline{B}$
→ 0	0	1	$m_1 = \overline{A}B$
2	1	0	$m_2 = A\overline{B}$
3	1	1	$m_3 = AB$

$$M_0 = A + B$$
$$M_1 = A + \overline{B}$$
$$M_2 = \overline{A} + B$$
$$M_3 = \overline{A} + \overline{B}$$

	A	B	C	Y
0	0	0	0	
1	0	0	1	
2	0	1	0	
3	0	1	1	
4	1	0	0	
5	1	0	1	
6	1	1	0	
7	1	1	1	

$m_0 = \overline{A} \cdot \overline{B}\,\overline{C}$

$m_1 = \overline{A}\,\overline{B}\,C$

$m_2 = \overline{A}\,B\,\overline{C}$

$m_3 = \overline{A}\,B\,C$

$m_4 = A\,\overline{B}\,\overline{C}$

$m_5 = A\,\overline{B}\,C$

$m_6 = A\,B\,\overline{C}$

$m_7 = A\,B\,C$

4	2	1	D

$M_0 = A + B + C$

$M_1 = A + B + \overline{C}$

$M_2 = A + \overline{B} + C$

$M_3 = A + \overline{B} + \overline{C}$

$M_4 = \overline{A} + B + C$

$M_5 = \overline{A} + B + \overline{C}$

$M_6 = \overline{A} + \overline{B} + C$

$M_7 = \overline{A} + \overline{B} + \overline{C}$

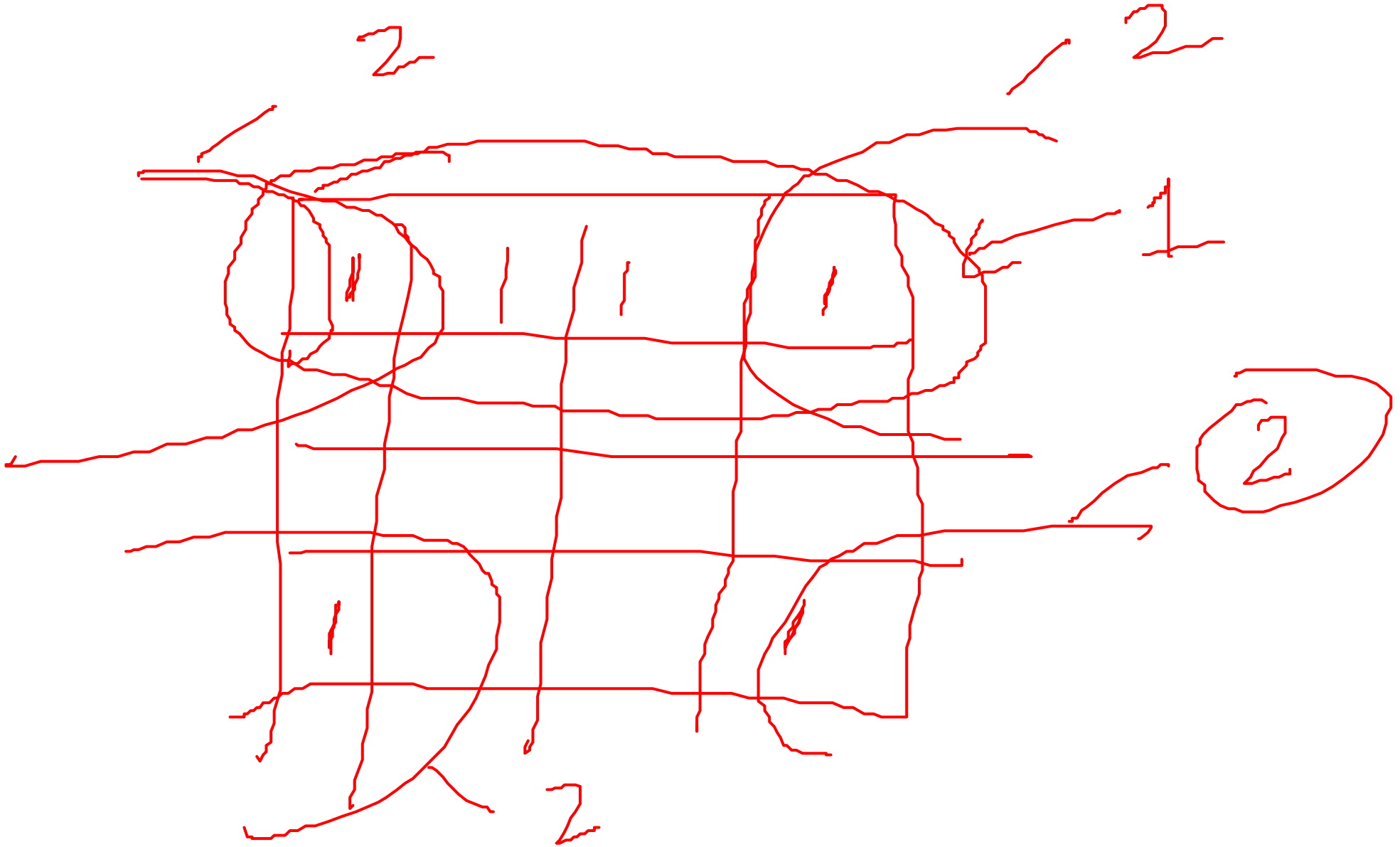
$$y = \bar{A}\bar{B}C + A\bar{B}C + AB\bar{C} + A\bar{B}\bar{C}$$

m_1 m_4 m_6 m_7

	A	B	C	y
0	0	0	0	
1	0	0	1	$\rightarrow m_1$
2	0	1	0	
3	0	1	1	
4	1	0	0	$\rightarrow m_4$
5	1	0	1	$\rightarrow m_6$
6	1	1	0	$\rightarrow m_7$
7	1	1	1	

$$y = \sum (m_1, m_4, m_6, m_7)$$

$$= \sum m(1, 4, 6, 7)$$


2
2
1
2
2

MANIPAL UNIVERSITY
JAIPUR
(University under Section 2(f) of the UGC Act)



FIRST STATE PRIVATE UNIVERSITY IN RAJASTHAN
A+ NAAC
3.28 SCORE

B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026

COURSE NAME: BASIC ELECTRONICS

COURSE CODE : ECE1001

LECTURE SERIES NO : 01(ONE)

CREDITS : 3

MODE OF DELIVERY : ONLINE (POWER POINT PRESENTATION)

FACULTY :

EMAIL-ID :

PROPOSED DATE OF DELIVERY:

MANIPAL UNIVERSITY
JAIPUR

VISION
Global Leadership in Higher Education and Human Development

MISSION
- Be the most preferred University for innovative and interdisciplinary learning
- Foster academic, research and professional excellence in all domains
- Transform young minds into competent professionals with good human values

VALUES
Integrity, Transparency, Quality,
Team Work, Execution with Passion, Humane Touch

SESSION OUTCOME

"A SYSTEMATIC WAY TO MINIMIZE THE GIVEN LOGIC "

ASSIGNMENT

QUIZ

MID TERM EXAMINATION –II

END TERM EXAMINATION

ASSESSMENT CRITERIA'S



PROGRAM OUTCOMES MAPPING WITH CO2

[PO1]

DEMONSTRATE DIFFERENT NUMBER SYSTEMS, BOOLEAN EXPRESSIONS AND DIFFERENT ELEMENTS OF COMMUNICATION SYSTEMS AND TO PROMOTE DIFFERENT SKILLS TOWARDS ELECTRONICS INDUSTRIES.

KARNAUGH MAP

The Karnaugh map, also known as the K-map, is a method to simplify Boolean algebra expressions.

The Karnaugh map reduces the need for extensive calculations by taking advantage of humans' pattern-recognition capability.

Karnaugh Map: A graphical technique for simplifying a Boolean expression into either form:

- minimal sum of products
- minimal product of sums

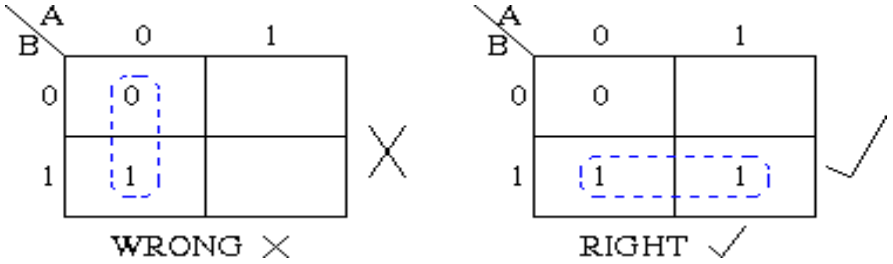
Goal of the simplification.

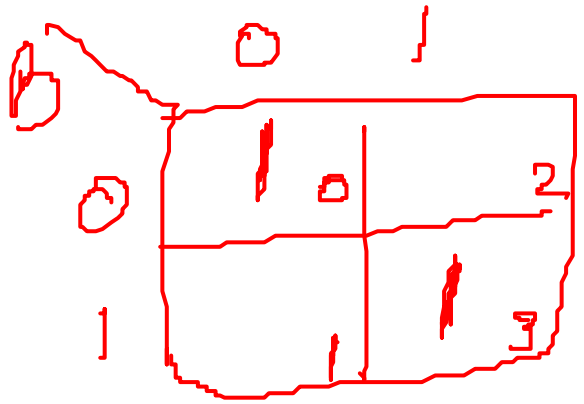
- There are a minimal number of product/sum terms
- Each term has a minimal number of literals

These terms can be used to write a minimal Boolean expression representing the required logic.

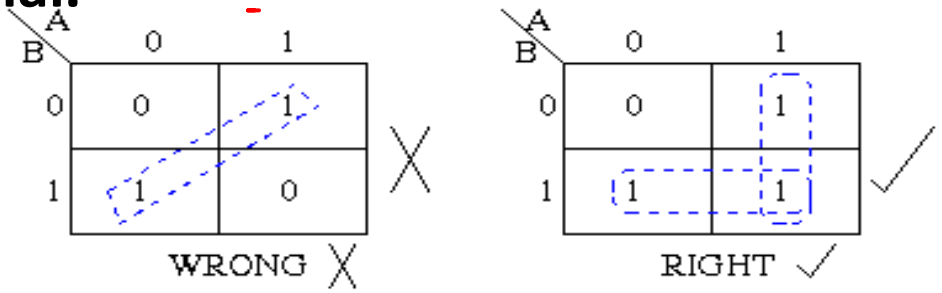
THE KARNAUGH MAP USES THE FOLLOWING RULES FOR THE SIMPLIFICATION OF EXPRESSIONS BY GROUPING TOGETHER ADJACENT CELLS CONTAINING *ONES.*

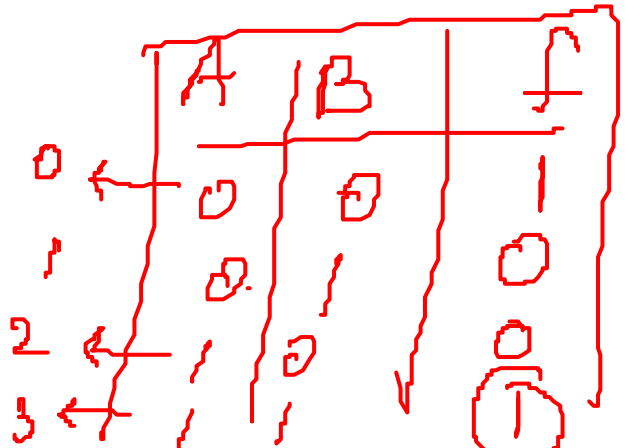
1.Groups may not include any cell containing a zero.



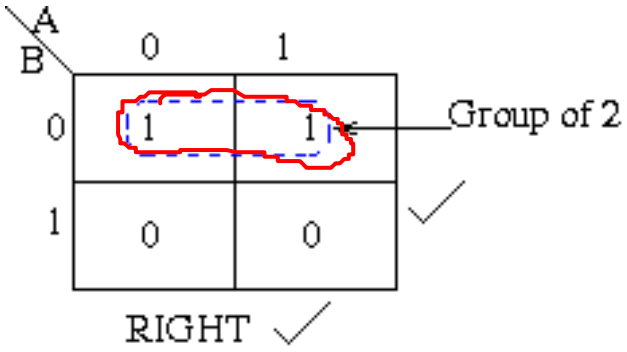


2.Groups may be horizontal or vertical, but not diagonal.

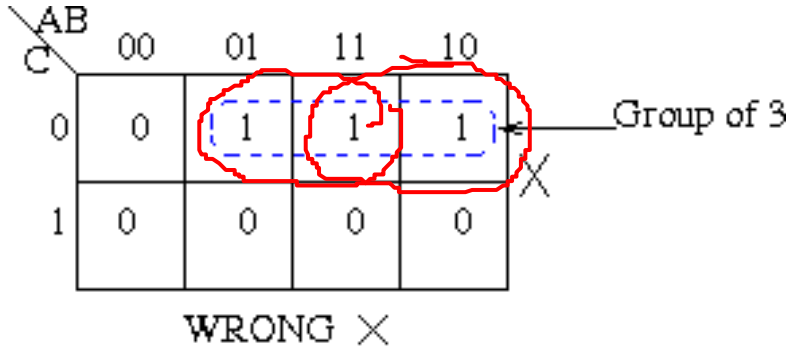




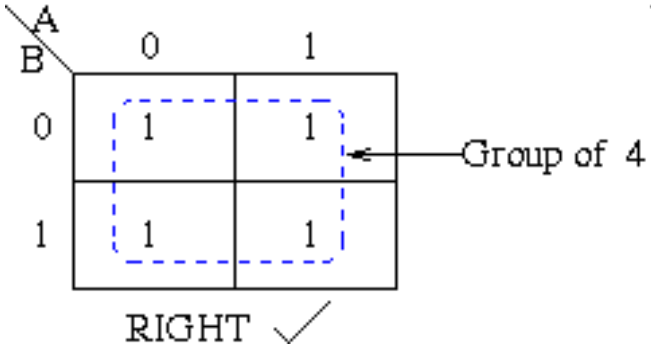
3. GROUPS MUST CONTAIN 1, 2, 4, 8 CELLS OR IN GENERAL 2^N CELLS. THAT IS IF N = 1, A GROUP WILL CONTAIN TWO 1'S SINCE $2^1 = 2$. IF N = 2, A GROUP WILL CONTAIN FOUR 1'S SINCE $2^2 = 4$.



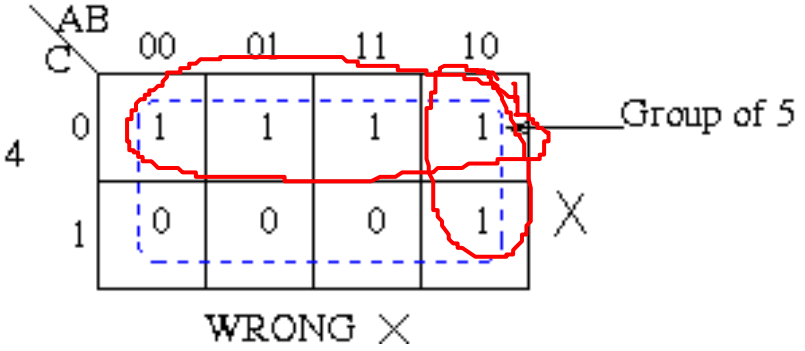
RIGHT ✓



WRONG ×

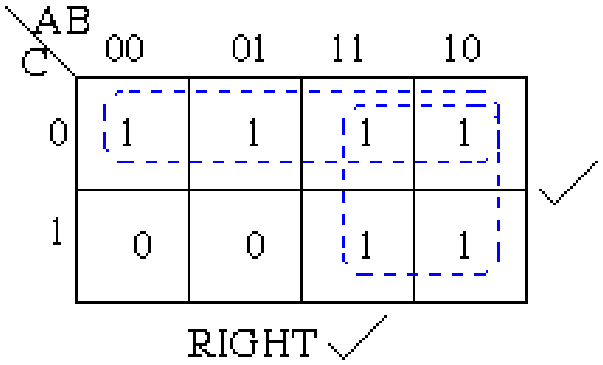


RIGHT ✓

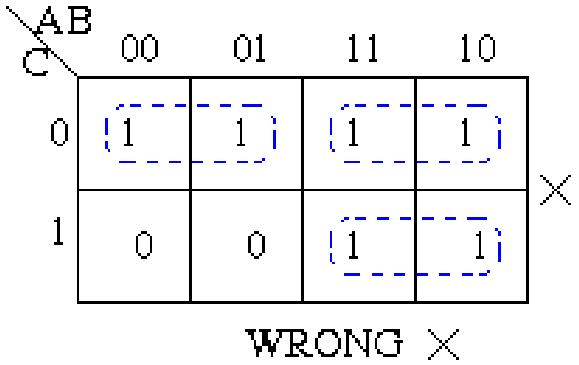


WRONG ×

4. EACH GROUP SHOULD BE AS LARGE AS POSSIBLE.



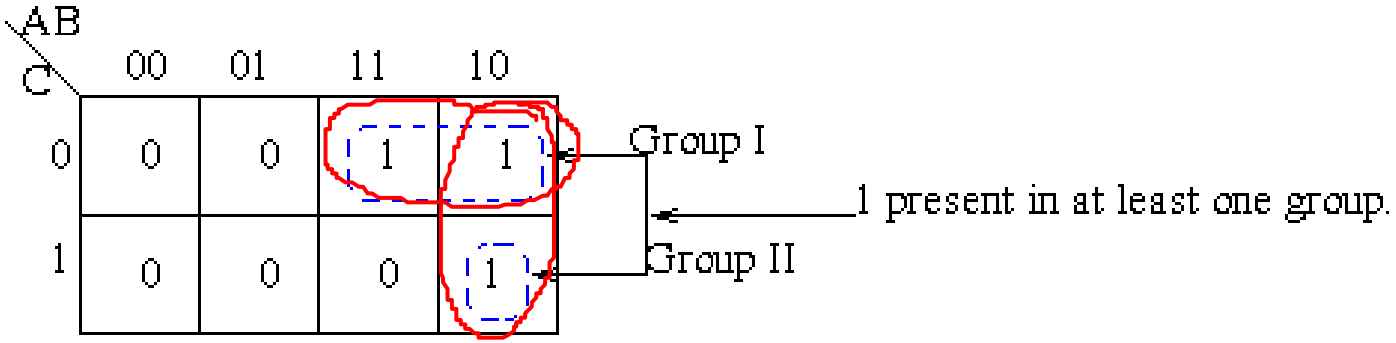
RIGHT ✓



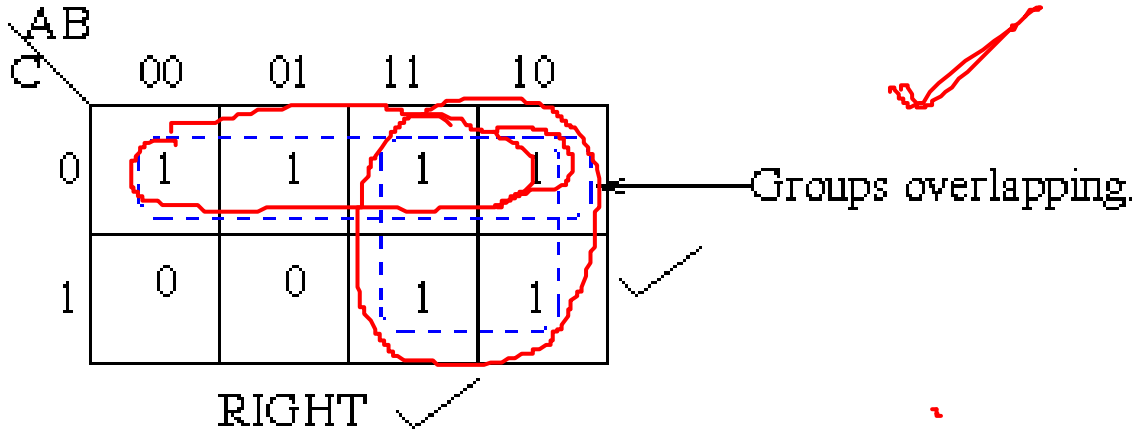
WRONG ×

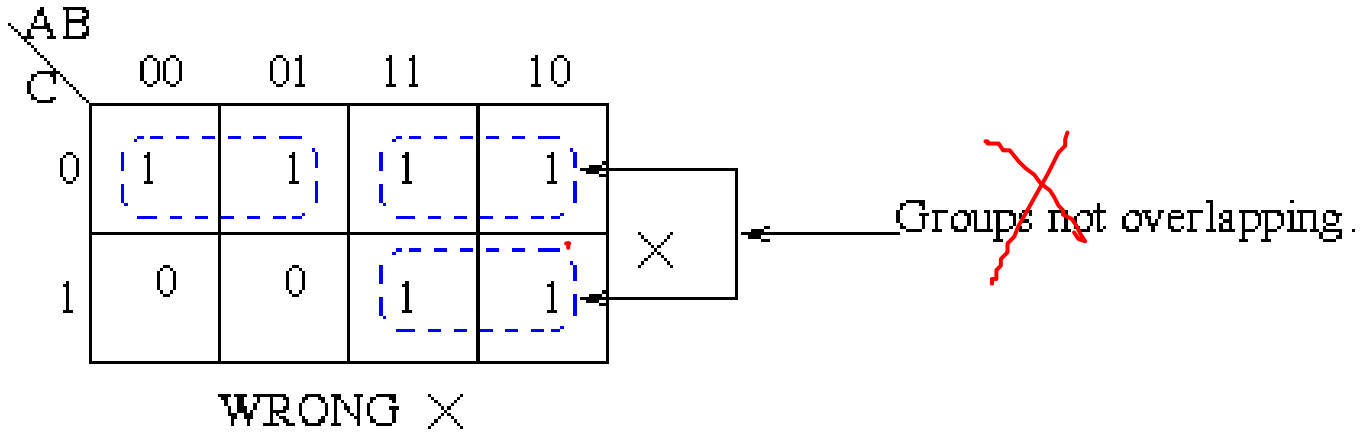
(Note that no Boolean laws broken,
but not sufficiently minimal)

5.Each cell containing a one must be in at least one group.



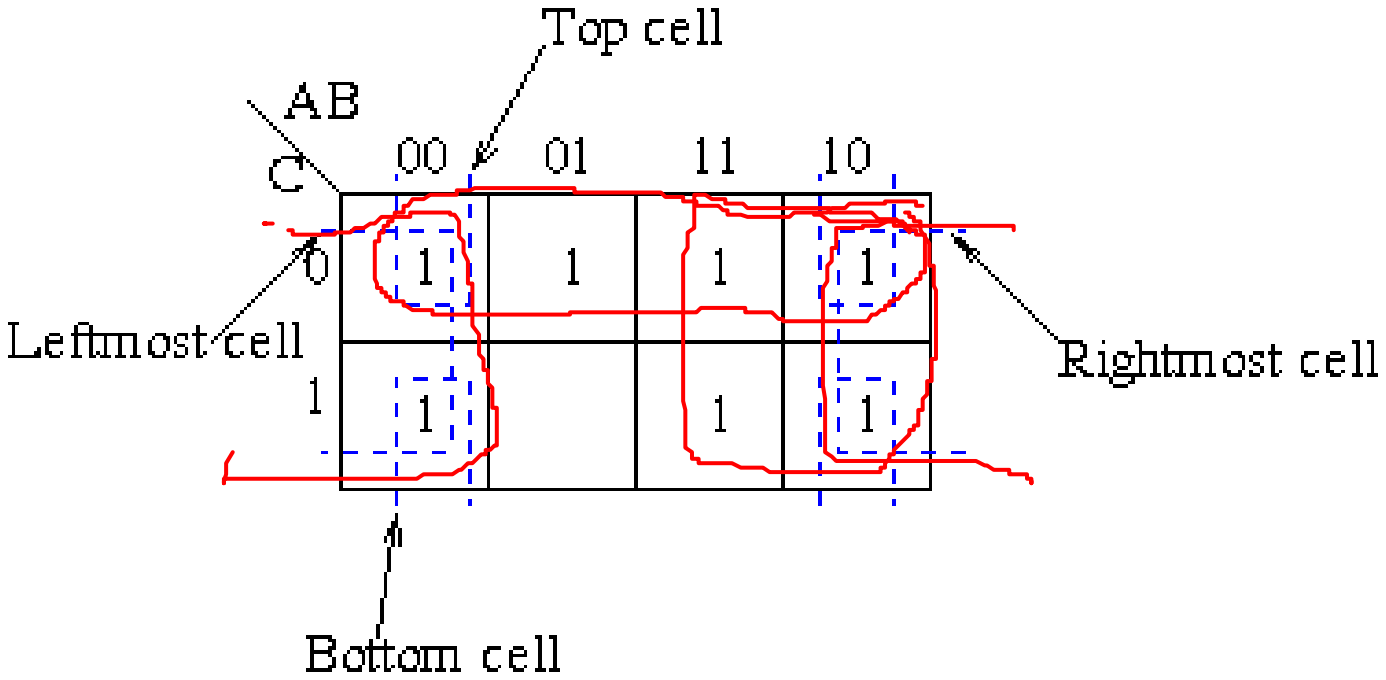
6.GROUPS MAY OVERLAP.



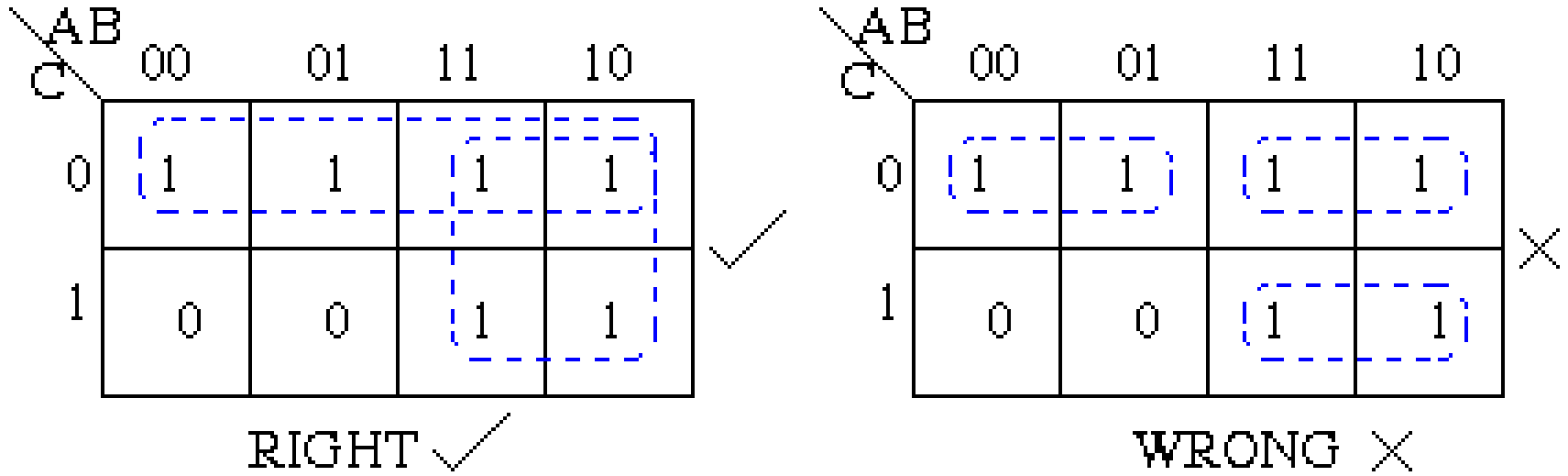


7.Groups may wrap around the table.

The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.



8. THERE SHOULD BE AS FEW GROUPS AS POSSIBLE, AS LONG AS THIS DOES NOT CONTRADICT ANY OF THE PREVIOUS RULES.



SUMMMARY:

- No zeros allowed.
- No diagonals.
- Only power of 2 number of cells in each group.
- Groups should be as large as possible.
- Every one must be in at least one group.
- Overlapping allowed.
- Wrap around allowed.
- Fewest number of groups possible.

KARNAUGH MAPS

- Karnaugh maps, or K-maps, are often used to simplify logic problems with 2, 3 or 4 variables.

Cell = 2^n ,where n is a number of variables

For the case of 2 variables, we form a map consisting of 2^2=4 cells as shown in Figure

B \ A	0	1
0	$A+B$	$\overline{A}+B$
1	$A+\overline{B}$	$\overline{A}+\overline{B}$

Maxterm

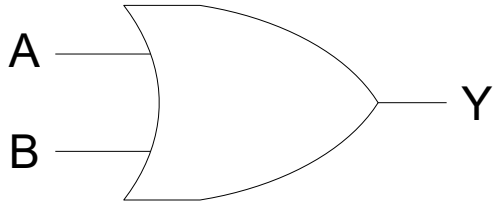
B \ A	0	1
0	00 0	10 2
1	01 1	11 3

B \ A	0	1
0	$\overline{A}\overline{B}$	$A\overline{B}$
1	$\overline{A}B$	AB

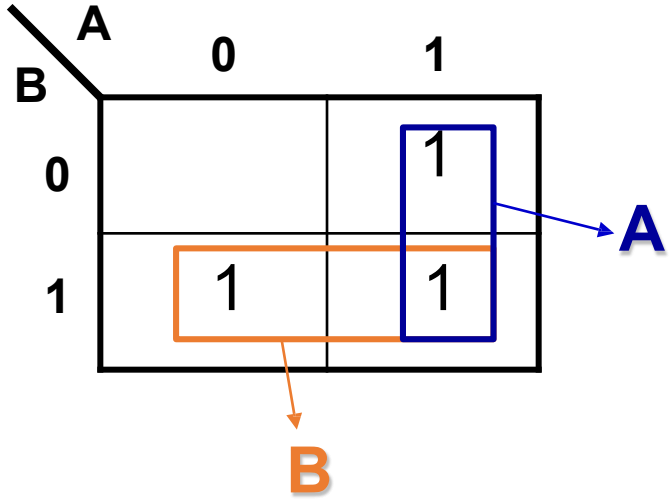
Minterm

EXAMPLE

2-variable Karnaugh maps are trivial but can be used to introduce the methods you need to learn. The map for a 2-input OR gate looks like this:

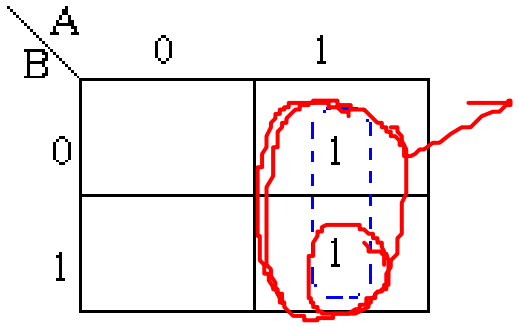


A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



A+B

EXAMPLE 1: Consider the following map. The function plotted is: Z = f(A,B) = AB' + AB



Note that values of the input variables form the rows and columns.
That is the logic values of the variables A and B (with one denoting true form and zero denoting false form) form the head of the rows and columns respectively.

The map displayed is a one dimensional type which can be used to simplify an expression in two variables.

There is a two-dimensional map that can be used for up to four variables, and a three-dimensional map for up to six variables.

REFERRING TO THE MAP ABOVE, THE TWO ADJACENT 1'S ARE GROUPED TOGETHER.

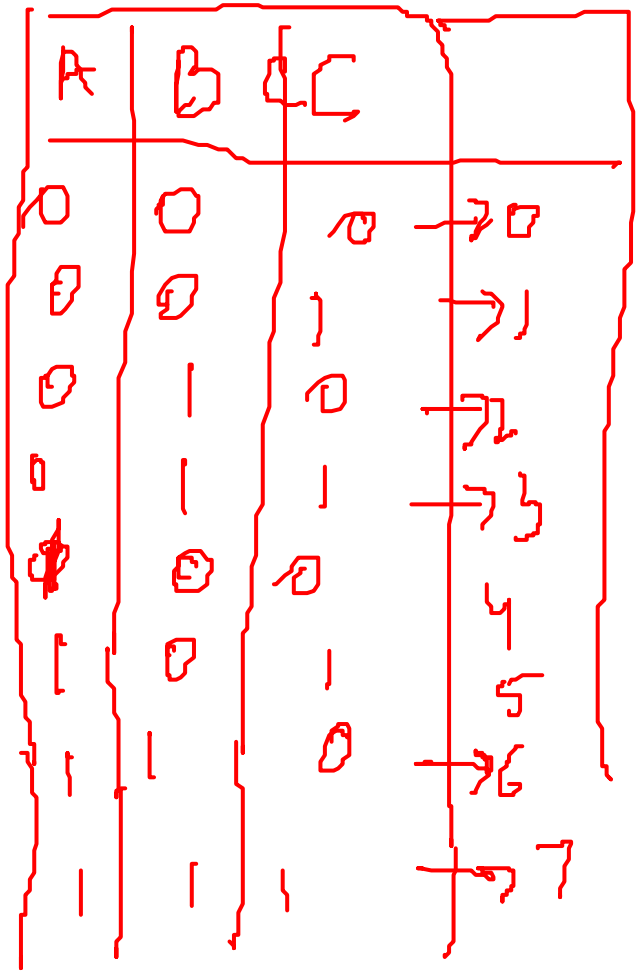
Through inspection it can be seen that variable B has its true and false form within the group.

This eliminates variable B leaving only variable A which only has its true form.

The minimized answer therefore is Z = A.

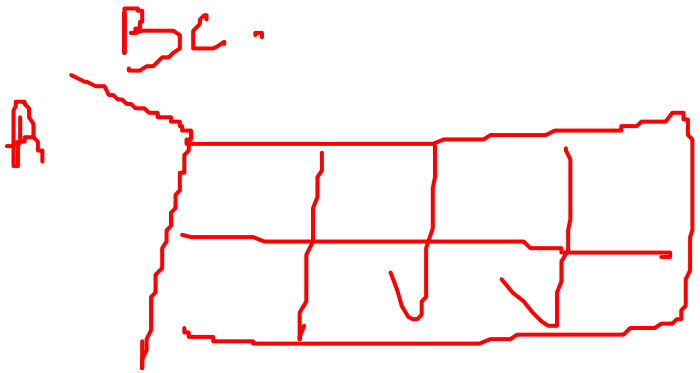
KARNAUGH MAPS

- 3 variables Karnaugh map

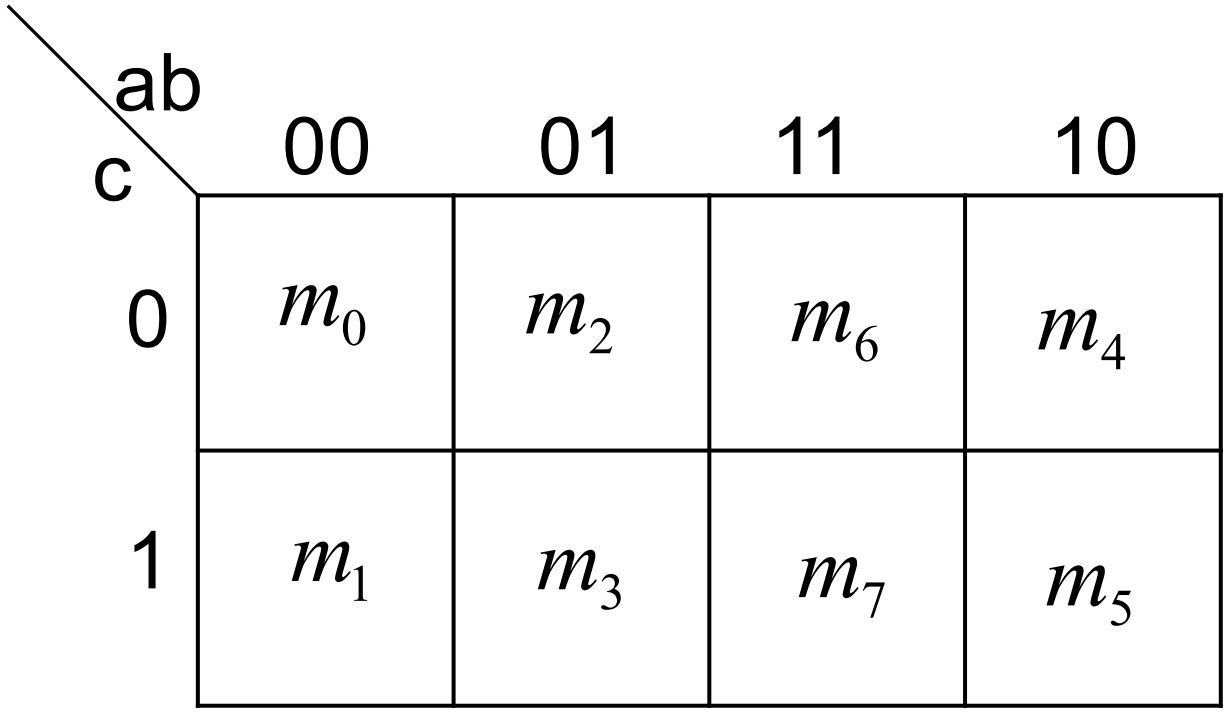


Cell = $2^3=8$

C \ AB	00	01	11	10
0	0 $\overline{A}\overline{B}\overline{C}$	2 $\overline{A}B\overline{C}$	6 $AB\overline{C}$	4 $A\overline{B}\overline{C}$
1	1 $\overline{A}\overline{B}C$	3 $\overline{A}BC$	7 ABC	5 $A\overline{B}C$



THREE-VARIABLE K-MAP

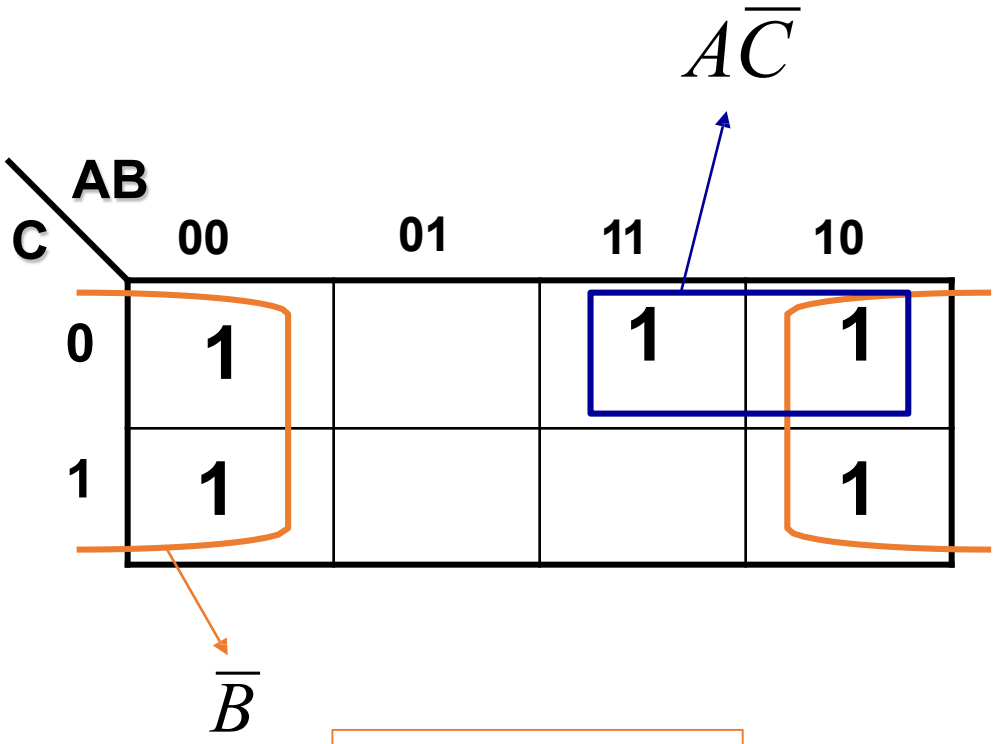


GUIDELINES FOR SIMPLIFYING FUNCTIONS

- Group as many squares as possible. This eliminates the most variables.
- Make as few groups as possible. Each group represents a separate product term.
- You must **cover** each minterm at least once. However, it may be covered more than once.

EXAMPLE

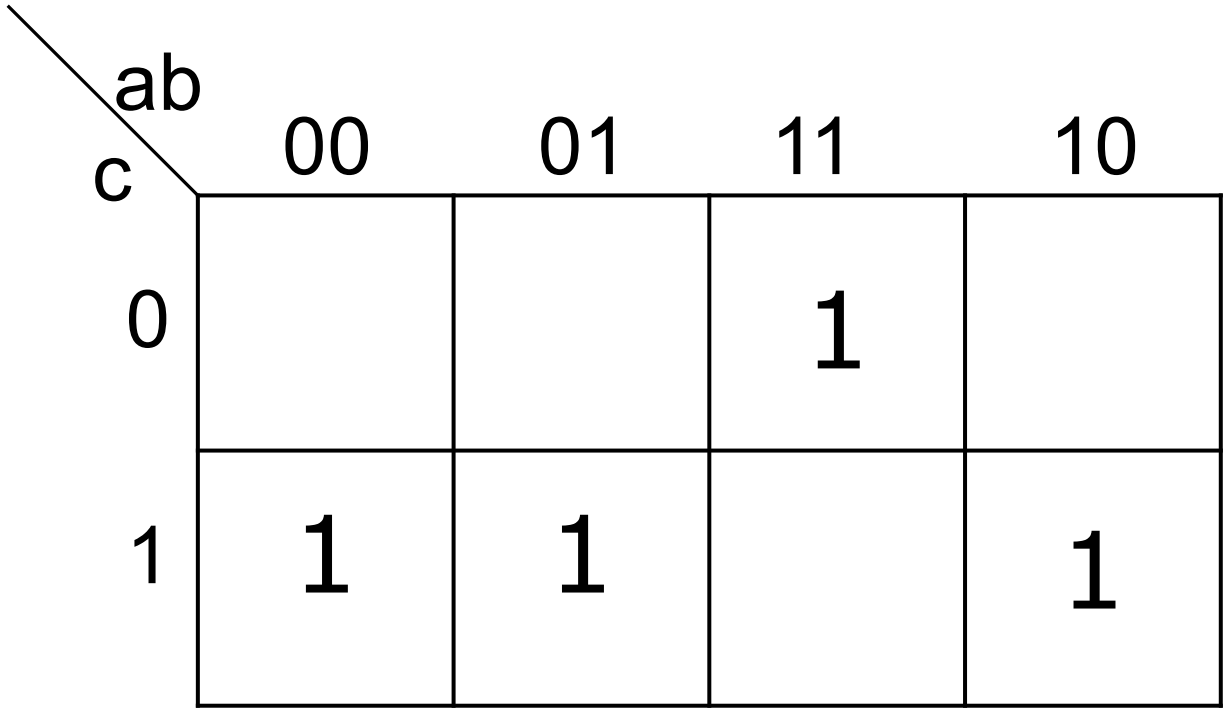
A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



$$\overline{B} + A\overline{C}$$

Three-Variable K-Map Example

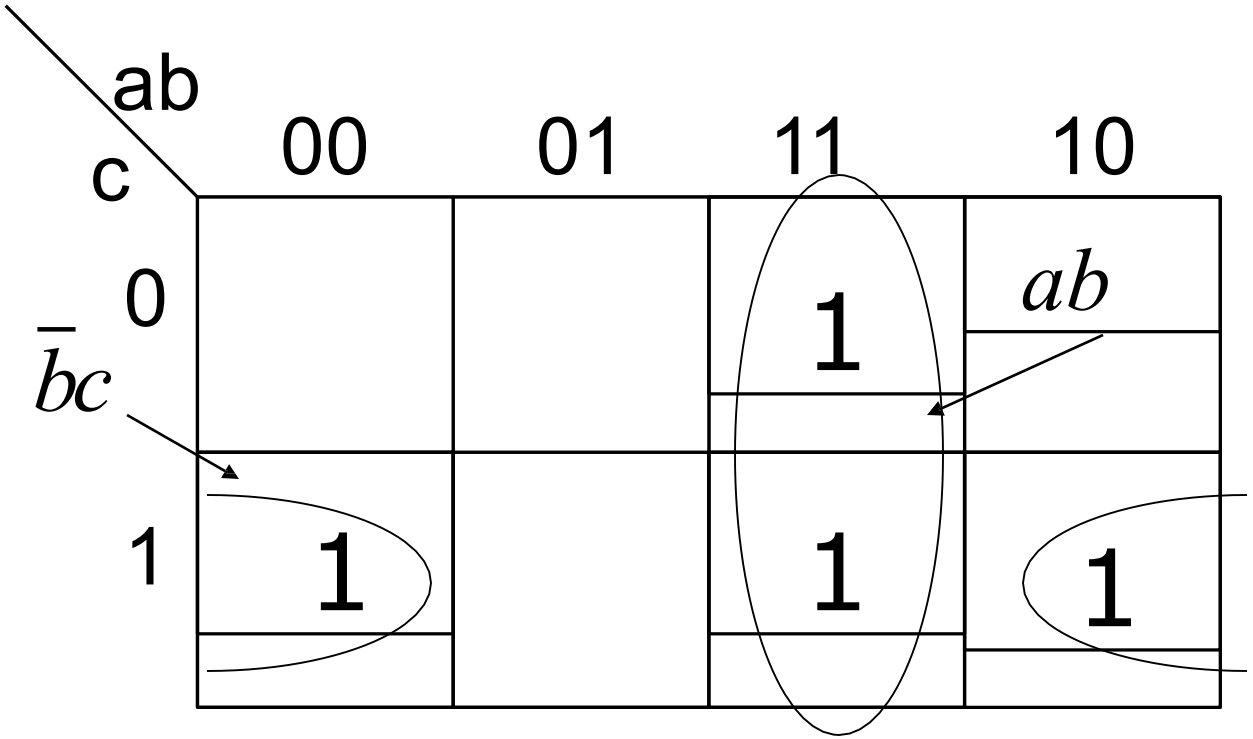
Plot 1's (minterms) of switching function



$$F(a,b,c)=\sum m(1,3,5,6)$$

Three-Variable K-Map Example

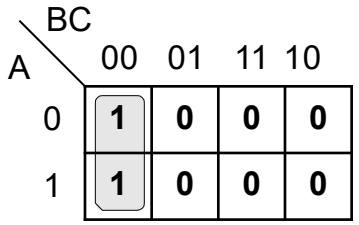
Plot 1's (minterms) of switching function



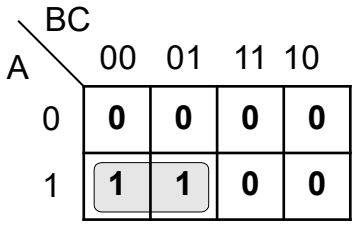
$$F(a,b,c) = ab + \bar{b}c$$

THREE-VARIABLE K-MAPS

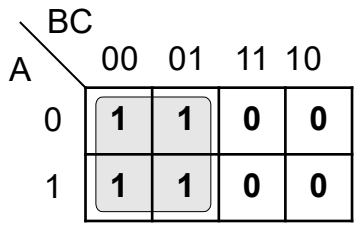
$f = \sum(0,4) = \overline{B}\,\overline{C}$



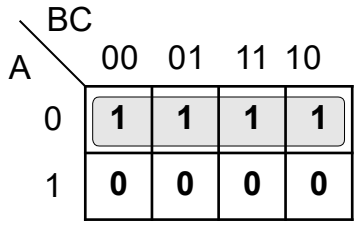
$f = \sum(4,5) = A\overline{B}$



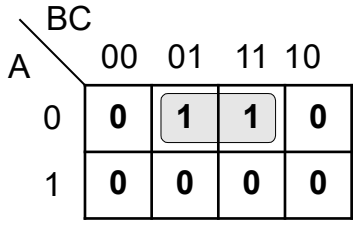
$f = \sum(0,1,4,5) = \overline{B}$



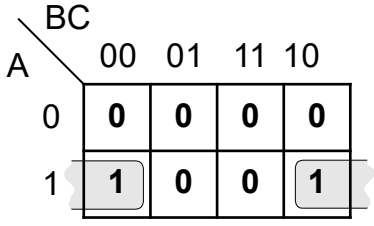
$f = \sum(0,1,2,3) = \overline{A}$



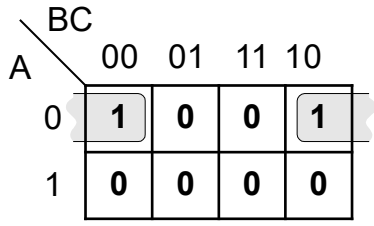
$f = \sum(0,4) = \overline{A}C$



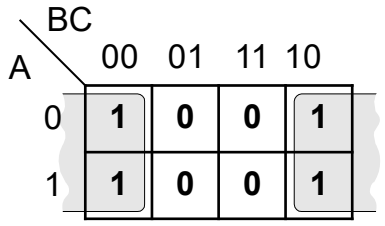
$f = \sum(4,6) = A\overline{C}$



$f = \sum(0,2) = \overline{A}\,\overline{C}$



$f = \sum(0,2,4,6) = \overline{C}$



THREE-VARIABLE K-MAP EXAMPLES

A \ BC	00	01	11	10
0	0	1	0	0
1	1	0	1	1

A \ BC	00	01	11	10
0			1	
1	1		1	1

A \ BC	00	01	11	10
0	1	1	1	1
1	1			1

A \ BC	00	01	11	10
0		1	1	1
1		1	1	

A \ BC	00	01	11	10
0			1	1
1	1	1		

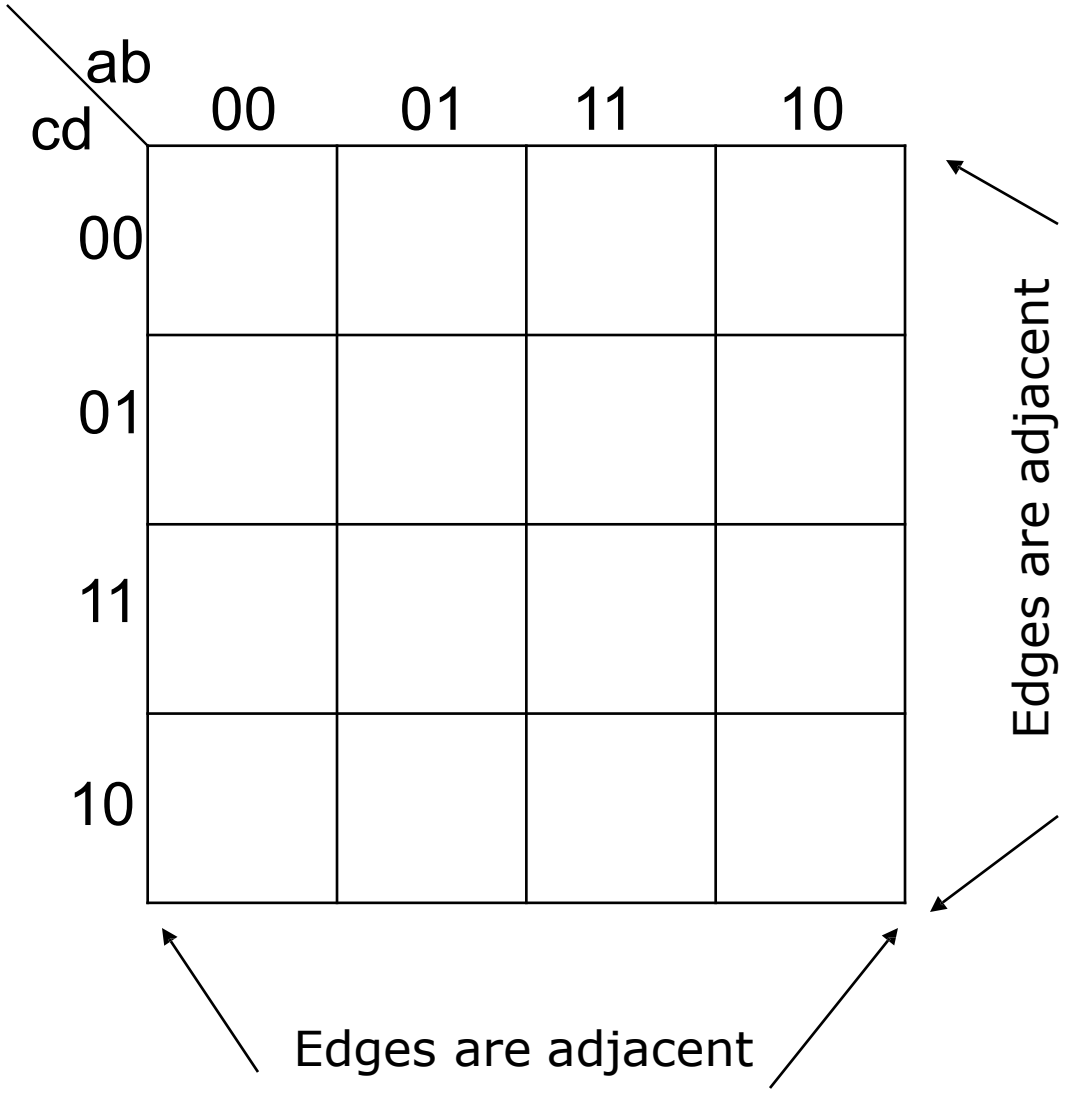
A \ BC	00	01	11	10
0	1	1	1	
1		1	1	1

FOUR VARIABLE EXAMPLES

FOUR-VARIABLE K-MAP

cd \ ab	00	01	11	10
00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

FOUR-VARIABLE K-MAP

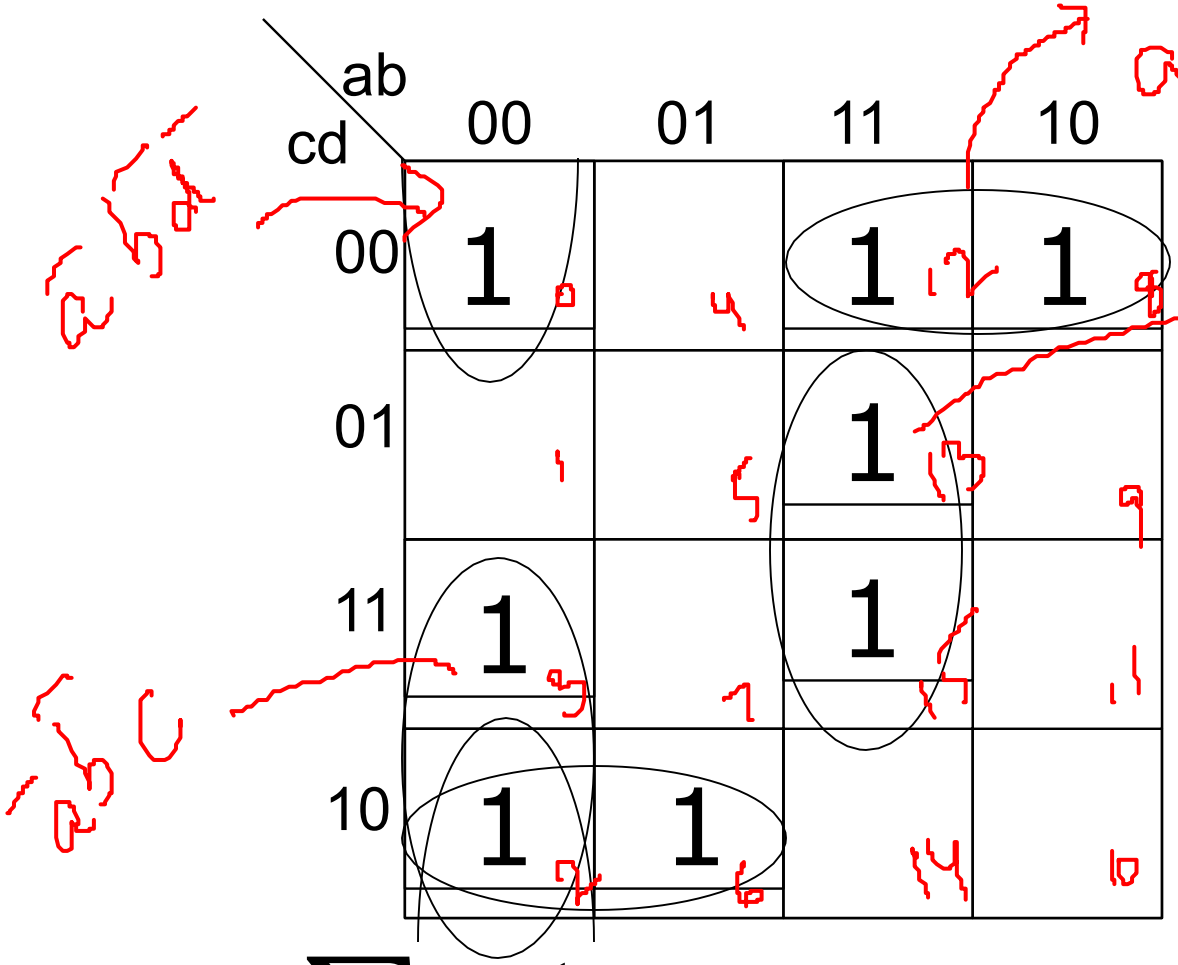


Example

- Use a K-Map to simplify the following Boolean expression

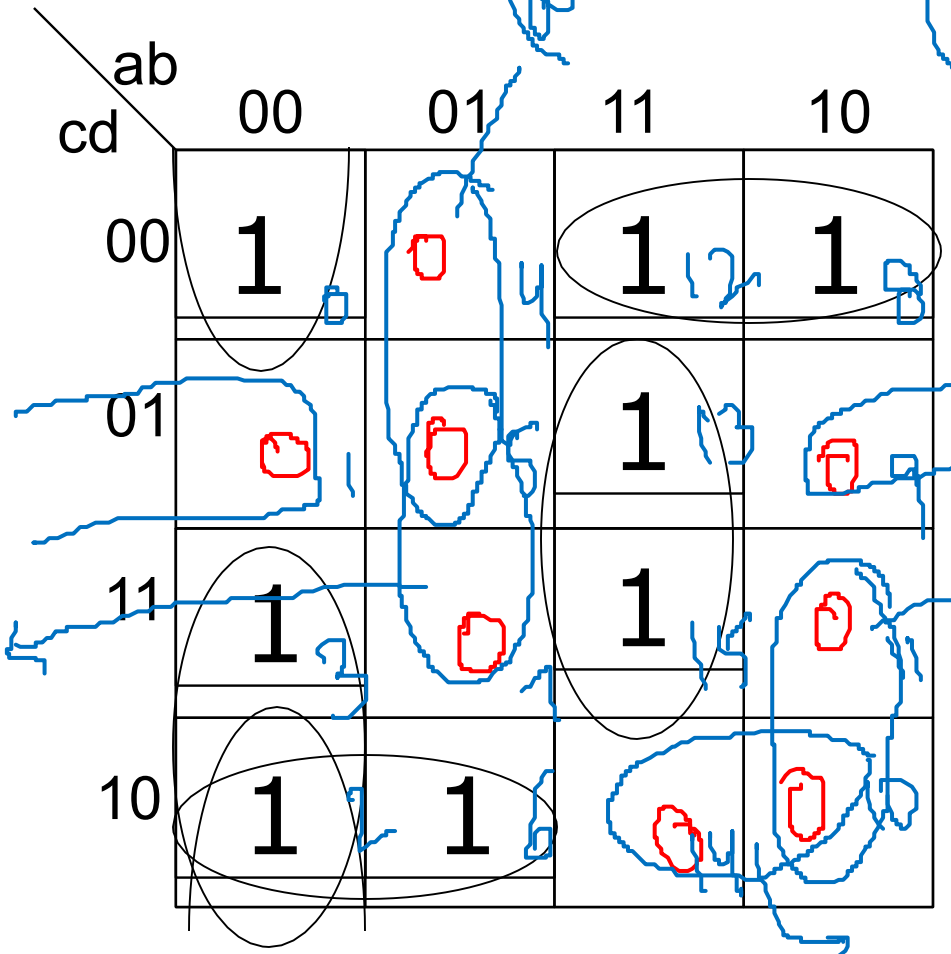
$$F(a,b,c,d)=\sum m(0,2,3,6,8,12,13,15)$$

Four-variable K-Map



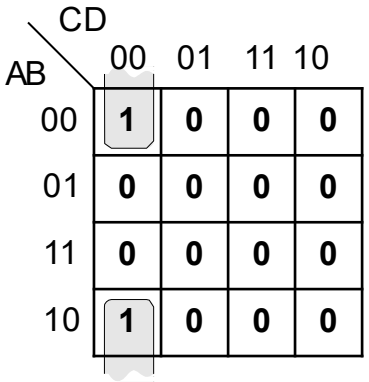
$$F = \sum m(0,2,3,6,8,12,13,15)$$

Four-variable K-Map

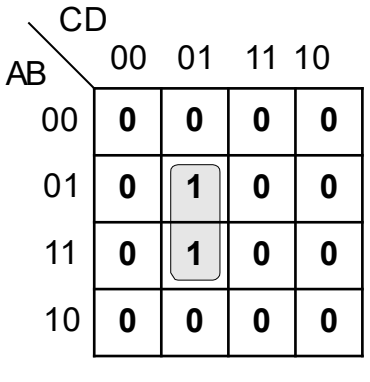


$$F = \bar{a}\bar{b}\bar{d} + \bar{a}bc + \bar{a}c\bar{d} + ab\bar{d} + ac\bar{d}$$

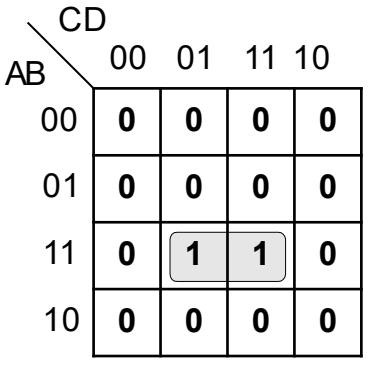
FOUR-VARIABLE K-MAPS



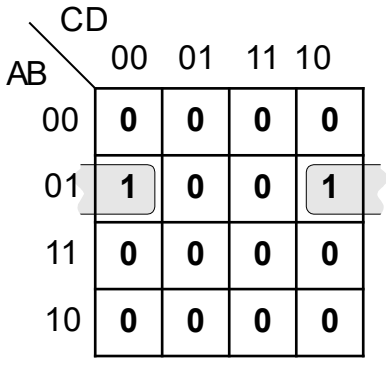
$f = \sum(0,8) = \overline{B} \bullet \overline{C} \bullet \overline{D}$



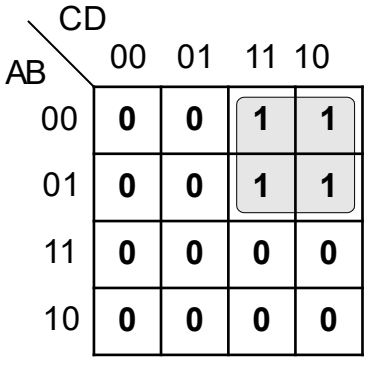
$f = \sum(5,13) = B \bullet \overline{C} \bullet D$



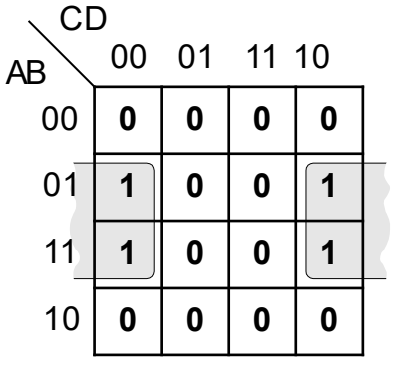
$f = \sum(13,15) = A \bullet B \bullet D$



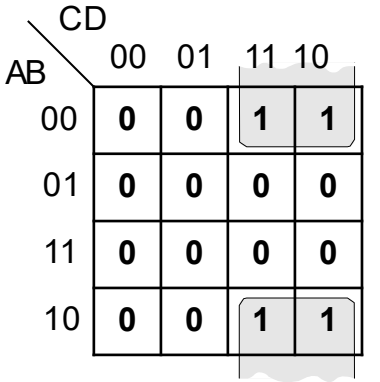
$f = \sum(4,6) = \overline{A} \bullet B \bullet \overline{D}$



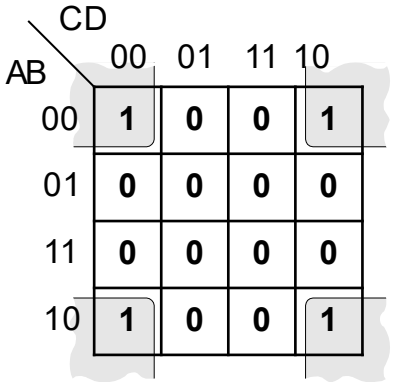
$f = \sum(2,3,6,7) = \overline{A} \bullet C$



$f = \sum(4,6,12,14) = B \bullet \overline{D}$

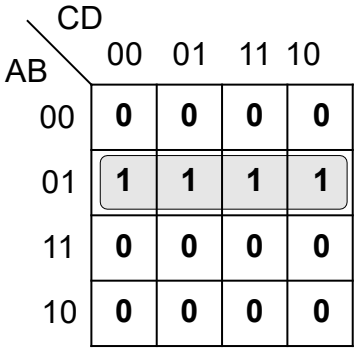


$f = \sum(2,3,10,11) = \overline{B} \bullet C$

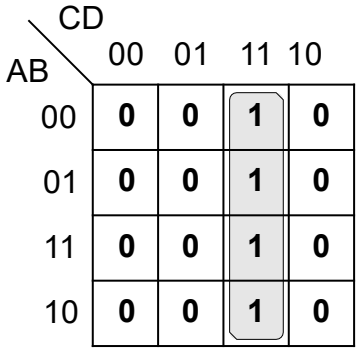


$f = \sum(0,2,8,10) = \overline{B} \bullet \overline{D}$

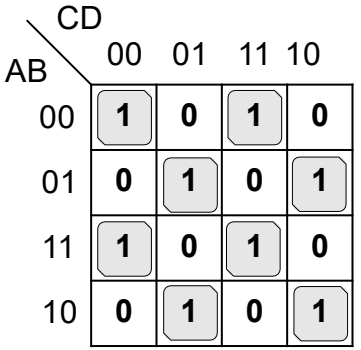
FOUR-VARIABLE K-MAPS



$f = \sum(4,5,6,7) = \overline{A} \bullet B$

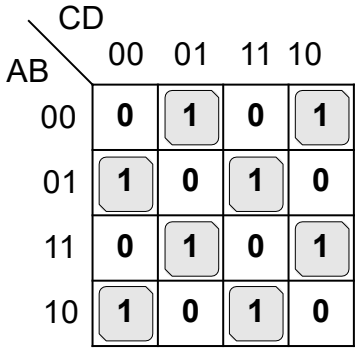


$f = \sum(3,7,11,15) = C \bullet D$



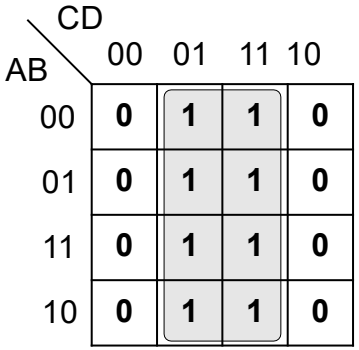
$f = \sum(0,3,5,6,9,10,12,15)$

$f = A \otimes B \otimes C \otimes D$



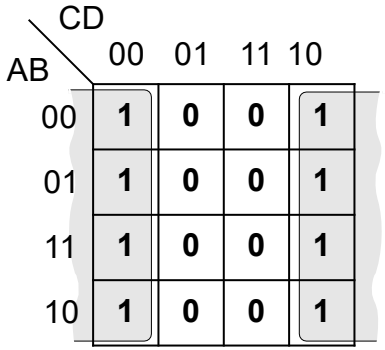
$f = \sum(1,2,4,7,8,11,13,14)$

$f = A \oplus B \oplus C \oplus D$



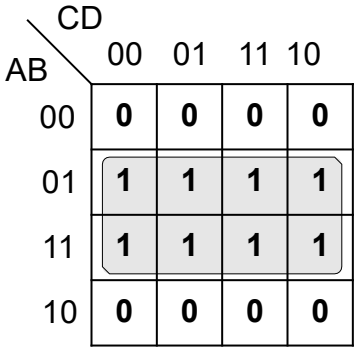
$f = \sum(1,3,5,7,9,11,13,15)$

$f = D$



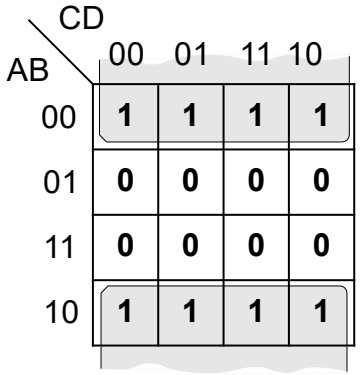
$f = \sum(0,2,4,6,8,10,12,14)$

$f = \overline{D}$



$f = \sum(4,5,6,7,12,13,14,15)$

$f = B$

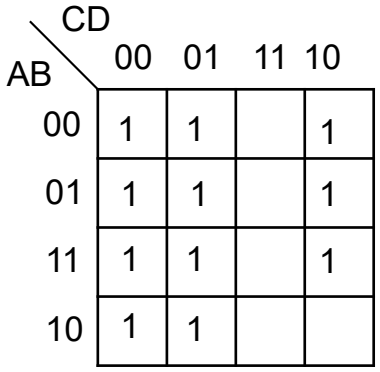


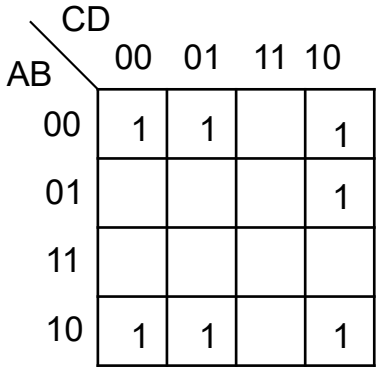
$f = \sum(0,1,2,3,8,9,10,11)$

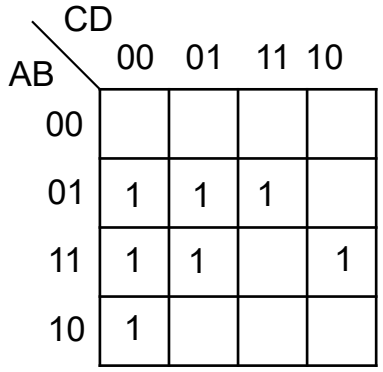
$f = \overline{B}$

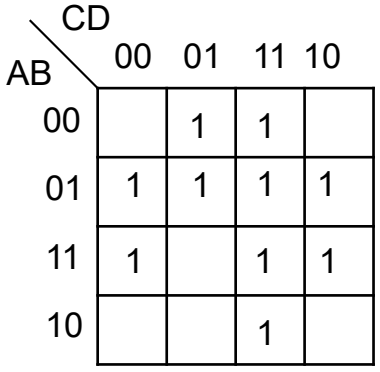
FOUR-VARIABLE K-MAPS

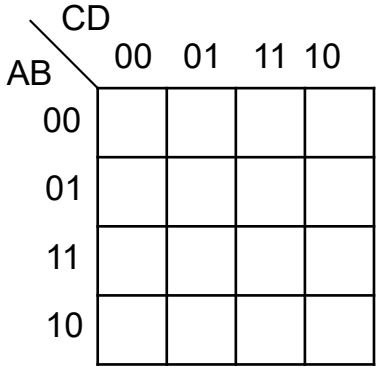
EXAMPLES

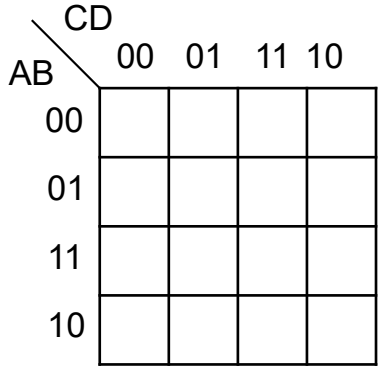












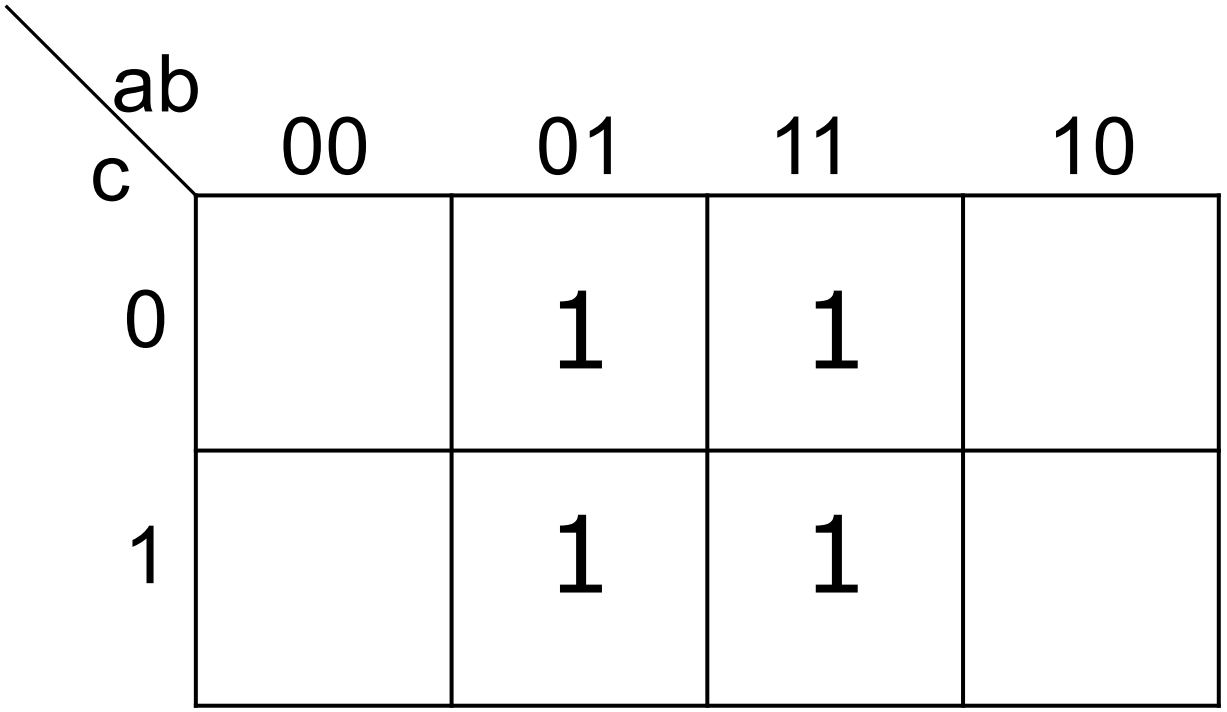
Example

- Use a K-Map to simplify the following Boolean expression

$$F(a,b,c) = \sum m(2,3,6,7)$$

Three-Variable K-Map Example

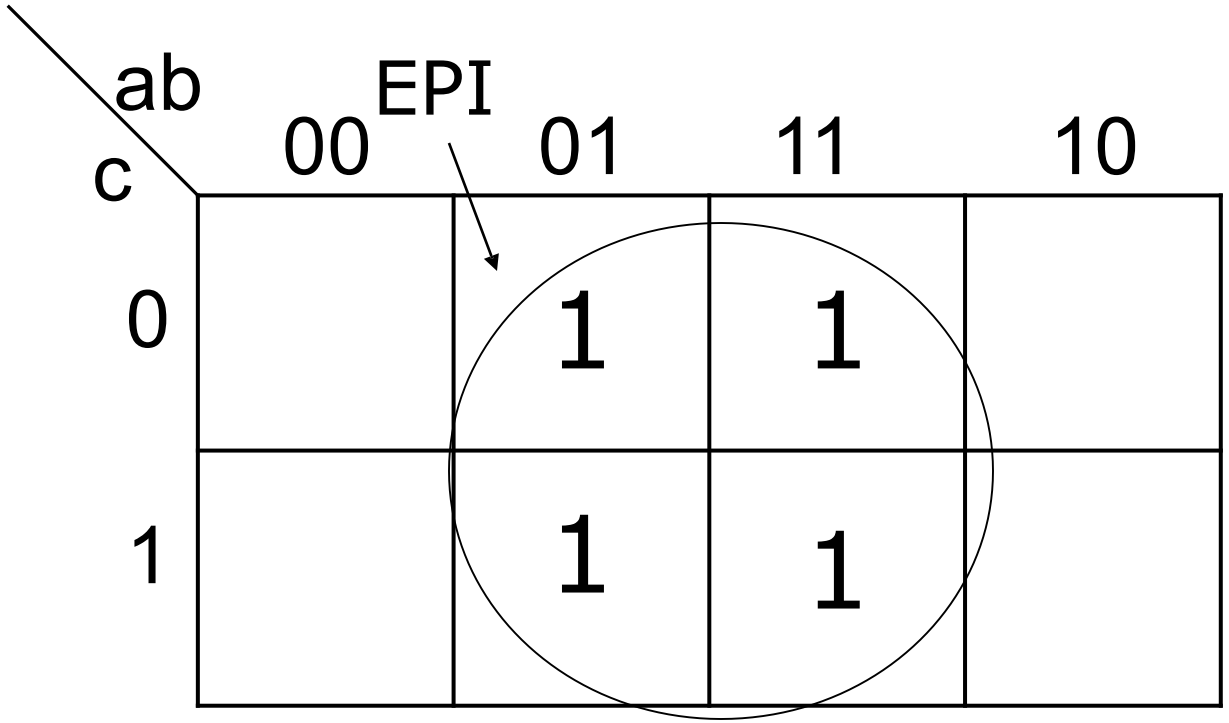
Step 1: Plot the K-map



$$F(a,b,c)=\sum m(2,4,5,7)$$

Three-Variable K-Map Example

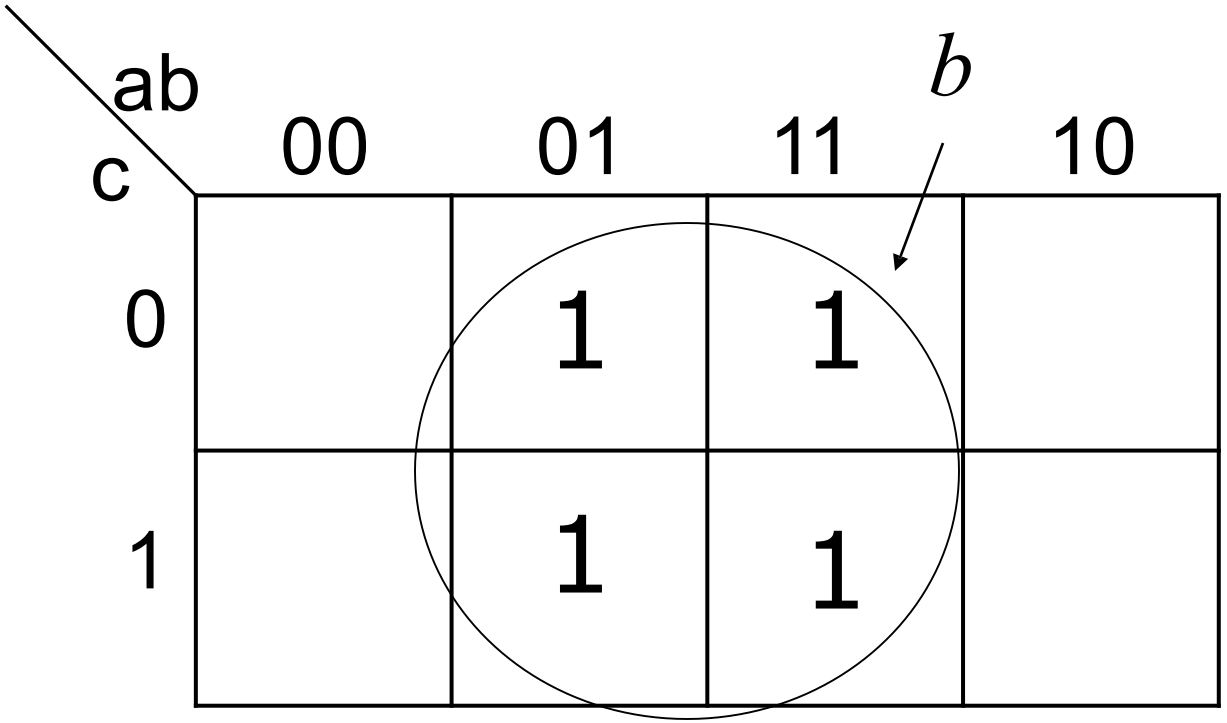
Step 3: Identify Essential Prime Implicants



$$F(a,b,c)=\sum m(2,3,6,7)$$

Three-Variable K-Map Example

Step 5: Read the map.



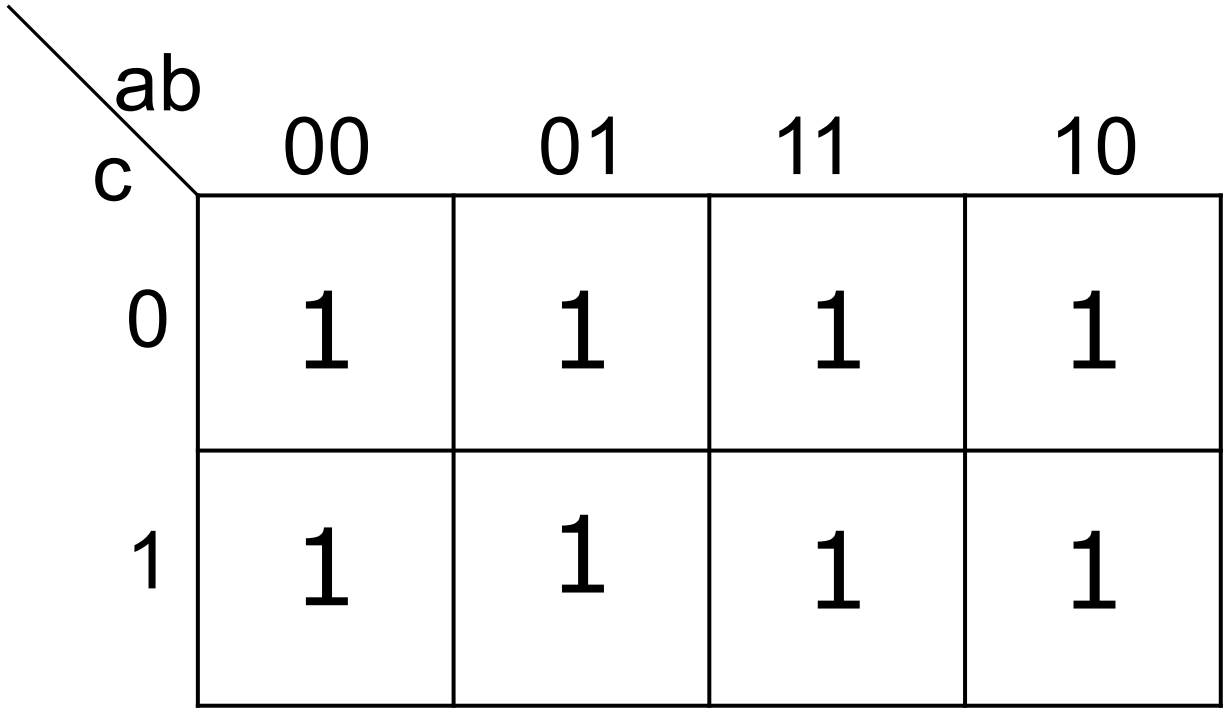
$$F(a,b,c)=\sum m(2,3,6,7)$$

Solution

$$F(a,b,c)=b$$

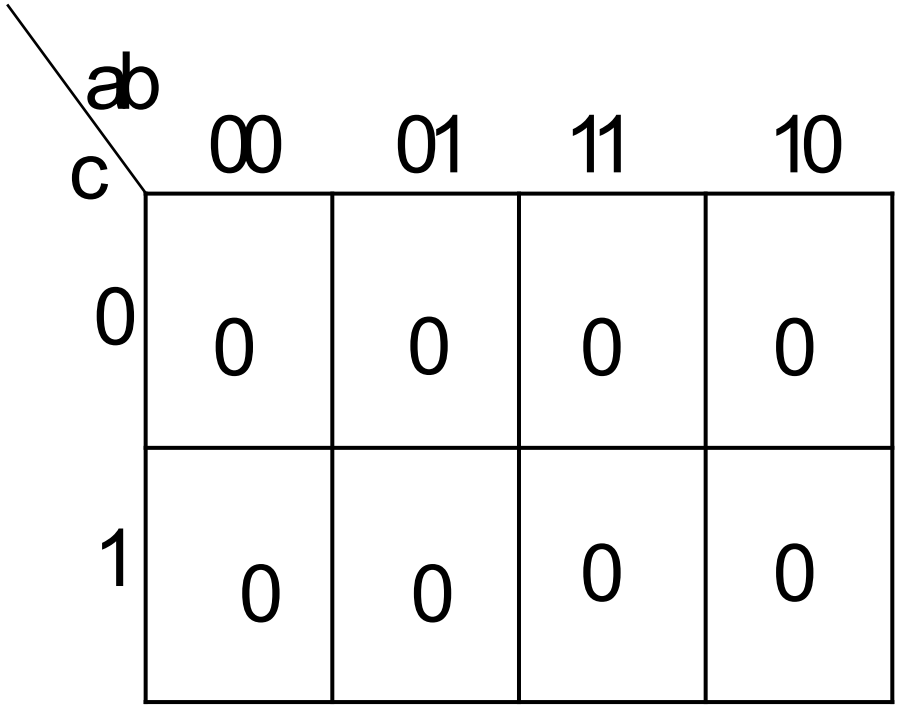
SPECIAL CASES

THREE-VARIABLE K-MAP EXAMPLE



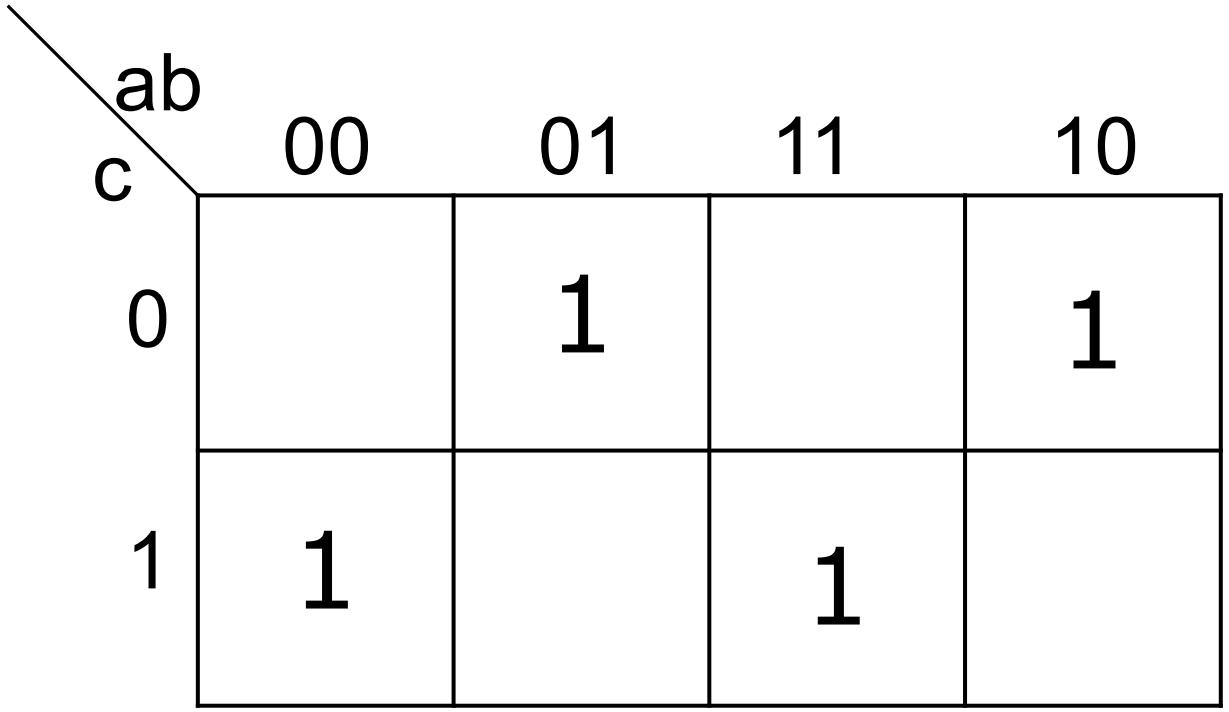
$$F(a,b,c)=1$$

THREE-VARIABLE K-MAP EXAMPLE



$$F(a,b,c)=0$$

THREE-VARIABLE K-MAP EXAMPLE



$$F(a,b,c) = a \oplus b \oplus c$$

Example

- Use a K-Map to simplify the following Boolean expression

$$F(a,b,c)=\sum m(1,2,3,5,6)$$

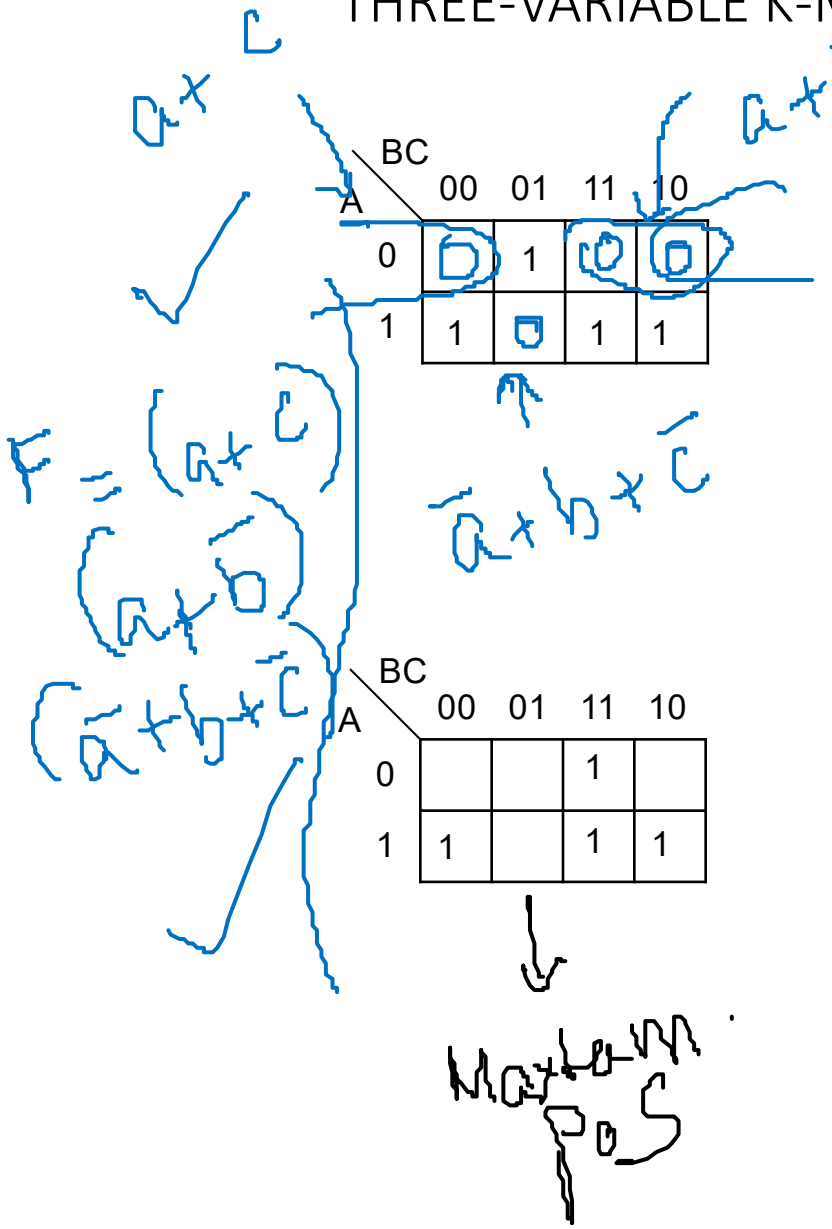
Three-Variable K-Map Example

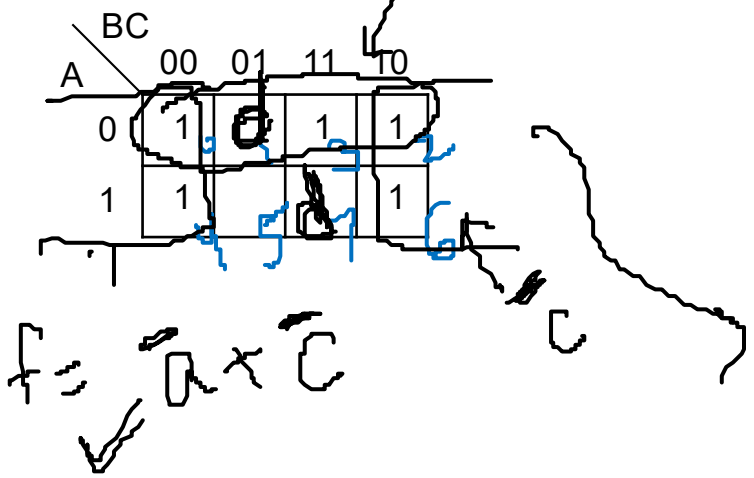
Step 1: Plot the K-map

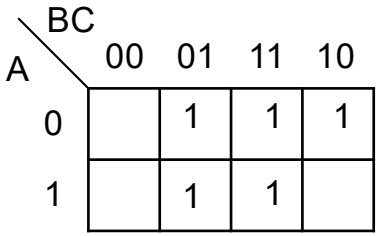
c \ ab	00	01	11	10
0		1	1	
1	1	1		1

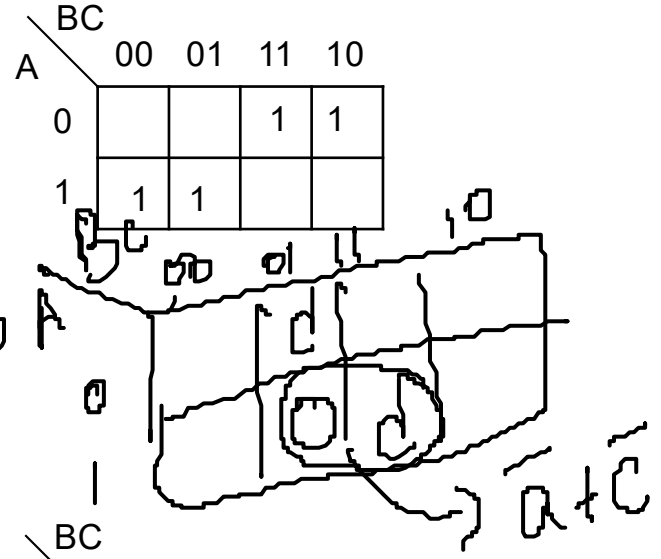
$$F(a,b,c)=\sum m(1,2,3,5,6)$$

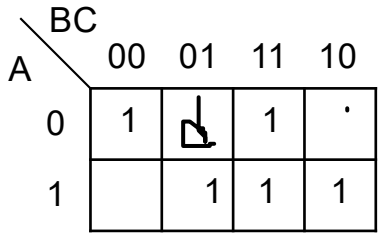
THREE-VARIABLE K-MAP MORE EXAMPLES













B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026

Course name:Digital system

COURSE CODE : ECE1001

LECTURE SERIES NO : 01(ONE)

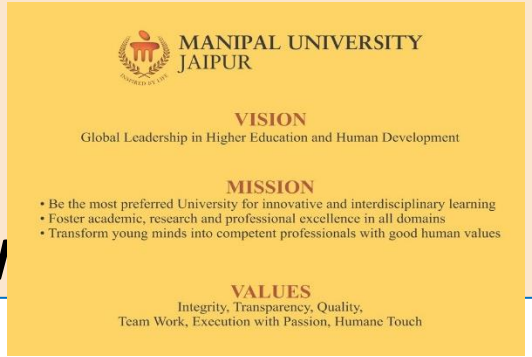
CREDITS : 3

MODE OF DELIVERY : ONLINE (POW

PRESENTATION)

FACULTY :

EMAIL-ID :



Session outcome

"DISCUSS AND UNDERSTAND VARIOUS COMBINATIONAL LOGIC CIRCUITS"

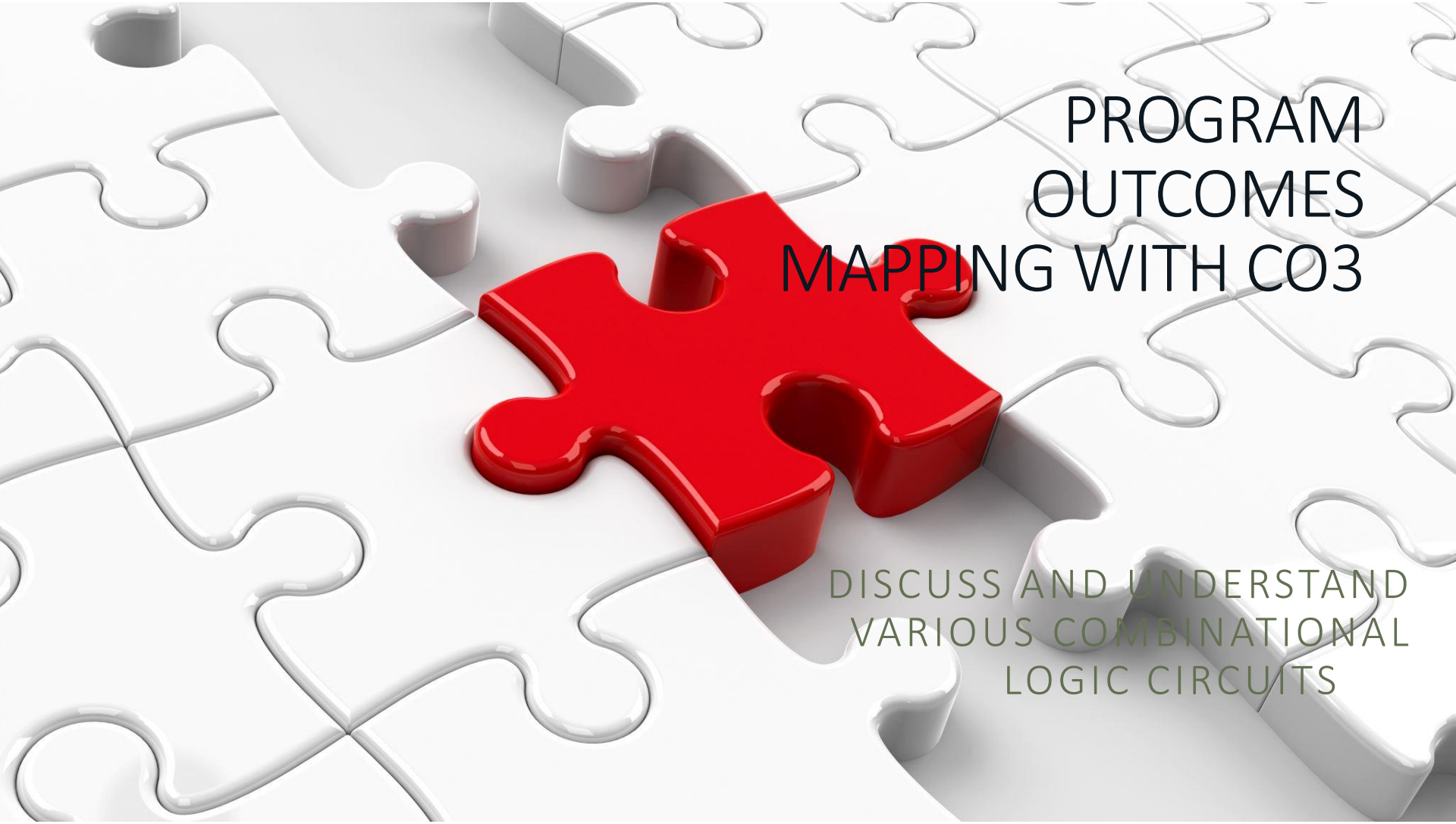
Assessment criteria’S

ASSIGNMENT

QUIZ

MID TERM EXAMINATION – II

END TERM EXAMINATION



PROGRAM OUTCOMES MAPPING WITH CO3

DISCUSS AND UNDERSTAND VARIOUS COMBINATIONAL LOGIC CIRCUITS

Combinational Logic

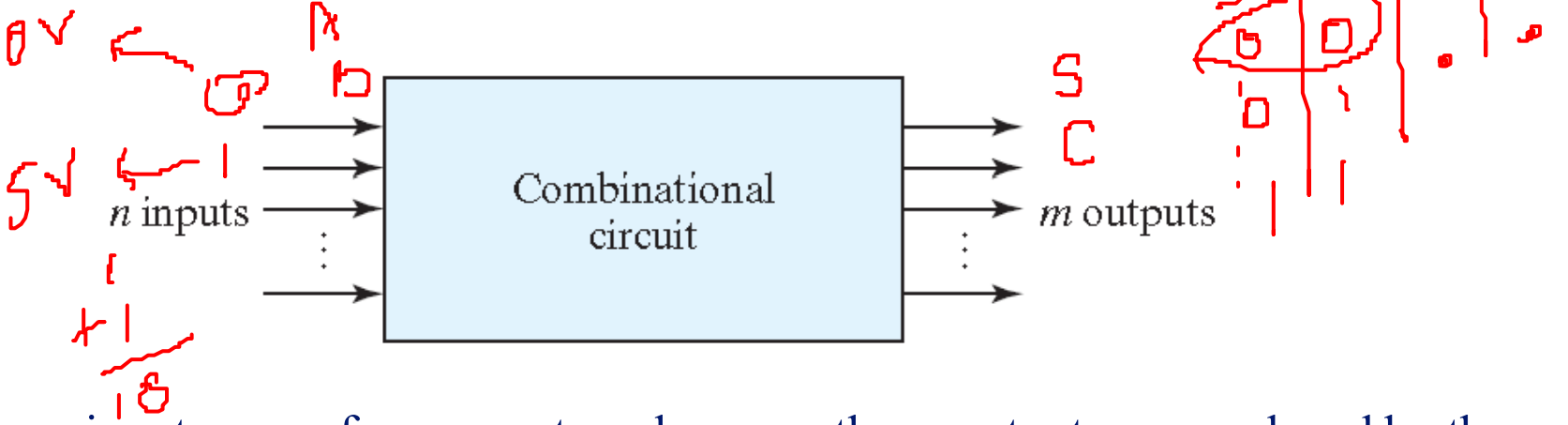
1. Introduction
2. Combinational Circuits
3. Binary Adder–Subtractor
4. Decimal Adder
5. Binary Multiplier
6. Magnitude Comparator
7. Decoders
8. Encoders
9. Multiplexers

Introduction

- Logic circuits may be ***combinational*** or ***sequential***.
- A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs.
- The operation of combinational circuits can be specified logically by a set of Boolean functions.
- Sequential circuits contain *storage* elements in addition to logic gates.
- The outputs of sequential circuits are a function of the inputs and the state of the storage elements.
- Because the state of the storage elements is a function of previous inputs, the outputs of a sequential circuit depend not only on present values of inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states.
- Sequential circuits are discussed in later in this course.

Combinational Circuits

- A combinational circuit consists of an interconnection of logic gates.
- Combinational circuits react to the values at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data.
- A block diagram of a combinational circuit is shown.



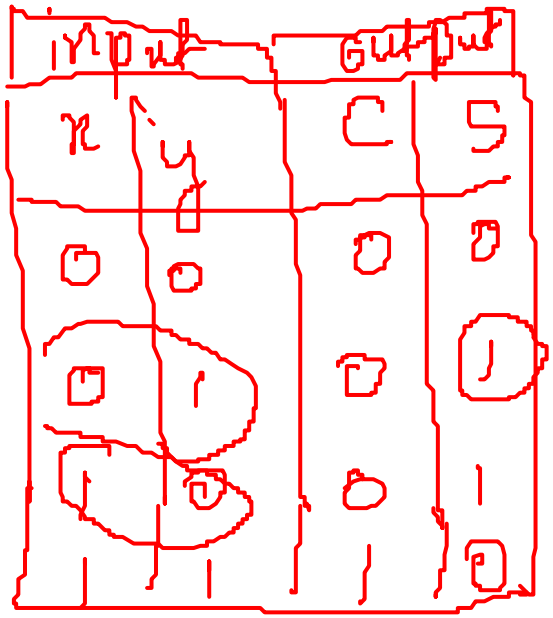
  - The *n* inputs come from an external source; the *m* outputs are produced by the combinational circuit and go to an external destination.
  - Each input and output variable is an analog electrical signal whose values are interpreted to be a binary signal that represents logic 1 and logic 0.

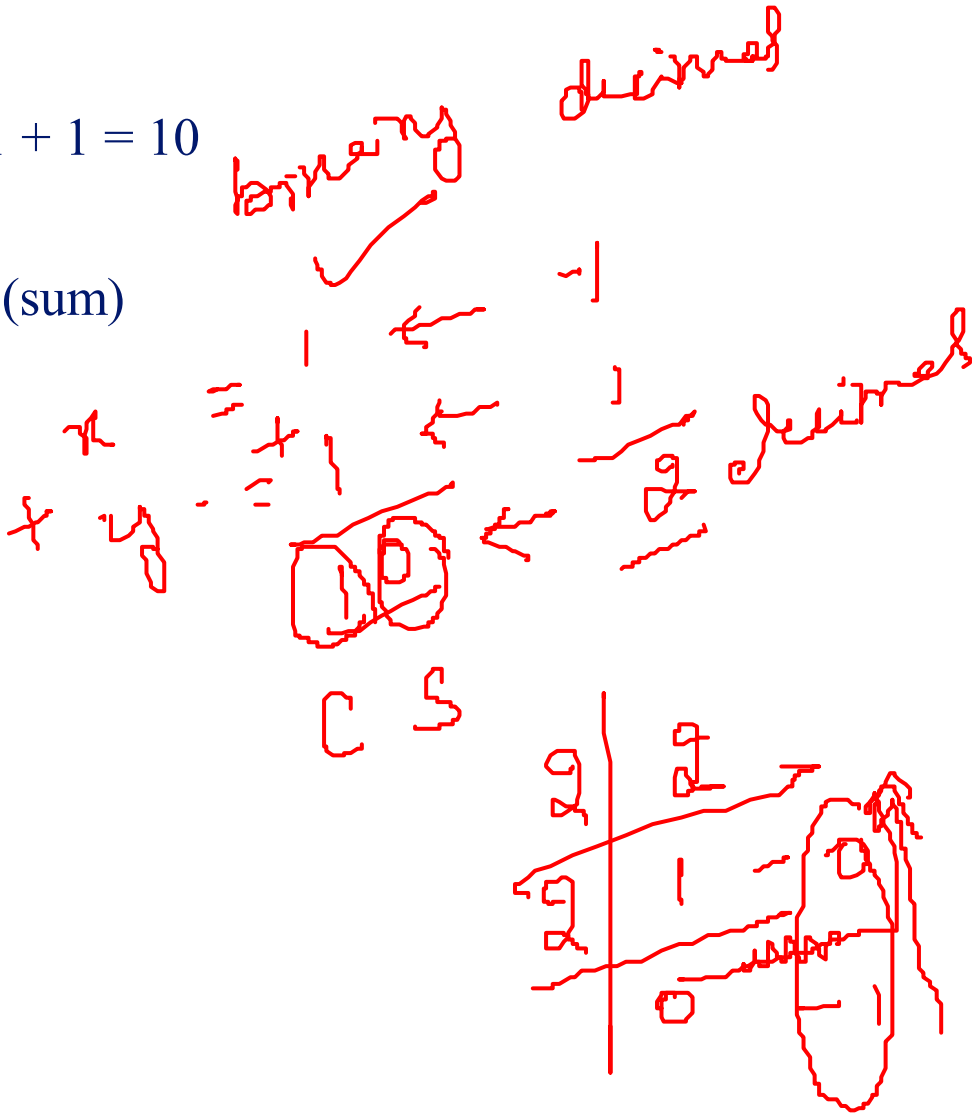
- For n input variables, there are 2^n possible combinations.
- For each possible input combination, there is one possible value for each output.
- Thus, a combinational circuit can be specified with a truth table that lists the output values for each combination of input variables.
- A combinational circuit also can be described by m Boolean functions, one for each output variable.
- Each output function is expressed in terms of the n input variables.

- Several extensively used combinational circuits, such as adders, subtractors, multipliers, comparators, decoders, encoders, and multiplexers, are available in standard integrated circuit components and used as *standard cells* in complex very large scale integrated (VLSI) circuits.

Binary Adder-Subtractor

- **Half adder**
  - 0 + 0 = 0; 0 + 1 = 1; 1 + 0 = 1; 1 + 1 = 10
  - two input variables: x, y
  - two output variables: C (carry), S (sum)
  - truth table



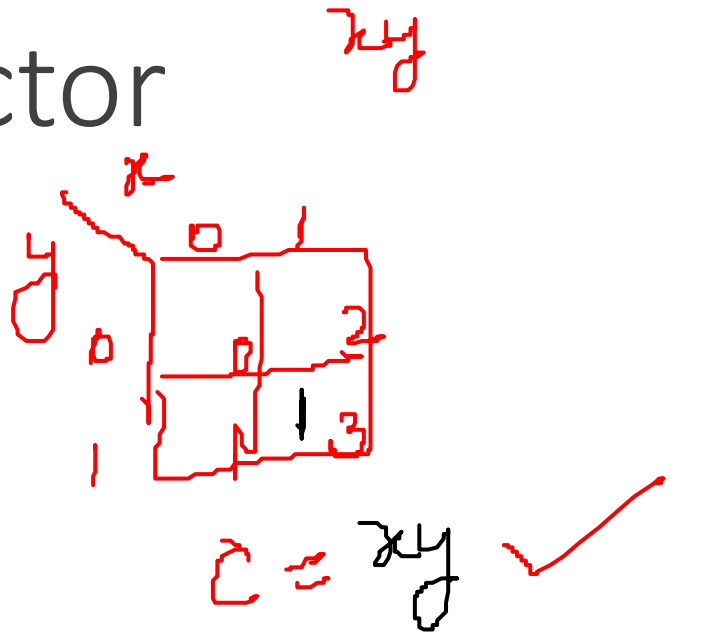


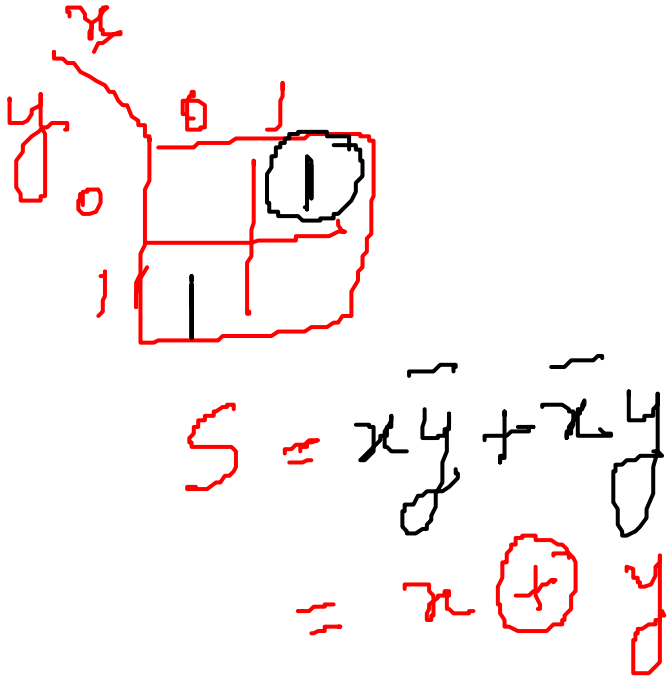
Binary Adder-Subtractor

- **Half adder**
  - 0 + 0 = 0; 0 + 1 = 1; 1 + 0 = 1; 1 + 1 = 10
  - two input variables: x, y
  - two output variables: C (carry), S (sum)
  - truth table

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0





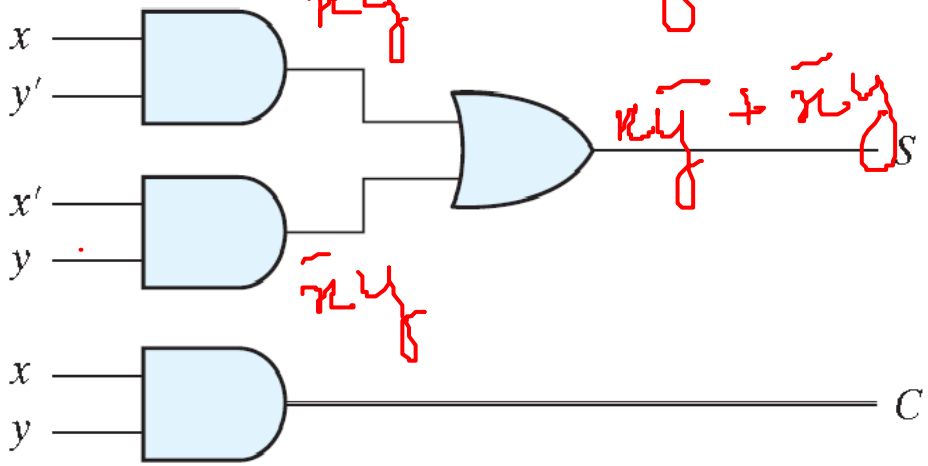
  - Boolean functions

$$S = x'y + xy'$$

$$C = xy$$

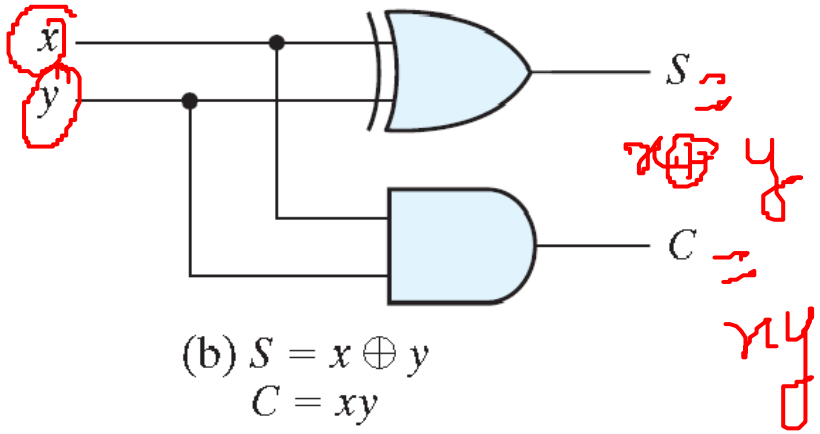
Implementation of Half Adder

- sum of products



(a) $S = xy' + x'y$
$C = xy$

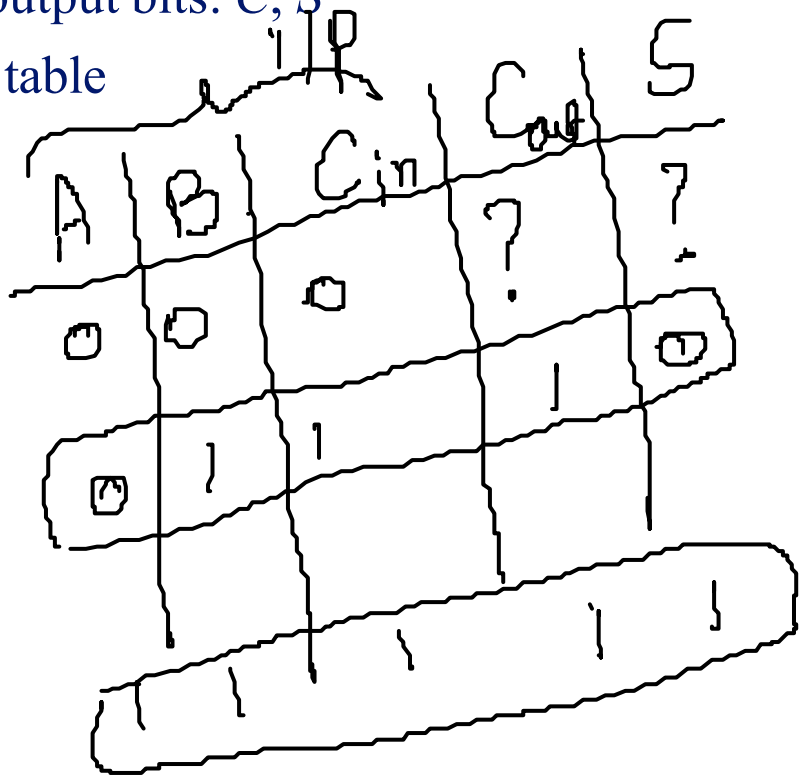
exclusive-OR and AND



(b) $S = x \oplus y$
$C = xy$

Full-Adder

- The arithmetic sum of three input bits
- Three input bits
  - x, y: two significant bits
  - z: the carry bit from the previous lower significant bit
- Two output bits: C, S
- Truth table



Full-Adder

- The arithmetic sum of three input bits
- Three input bits
  - x, y: two significant bits
  - z: the carry bit from the previous lower significant bit
- Two output bits: C, S
- Truth table

Full Adder

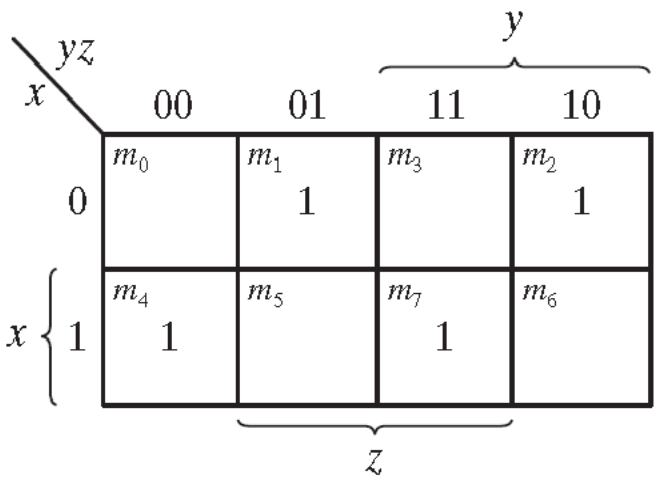
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Boolean Functions of Full Adder

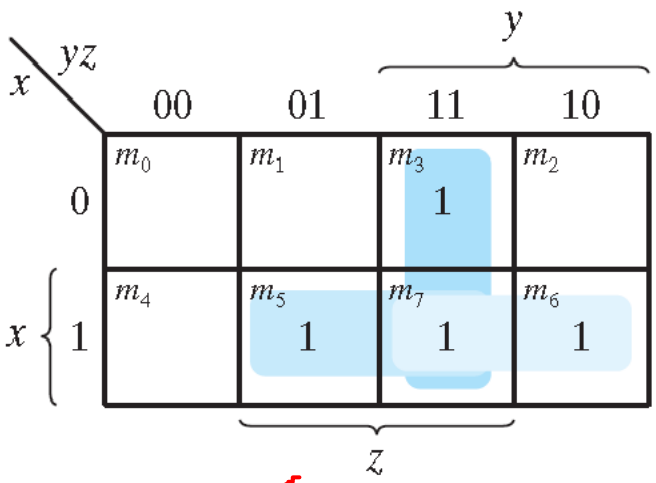
$$S = x'y'z + x'yz' + xy'z' + xyz$$
$$C = xy + xz + yz$$

Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



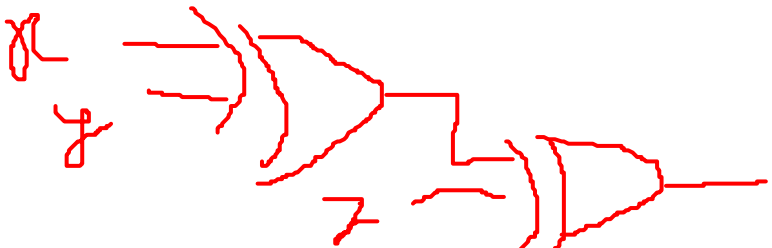
(a) $S = x'y'z + x'yz' + xy'z' + xyz$



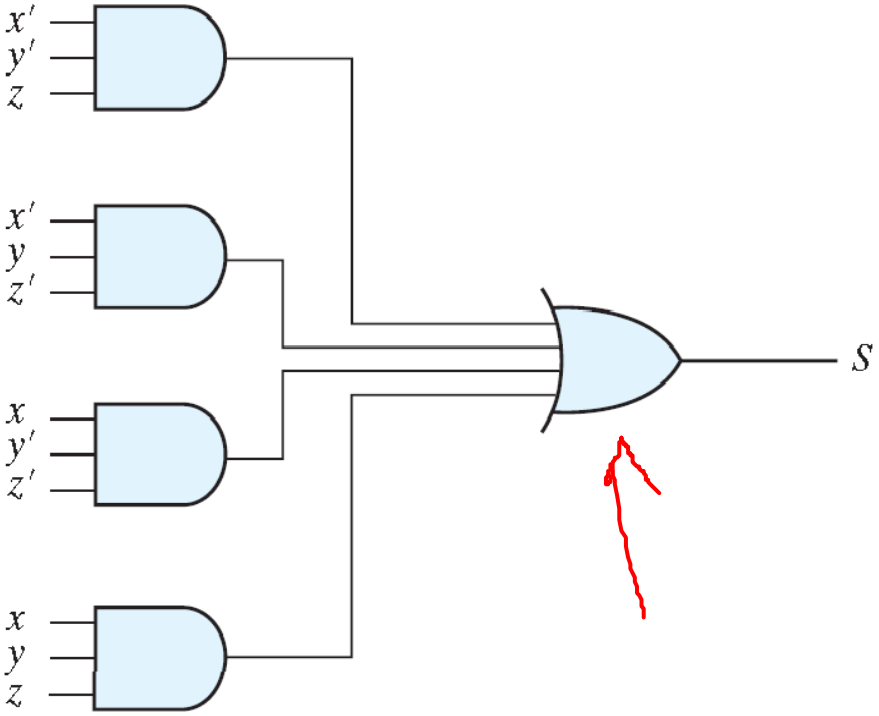
(b) $C = xy + xz + yz$

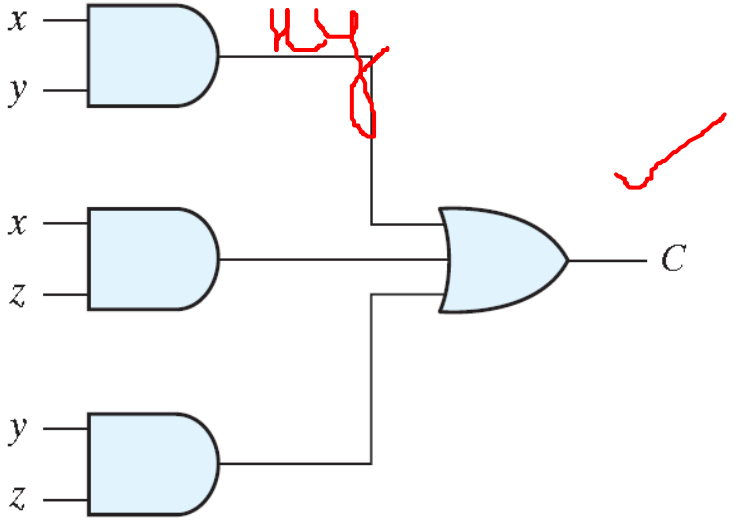
$$S = x \oplus y \oplus z$$

$$C = xy + (x + y)\,z$$

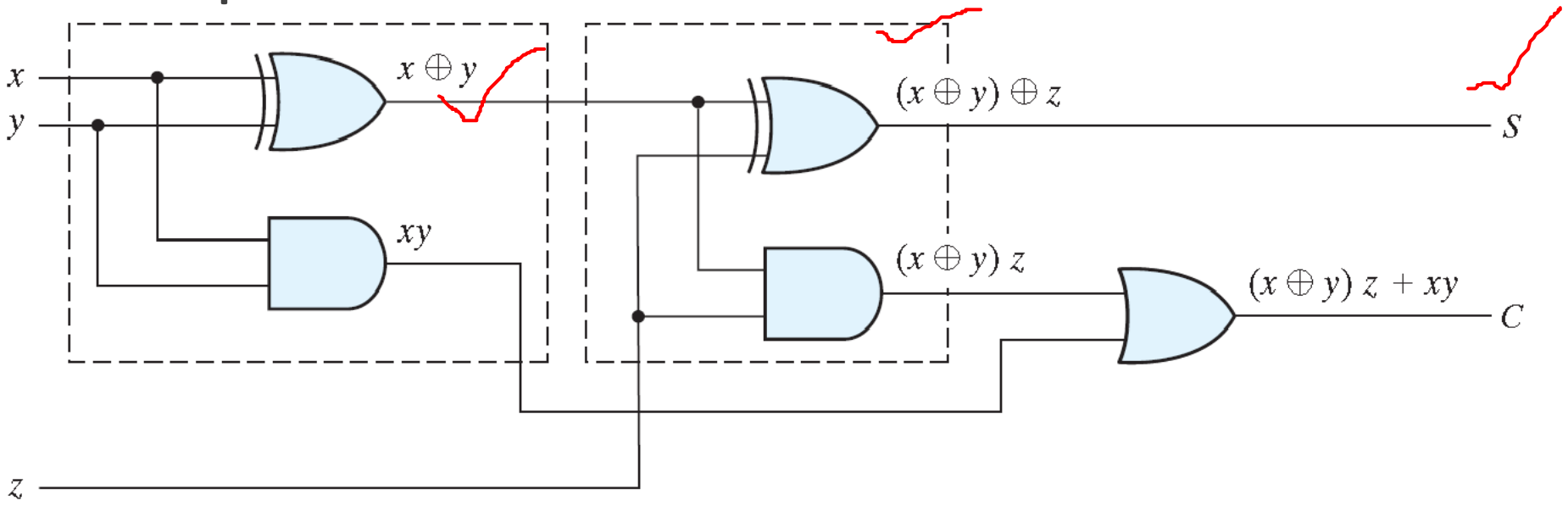


Sum-of-Product Implementation of Full Adder





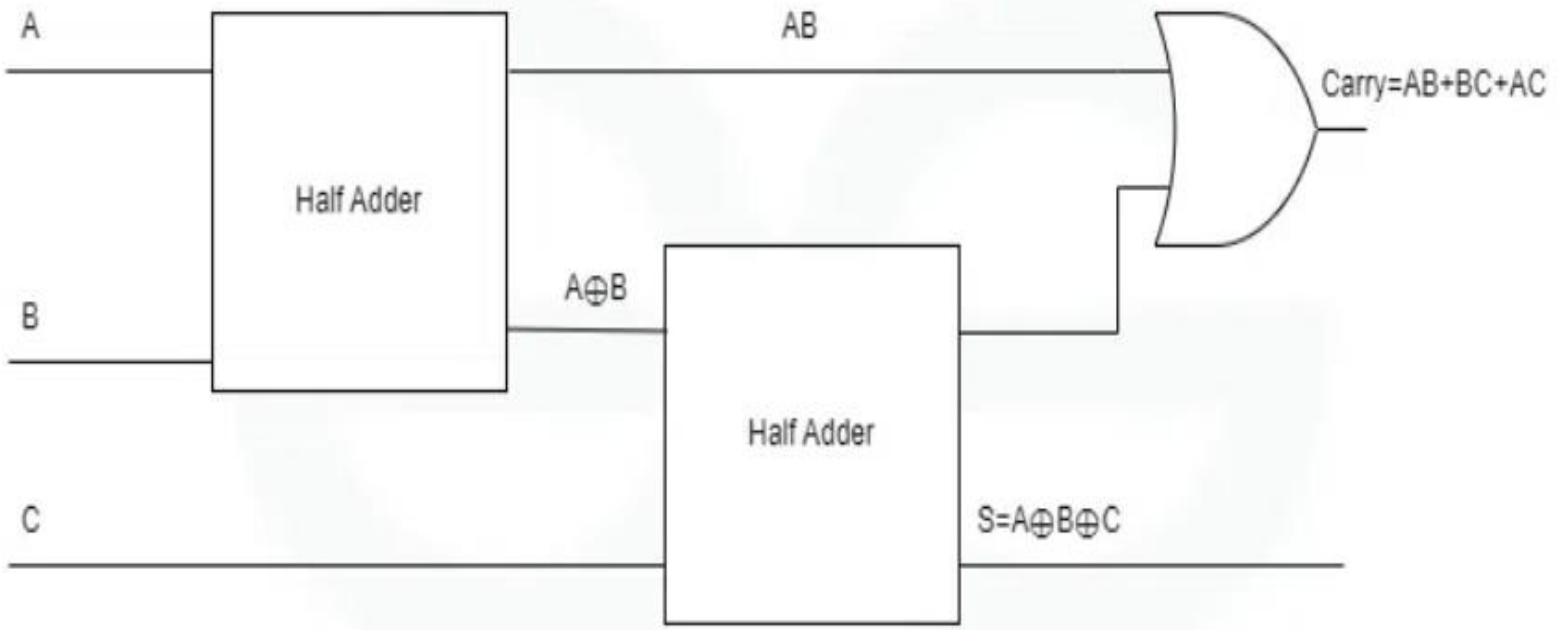
Implementation of Full Adder



$$S = x \oplus y \oplus z$$

$$C = xy + (x + y)\,z$$
$$= xy + (x \oplus y)\,z$$

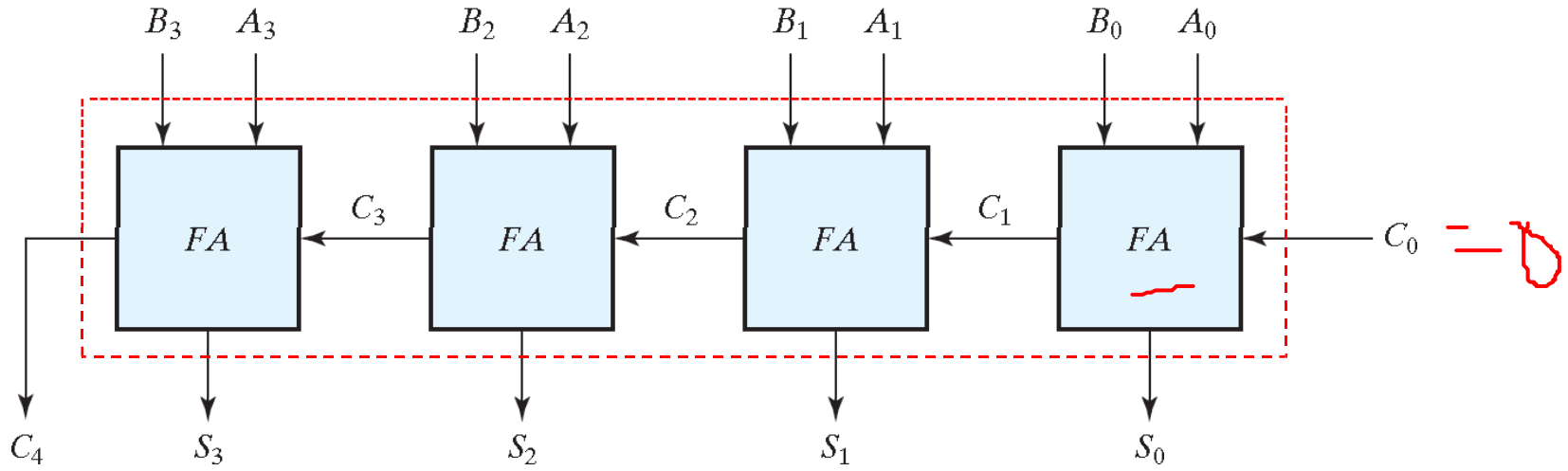
Implementation of Full Adder Using Half Adders



Binary Adder

- A binary adder can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain (called *ripple-carry* adder).
  - Example: 4-bit binary adder

Is it possible to design it with truth table?



$A = 1011$

$B = 0011$

$S = 1110$

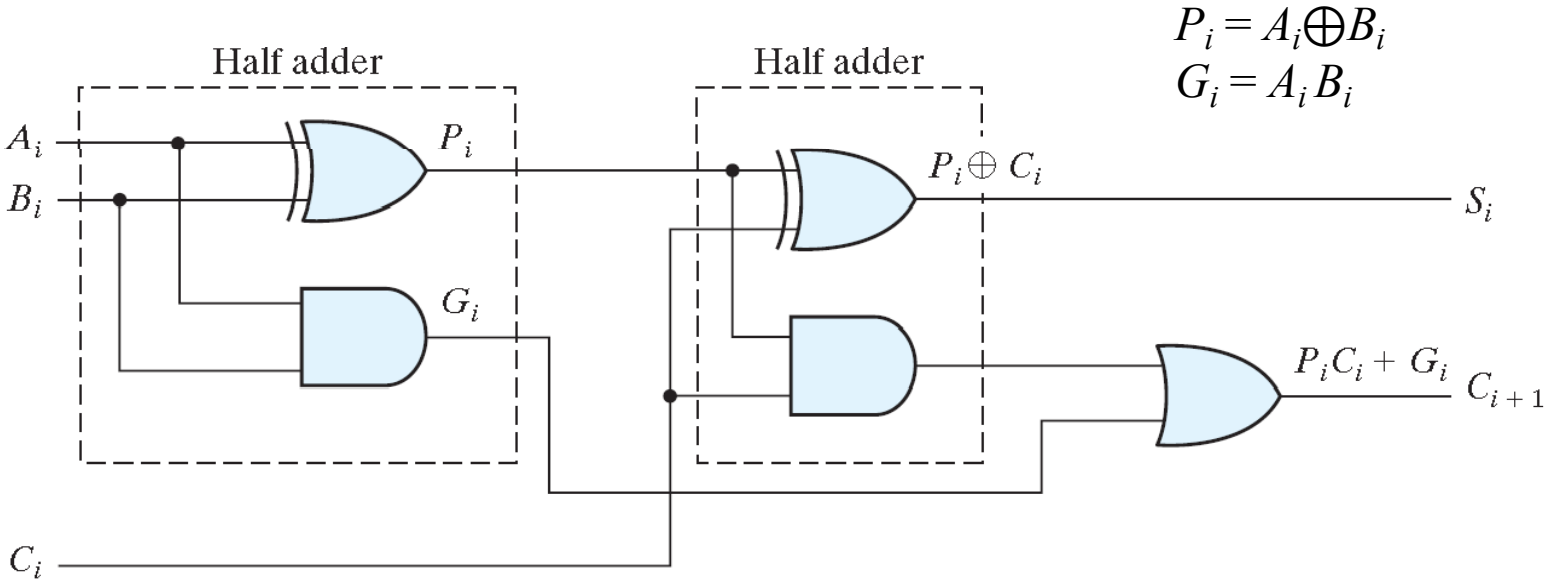
Subscript *i*:	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

Carry Propagation

- The total propagation time is equal to the sum of the propagation delay of logic gates along a path from input to output.
- The longest propagation delay time in an adder is the time it takes the carry to propagate through the full adders. (called *critical path*)
- Each bit of the sum output, S_i, in the adder will be in its steady-state final value only after the input carry to that stage has been propagated.
- Consider output S_3 in the 4-bit adder, inputs A_3 and B_3 are available as soon as input signals are applied to the adder.
- However, input carry C_3 does not settle to its final value until C_2 is available from the previous stage.
- Similarly, C_2 has to wait for C_1 and so on down to C_0.
- Thus, only after the carry propagates and ripples through all stages will the last output S_3 and carry C_4 settle to their final correct value.
- This will be a serious problem if the adder has a long bit length.

Carry Propagation in Binary Adder

- Re-label the half-adder implementation with $P_i = A_i \oplus B_i$ and $G_i = A_i B_i$
- P_i and G_i settle to steady-state values after A_i and B_i propagate through their respective gates.
- The input carry C_i to the output carry C_{i+1} propagates through an AND gate and an OR gate. If there are four full adders in the adder, the output carry C_4 would have 2 * 4 = 8 gate levels from C_0 to C_4.
- For an n-bit adder, there are $2n$ gate levels for the carry to propagate from input to output.



Half Subtractor

- A **half subtractor** is a digital logic circuit that performs the binary subtraction of two single-bit binary numbers.
- It has two inputs, **A** and **B**, and two outputs, **Difference** and **Borrow**.

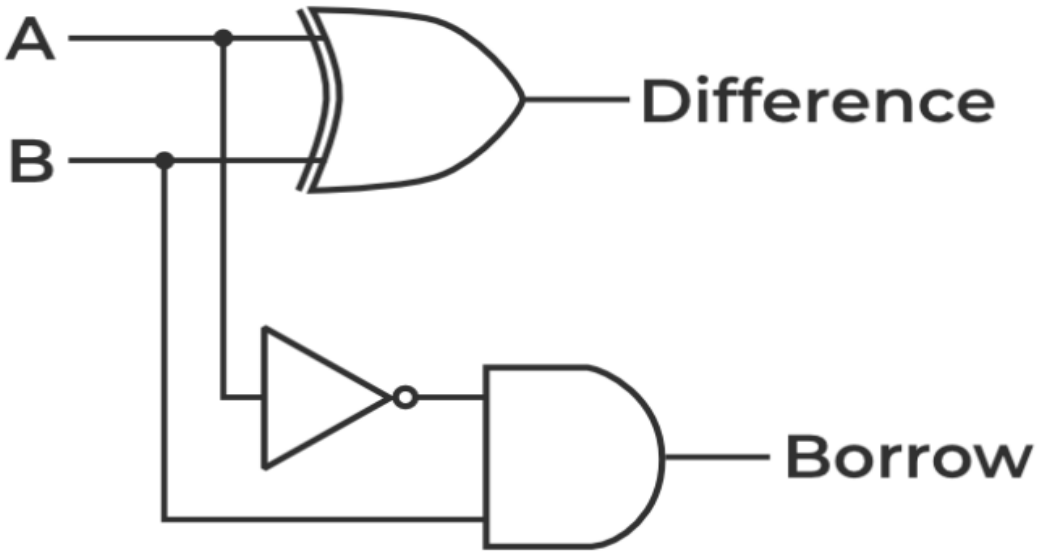
Half Subtractor

- A **half subtractor** is a digital logic circuit that performs the binary subtraction of two single-bit binary numbers.
- It has two inputs, **A** and **B**, and two outputs, **Difference** and **Borrow**.

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Difference = A'B+AB’
Borrow = A'B

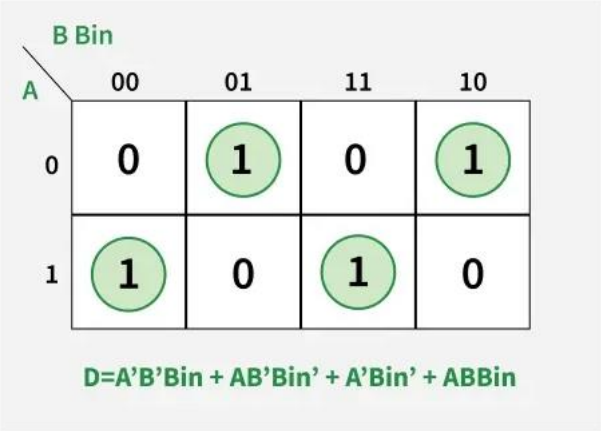
Implementation of Half Subtractor



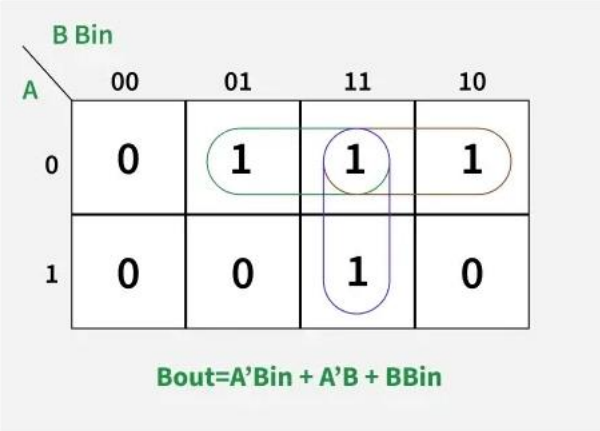
Full Subtractor

Input			Output	
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Full Subtractor



K-Map for Difference



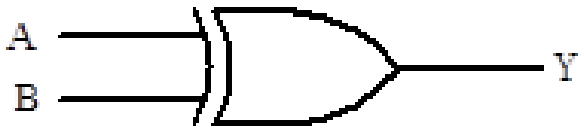
K-Map for Borrow

D = A'B'Bin + A'BBin' + AB'Bin' + ABBin

Bout = A'Bin + A'B + BBin

Binary Adder-Subtractor

- ***Working of XOR Gate***



Eg

3-2=?

9-15=?

$Y = A \oplus B = A\overline{B} + \overline{A}B$

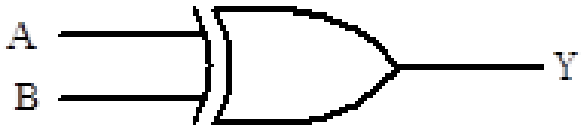
Input		Output
A	**B**	**Y =**
0	**0**	**0**
0	**1**	**1**
1	**0**	**1**
1	**1**	**0**

output is HIGH only when ODD no of inputs are HIGH.

Inequality comparator, as output is high when two inputs are different(for 2-input XOR).

Binary Adder-Subtractor

- ***Working of XOR Gate***



Y = $A \oplus B$ = A$\overline{B}$ + $\overline{A}$ B

Input		Output
A	B	Y =
0	0	0
0	1	1
1	0	1
1	1	0

output is HIGH only when ODD no of inputs are HIGH.

Inequality comparator, as output is high when two inputs are different(for 2-input XOR).

Eg

3-2=?

9-15=?

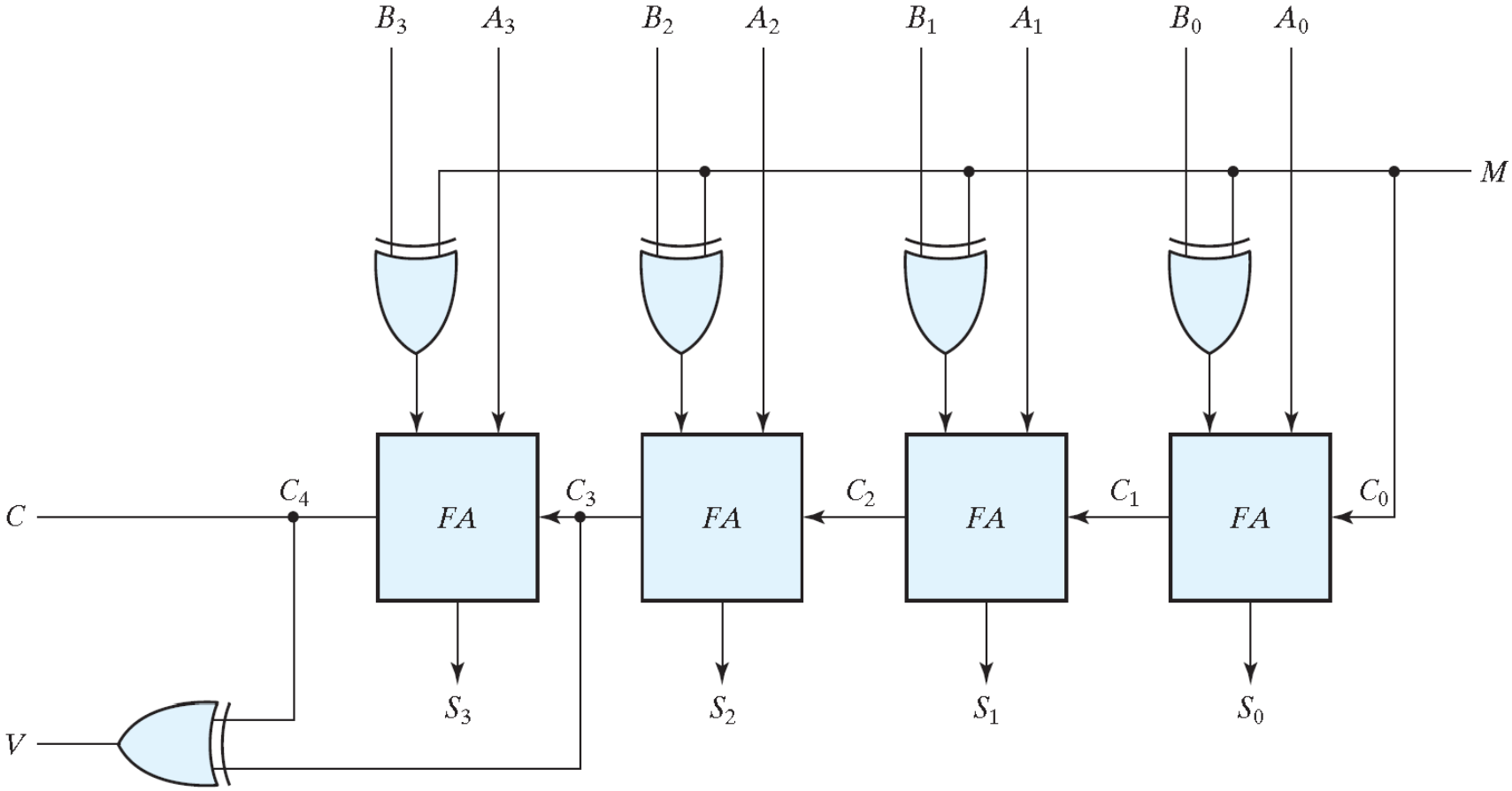
Binary Adder-Subtractor

- A - B = A + (2's complement of B)
- 4-bit adder-subtractor
  - $M = 0$ ➔ $A + B$; $M = 1$ ➔ $A + B' + 1$
- Output V is for detecting an *overflow*.

Eg

3-2=?

9-15=?

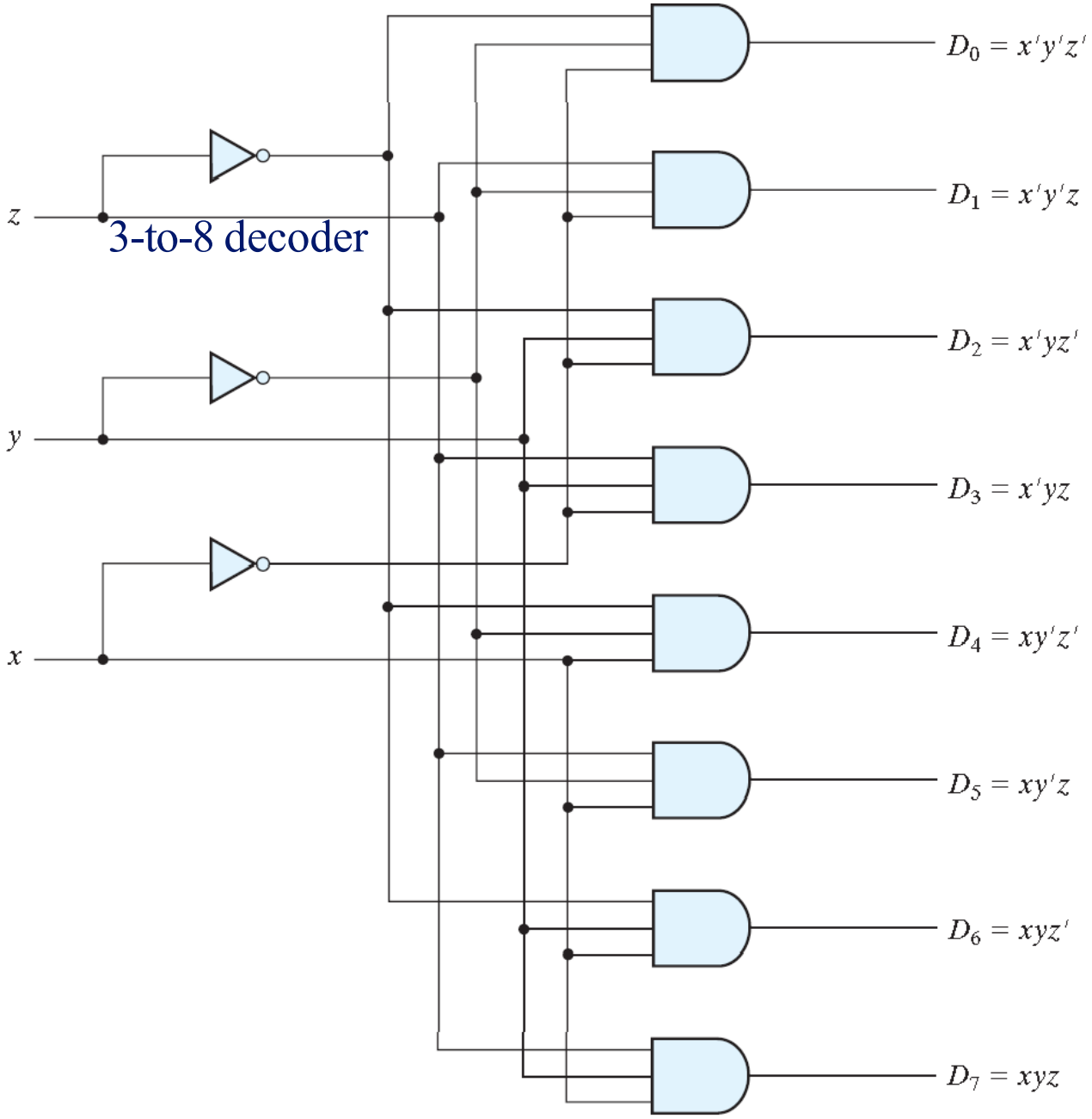


Decoders

- A *decoder* converts binary information from n input lines to a maximum of 2^n unique output lines.
- A n-to-m decoder ($m \leqq 2^n$)
  - a binary code of n bits has 2^n distinct information
  - n input variables; up to 2^n output lines
  - only one output can be active (high) at any time

Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

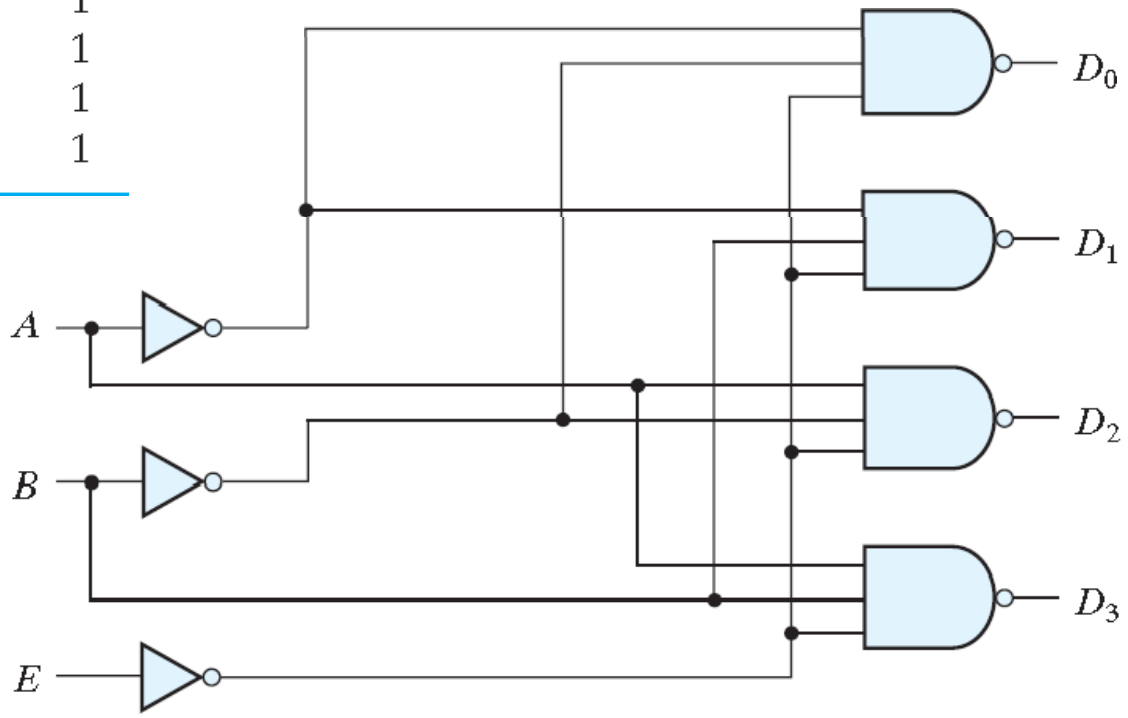


each output = a minterm

Two-to-four Decoder with Enable

- *Enable* input is added to control the circuit operation.

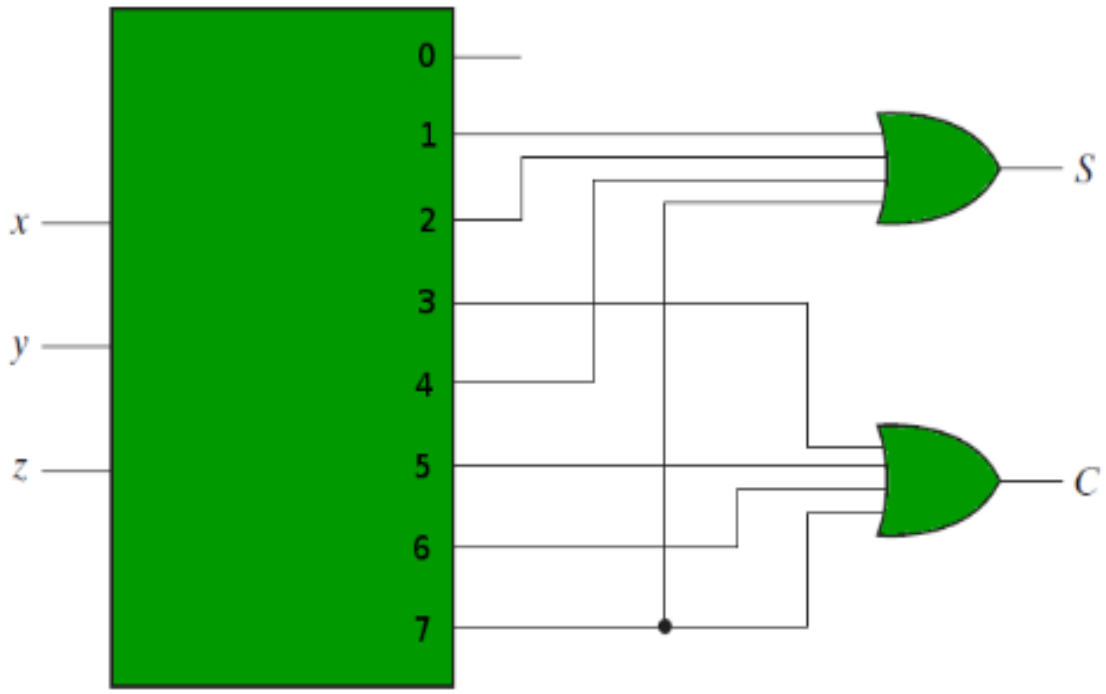
E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	1

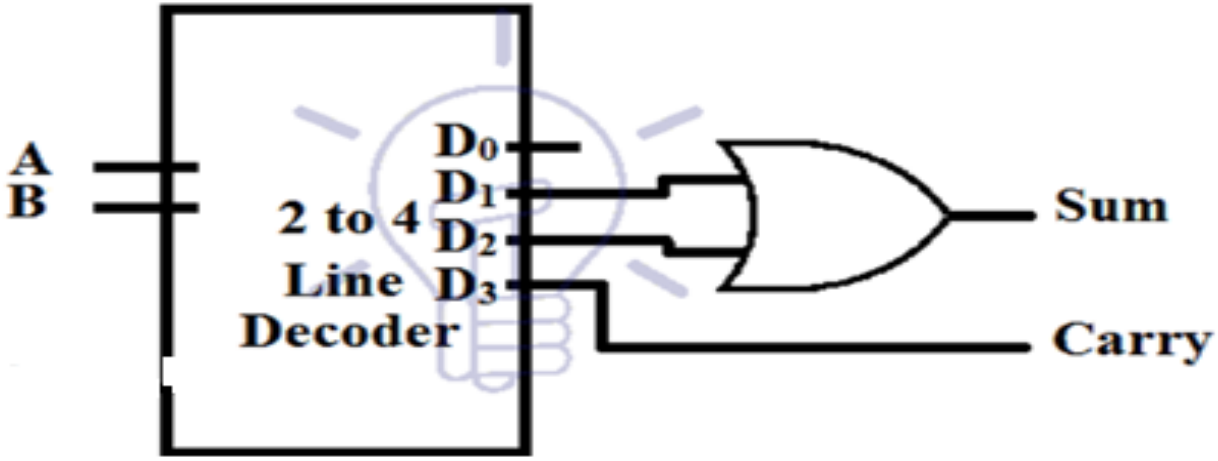


Design Full Adder Using 3:8 Decoder

Input bit for number A	Input bit for number B	Carry bit input C_{IN}	Sum bit output S	Carry bit output C_{OUT}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

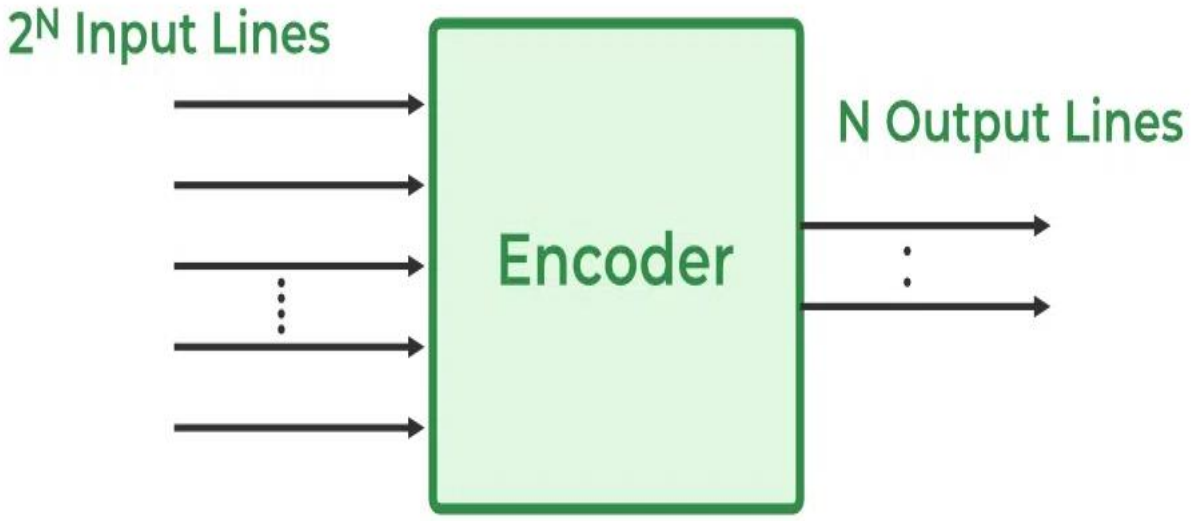
$$S = \sum(1, 2, 4, 7)$$
$$C = \sum(3, 5, 6, 7)$$




A
B
2 to 4
Line
Decoder
D0
D1
D2
D3
Sum
Carry

Encoders

- The inverse function of a decoder
- 2^n input lines and n output lines
- The output lines generate the binary code corresponding to the input value.
- Example:



Types of Encoders

There are different types of Encoders which are mentioned below.

4 to 2 Encoder

Octal to Binary Encoder (8 to 3 Encoder)

Decimal to BCD Encoder

Priority Encoder

4 to 2 Encoder

- The 4 to 2 Encoder consists of **four inputs Y3, Y2, Y1 & Y0, and two outputs A1 & A0**.
- At any time, only one of these 4 inputs can be ‘1’ in order to get the respective binary code at the output.



The Truth table of 4 to 2 encoder

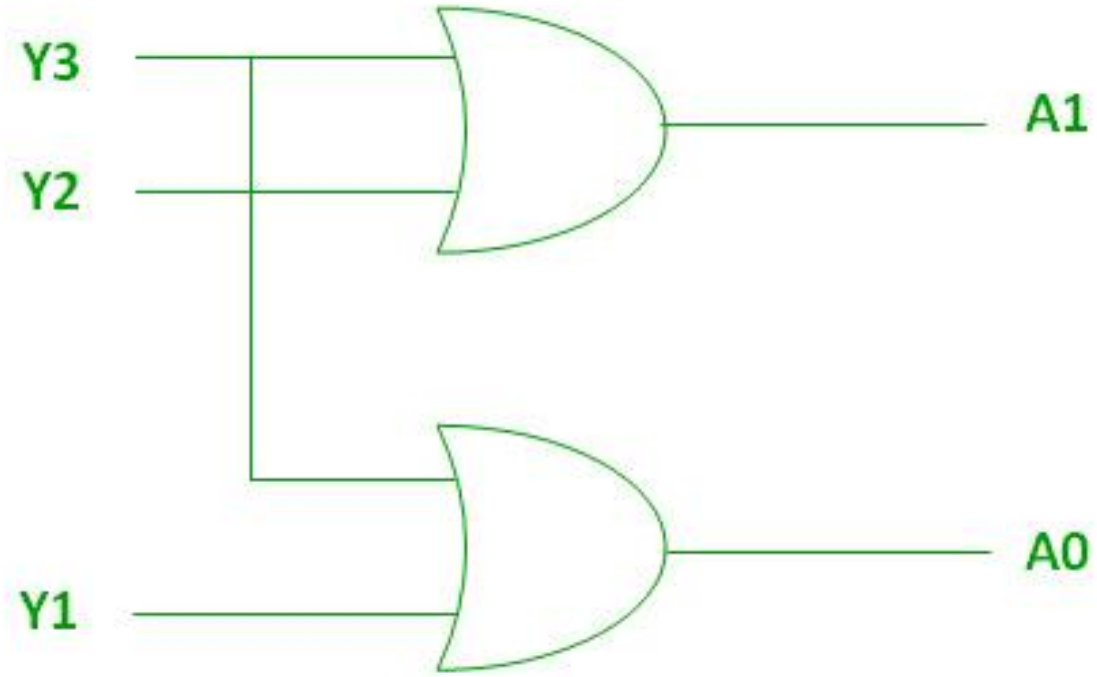
INPUTS				OUTPUTS	
Y3	**Y2**	**Y1**	**Y0**	**A1**	**A0**
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Logical expression for A1 and A0:

$$A1 = Y3 + Y2$$

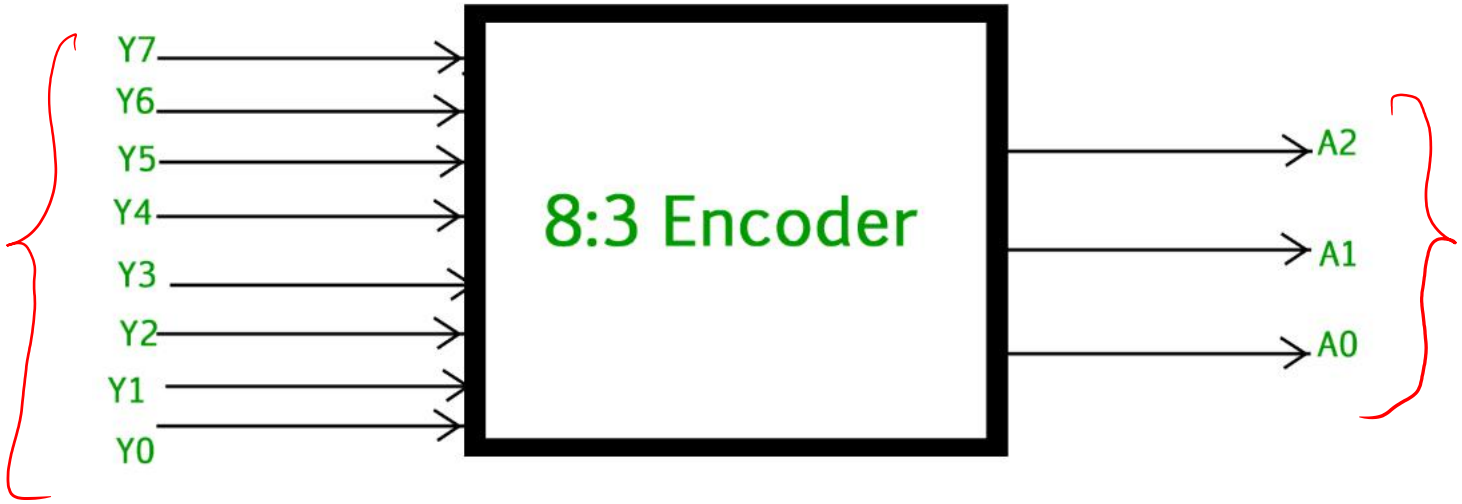
$$A0 = Y3 + Y1$$

Implementation using OR Gate



Octal to Binary Encoder (8 to 3 Encoder)

- The 8 to 3 Encoder or octal to Binary encoder consists of **8 inputs**: Y7 to Y0 and **3 outputs**: A2, A1 & A0.
- Each input line corresponds to each octal digit value and three outputs generate corresponding binary code.



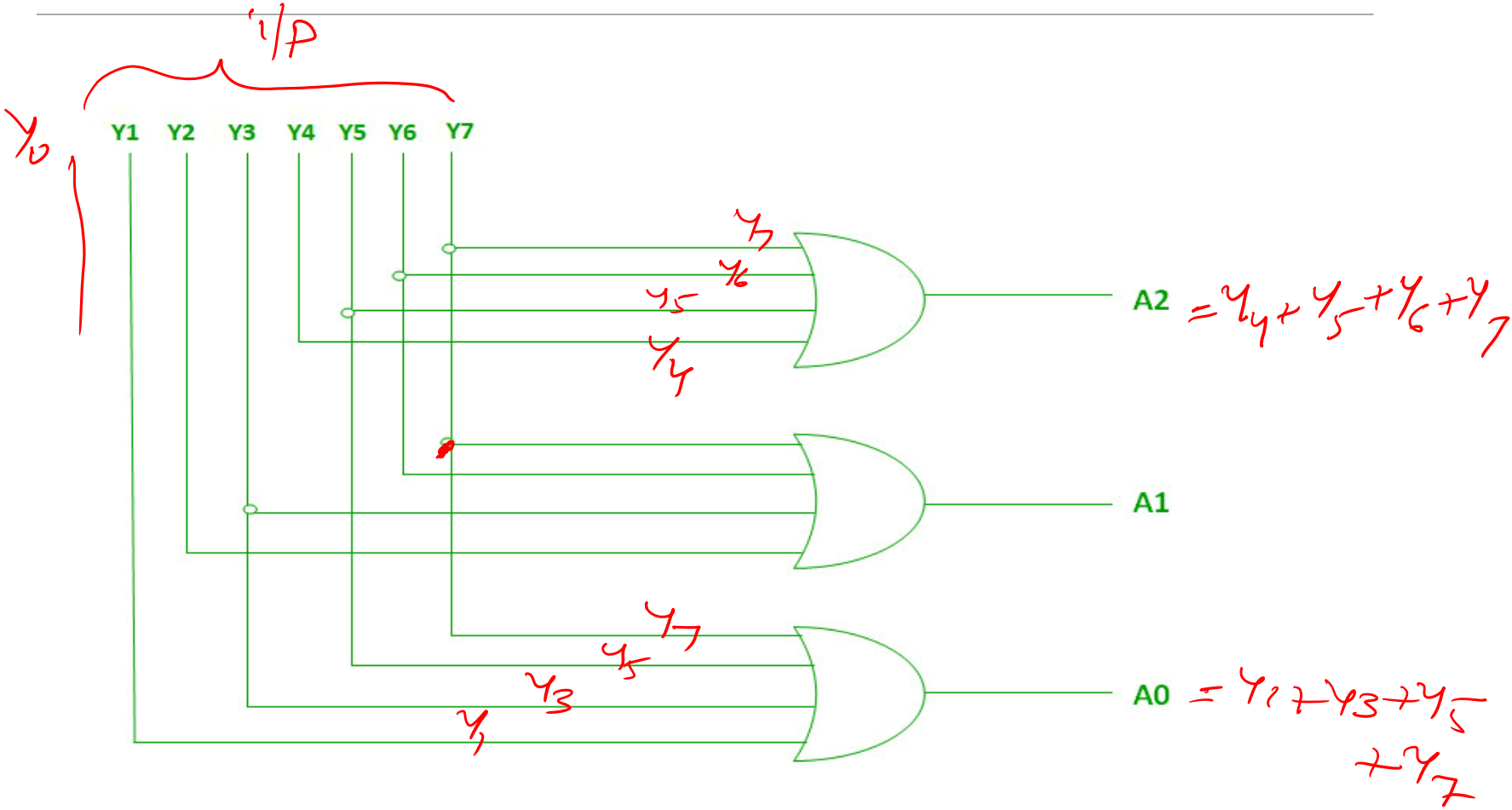
Truth table for the 8 to 3 encoder

INPUTS								OUTPUTS		
Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A2	A1	A0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

Logical expression for A2, A1, and A0.

$$A2 = Y7 + Y6 + Y5 + Y4$$
$$A1 = Y7 + Y6 + Y3 + Y2$$
$$A0 = Y7 + Y5 + Y3 + Y1$$

Implementation using OR Gate



0-9

Binary - coded - decimal

Decimal to BCD Encoder

BCD

32 → 0011 0010

3 2

e.g. → 65 → 0110 0101

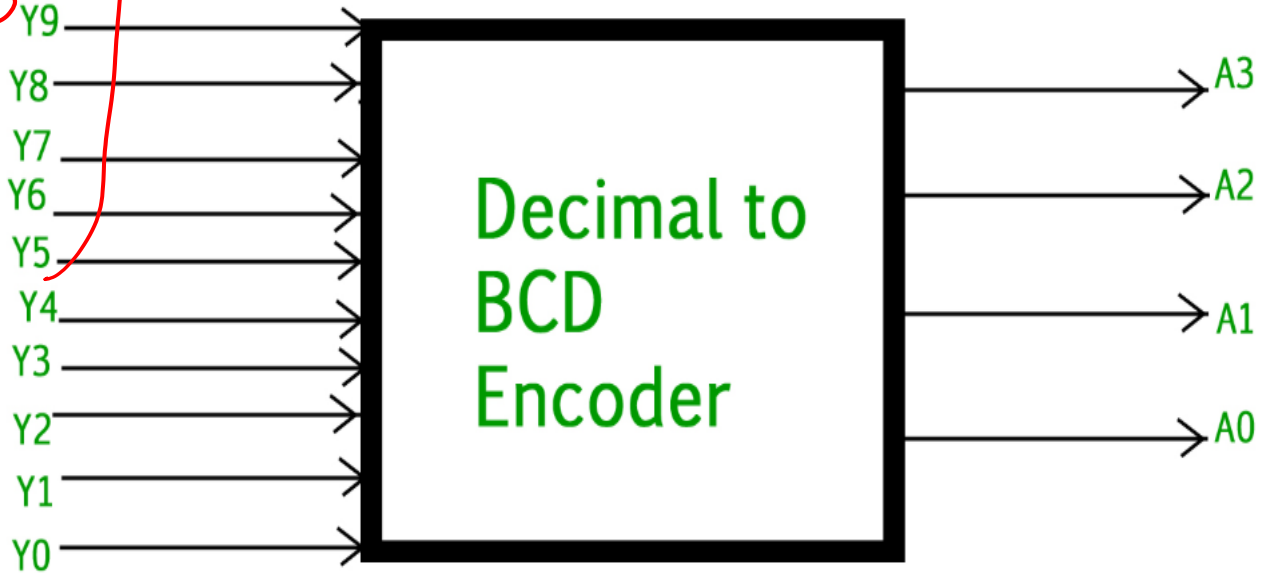
6 5

98 → 1001 1000

9 8

2 | 32 - 1

2 | 16 - 0

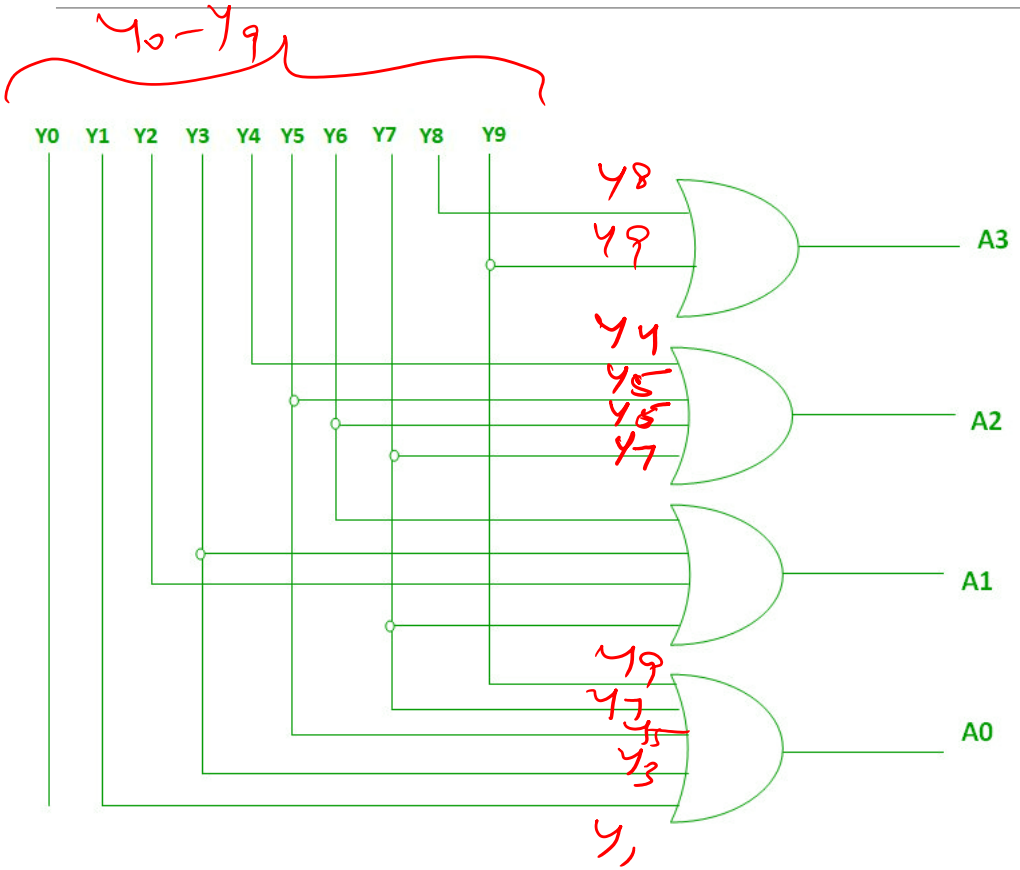


4 o/p

Truth table for decimal to BCD encoder

INPUTS										OUTPUTS			
Y9	Y8	Y7	Y6	Y5	Y4	Y3	Y2	Y1	Y0	A3	A2	A1	A0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	0	1	0	0	1

Implementation using OR Gate



Logical expression for A3, A2, A1, and A0.

```
A3 = Y9 + Y8
A2 = Y7 + Y6 + Y5 +Y4
A1 = Y7 + Y6 + Y3 +Y2
A0 = Y9 + Y7 +Y5 +Y3 + Y1
```

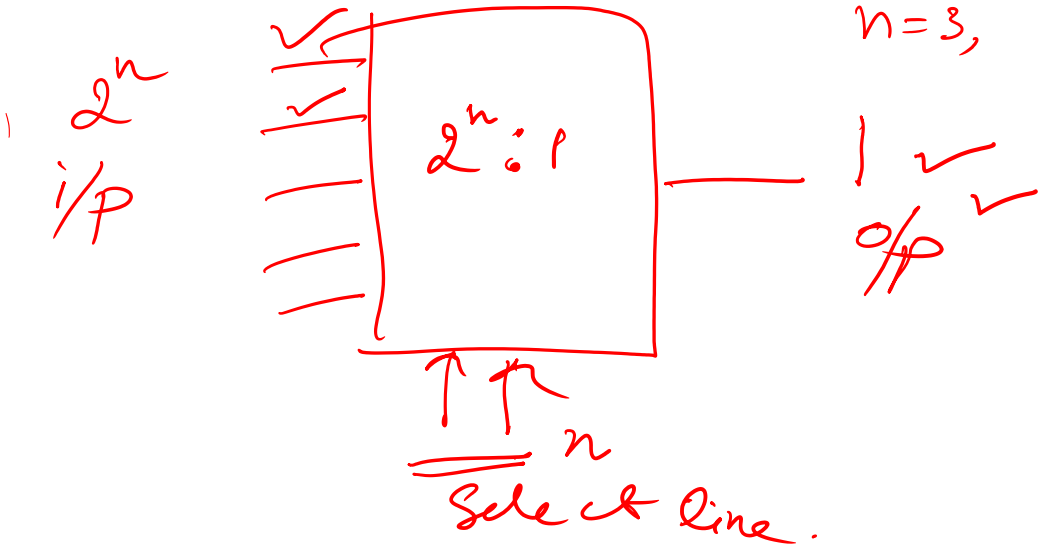
Multiplexers

Many to one

Encoder
2^n i/p : n o/p
8:3

- Select from one of many inputs and directs it to a single output, controlled by a set of selection lines. A multiplexer is also called a *data selector*.
- Normally, there are 2^n inputs and n selection lines whose bit combinations determine which input is selected.
- Example: (two-to-one multiplexer)
  - $Y = I_0$ if $S = 0$, and $Y = I_1$ if $S = 1$.
  - $Y = S'I_0 + SY_1$

n=2, 4:1 Mux
n=3,



Multiplexer

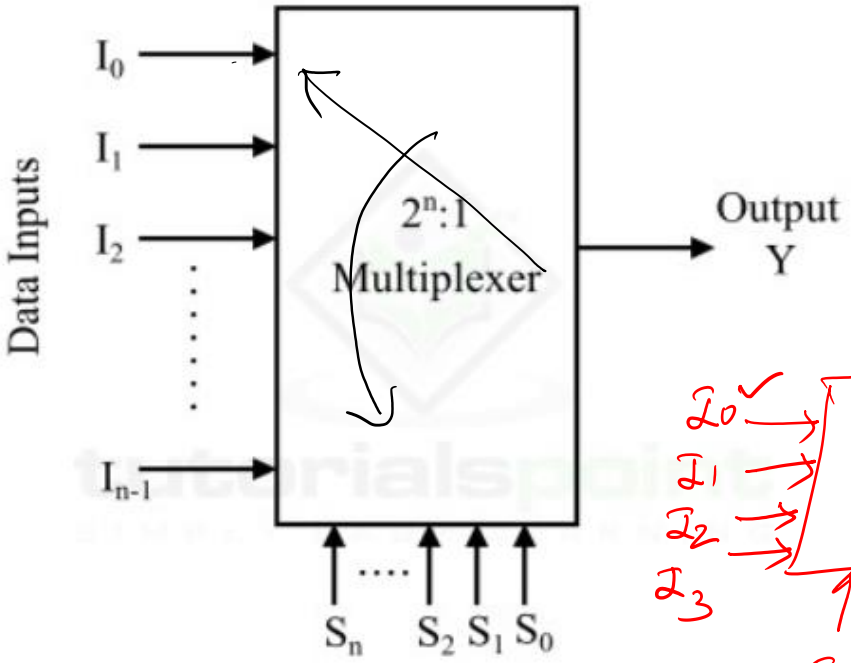
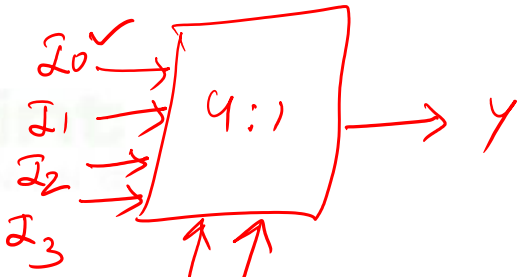


Figure 1 - Digital Multiplexer



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

2×1 Multiplexer

2 : 1 Mux

$2^n : 1$

$n = 1$

Select Line (S)	Output (Y)
0	I_0
1	I_1

$$Y = \overline{S_0}.I_0 + S_0.I_1$$

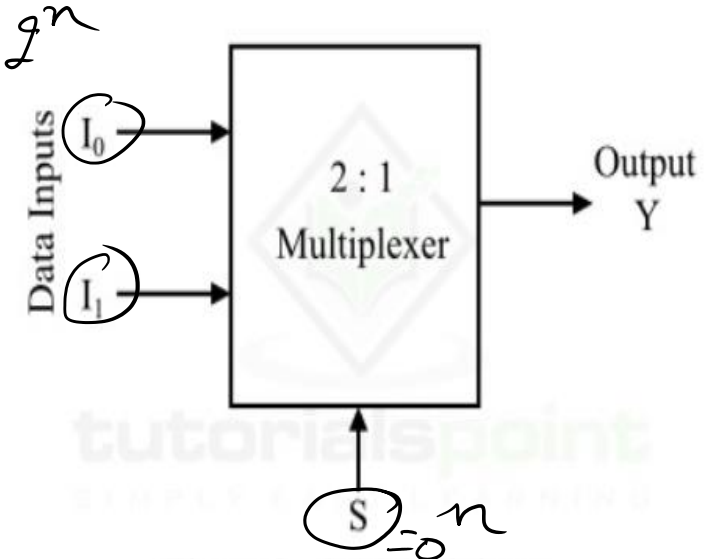
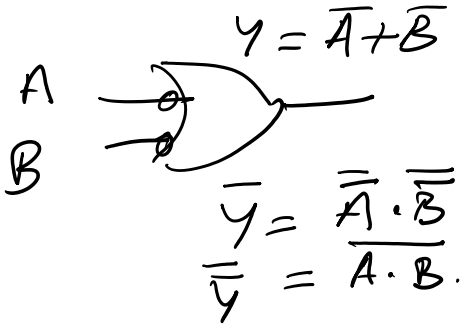


Figure 2 - 2:1 Multiplexer

2×1 Multiplexer

$\overline{\overline{A}} = A$

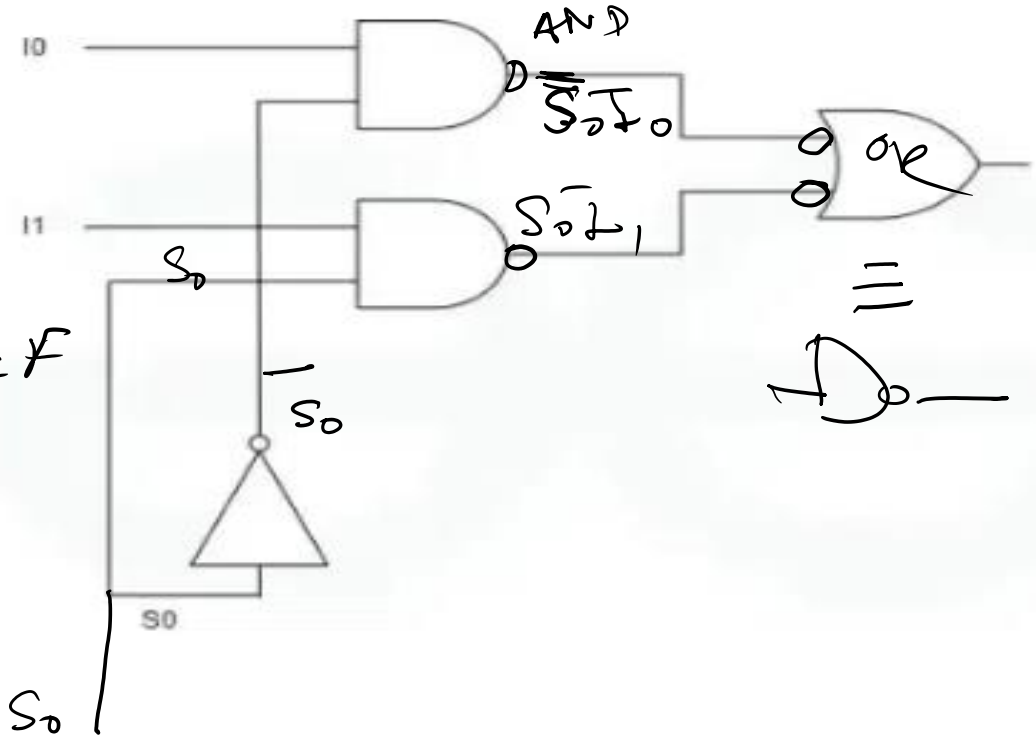
NAND



AND OR



Circuit Diagram of 2x1 Multiplexers



eg

$(\overline{A} + B + C)$

$(A + \overline{B} + C) = F$

4×1 Multiplexer

Selection Lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

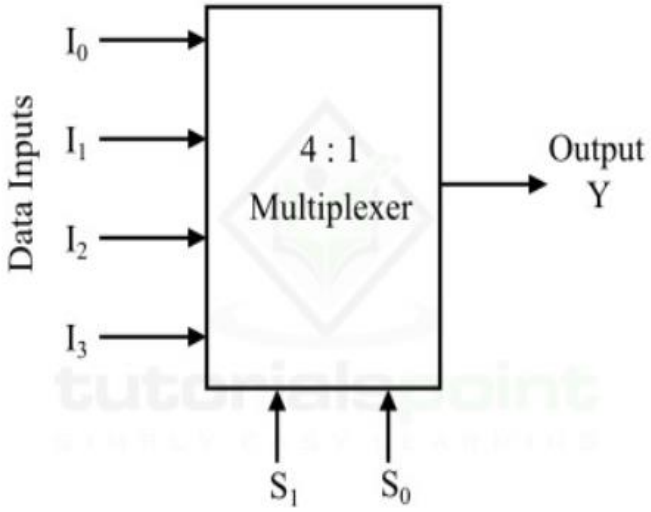
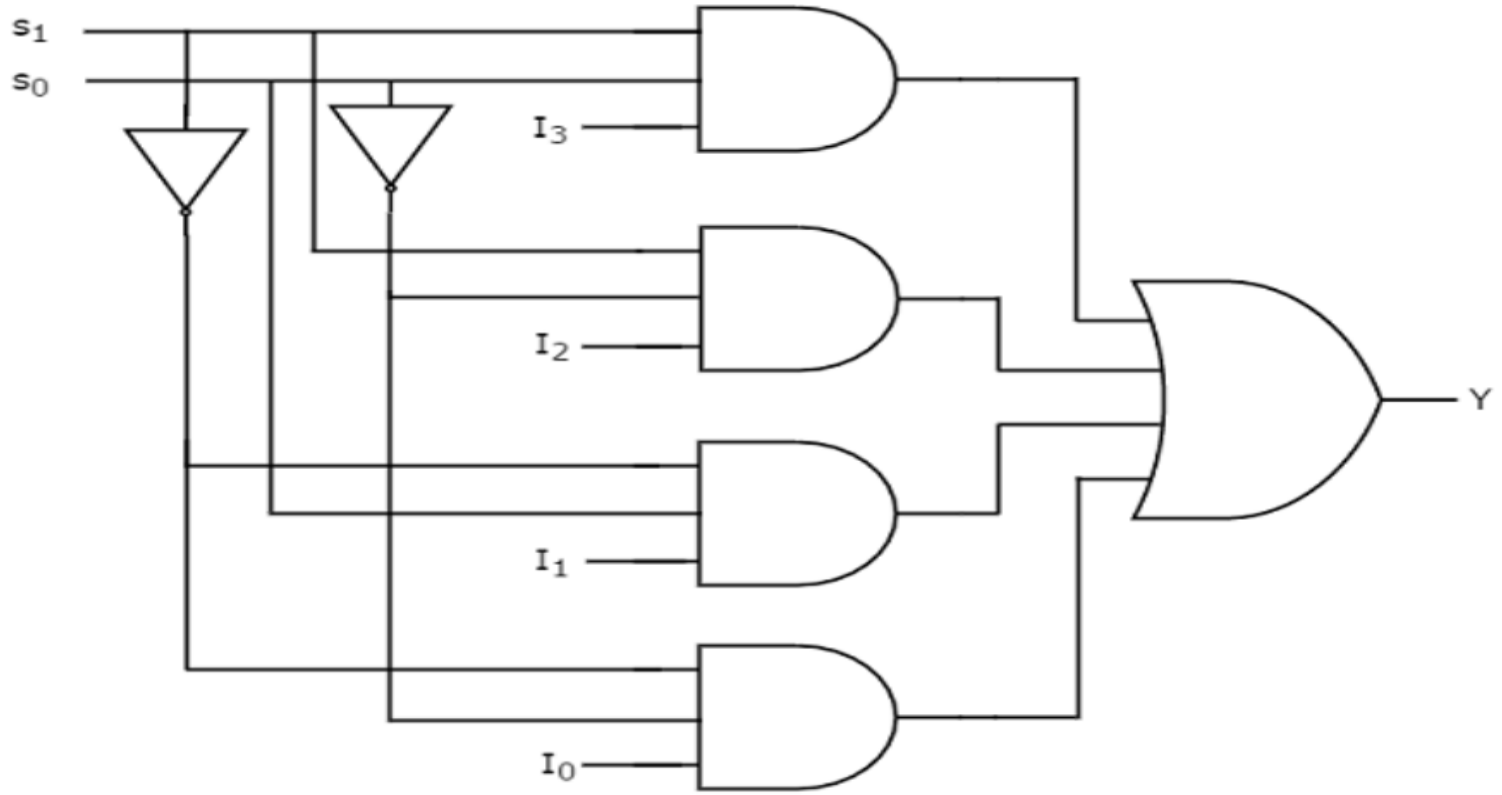


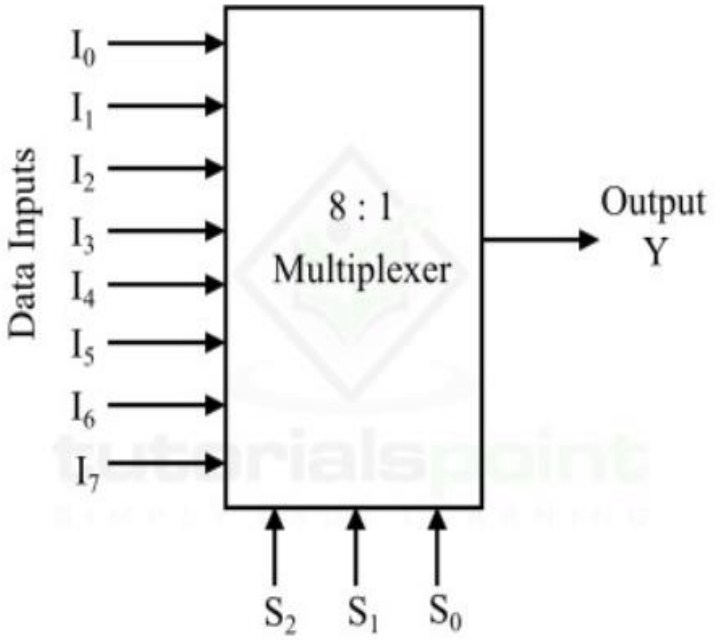
Figure 3 - 4:1 Multiplexer

$$Y = S_1'S_0'I_0 + S_1'S_0I_1 + S_1S_0'I_2 + S_1S_0I_3$$

4×1 Multiplexer



8×1 Multiplexer



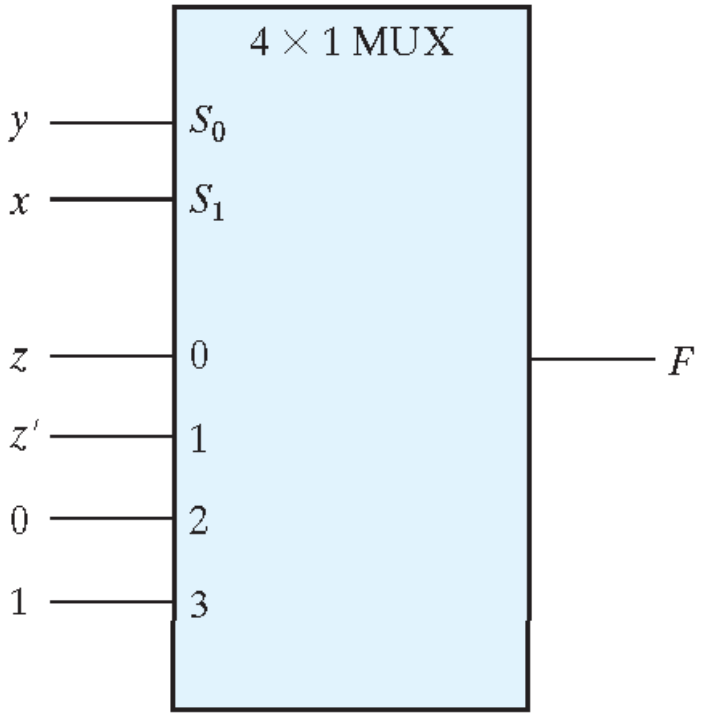
Select Lines			Output
S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

Boolean Function Implementation

- MUX has a structure composed of a decoder and an OR gate
- 2^n-to-1 MUX can implement any Boolean function of n+1 input variables
- n of these input variables are used as the selection lines
- The remaining single variable is used for the data inputs.
- If the single variable is denoted by z, each data input of the multiplexer will be z, z', 1, or 0.
- Example: $F(x, y, z) = \Sigma(1, 2, 6, 7)$

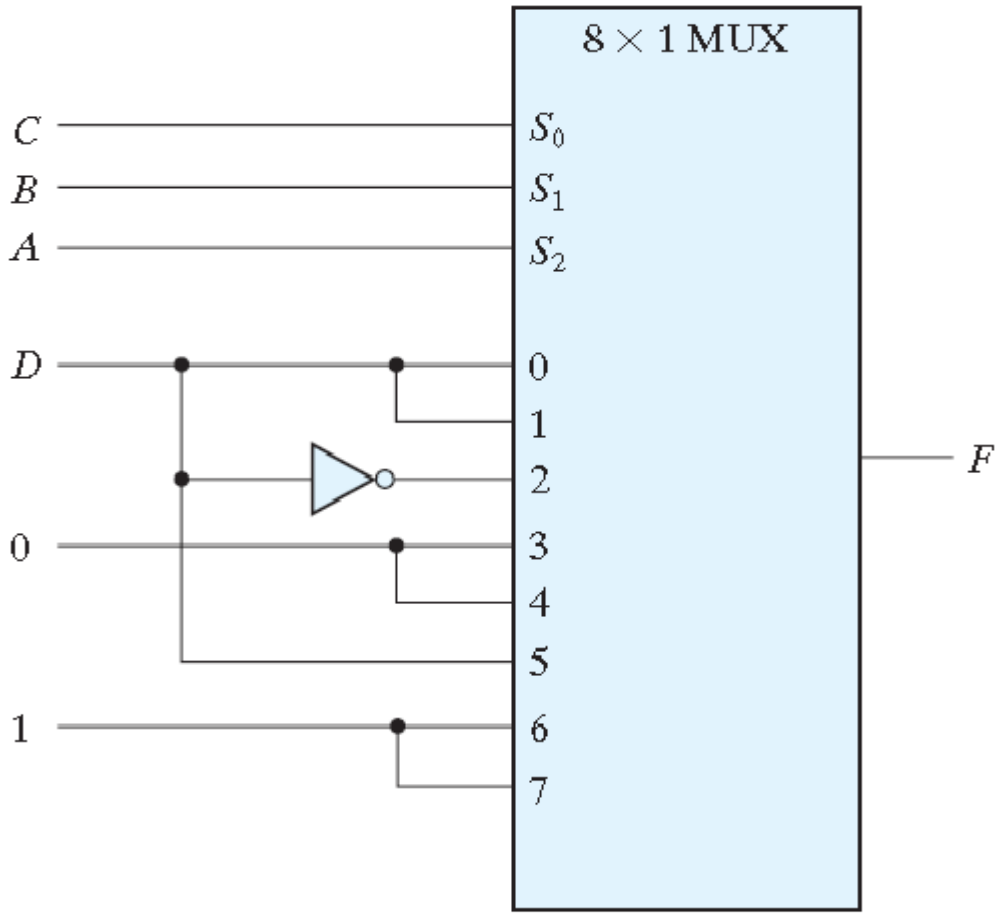
x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table



Another example: $F(A, B, C, D) = \Sigma\,(1, 3, 4, 11, 12, 13, 14, 15)$

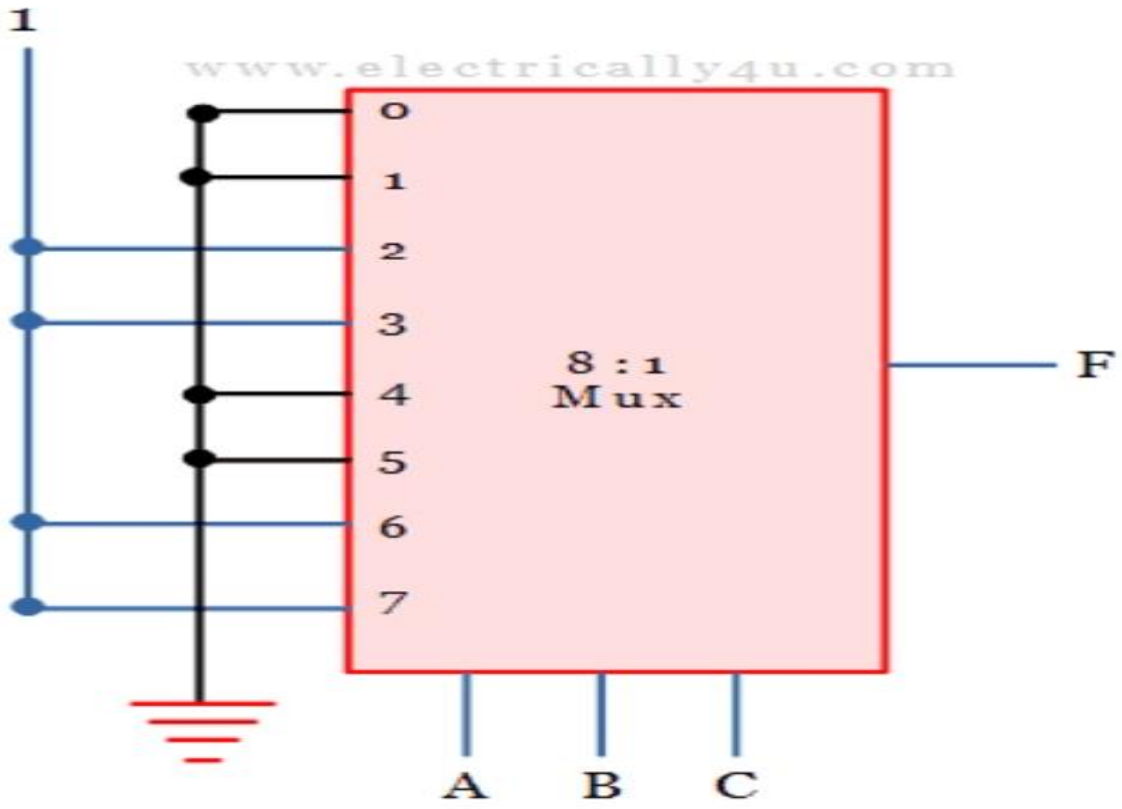
A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



Implement the boolean expression $F(A, B, C) = \sum m(2, 3, 6, 7)$ using a multiplexer.

- There are 3 variables in the given expression, hence $2^n = 2^3 = 8 : 1$ multiplexer. So, the mux has 8 input lines, 3 selection lines, and one output.

- The inputs, corresponding to the minterms (2, 3, 6, 7) are connected to logic 1 and the remaining terms to logic 0(grounded). The given input variables are connected as three selection lines.

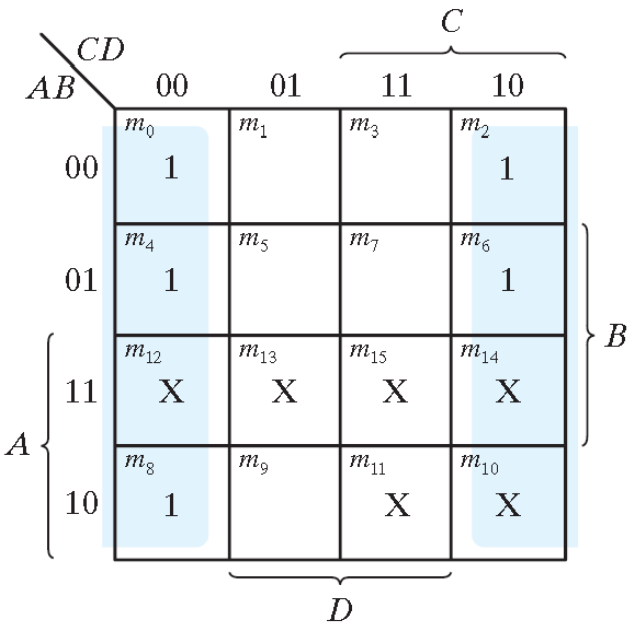

1
www.electrically4u.com
0
1
2
3
4
5
6
7
8 : 1
Mux
F
A
B
C

Code Conversion Example

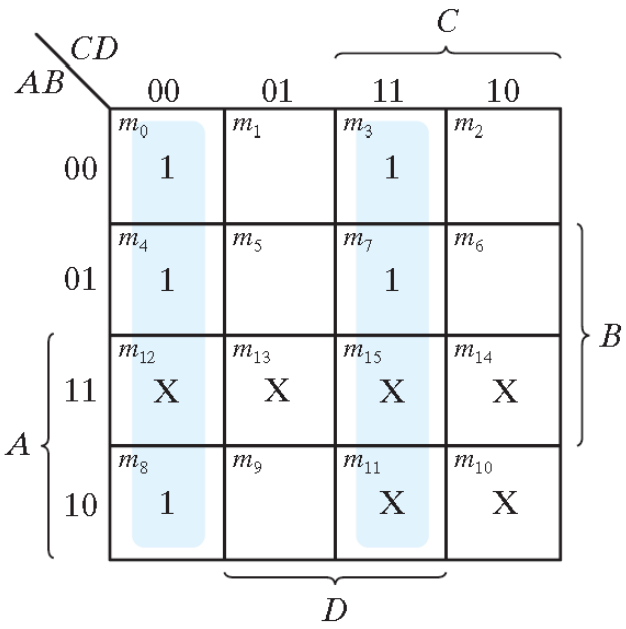
- BCD to excess-3 code
  - The truth table

Table 4.2
Truth Table for Code Conversion Example

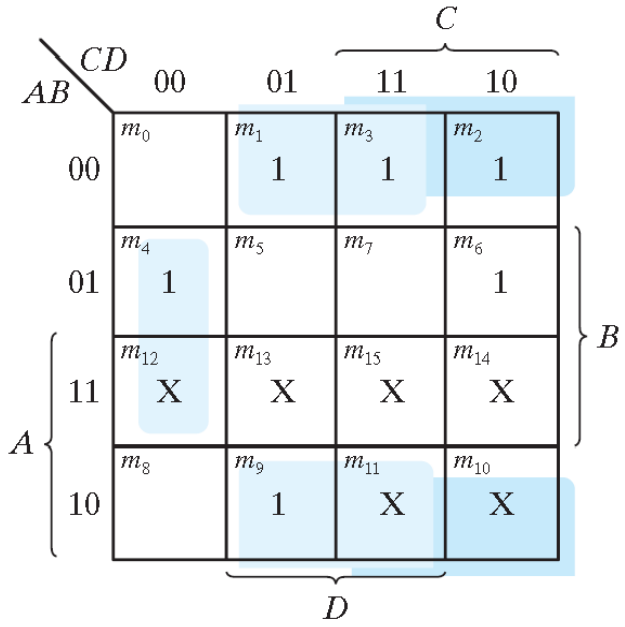
Input BCD				Output Excess-3 Code			
A	*B*	*C*	*D*	*w*	*x*	*y*	*z*
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0



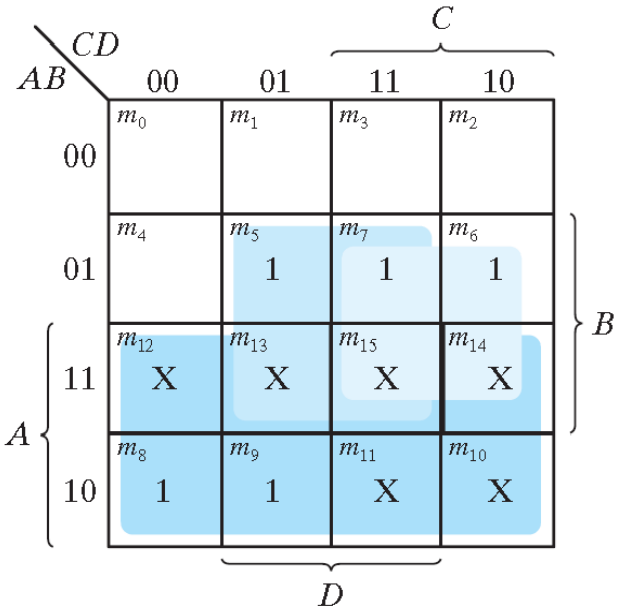
$z = D'$



$y = CD + C'D'$



$x = B'C + B'D + BC'D'$



$w = A + BC + BD$

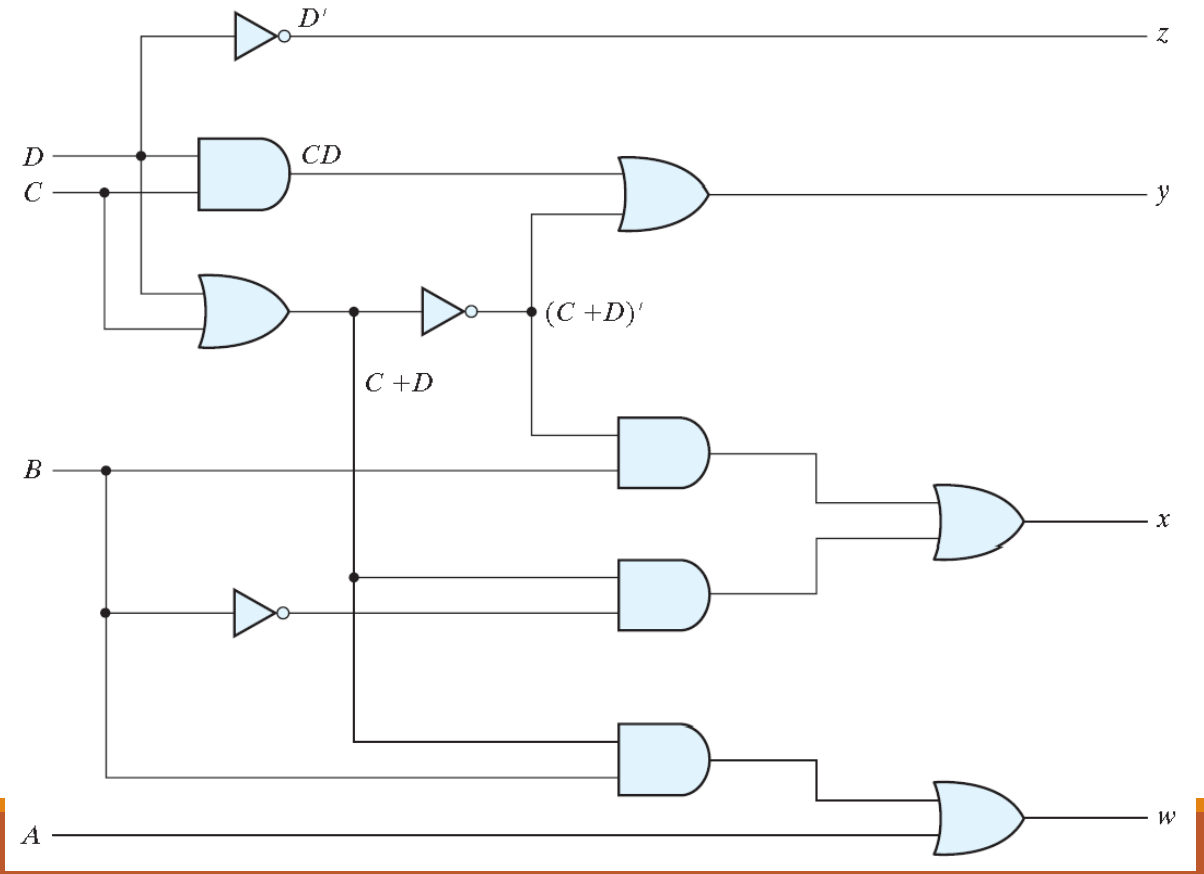
Simplified Functions and Logic Diagram

$$z = D'$$

$$y = CD + C'D' = CD + (C + D)'$$

$$x = B'C + B'D + BC'D' = B'(C + D) + BC'D'$$
$$= B'(C + D) + B(C + D)'$$

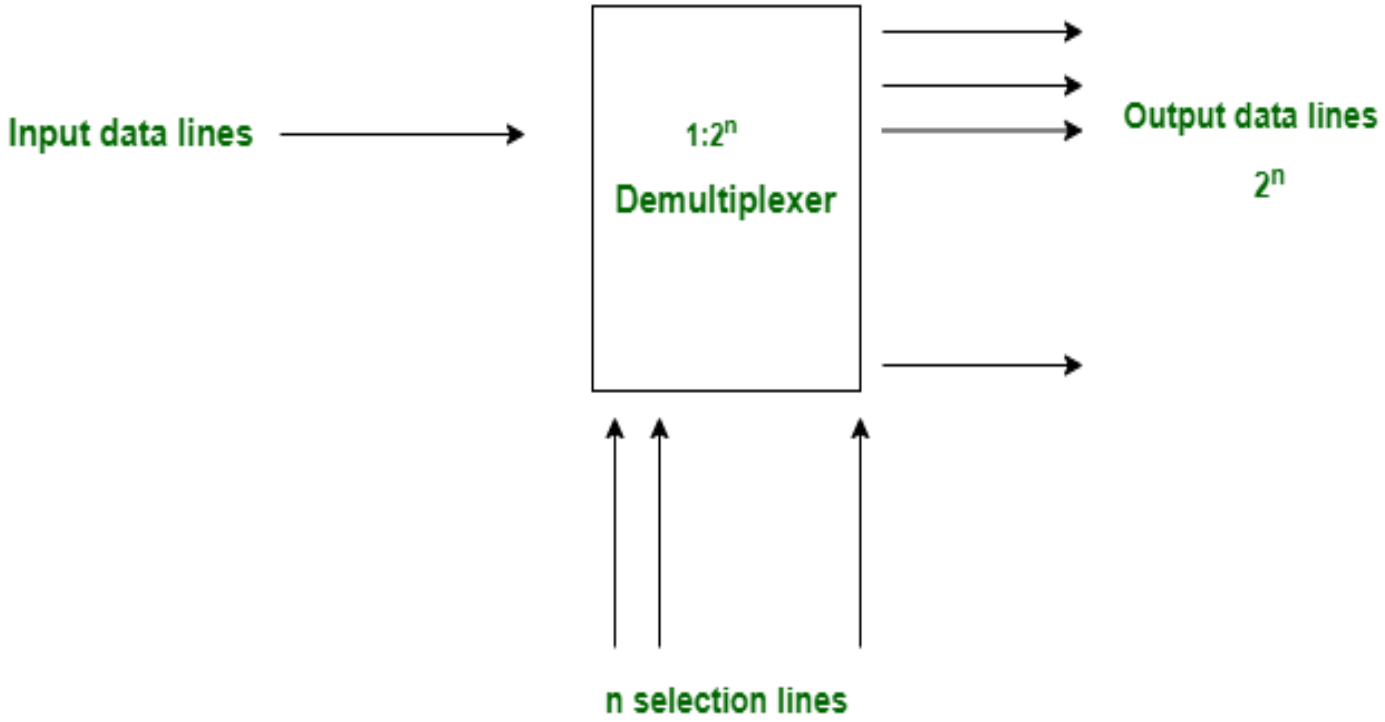
$$w = A + BC + BD = A + B(C + D)$$



Demultiplexer(DEMUX)

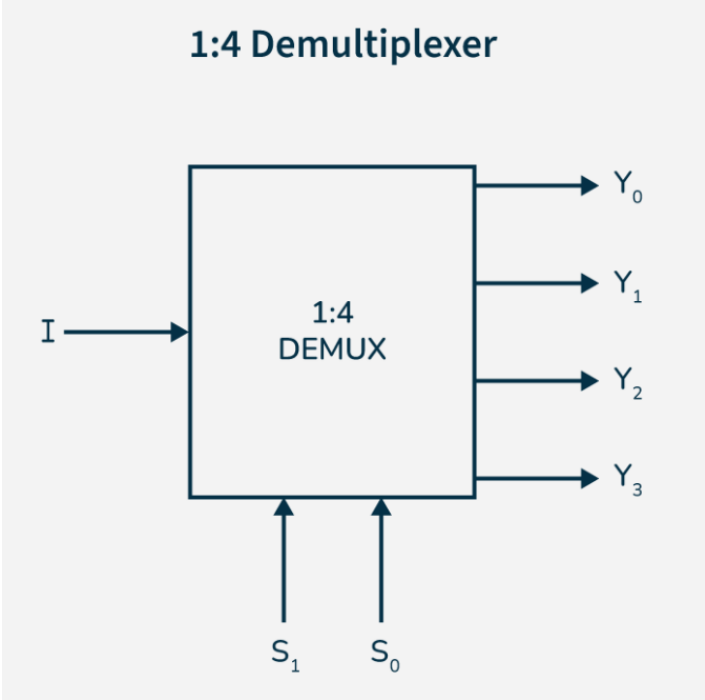
- DEMUX or De-Multiplexer is a data distributor combinational circuit.
- It works in a reverse way of the Multiplexer.
- The DEMUX has 1 input port and 2^n output lines.
- Here n signifies the selection line for a DEMUX.
- As per the selection line value, the DEMUX input lines will be connected to receive the output.

Demultiplexer(DEMUX)



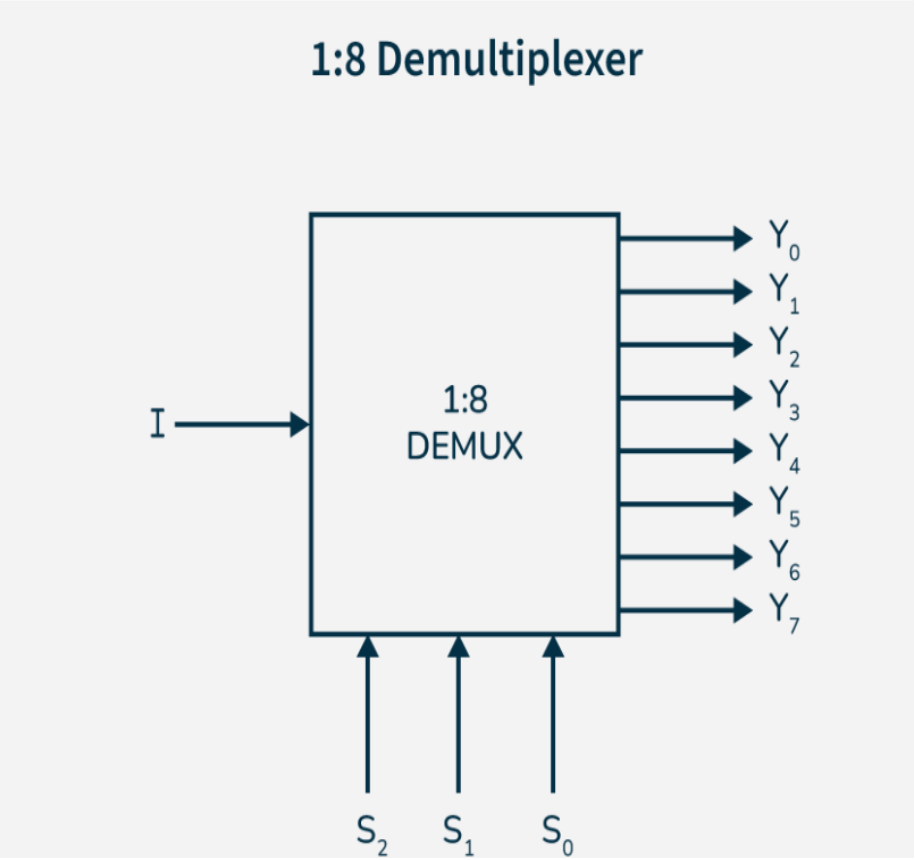
1X4 DEMUX

Selection Inputs		Outputs			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

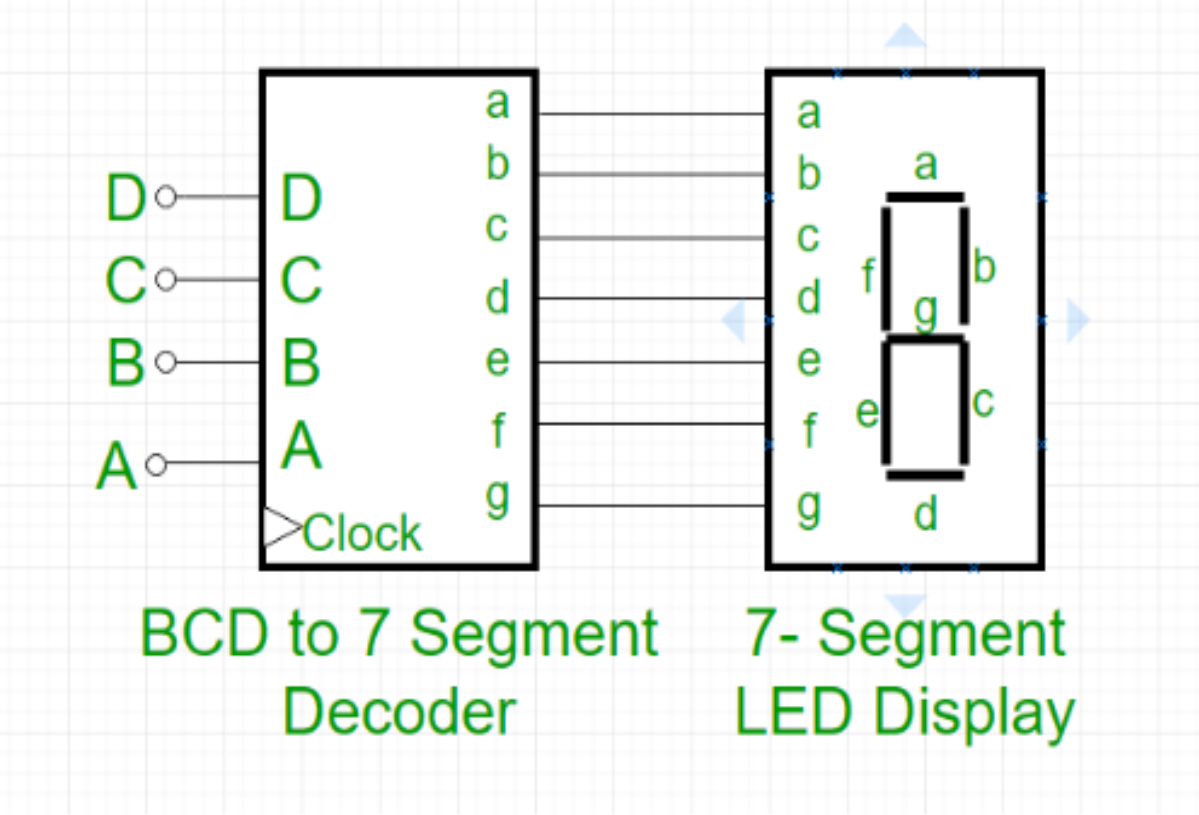


1x8 De-Multiplexer

Selection Inputs			Outputs							
S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	I
0	0	1	0	0	0	0	0	0	I	0
0	1	0	0	0	0	0	0	I	0	0
0	1	1	0	0	0	0	I	0	0	0
1	0	0	0	0	0	I	0	0	0	0
1	0	1	0	0	I	0	0	0	0	0
1	1	0	0	I	0	0	0	0	0	0
1	1	1	I	0	0	0	0	0	0	0

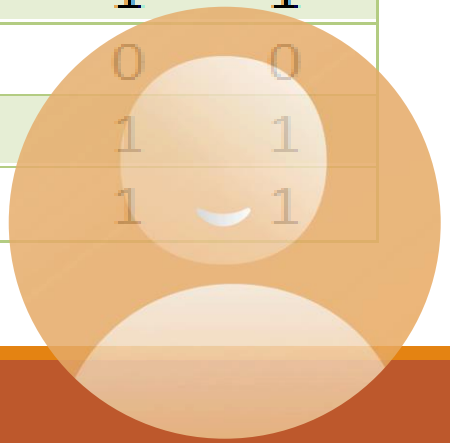


BCD to seven segment Decoder

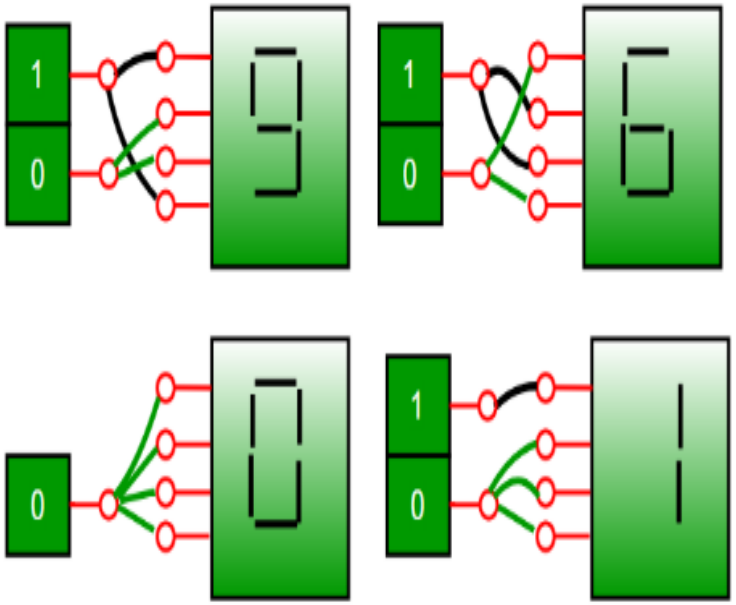


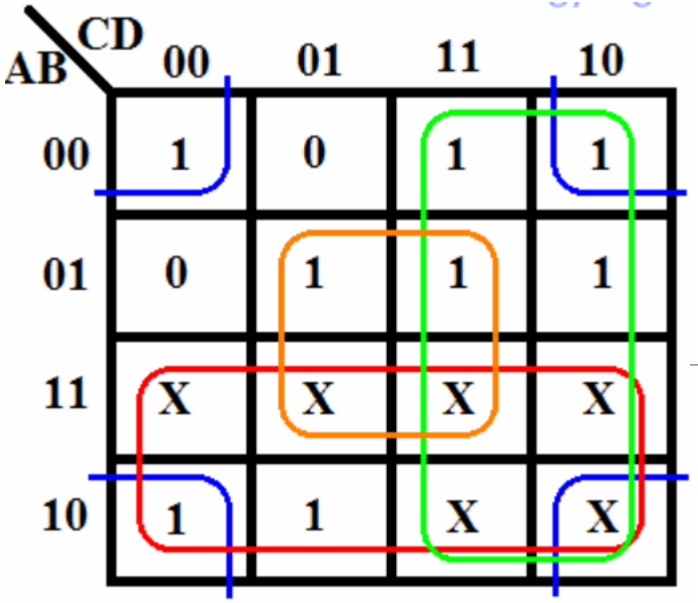
BCD to seven segment Decoder

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

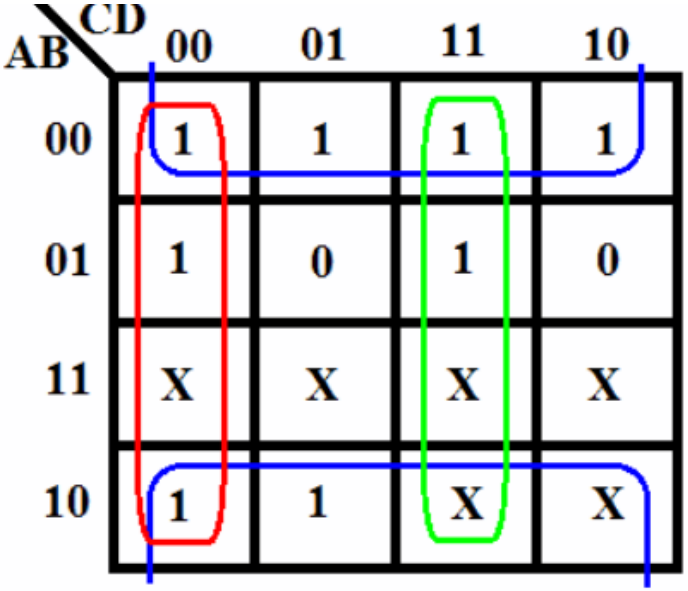


BCD to seven segment Decoder

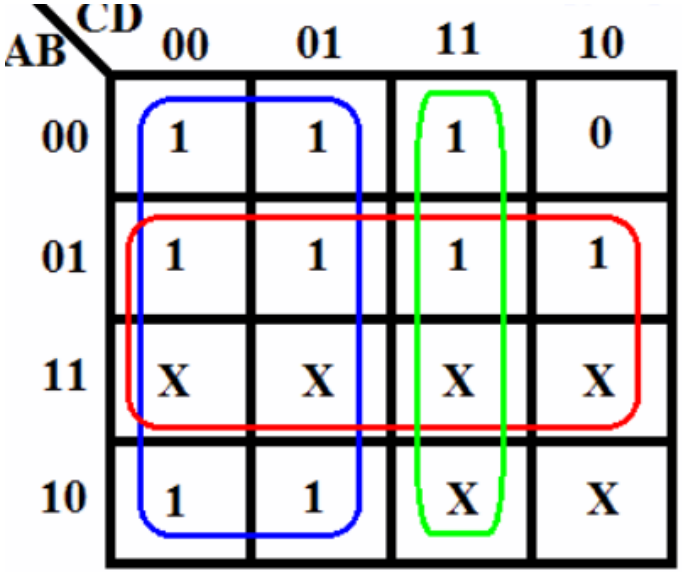




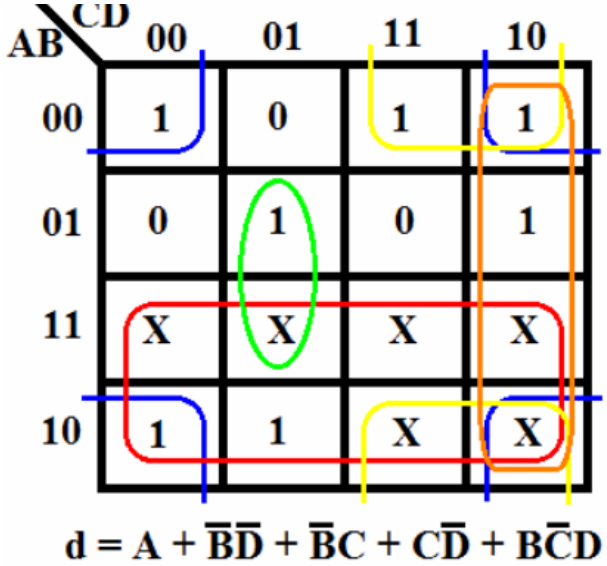
$a = A + C + BD + \overline{B}\overline{D}$



$b = \overline{B} + \overline{C}\overline{D} + CD$

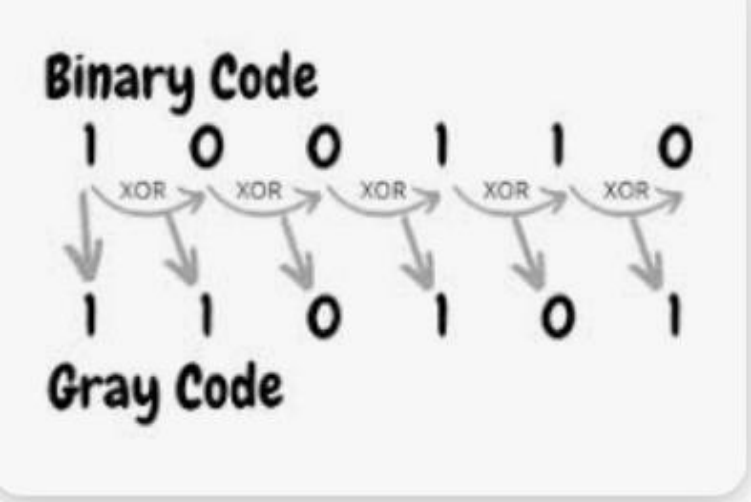


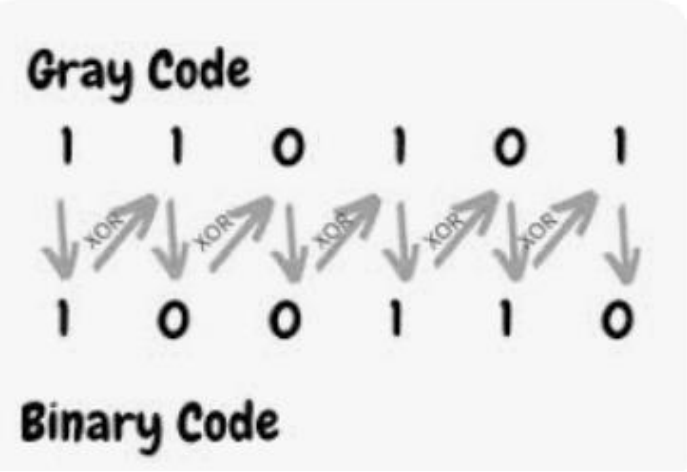
$c = B + \overline{C} + D$



$d = A + \overline{B}\overline{D} + \overline{B}C + C\overline{D} + B\overline{C}D$

Code Converters - Binary to/from Gray Code





Binary (ABC)	Gray (G2 G1 G0)
000	000
001	001
010	011
011	010
100	110
101	111
110	101
111	100

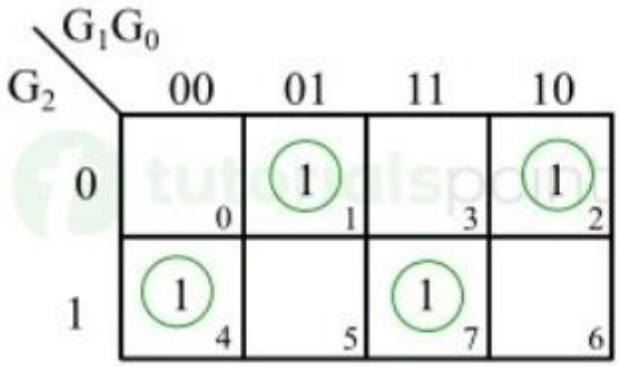
$$\begin{cases} G_2 = A \\ G_1 = A \oplus B \\ G_0 = B \oplus C \end{cases}$$

Implementation: Use XOR gates as per the expressions above.

3 bit gray to binary code converter

Gray Code			Binary Code		
G_2	G_1	G_0	B_2	B_1	B_0
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	0	0
1	1	1	1	0	1

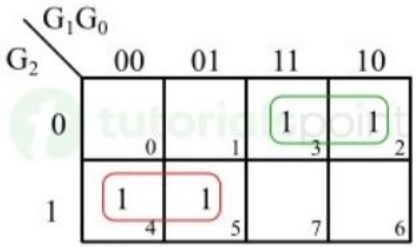
The K-map simplification for the binary output bit B_0 is shown in the following figure.



$$B_0 = G_0 \oplus G_1 \oplus G_2$$

K-Map for Binary Bit B_1

The K-map simplification for the binary output B_1 is shown below.

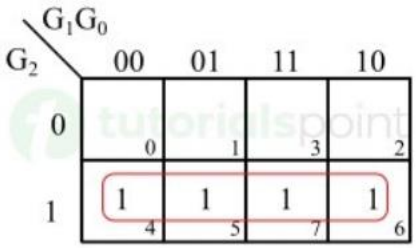


The Boolean expression for the binary bit B_1 is,

$$B_1 = G_2\,\overline{G_1} + \overline{G_2}\,G_1 = G_1 \oplus G_2$$

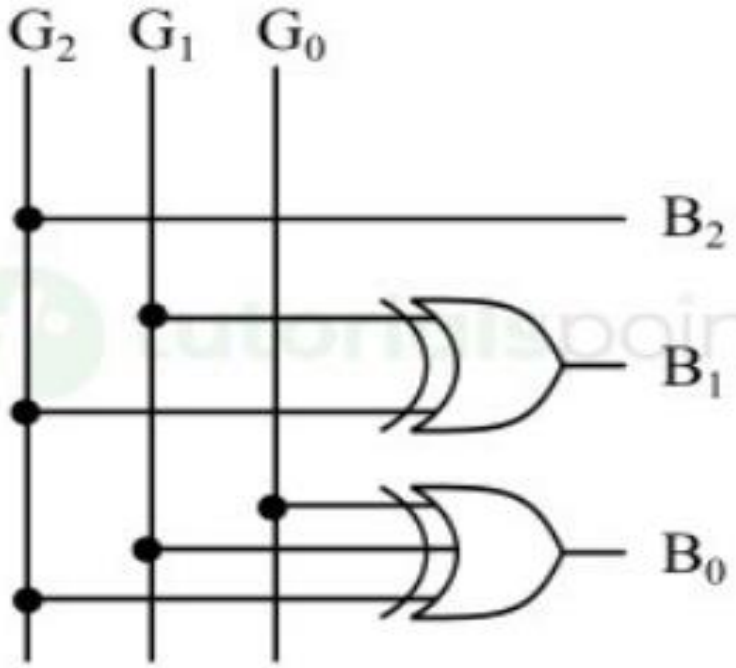
K-Map for Binary Bit B_2

The following figure shows the K-map simplification for the binary bit B_2.



From this K-Map, we obtain the following Boolean expression –

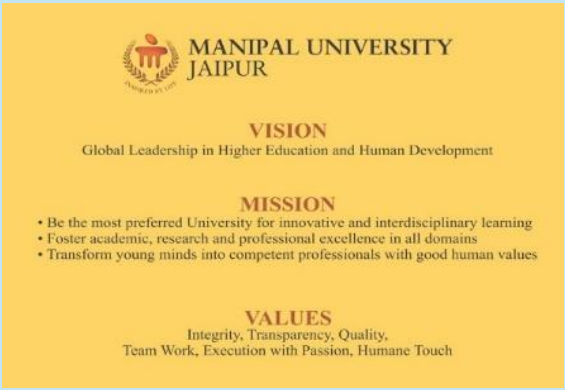
$$B_2 = G_2$$


G2
G1
G0
B2
B1
B0



Course name: Digital system

Course code : ECE1001
lecture series no : 04(four)
Credits : 3
Mode of delivery : presentation
Faculty :

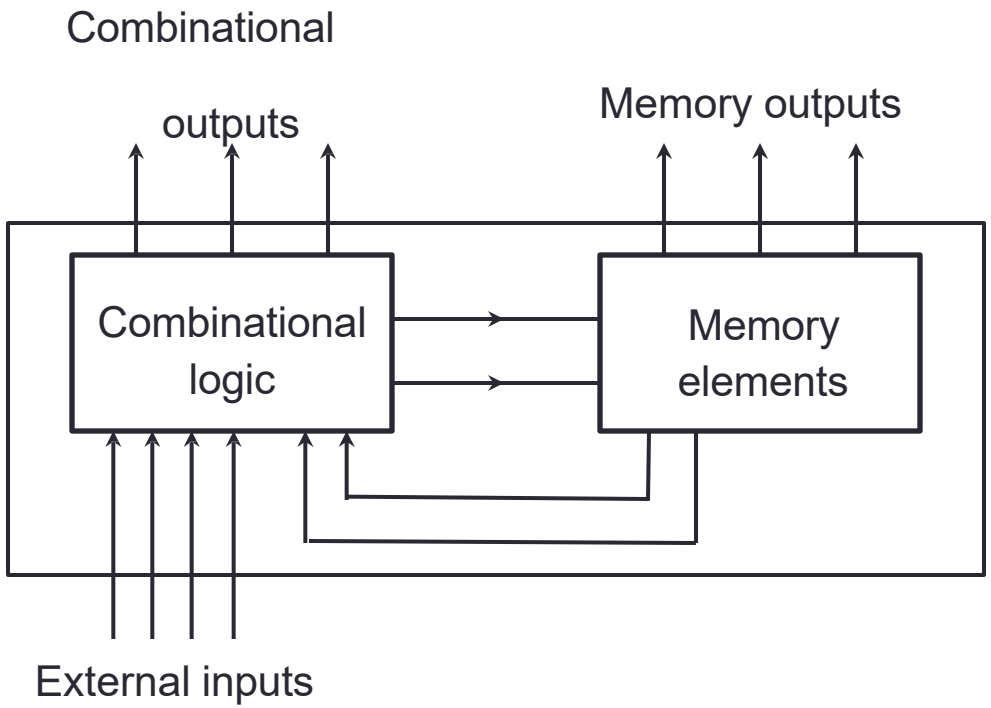


Sequential Logic Latches & Flip-flops

- Introduction
- Memory Elements
- **Latch**
  - S-R Latch
  - J-K Latch
- **Flip-flops**
  - S-R Flip-flop
  - J-K Flip-flop
  - D Flip-flop
  - T Flip-flop

Introduction

- A **sequential circuit** consists of a *feedback path*, and employs some *memory elements*.



Sequential circuit = Combinational logic + Memory Elements

Introduction (cont.)

- There are two types of sequential circuits:
  - *synchronous*: outputs change only at specific time
  - *asynchronous*: outputs change at any time
- *Multivibrator*: a class of sequential circuits
  They can be:
  - *bistable* (2 stable states) : **stable in each cases**
  - *monostable* or *one-shot* (1 stable state) : **return to stable**
  - *astable* (no stable state) : **continuously switching**
- Bistable logic devices: *latches* and *flip-flops*.
- Latches and flip-flops differ in the method used for changing their state.

Memory Elements

- **Memory element**: a device which can remember value indefinitely, or change value on command from its inputs.



- **Characteristic table**:

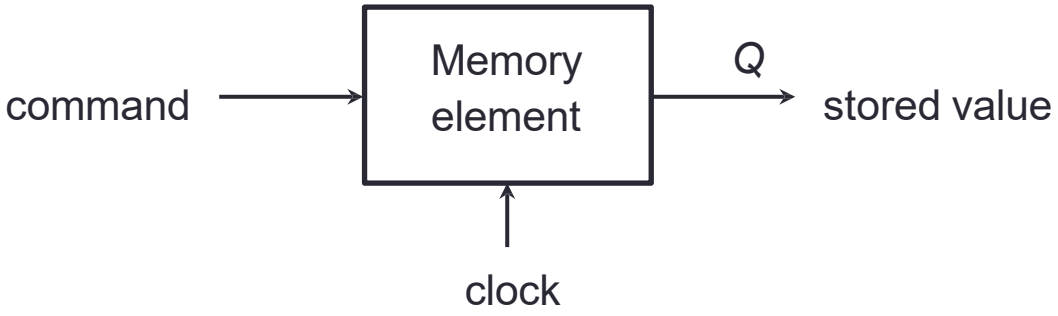
Command	*Q(n)*	*Q(n+1)*
Set	X	1
Reset	X	0
Memorise / No Change	0	0
	1	1

Q(n): current state

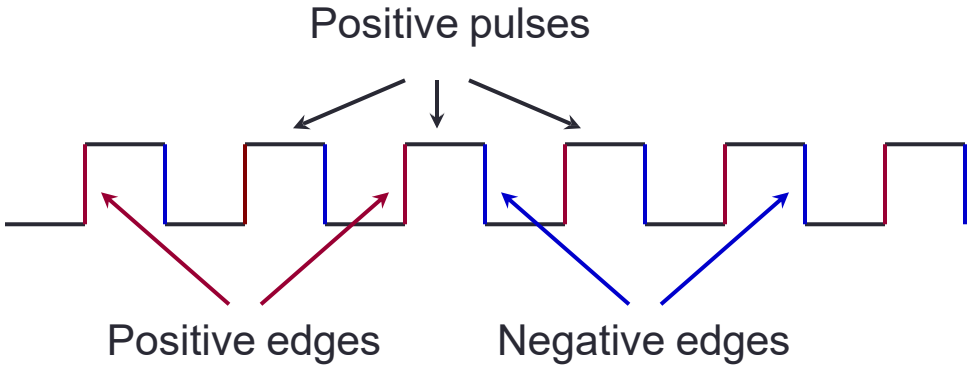
Q(n+1) : next state

Memory Elements (cont.)

- Memory element with clock. Flip-flops are memory elements that change state on clock signals.



- Clock is usually a square wave.



Memory Elements (cont.)

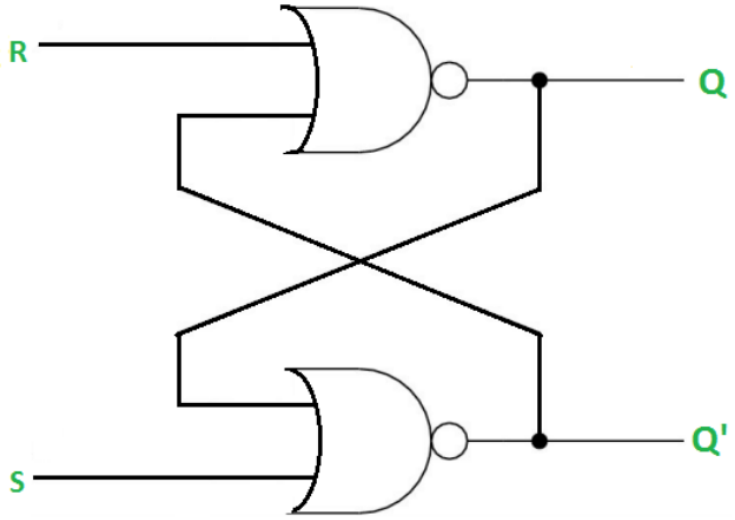
- Two types of triggering/activation:
  - **Pulse-triggered**
  - **Edge-triggered**
- **Pulse-triggered**
  - latches
  - ON = 1, OFF = 0
- **Edge-triggered**
  - flip-flops
  - positive edge-triggered (ON = from 0 to 1; OFF = other time)
  - negative edge-triggered (ON = from 1 to 0; OFF = other time)

Latches

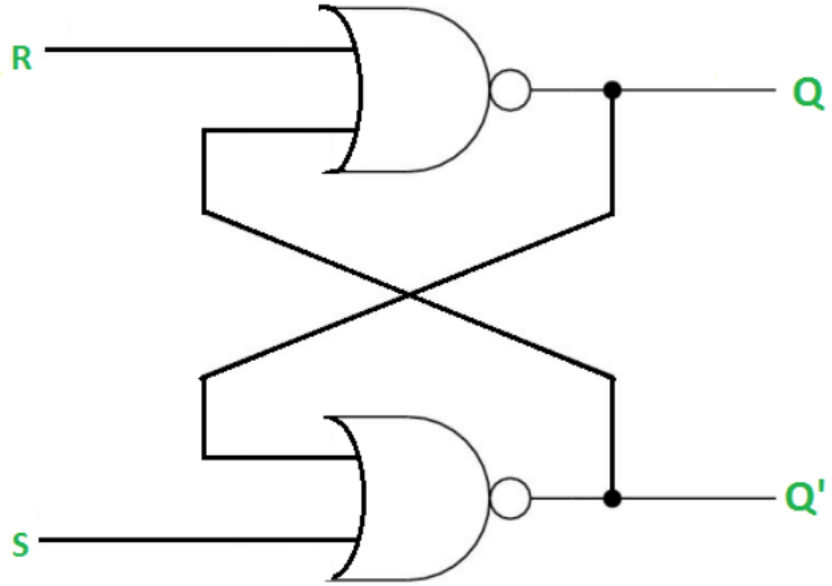
- **Asynchronous**: no need of clock signal
- Store a single bit of information
- Hold its value until it is updated by new input signals
- Temporary storage element in digital system
- Implement using various digital logic gates: AND, NAND….

Types of Latches

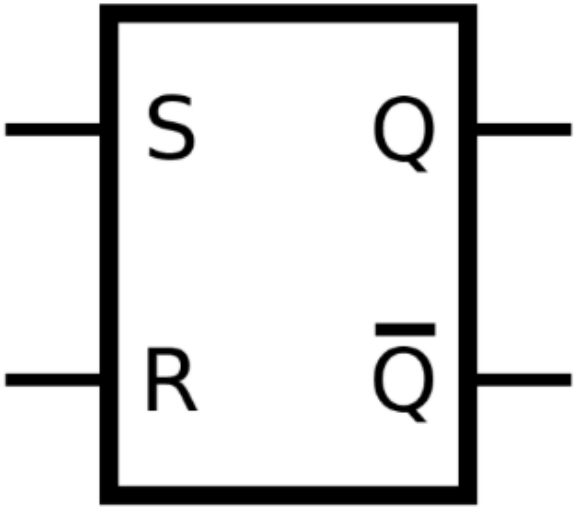
- **S-R Latches**
- **J-K Latches**
- T Laches
- D Latches



S-R Latch

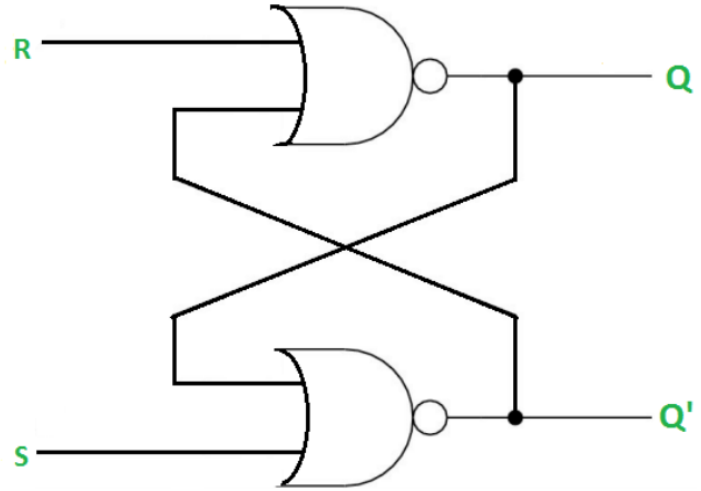


S-R Latch using NOR gates



Symbol of S-R Latch

Working of S-R Latch (using NOR gate)

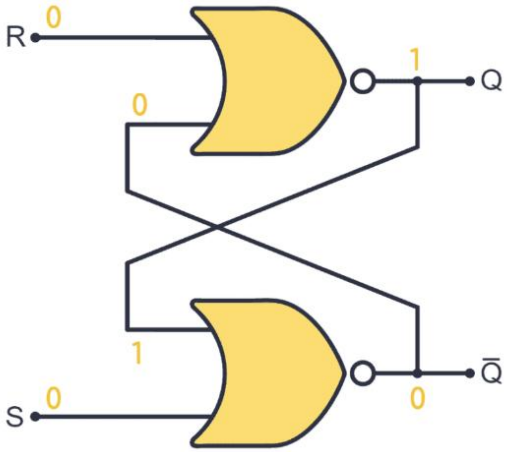


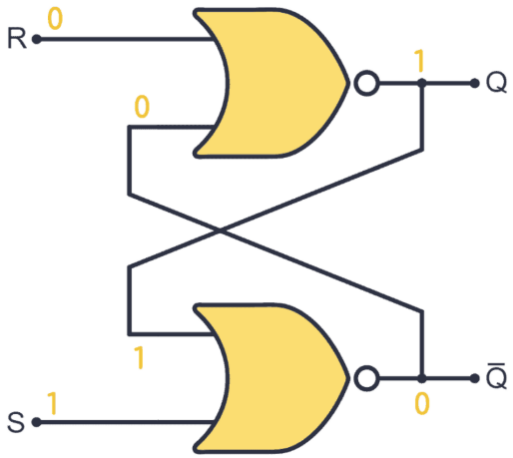
S-R Latch

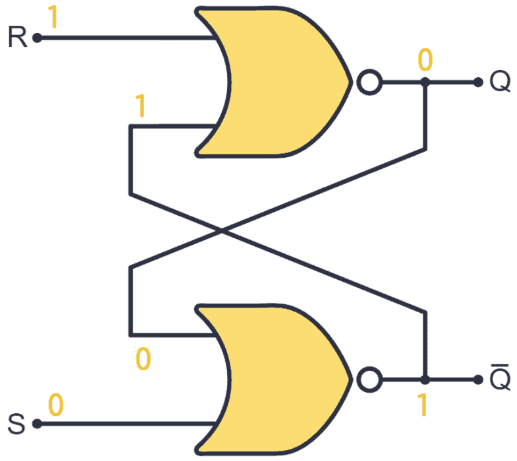
Truth Table

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Invalid

Q_n: current state Q_{n+1} : next state







S-R Latch (using NOR gate).....

Truth Table

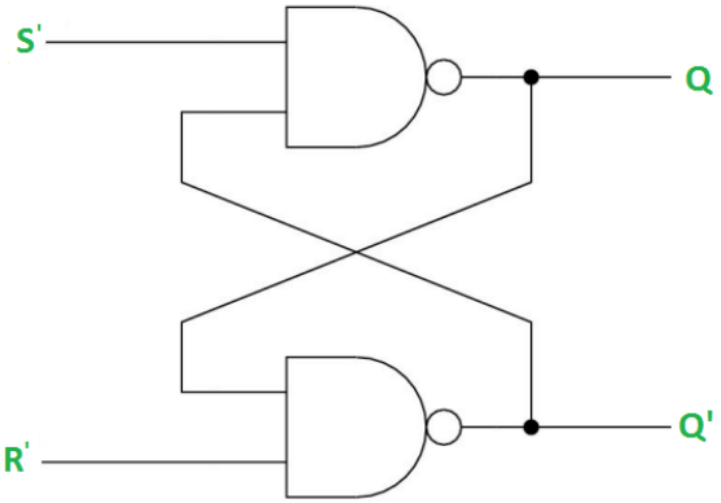
S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Invalid

Function Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	X
1	1	1	X

Q_n: current state Q_{n+1} : next state

Working of S-R Latch (using NAND gate)

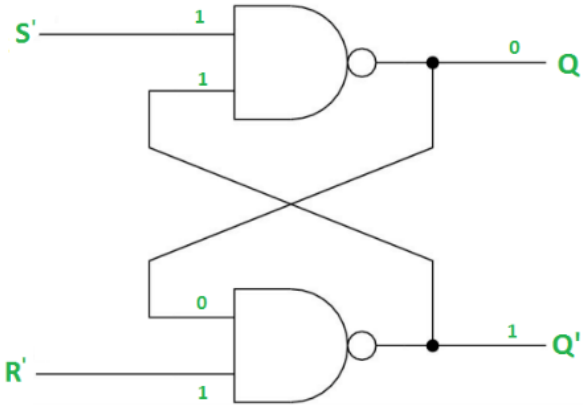


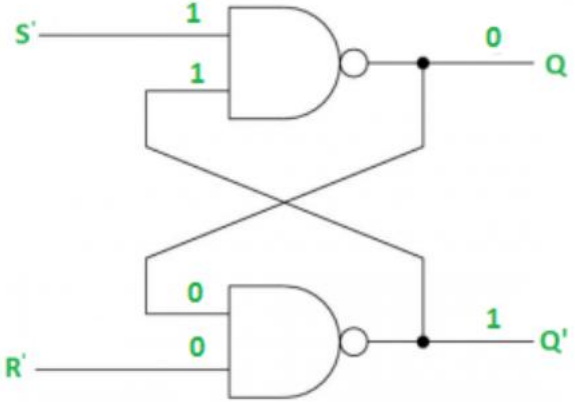
S-R Latch

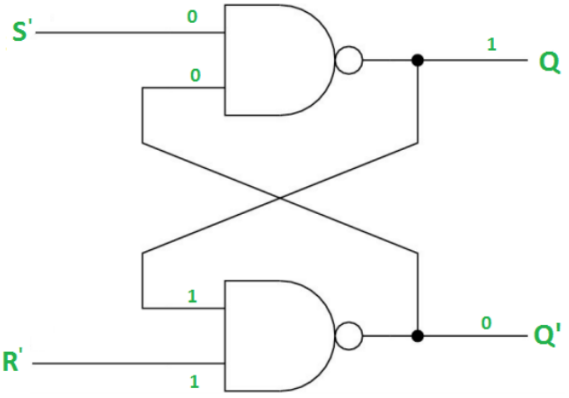
Truth Table

S’	R’	Q_{n+1}
1	1	Q_n
1	0	0
0	1	1
0	0	Invalid

Q_n: current state Q_{n+1} : next state







S-R Latch (using NAND gate)....

Truth Table

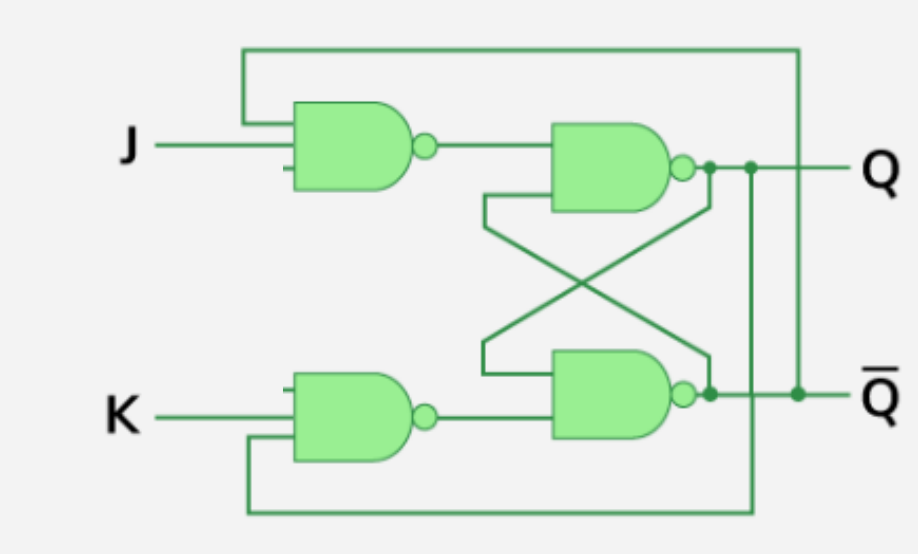
S’	R’	Q_{n+1}
1	1	Q_n
1	0	0
0	1	1
0	0	Invalid

Function Table

S’	R’	Q_n	Q_{n+1}
1	1	0	0
1	1	1	1
1	0	0	0
1	0	1	0
0	1	0	1
0	1	1	1
0	0	0	X
0	0	1	X

Q_n: current state Q_{n+1} : next state

J-K Latch (using NAND gate)



J-K Latch using NAND gates

Truth Table

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Q_n'

J-K Latch (using NAND gate)....

Truth Table

S	R	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Q_n'

Function Table

S	R	Q_n	Q_{n+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Q_n: current state Q_{n+1} : next state

Advantages of Latches

- **Easy to Implement** using basic digital logic gates.
- **Low Power Consumption** than other sequential circuits such as flip-flops.
- **High Speed** making them suitable for use in high-speed digital systems.
- **Low Cost** digital systems.
- **Versatile** for various applications, such as data storage, control circuits, and flip-flop circuits

Disadvantages of Latches

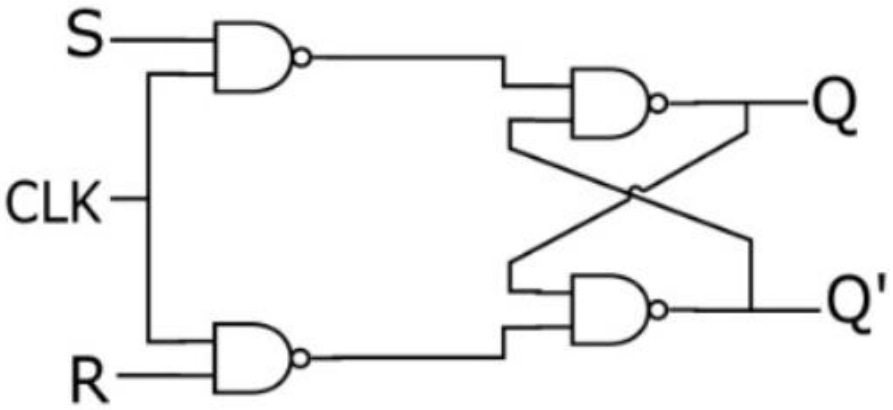
- Do not have a **clock signal to synchronize** their operations, making their behavior unpredictable.
- Latches can sometimes enter into an **unstable state**.
- Less suitable for real-time control applications

Flip-Flop

- **Synchronous: need of clock signal**
- Store a single bit of information
- Latching a value and changing it when triggered by a clock signal
- Temporary storage element in digital system
- Implement using various digital logic gates: AND, NAND….

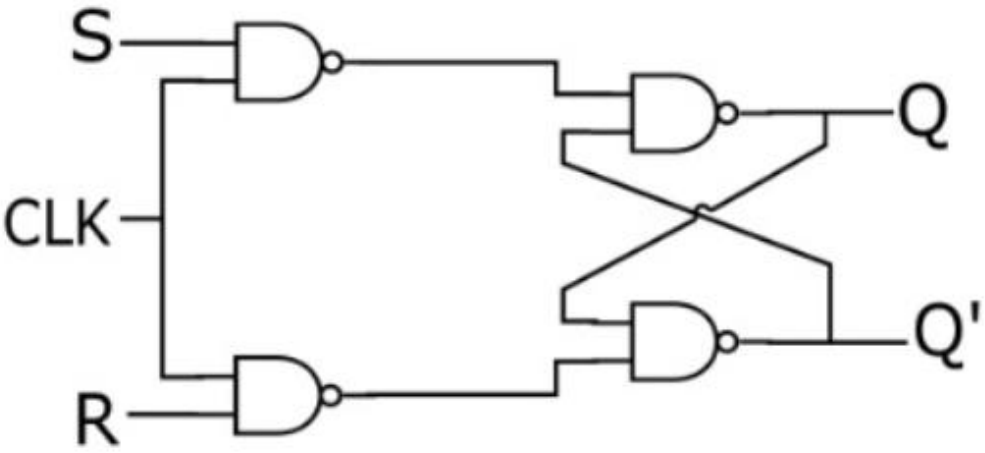
Types of Flip-Flops

- S-R Flip Flop
- J-K Flip Flop
- D Flip Flop
- T Flip Flop

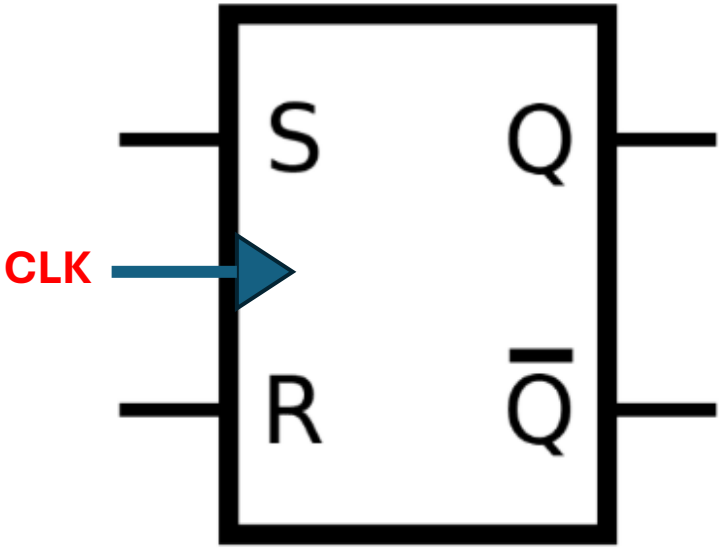


S-R Flip-flop

S-R Flip-Flop

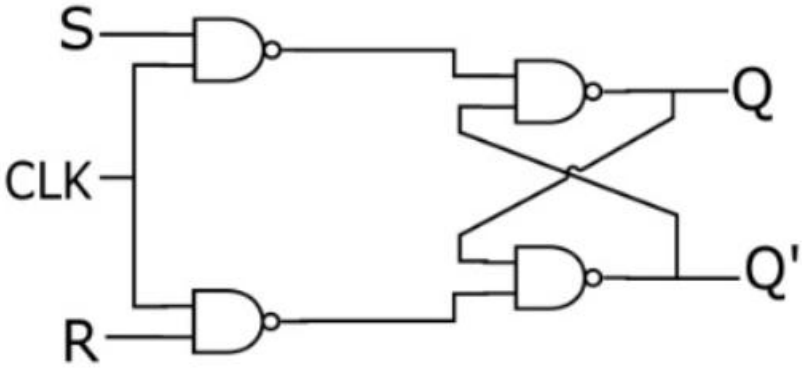


S-R Flip-flop using NAND gates



Symbol of S-R Flip-flop

S-R Flip-Flop



S-R Flip-flop using NAND gates

Truth Table

S	R	Qn+1	State
0	0	Qn	Hold
0	1	0	Reset
1	0	1	Set
1	1	X	Invalid

Function Table

S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Invalid
1	1	1	Invalid

Q_n: current state Q_{n+1} : next state

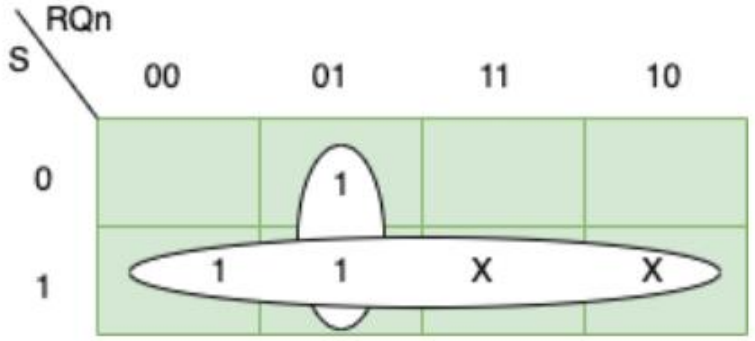
S-R Flip-Flop

Function Table

S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Invalid
1	1	1	Invalid

Characteristic Equation

- next state of flip flop in terms of present state



$$Q_{n+1} = S + Q_n.R'$$

S-R Flip-Flop

Function Table

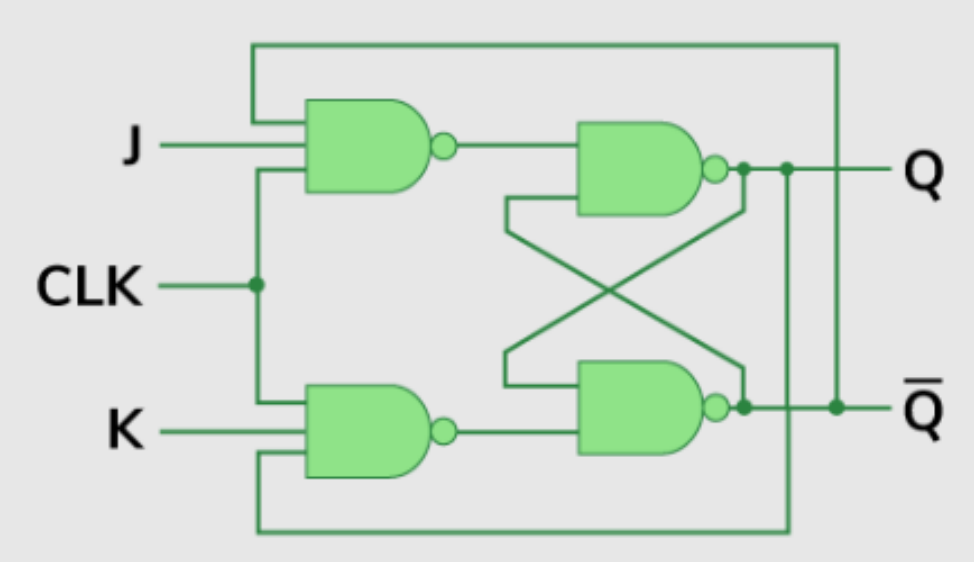
S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	Invalid
1	1	1	Invalid

Excitation Table

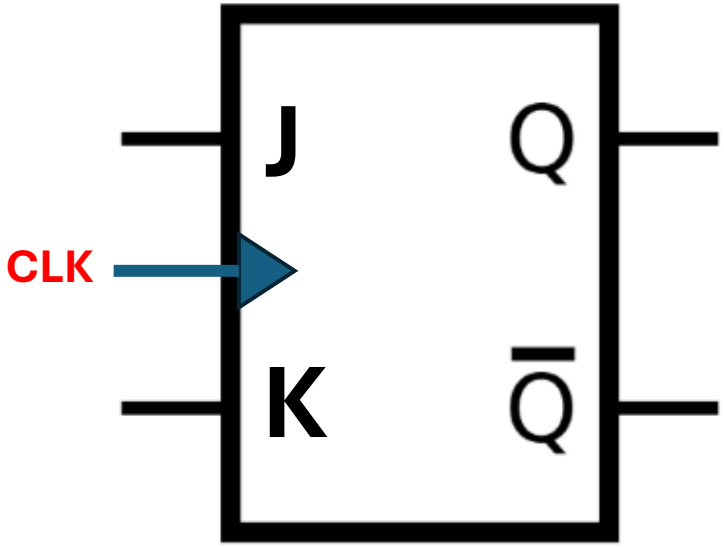
- Tells about the excitation to move from current state to next state

Qn	Qn+1	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

JK Flip-Flop

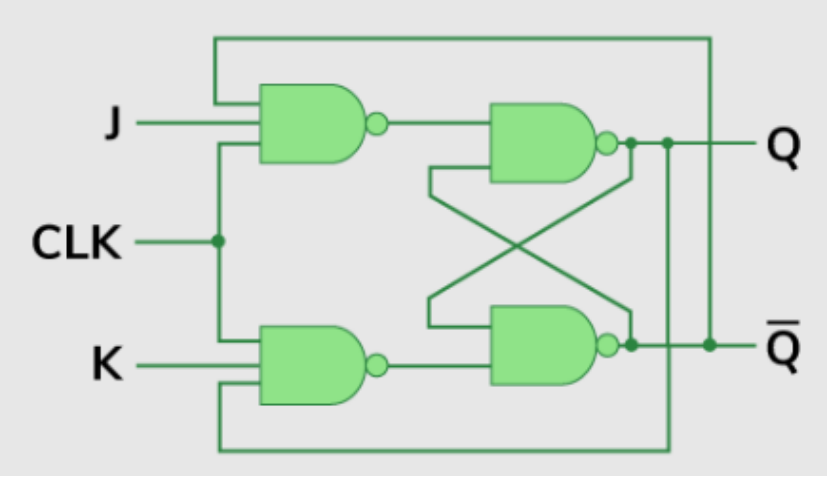


J-K Flip-flop using NAND gates



Symbol of J-K Flip-flop

JK Flip-Flop



Truth Table

J	K	Q_{N+1}
0	0	Q_N
0	1	0
1	0	1
1	1	Q_N'

Function Table

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Q_N: current state Q_{N+1} : next state

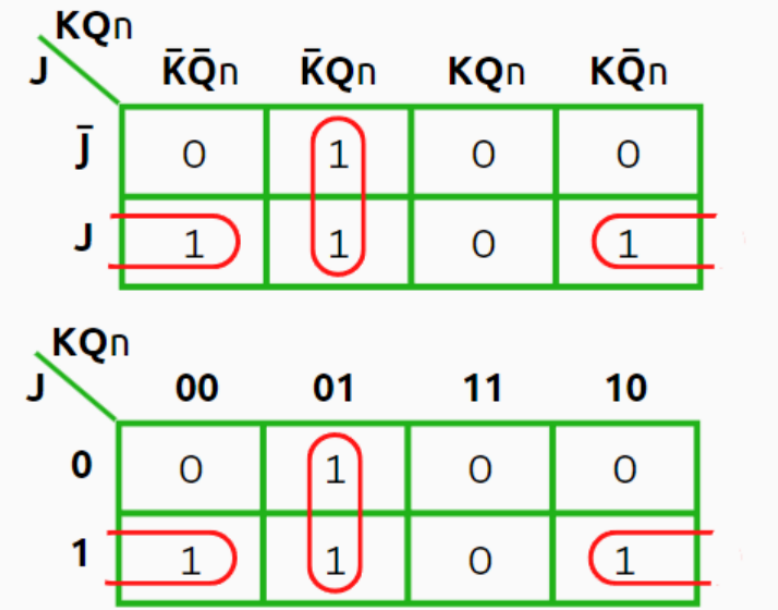
JK Flip-Flop

Function Table

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Characteristic Equation

- next state of flip flop in terms of present state



$$Q_{n+1} = J.\,Q_n' + K'.Q_n$$

JK Flip-Flop

Function Table

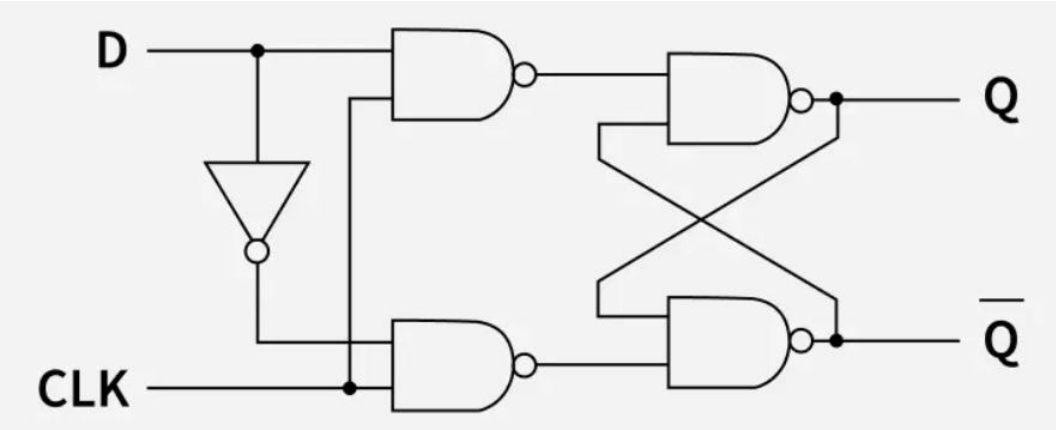
J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Excitation Table

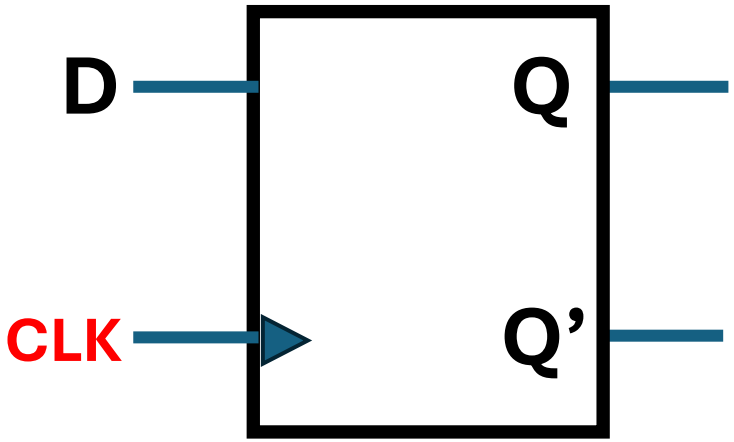
- Tells about the excitation to move from current state to next state

INPUT		OUTPUT	
Qn	Q(n+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

D Flip-Flop

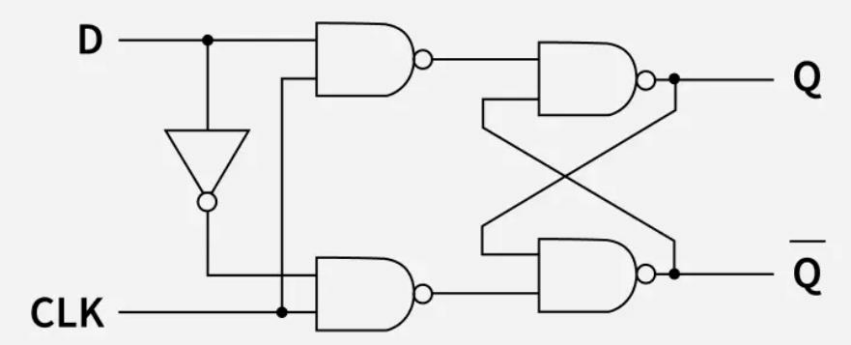


D Flip-flop using NAND gates



Symbol of D Flip-flop

D Flip-Flop



J-K Flip-flop using NAND gates

Truth Table

D	CLK	Q_{N+1}
0	1	0
1	1	1

Function Table

D	Q(Current)	Q(n+1) (Next)
0	0	0
0	1	0
1	0	1
1	1	1

D Flip-Flop

Function Table

D	Q(Current)	Q(n+1) (Next)
0	0	0
0	1	0
1	0	1
1	1	1

Characteristic Equation

- next state of flip flop in terms of present state

$$Q_{n+1} = D$$

D Flip-Flop

Function Table

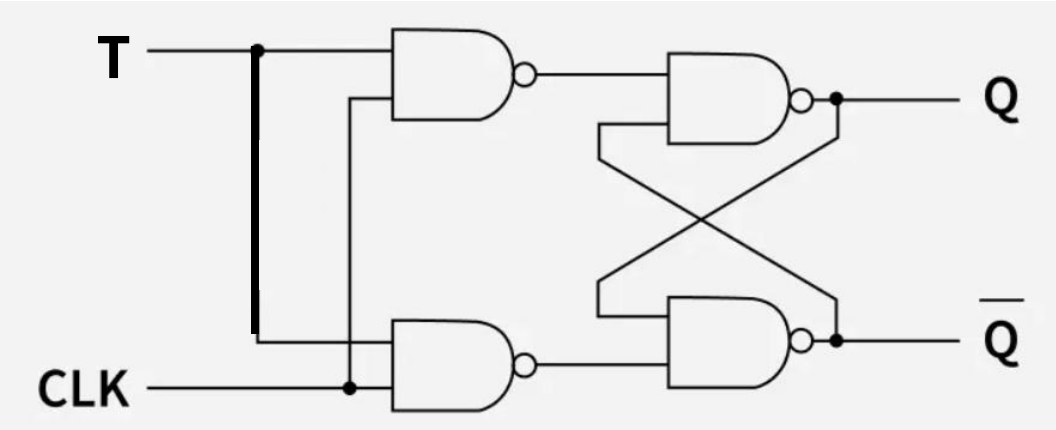
D	Q(Current)	Q(n+1) (Next)
0	0	0
0	1	0
1	0	1
1	1	1

Excitation Table

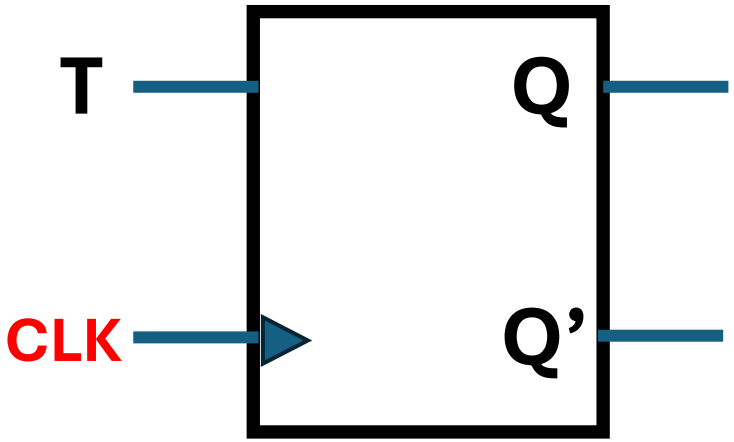
- Tells about the excitation to move from current state to next state

Q(n)	D(n)	Q(n+1) (Next)
0	0	0
0	1	1
1	0	0
1	1	1

T Flip-Flop

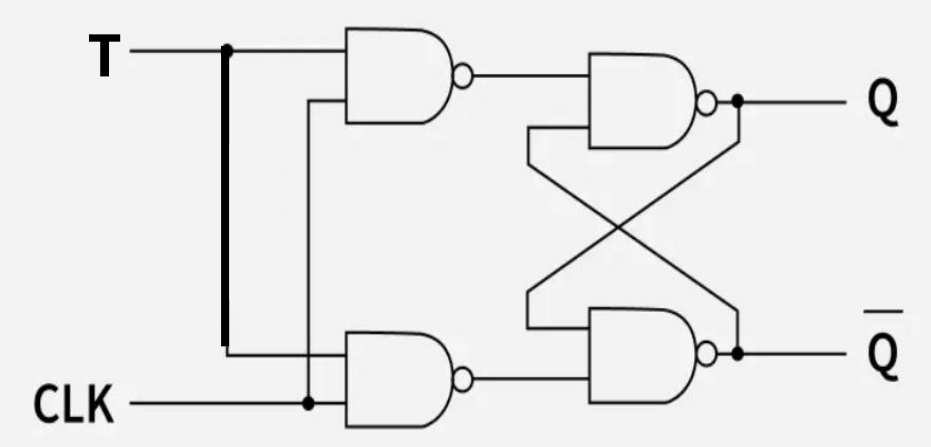


T Flip-flop using NAND gates



Symbol of T Flip-flop

T Flip-Flop



T Flip-flop using NAND gates

Function Table

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1
1	1	0

Truth Table

T	CLK	Q_{t+1}
0	1	1
1	1	0

Q: current state Q_{t+1} : next state

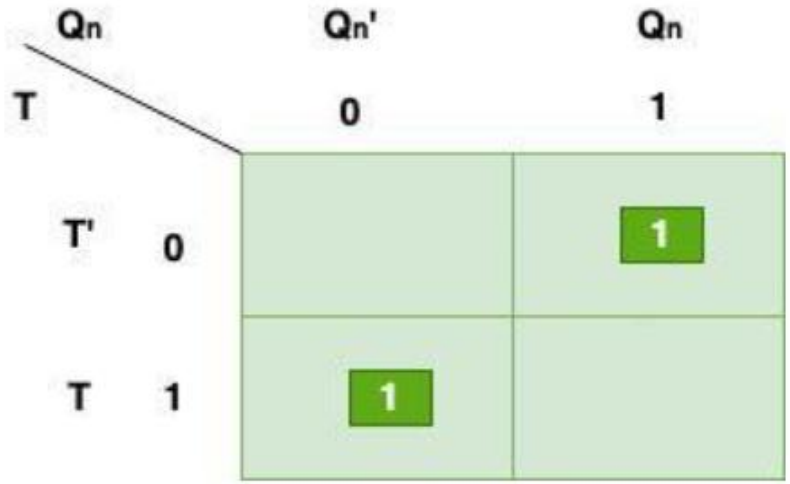
T Flip-Flop

Function Table

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1
1	1	0

Characteristic Equation

- next state of flip flop in terms of present state



$$Q_{n+1} = T.\,Q_n' + T'.Q_n$$

Q_n *or* Q: current state Q_{n+1} *or* Q_{t+1} : next state

T Flip-Flop

Function Table

T	Q	Q(t+1)
0	0	0
1	0	1
0	1	1
1	1	0

Excitation Table

- Tells about the excitation to move from current state to next state

Q_t	Q(t+1)	T
0	0	0
1	0	1
0	1	1
1	1	0

Q_n *or* Q: current state Q_{n+1} *or* Q_{t+1} : next state

Flip-Flop Conversions

- Follow these **steps** for converting one flip-flop to the other.

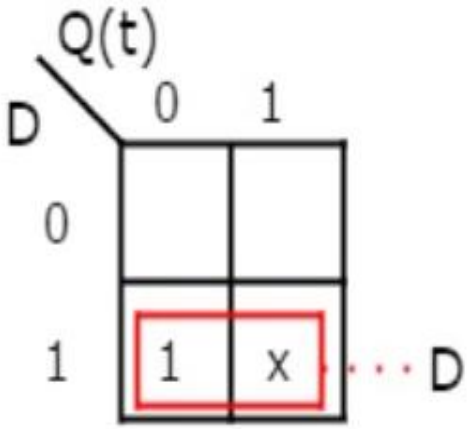
1. Consider the **characteristic table** of desired flip-flop.

2. Fill the excitation values (inputs) of given flip-flop for each combination of present state and next state. The **excitation table** for all flip-flops is shown below.

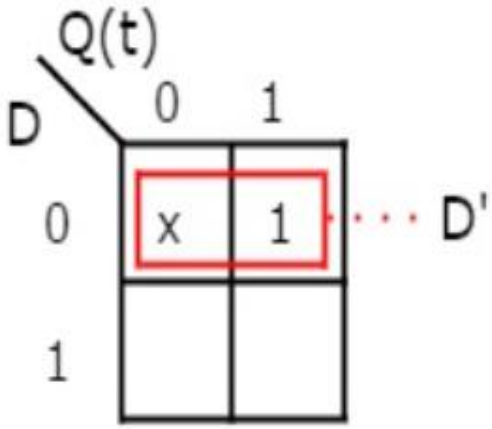
SR Flip-Flop to D Flip-Flop

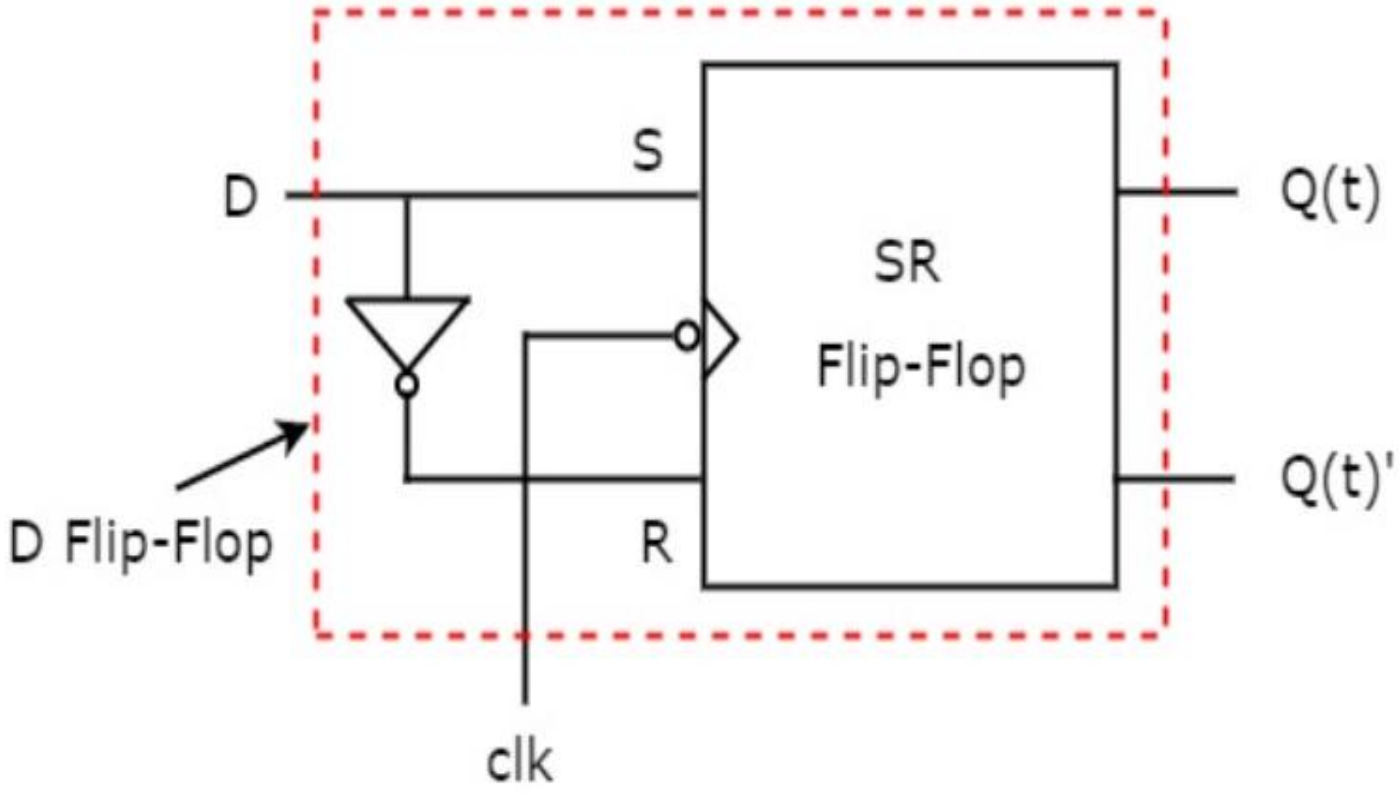
D Flip-Flop Input	Present State	Next State	SR Flip-Flop Inputs	
D	Q(t)	Q(t + 1)	S	R
0	0	0	0	x
0	1	0	0	1
1	0	1	1	0
1	1	1	x	0

K-Map for S



K-Map for R



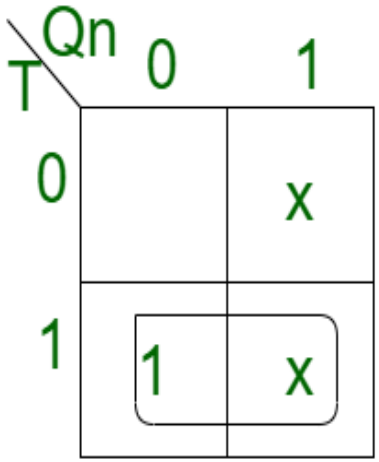

D
S
Q(t)
SR
Flip-Flop
Q(t)'
D Flip-Flop
R
clk

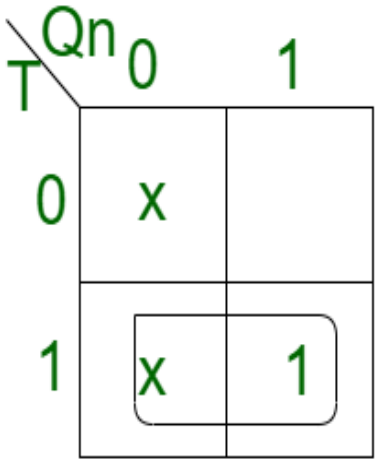
Conversion of J-K Flip-Flop into T Flip-Flop

- **Step-1:** Construct the characteristic table of T flip-flop and excitation table of the J-K flip-flop.

T	Qn	Qn+1	J	K
0	0	0	0	x
0	1	1	x	0
1	0	1	1	x
1	1	0	x	1

- **Step-2:** Using the K map, find the boolean expression for J and K in terms of T and Q.





J = T

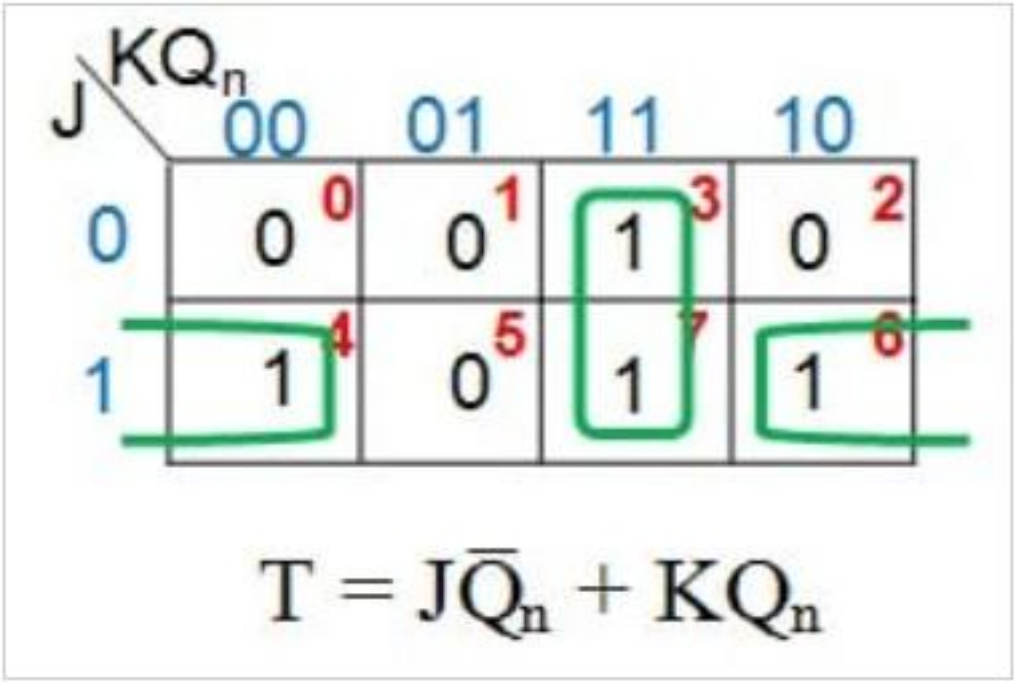
K = T

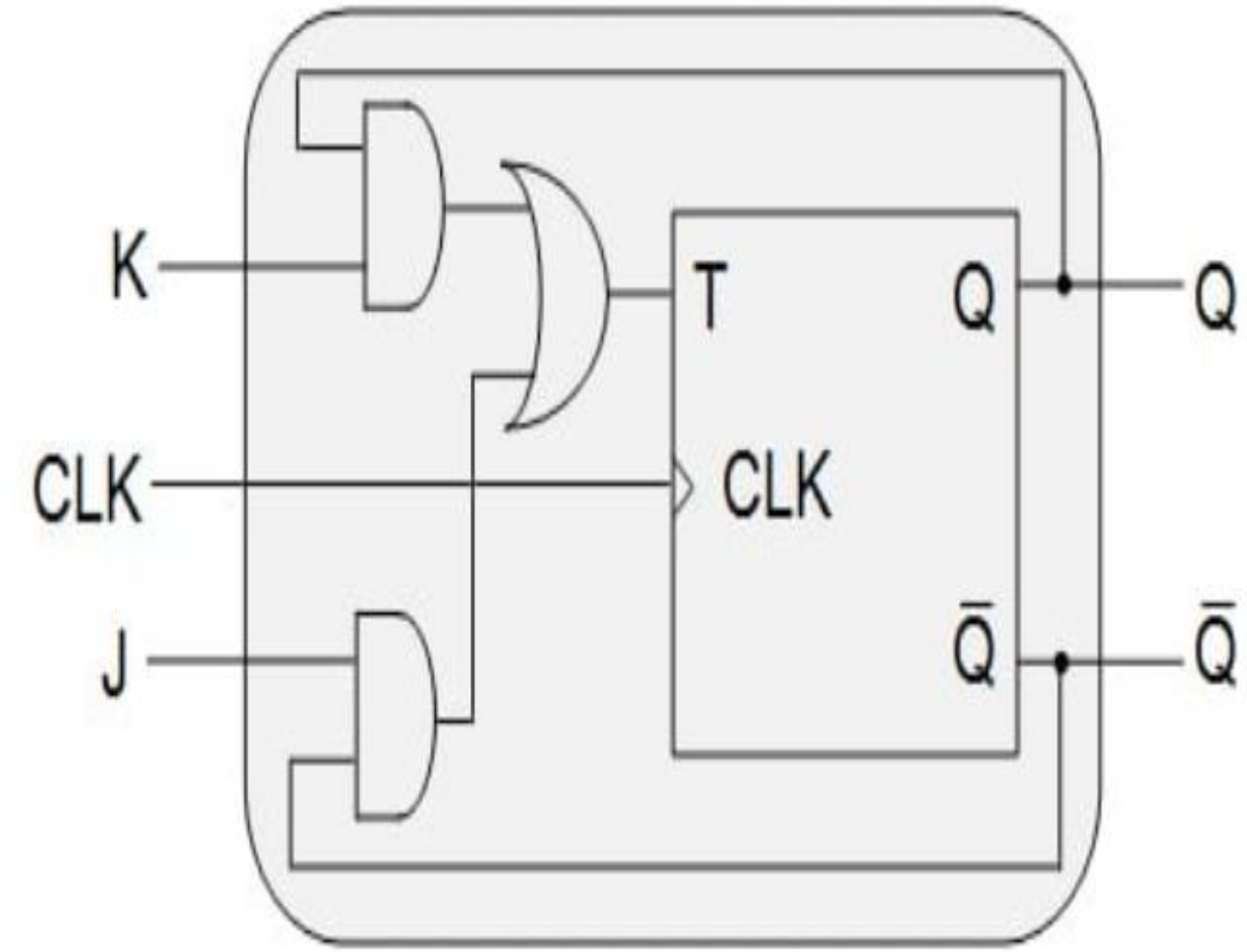
JK Inputs		Outputs		T Input
		Present State	Next State	
J	K	Q_n	Q_{n+1}	T
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	1

T to JK Conversion Table

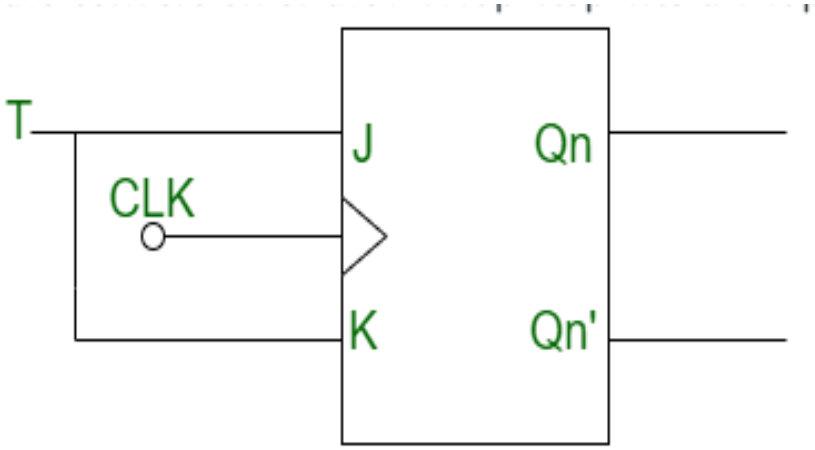
Excitation Table of T Flip-flop

Outputs		Input
Present State	Next State	
Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0


J
KQn
00
01
11
10
0
1
0 0
0 1
1 3
0 2
1 4
0 5
1 7
1 6
T = JQ̄n + KQn


K
CLK
J
T
CLK
Q
$\bar{Q}$
Q
$\bar{Q}$

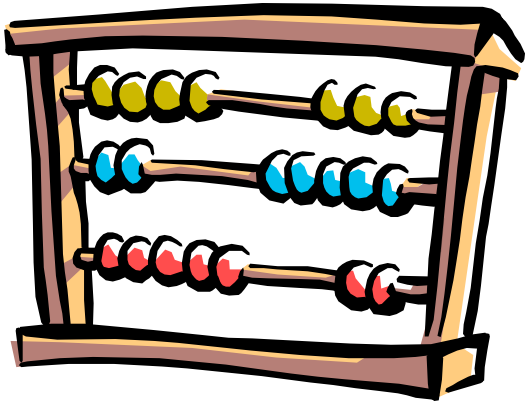
- **Step-3:** Construct the circuit diagram for the conversion of the J-K flip-flop into a T flip-flop.



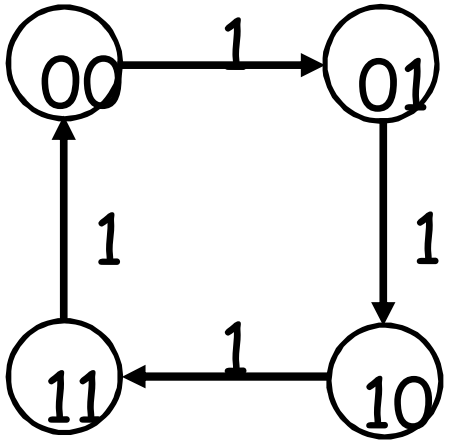
Counter

Counters

- Counters are a specific type of sequential circuit.
- Like registers, the state, or the flip-flop values themselves, serves as the "output."
- The output value increases by one on each clock cycle.
- After the largest value, the output "wraps around" back to 0.
- Using two bits, we'd get something like this:



Present State		Next State	
A	B	A	B
0	0	0	1
0	1	1	0
1	0	1	1
1	1	0	0



Benefits of counters

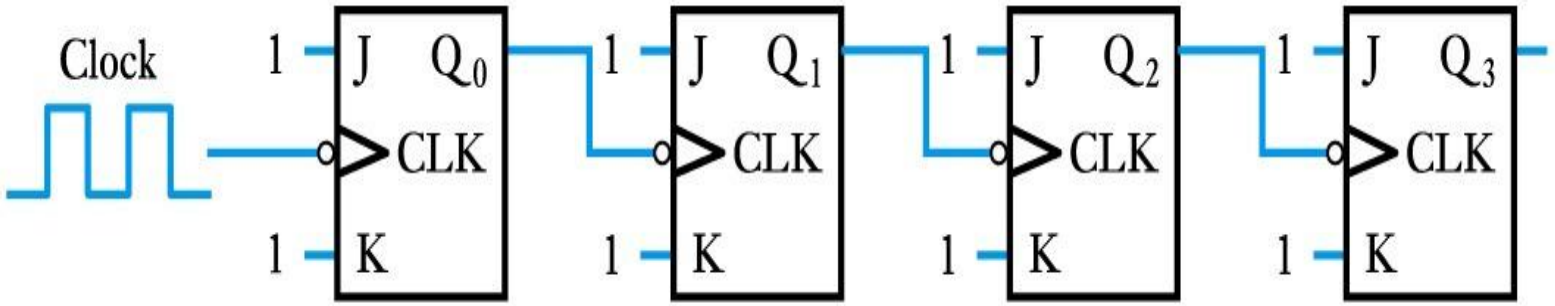
- Counters can act as simple clocks to keep track of “time.” You may
- need to record how many times something has happened.
  - –How many bits have been sent or received? –How many steps have been performed in some computation?
- All processors contain a program counter, or PC.
  - –Programs consist of a list of instructions that are to be executed one after another (for the most part).
  - –The PC keeps track of the instruction currently being executed.
  - –The PC increments once on each clock cycle, and the next program instruction is then executed.

- In digital logic and computing, a **counter**is a device which stores (and sometimes displays) the number of times a particular event or process has occurred, often in relationship toa clock signal.

Classifications of Counters

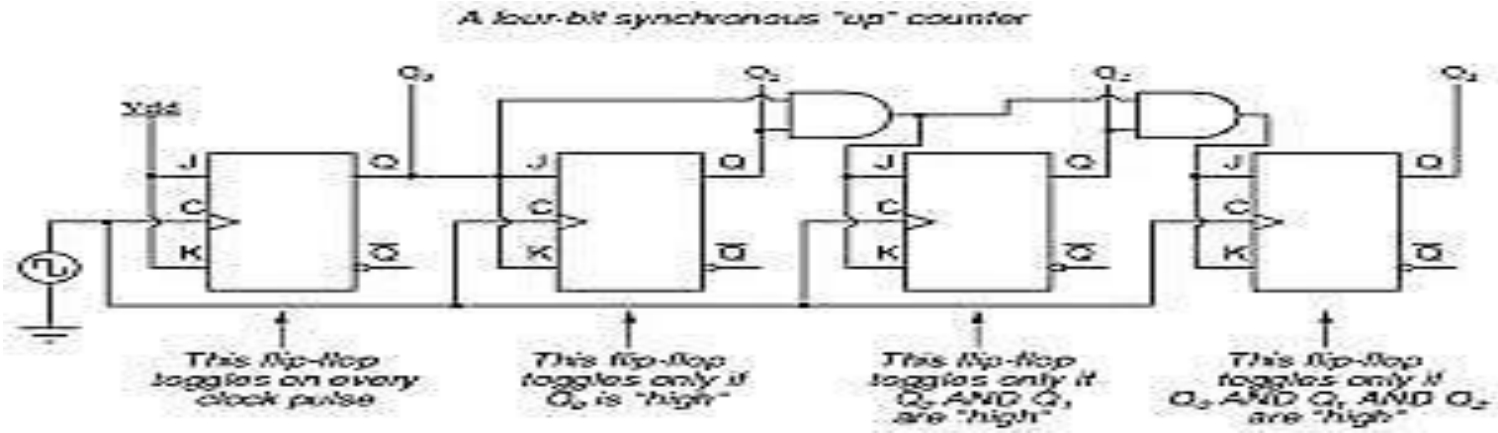
Asynchronous Counters

- Onlythe first flip-flop is clocked by an external clock. All subsequent flip-flops are clocked by the output of the preceding flip-flop.means output of previous flip-flop is connected to clock input of next flip flop.
- Asynchronous counters are slower than synchronous counters because of the delay in the transmission of the pulses from flip-flop to flip-flop.
- Asynchronous counters are also **called *ripple-counters*** because of the way the clock pulse ripples it way through the flip-flops.



Synchronous Counters

- Allflip-flops are clocked simultaneously by an external clock. Means clock input of all flip flops are connected to same external clock.
- Synchronous counters are faster than asynchronous counters because of the simultaneous clocking.
- Synchronous counters are an example of *state machine* design because they have a set of states and a set of transition rules for moving between those states after each clocked event.



States / Modulus / Flip-Flops

- The number of flip-flops determines the count limit or number of states.
  (STATES = 2 *# of flip flops*)
- The number of states used is called the *MODULUS.*
- For example, a Modulus-12 counter would count from 0 (0000) to 11 (1011) and requires four flip-flops (16 states -12 used).

Electronic counters --Examples

1. Up/down counter –counts both up and down, under command of a control input

2. Ring counter –formed by a shift register with feedback connection in a ring

3. Johnson counter –a *twisted*ring counter

4. Cascaded counter

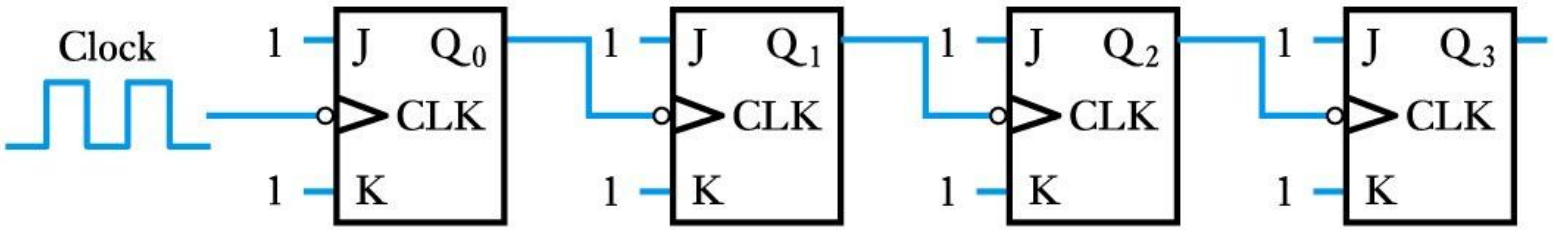
5. Decade Counter

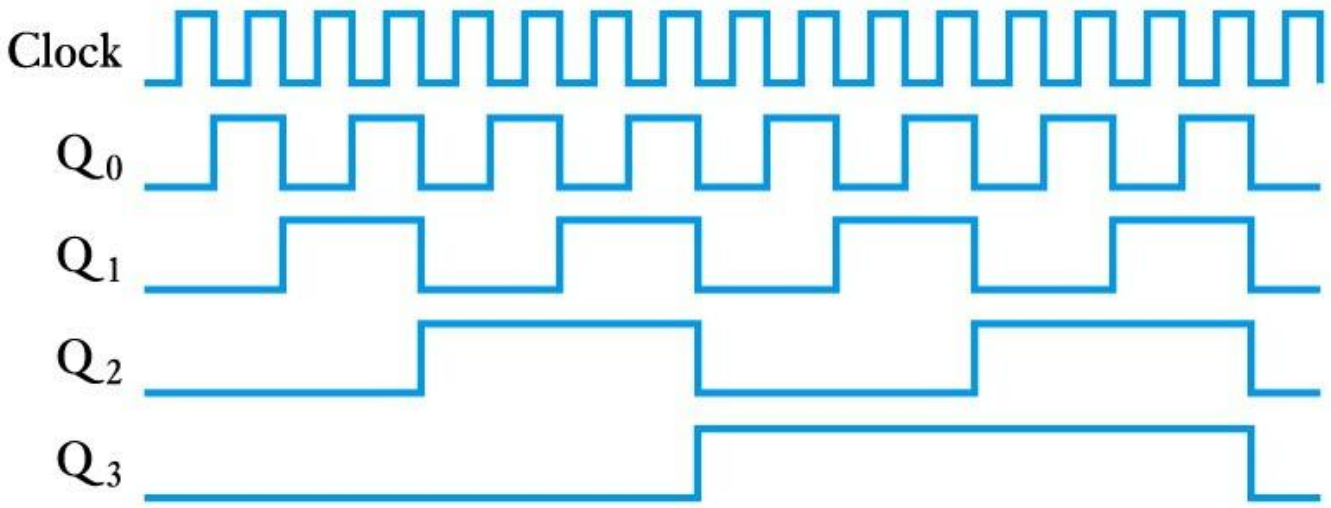
Asynchronous Counters

Asynchronous Counter/Ripple counters

–can be constructed usingseveralflip flops

–consider the following arrangement –with *J*= *K*= 1 each flip flop toggles on the falling edge of its clock input



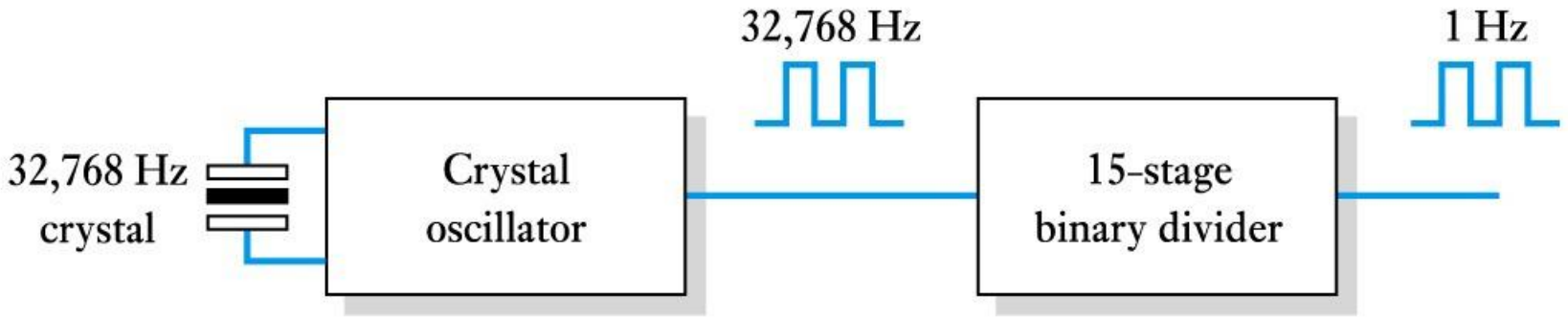


- Each stage toggles at half the frequency of the previous stage
  –acts as a frequency divider –divides frequency by 2*n*(*n*is the number of stages)

- Application of a frequency divider

Clock generator for a digital watch

  – 15-stage counter divides signal from a crystal oscillator by 32,768 to produce a 1 Hz signal to drive stepper motor or digital display



Consider the pattern on the outputs of the counter as shown –displayed on the right

the outputs count in binary from 0 to 2*n*-1 and then repeat

Number of clock pulses	Q_3	Q_2	Q_1	Q_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1
16	0	0	0	0
17	0	0	0	1
18	0	0	1	0
19	0	0	1	1
20	0	1	0	0

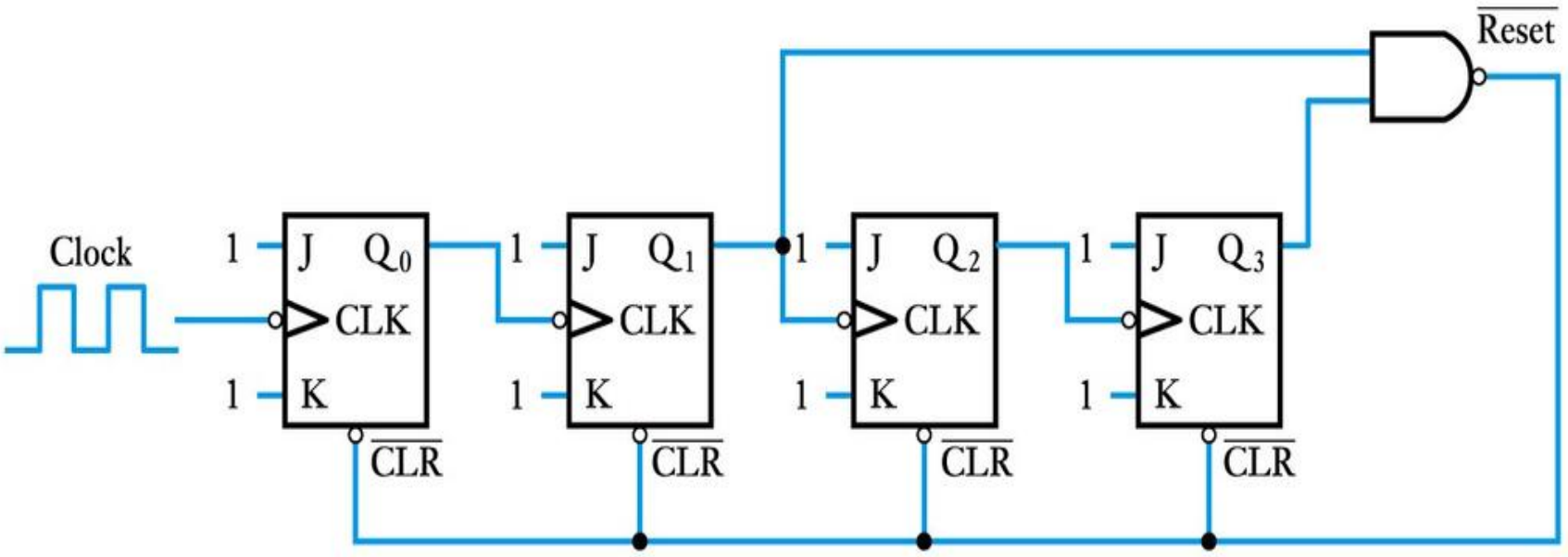
- the circuit actsas a **modulo-2^ncounter**
- since the counting process propagates from one bistable to the next this is called a **ripple counter**

–circuit shown is a **4-bit**or **modulo-16** (or **mod-16**) ripple counter

Modulo-*N* counters

–by using an appropriate number of stages the earlier counter can count modulo any power of 2

–to count to any other base we add reset circuitry
–e.g. the modulo-10 or decade counter shown here



Down and up/down Counters

–aslight modification totheearlier circuit will produce a counter that counts from 2*n*-1 to 0 and then restarts

–this is a **down counter**

–a further modification can produce an **up/downcounter** which counts up or down depending on the state of a control line (usually labelled u p $\overline{\text{/ d o w}}$ n)

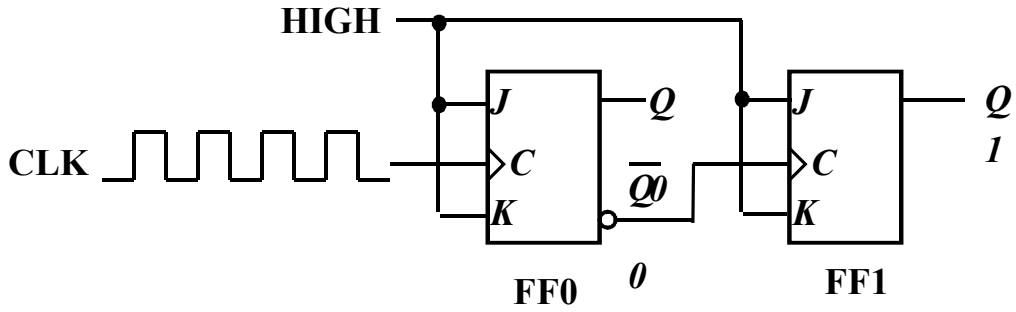
- when this is 1 the counter counts up
- when this is 0 the counter counts down

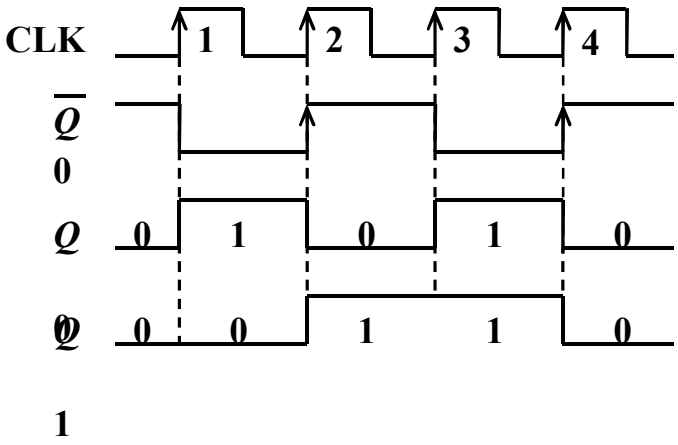
Drawbacks/Limitation of Ripple Counter

- **Propagation delay in counters**
  - –while ripple countersarevery simple they suffer from problems at high speed
  - –since the output of one flip-flop is triggered by the change of the previous device, delays produced by each flip-flop are summed along the chain
  - –the time for a single device to respond is termed its **propagation delay time***tPD*
  - –an *n*-bit counter will take *n* *tPD*to respond –if read before this time the result will be garbled

Asynchronous (Ripple) Counters

- Example: 2-bit ripple binarycounter.
- Output of one flip-flop is connected to the clock input of the next more-significant flip-flop.



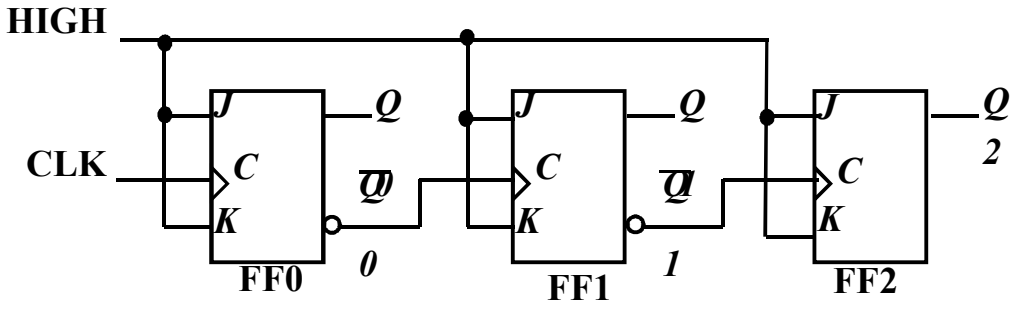


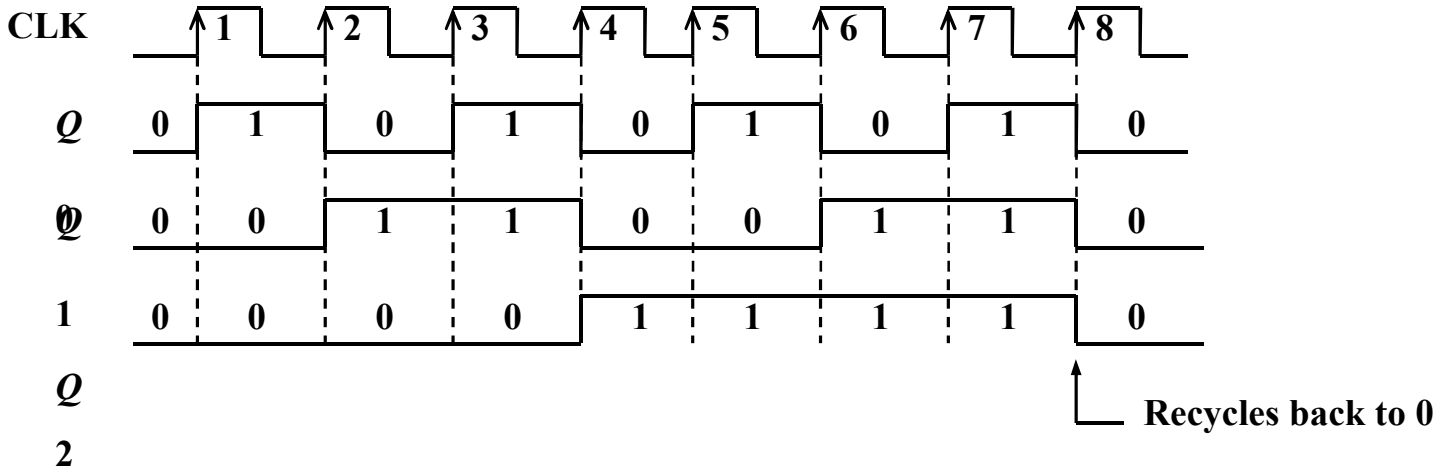
Timing diagram

00 → 01 → 10 → 11 → 00 ...

Asynchronous (Ripple) Counters

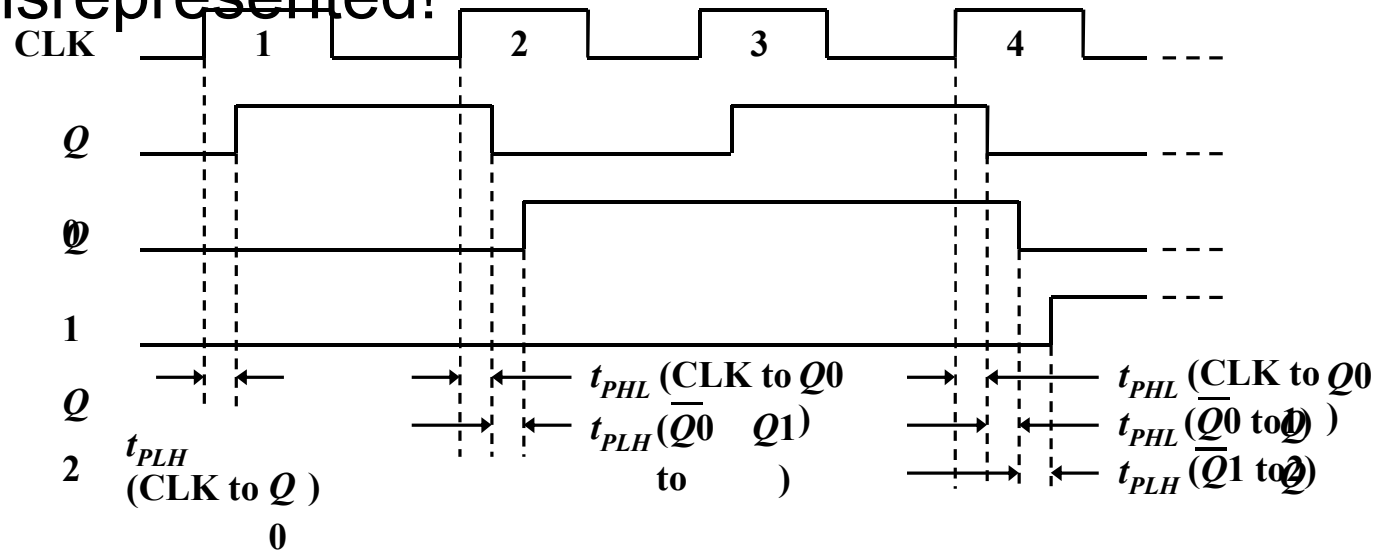
- Example: 3-bit ripple binary counter.





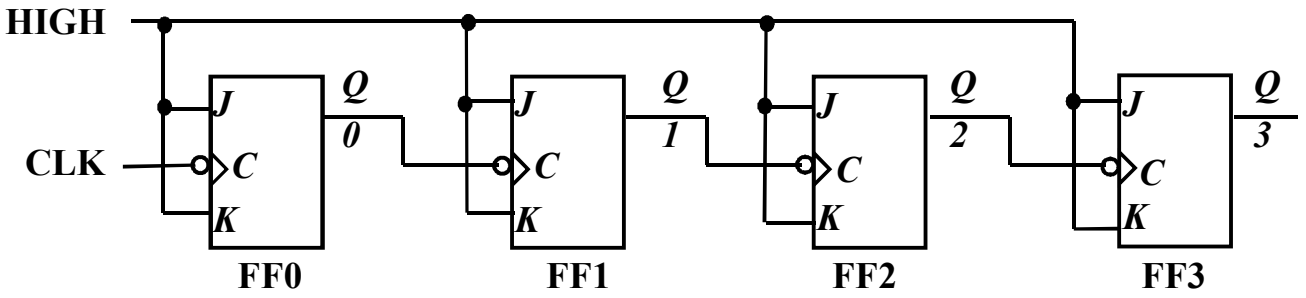
Asynchronous (Ripple) Counters

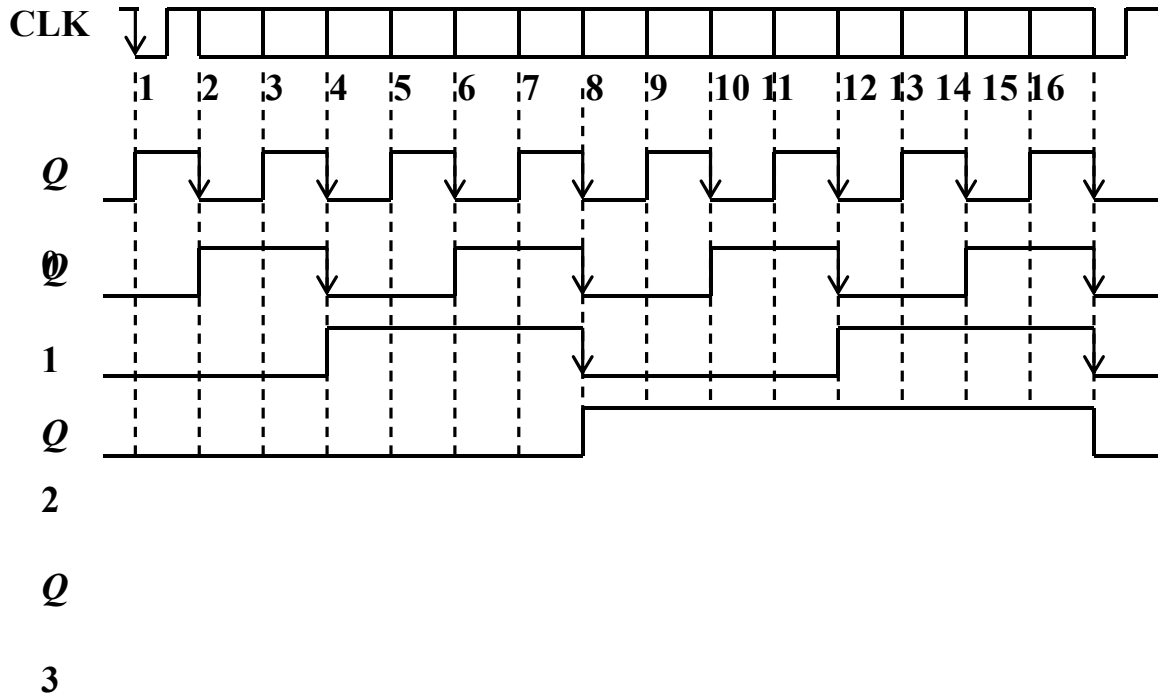
- Propagationdelays in an asynchronous (ripple-clocked) binary counter.
- If the accumulated delay is greater than the clock pulse, some counter states may be misrepresented!



Asynchronous (Ripple) Counters

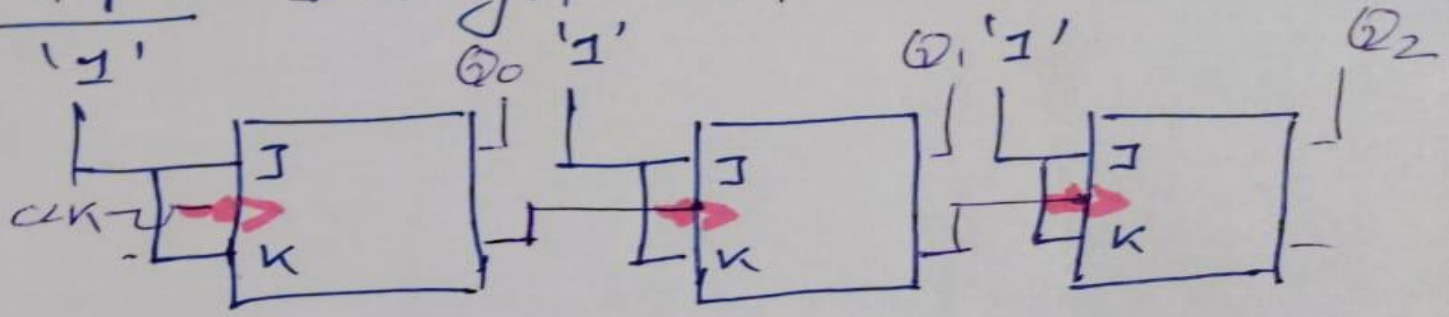
- Example:4-bitripplebinarycounter (negative-edge triggered).





Design MOD-N counter

step1 Design Full MOD counter

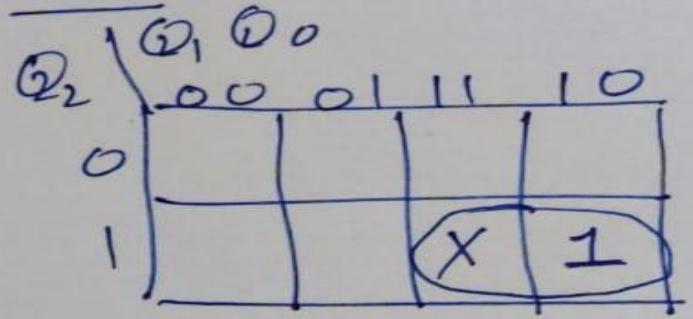


step2:- Draw the Full Table

Q_2	Q_1	Q_0	R
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	X

K map for R

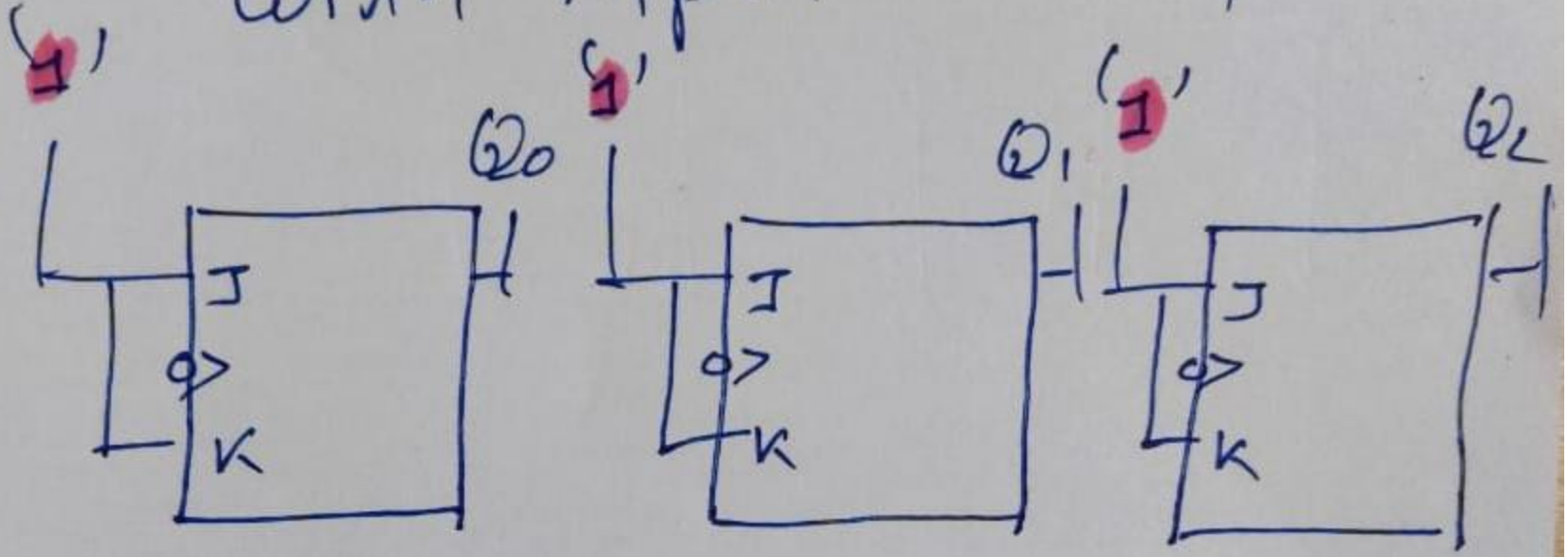
R



$R = Q_2 Q_1$

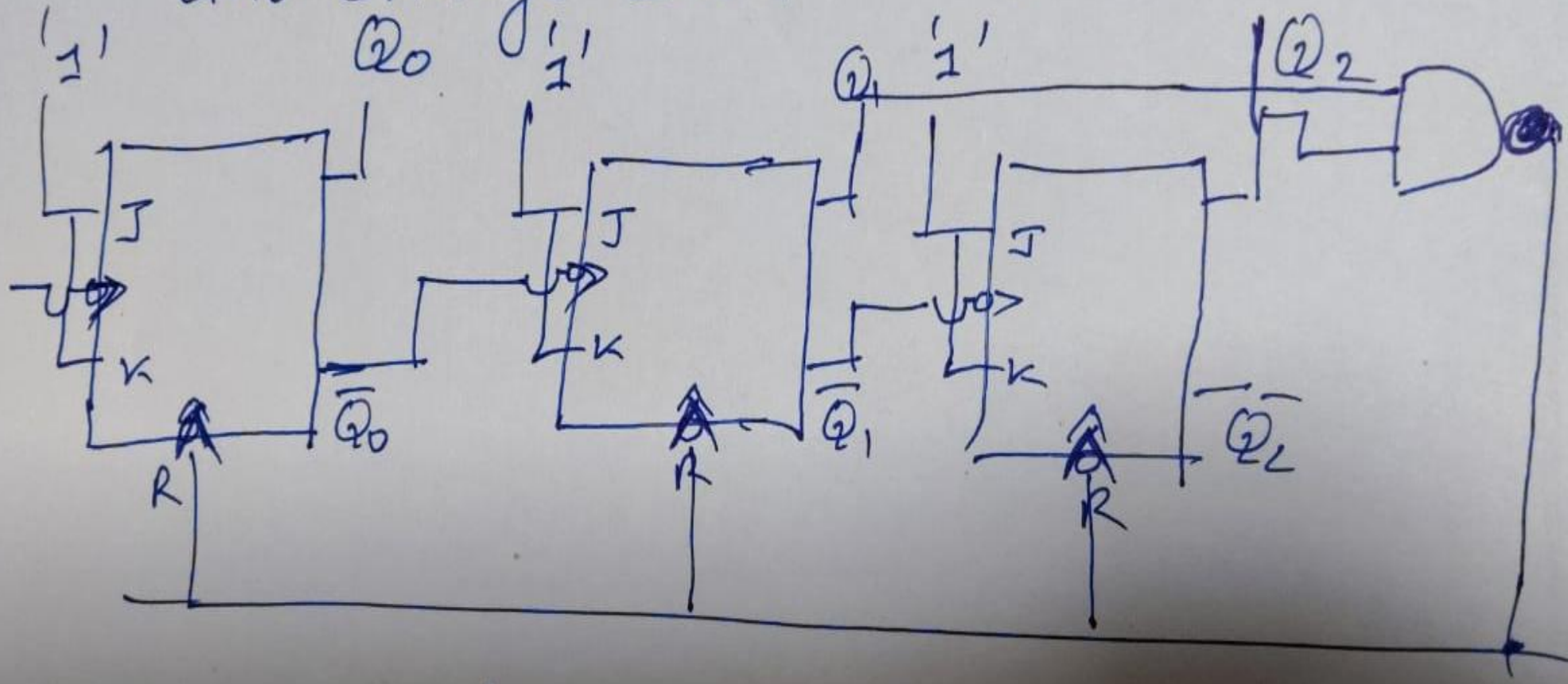
Step 3:- Modified counter

* Add the MAMD gate with input Q_2 Q_1



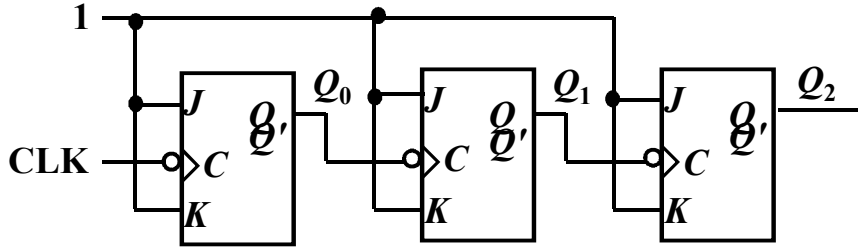
Step3:- modified Counter

* Add the AND GATE with input Q_2 Q_1 and change with MAND

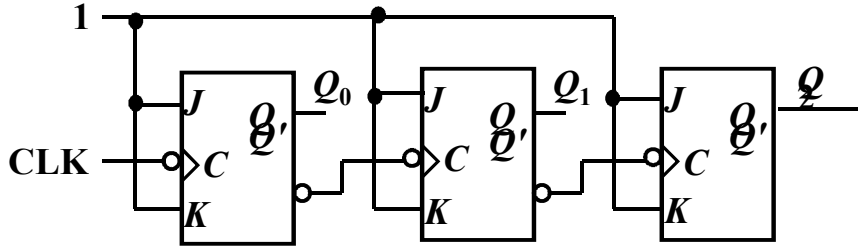


Asynchronous Down Counters

- So far we are dealing with *up counters*. *Down counters*, on the other hand, count downward from a maximum value to zero, and repeat.
- Example: A 3-bit binary (MOD-23) down counter.



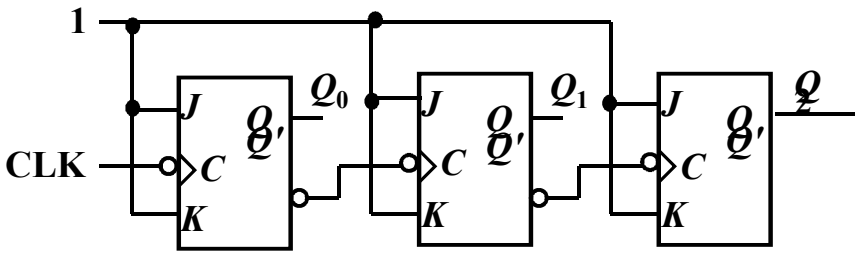
3-bitbinary up counter

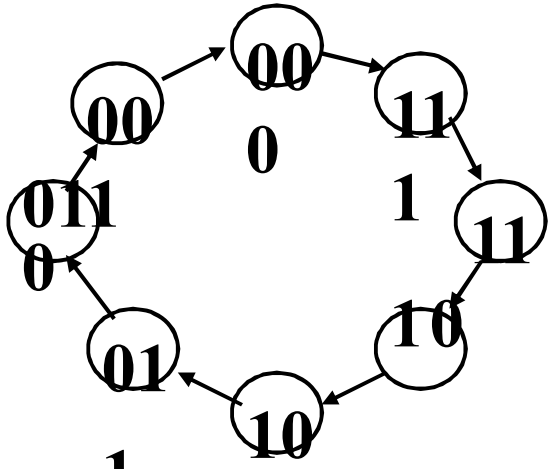


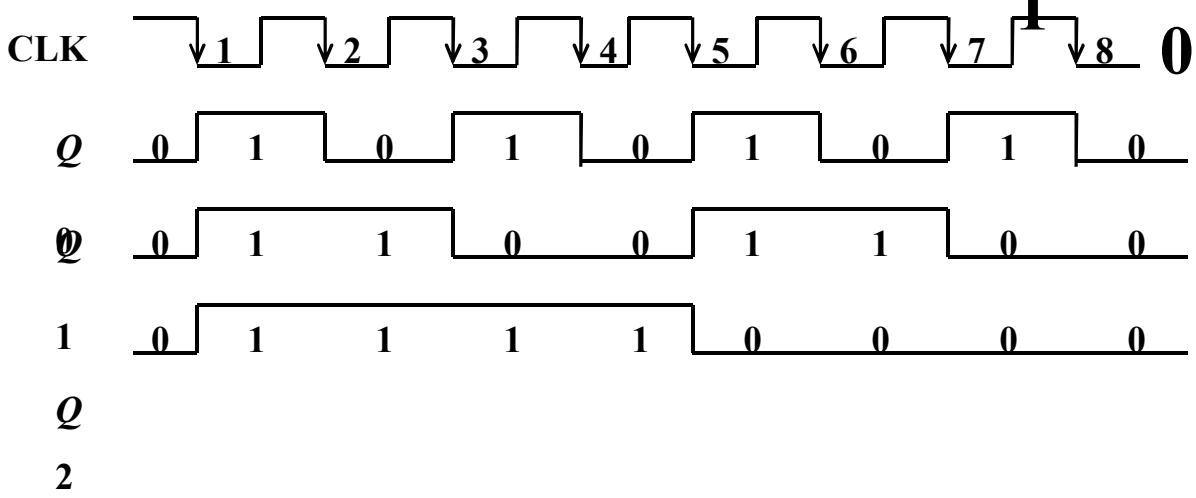
3-bit binary down counter

Asynchronous Down Counters

- Example:A3-bitbinary(MOD-8) down counter.



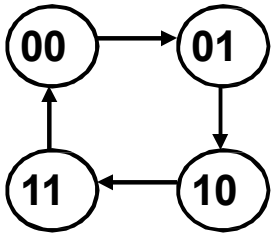




Synchronous Counter

Synchronous (Parallel) Counters

- Synchronous (parallel) counters: the flip-flops are clocked at the same time by a common clock pulse.
- We can design these counters using the sequential logic design process (covered in Lecture #12).
- Example: 2-bit synchronous binary counter (using T flip-flops, or JK flip-flops with identical J,K inputs).



Present state		Next state		Flip-flop inputs	
A_1	A_0	A_1^+	A_0^+	TA_1	TA_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

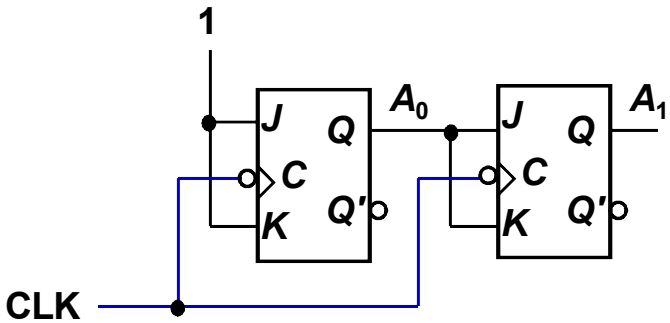
Synchronous (Parallel) Counters

- Example: 2-bit synchronous binary counter (using T flip-flops, or JK flip-flops with identical J,K inputs).

Present state		Next state		Flip-flop inputs	
A_1	A_0	A_1^+	A_0^+	TA_1	TA_0
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1

$TA_1 = A_0$

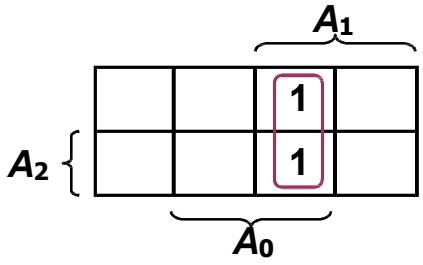
$TA_0 = 1$



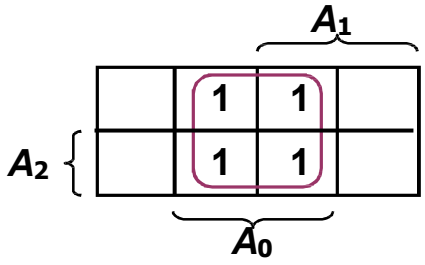
Synchronous (Parallel) Counters

- Example: 3-bit synchronous binary counter (using T flip-flops, or JK flip-flops with identical J, K inputs).

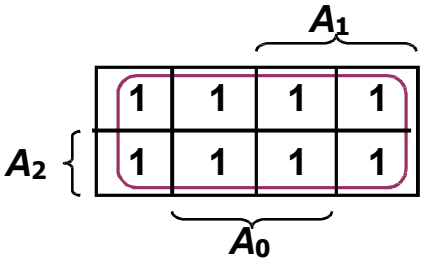
Present state			Next state			Flip-flop inputs		
A_2	A_1	A_0	A_2^+	A_1^+	A_0^+	TA_2	TA_1	TA_0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0	1
0	1	1	1	0	0	1	1	1
1	0	0	1	0	1	0	0	1
1	0	1	1	1	0	0	1	1
1	1	0	1	1	1	0	0	1
1	1	1	0	0	0	1	1	1



$TA_2 = A_1.A_0$



$TA_1 = A_0$

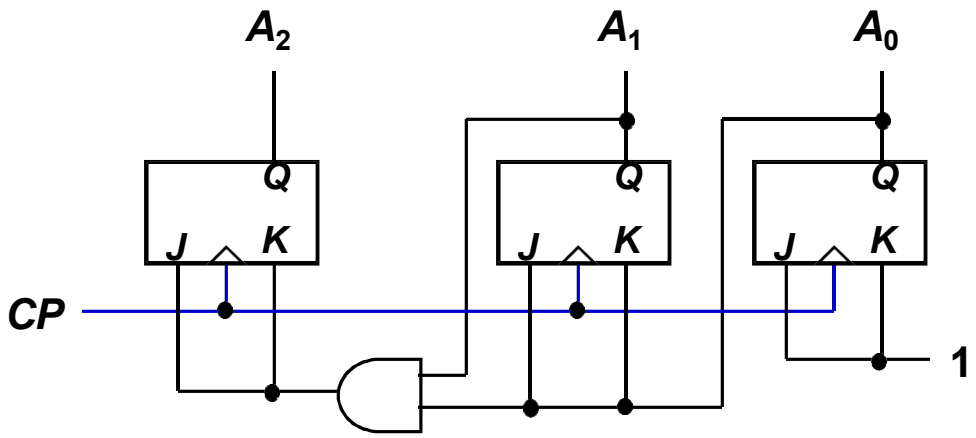


$TA_0 = 1$

Synchronous (Parallel) Counters

- Example: 3-bit synchronous binary counter (cont'd).

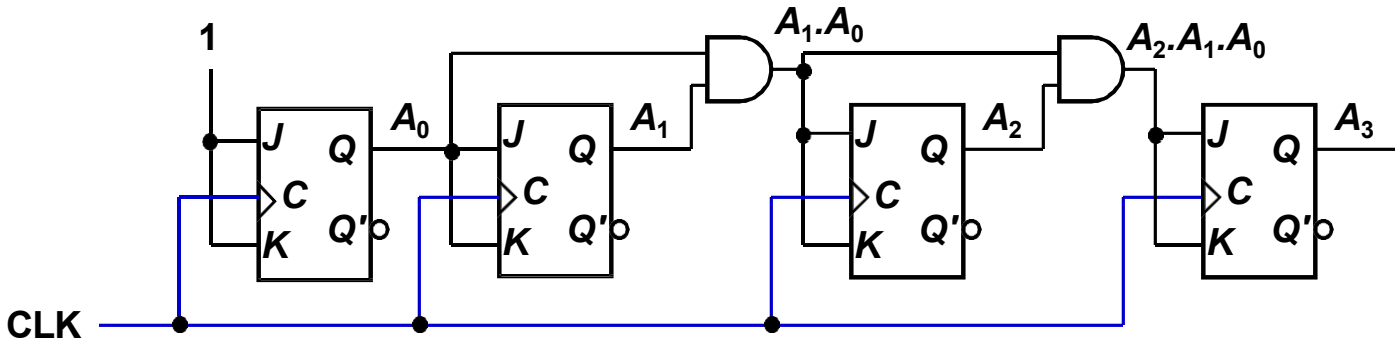
$TA_2 = A_1.A_0$ $TA_1 = A_0$ $TA_0 = 1$



Synchronous (Parallel) Counters

- Example: 4-bit synchronous binary counter.

$$TA_3 = A_2 \cdot A_1 \cdot A_0$$
$$TA_2 = A_1 \cdot A_0$$
$$TA_1 = A_0$$
$$TA_0 = 1$$



Try :

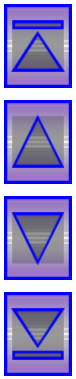
Design of 3 BIT Synchronous counter using

(a)T (Down counter)

(b)D(Up Counter)

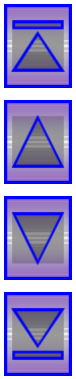
Introduction: Registers

- An *n*-bit register has a group of *n* flip-flops and some logic gates and is capable of storing *n* bits of information.
- The flip-flops store the information while the gates control when and how new information is transferred into the register.
- Some functions of register:
  - retrieve data from register
  - store/load new data into register (serial or parallel)



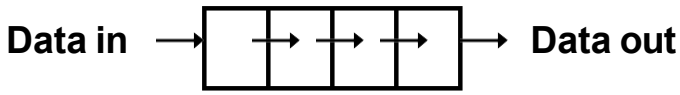
Shift Registers

- Another function of a register, besides storage, is to provide for *data movements*.
- Each *stage* (flip-flop) in a shift register represents one bit of storage, and the shifting capability of a register permits the movement of data from stage to stage within the register, or into or out of the register upon application of clock pulses.

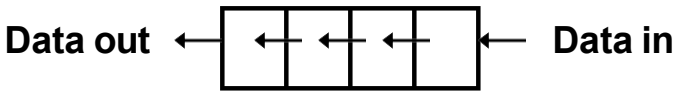


Shift Registers

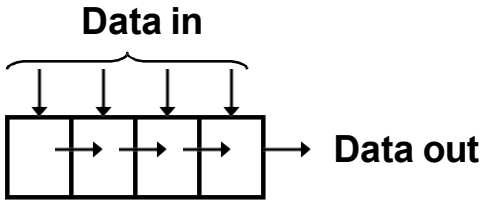
- Basic data movement in shift registers (four bits are used for illustration).



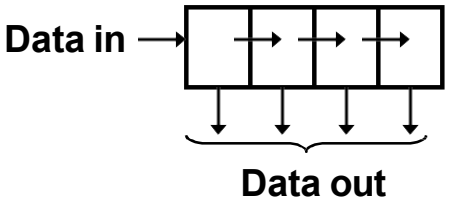
(a) Serial in/shift right/serial out



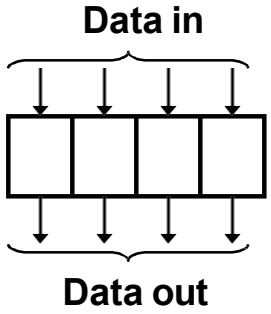
(b) Serial in/shift left/serial out



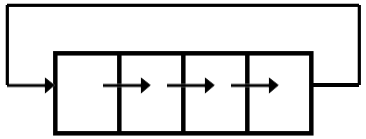
(c) Parallel in/serial out



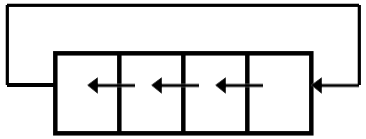
(d) Serial in/parallel out



(e) Parallel in / parallel out



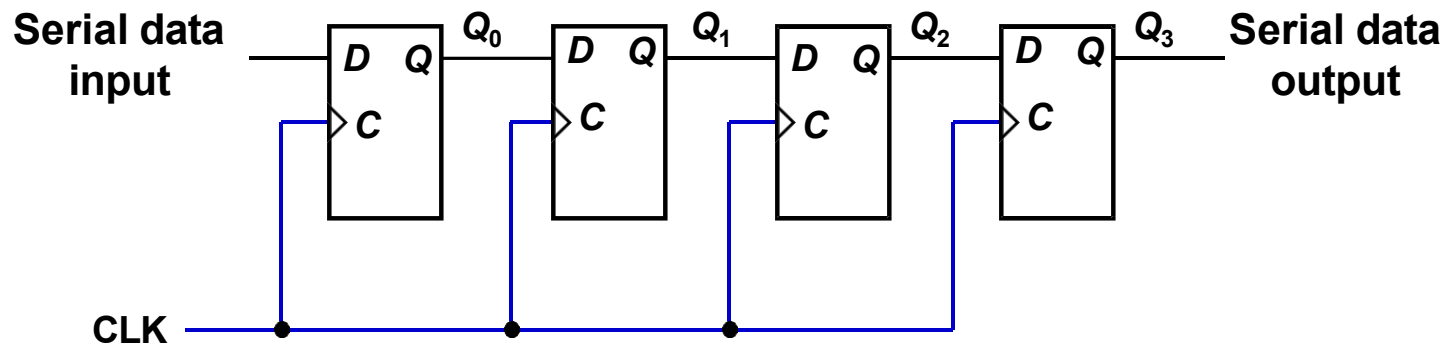
(f) Rotate right



(g) Rotate left

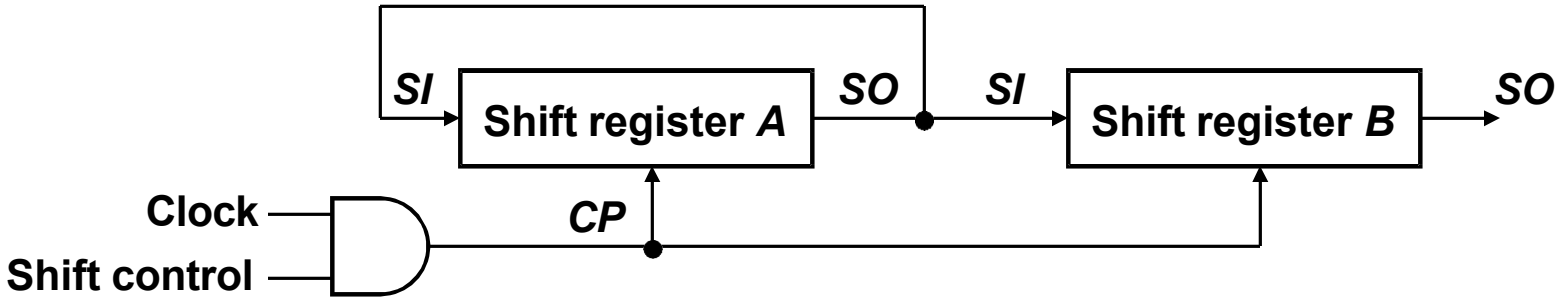
Serial In/Serial Out Shift Registers

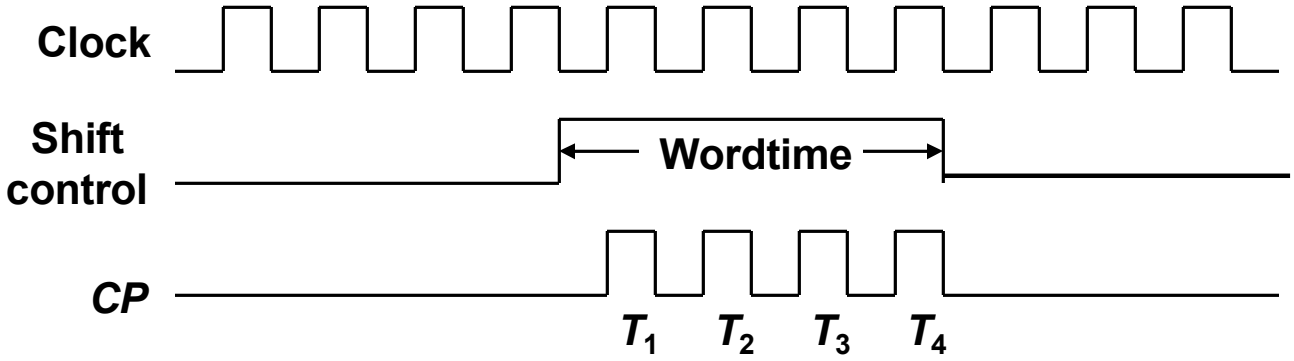
- Accepts data serially – one bit at a time – and also produces output serially.



Serial In/Serial Out Shift Registers

- Application: Serial transfer of data from one register to another.

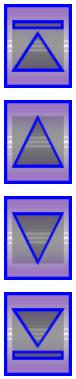




Serial In/Serial Out Shift Registers

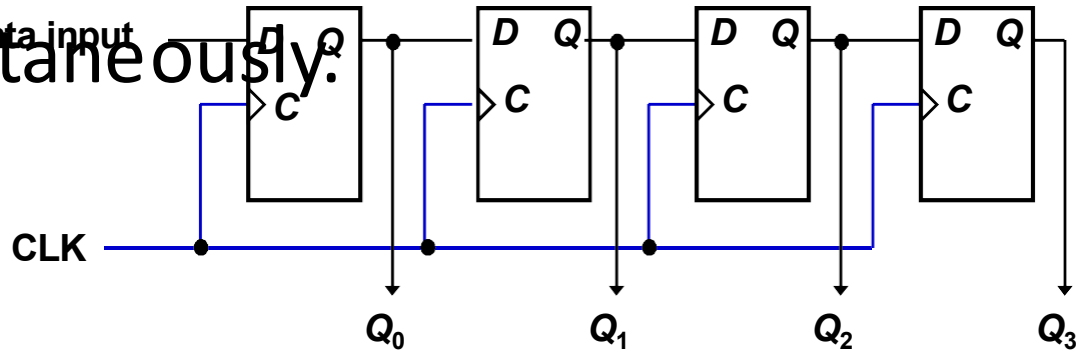
- Serial-transfer example.

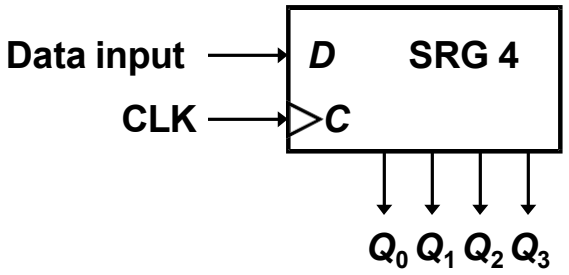
Timing Pulse	Shift register *A*				Shift register *B*				Serial output of *B*
Initial value	1	0	1	1	0	0	1	0	0
After T_1	1	1	0	1	1	0	0	1	1
After T_2	1	1	1	0	1	1	0	0	0
After T_3	0	1	1	1	0	1	1	0	0
After T_4	1	0	1	1	1	0	1	1	1



Serial In/Parallel Out Shift Registers

- Accepts data serially.
- Outputs of all stages are available simultaneously.

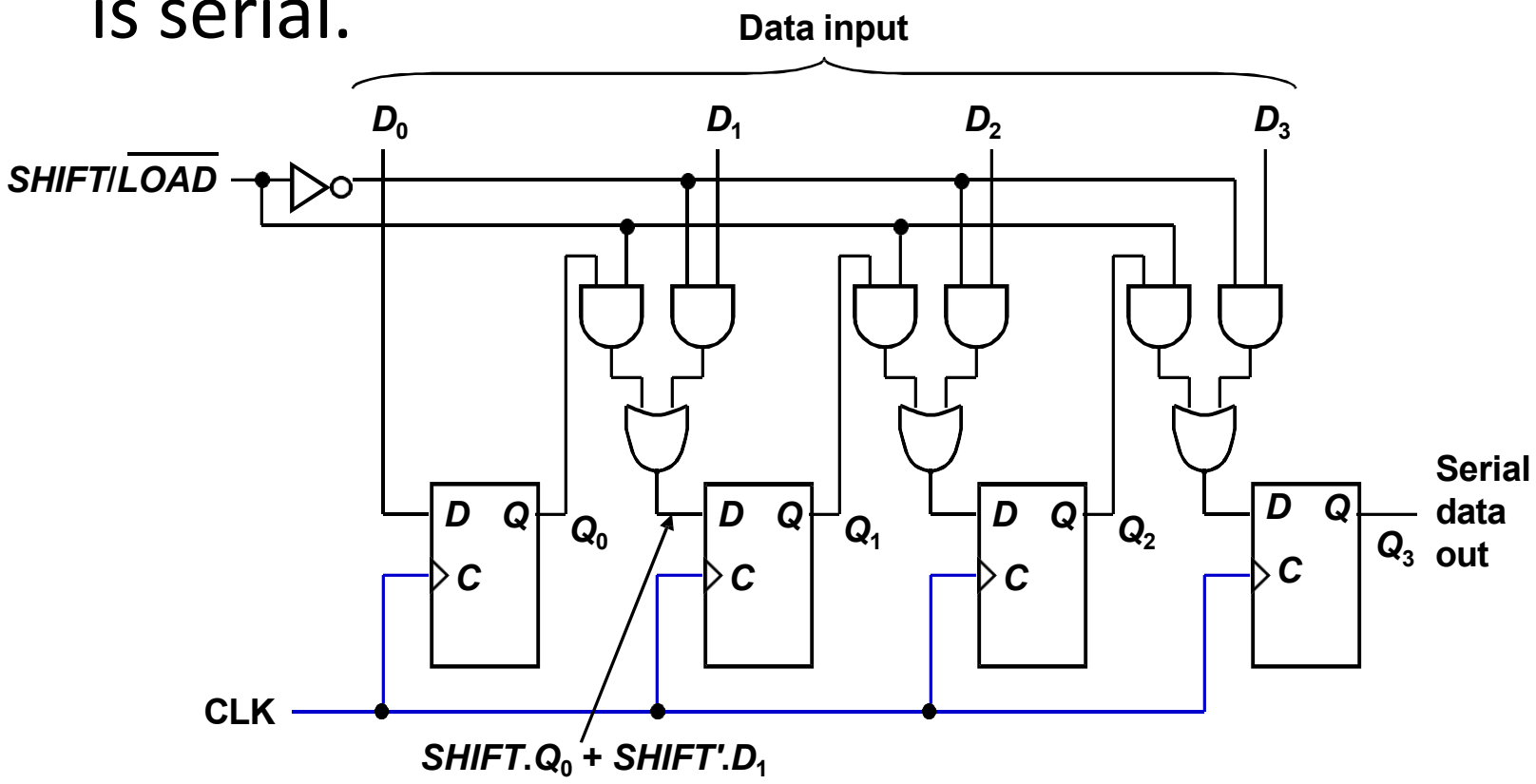




Logic symbol

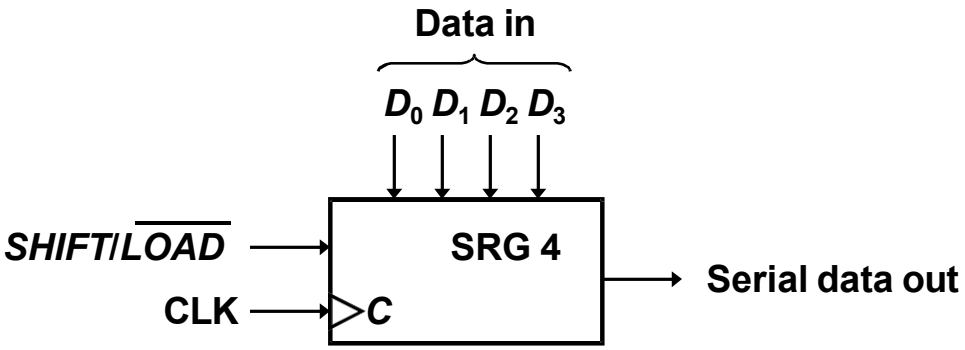
Parallel In/Serial Out Shift Registers

- Bits are entered simultaneously, but output is serial.



Parallel In/Serial Out Shift Registers

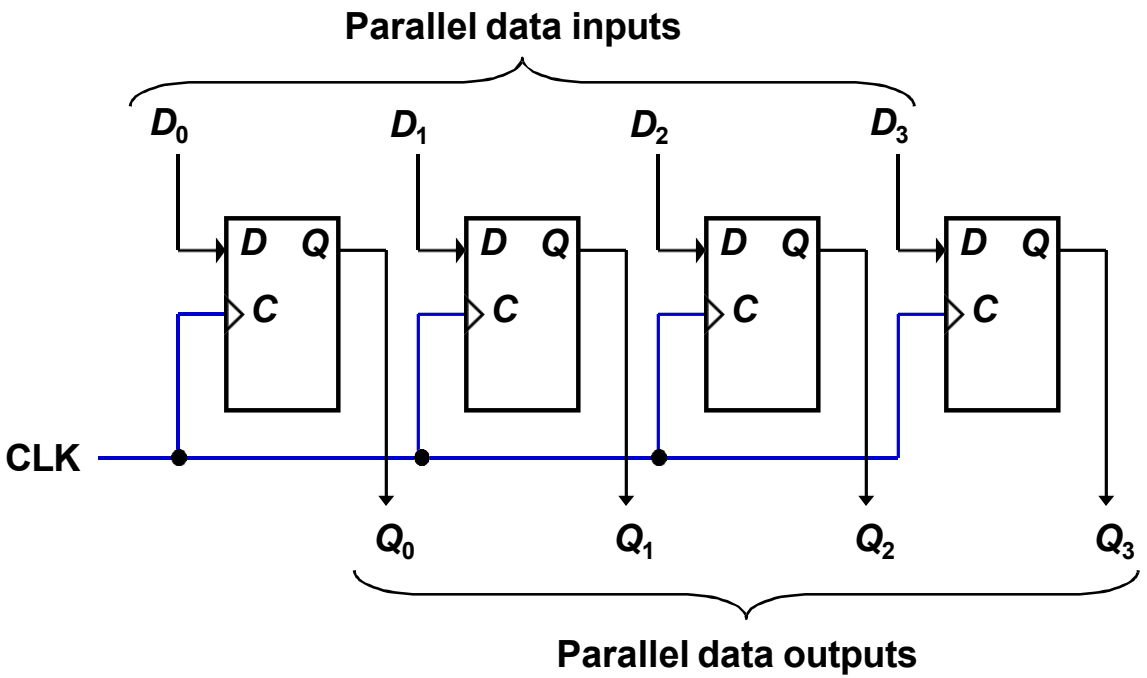
- Bits are entered simultaneously, but output is serial.



Logic symbol

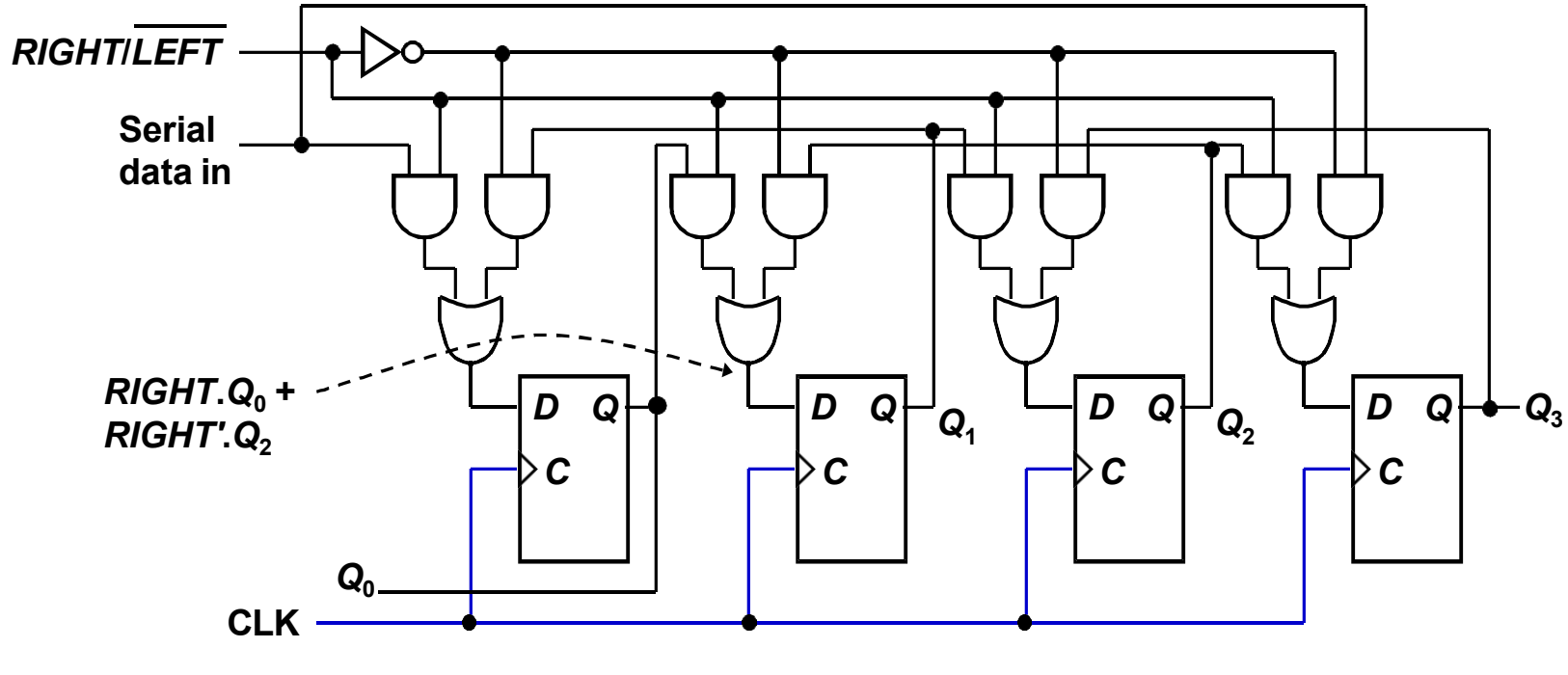
Parallel In/Parallel Out Shift Registers

- Simultaneous input and output of all data bits.



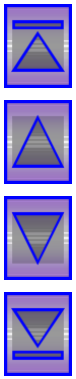
Bidirectional Shift Registers

- Data can be shifted either left or right, using a control line *RIGHT*/*LEFT* (or simply *RIGHT*) to indicate the direction.



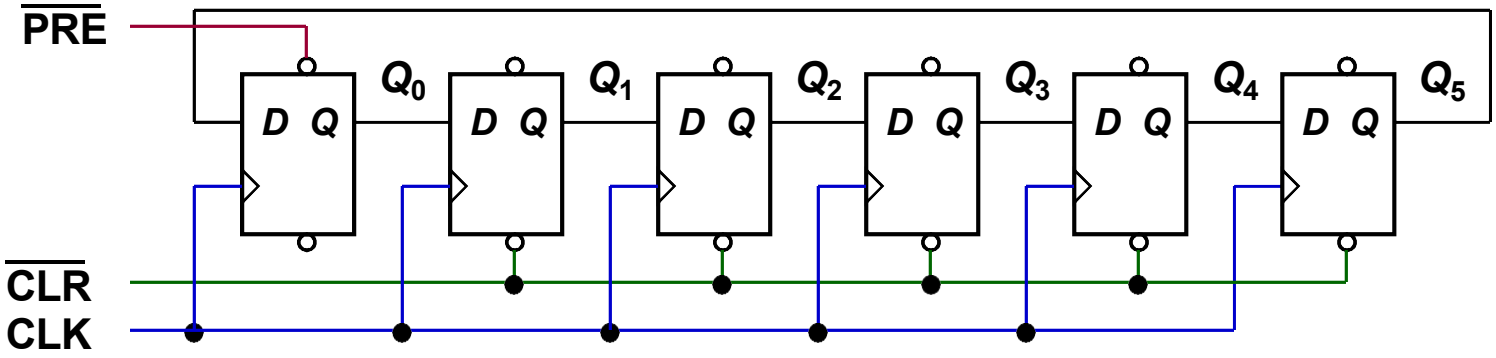
Ring Counters

- One flip-flop (stage) for each state in the sequence.
- The output of the last stage is connected to the D input of the first stage.
- An *n*-bit ring counter cycles through *n* states.
- No decoding gates are required, as there is an output that corresponds to every state the counter is in.

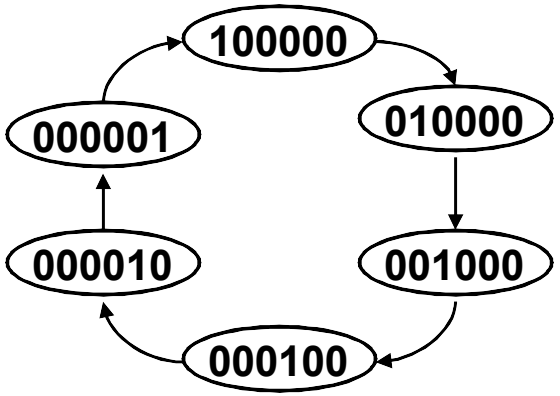


Ring Counters

- Example: A 6-bit (MOD-6) ring counter.

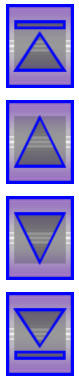


Clock	Q_0	Q_1	Q_2	Q_3	Q_4	Q_5
0	1	0	0	0	0	0
1	0	1	0	0	0	0
2	0	0	1	0	0	0
3	0	0	0	1	0	0
4	0	0	0	0	1	0
5	0	0	0	0	0	1



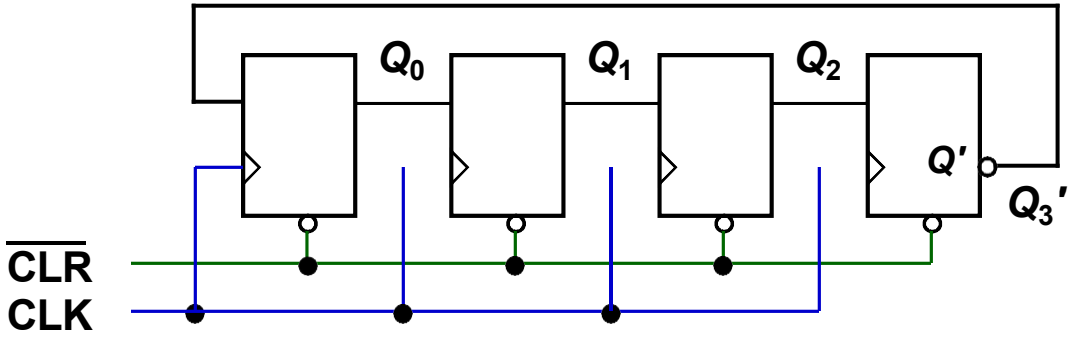
Johnson Counters

- The complement of the output of the last stage is connected back to the D input of the first stage.
- Also called the *twisted-ring counter*.
- Require fewer flip-flops than ring counters but more flip-flops than binary counters.
- An *n*-bit Johnson counter cycles through 2*n* states.
- Require more decoding circuitry than ring counter but less than binary counters.

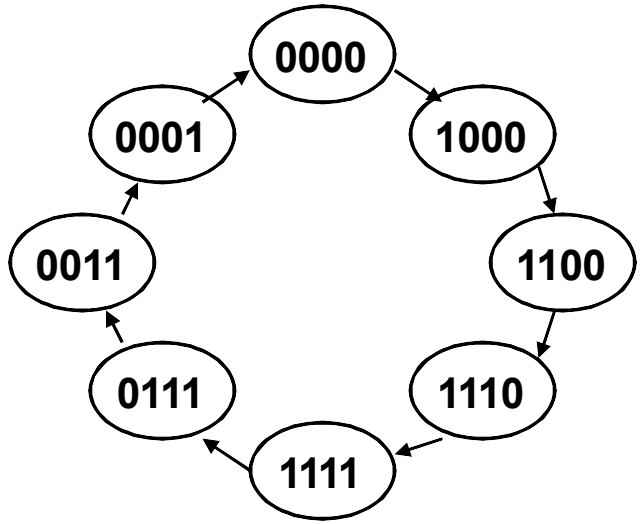


Johnson Counters

- Example: A 4-bit (MOD-8) Johnson counter.

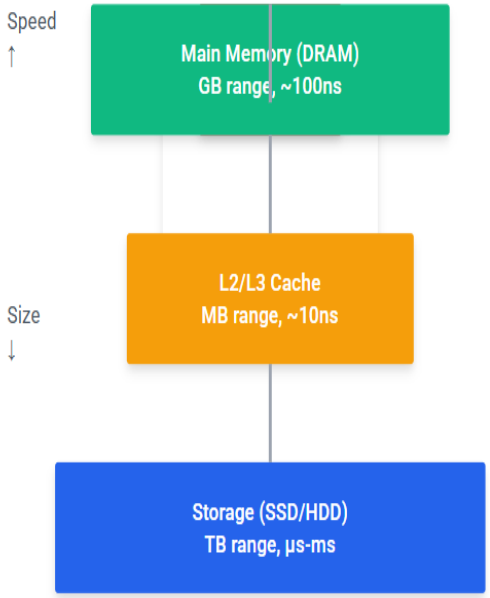


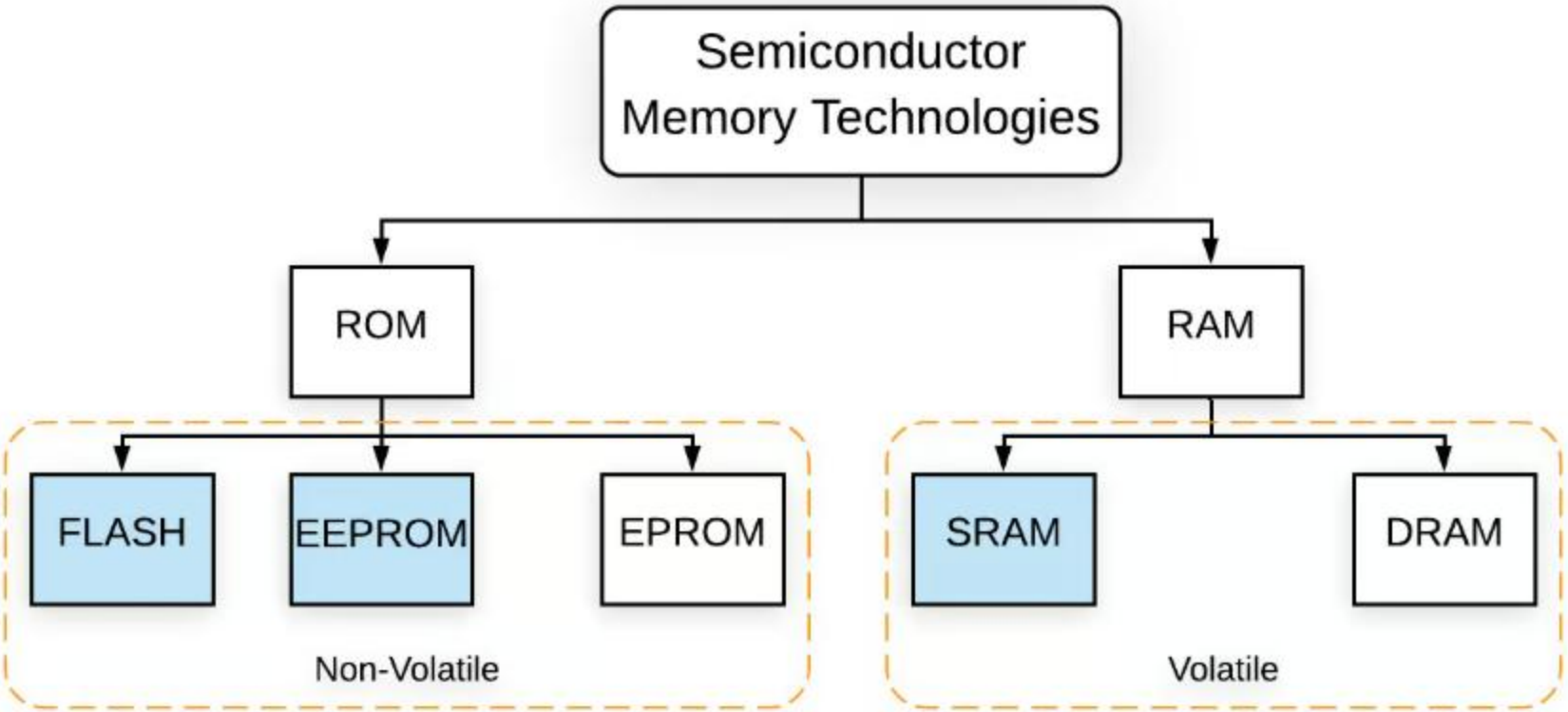
Clock	Q_0	Q_1	Q_2	Q_3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1



What is Computer Memory?

- **Primary (main) memory** powers active computing: RAM (volatile) and ROM (non-volatile)
- **Secondary storage** holds data long term: SSD/HDD, memory cards, etc.
- **Memory hierarchy:** CPU registers → Cache (SRAM) → Main memory (DRAM) → Storage
- **Role in performance:** latency and bandwidth closer to CPU are critical




Semiconductor
Memory Technologies
ROM
RAM
FLASH
EEPROM
EPROM
SRAM
DRAM
Non-Volatile
Volatile

Primary Memory

Types of RAM and ROM

CONTENTS INSIDE:

STATIC, DYNAMIC AND NV-RAM

MASKED ROM, PROM, EPROM, EEPROM

Primary Memory

- Primary storage (or main memory or internal memory), often referred to simply as memory, is the only one directly accessible to the CPU.
- The CPU continuously reads instructions stored there and executes them as required.
- Main memory is directly or indirectly connected to the CPU via a *memory bus*. It is actually two buses an address bus and a data bus.
- The CPU firstly sends a number through an address bus, a number called memory address, that indicates the desired location of data.
- Then it reads or writes the data itself using the data bus.

Primary Memory

- It is the only memory that is directly accessible to CPU.
- The CPU continuously reads instructions stored there and executes them as required.
- Various "Primary storage devices" are:
  - RAM
  - ROM
  - Cache Memory

Primary Memory Types

RAM (Random access memory):

- SRAM (Static RAM) (flip-flop gates)
- DRAM (Dynamic RAM)
- NvRAM (Non-volatile RAM)

ROM (Read only memory)

- Masked ROM
- PROM (programmable)
- EPROM (erasable programmable)
- EEPROM (electronically erasable programmable)

RAM(Random Access Memory)

- Random Access Memory allows stored data to be accessed directly by CPU in any random order.
- It is so called random access as data and instructions can read from or written into RAM in random order. For e.g. to read data stored in memory location 11111111, RAM doesn't need to scan all the memory locations starting from 00000000 index.
- It stores temporary data of currently running applications and play vital role in computers operation.
- It is a Volatile Memory i.e. the data vanishes/erased when the power supply is cut off.
- These are of 2 types:
  - SRAM (Static RAM)
  - DRAM (Dynamic RAM)

DRAM (Dynamic random access memory)

- This is a Random Access Memory that stores each bit of data in a separate capacitor.
- The capacitor can be either charged or discharged; these two states are taken to represent the two values of bit, that are 0 and 1.
- Main characteristics of capacitors is that they leaks electric charge gradually. This will result in turning back the 1 state gradually to 0 state i.e. data may get lost in long run.
- Because of this, there is need of extra power supply for DRAMs using extra circuit. Extra circuit is known as refresh circuit and process of providing electricity is known as refreshing.

SRAM (Static Random Access Memory)

- **Static random-access memory** (static RAM or **SRAM**) is a type of semiconductor memory that uses bistable latching circuitry (flip-flop) and transistors to store each bit.
- **V**olatile in the conventional sense that data is eventually lost when the memory is not powered.
- It is faster than the DRAM as it uses a six transistor configuration.
- In this the direction of current depicts the value 0 or 1.
- It is expensive than DRAM
- This memory need not to be refreshed frequently.

Types of RAM: SRAM vs DRAM

Attribute	SRAM	DRAM
Memory Cell	6-transistor flip-flop circuit (6T)	1 transistor + 1 capacitor (1T1C)
Speed/Latency	Very fast, low latency	Slower than SRAM, higher latency
Refresh	Not required	Requires periodic refresh (~64ms)
Cost/Density	High cost, lower density	Lower cost, high density
Typical Use	CPU caches (L1/L2/L3)	Main memory (SDRAM/DDR)
Variants	–	SDRAM, DDR/DDR2/DDR3/DDR4/DDR5, LPDDR, GDDR (VRAM)

RAM: Examples and Applications

- **PCs/Laptops:** 8–64 GB DDR4/DDR5 for multitasking and performance
- **Smartphones/Tablets:** LPDDR4/LPDDR5 for power-efficient speed
- **Graphics:** GDDR6/GDDR6X (VRAM) for rendering and gaming
- **Caches:** SRAM-based CPU caches for nanosecond access
- **Servers:** ECC RDIMMs/LRDIMMs for reliability and uptime



Desktop DDR4 DIMM



LPDDR Mobile RAM

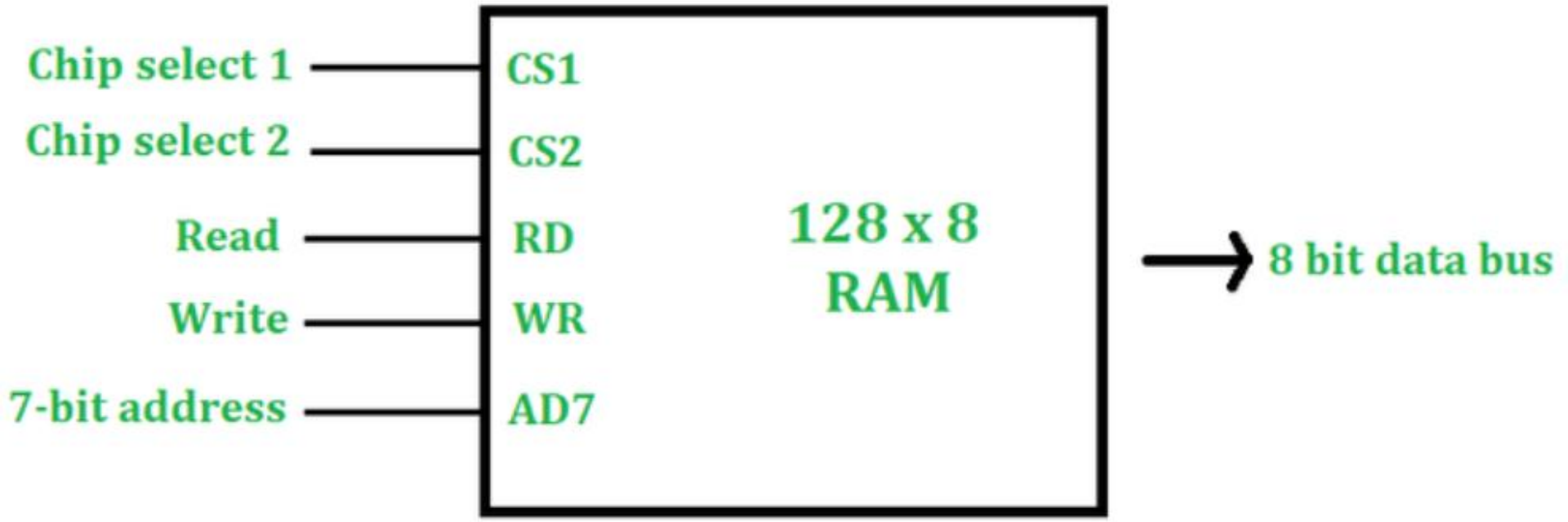


Graphics Card VRAM

Tips:

- ✓ More RAM improves multitasking and reduces disk swapping
- ✓ Match RAM type and generation to motherboard support

512×8



Block diagram of typical RAM chip

Non-Volatile RAM

- Only RAM that can retain data even after electricity is cut off.
- In fact it is volatile RAM equipped with external power supply.
- It takes power from system battery at the time when computer is turned off.
- Currently, the best-known forms of both NV-RAM and EEP-ROM memory are flash memory.
- Floating gate transistors are commonly used in this category of RAM.

ROM(Read Only Memory)

- The DATA stored in Read Only Memory cannot be modified easily i.e. can be modified only slowly or with difficulty.
- It is non-volatile memory i e it does not loose data when the power is cut off.
- Its various types are :
  - Masked ROM
  - PROM
  - EPROM
  - EEPROM

ROM(Read Only Memory)

- **Mask ROM (MROM)** is a type of read-only memory (ROM) whose contents are programmed by the integrated circuit manufacturer (rather than by the user).
- User cannot erase content and they are not allowed to write new contents in masked ROM.
- Also known commonly as manufacturer programmed ROM.

Type	Erasure Method	Reprogrammable	Typical Uses
Mask ROM (MROM)	None (hardwired during manufacturing)	No	High-volume fixed firmware, early embedded systems
PROM	None after programming (fuse burning)	No	One-time custom firmware, permanent configurations
EPROM	UV light (bulk erasure)	Yes (limited cycles)	Legacy BIOS, development prototyping, arcade games
EEPROM	Electrical signals (byte/page level)	Yes	Microcontrollers, BIOS settings storage, smart cards
Flash ROM	Electrical signals (block level)	Yes	SSDs, USB drives, smartphones, modern BIOS/UEFI

PROM (Programmable Read Only Memory)

- On Programmable Read Only Memory the Data can be written only once.
- It is Manufactured as a blank memory and then the data is written on it , also called as **Burning the PROM.**
- One step up from the masked ROM is the PROM (programmable ROM), which is purchased in an unprogrammed state.
- If you look at the contents of an unprogrammed PROM, the data is made up entirely of 1's.
- The process of writing your data to the PROM involves a special piece of equipment called a **PROM burner.**
- The device programmer writes data to the device one word at a time by applying an electrical charge to the input pins of the chip. Once a PROM has been programmed in this way, its contents can never be changed.
- If the code or data stored in the PROM must be changed, the current device must be discarded. As a result, PROMs are also known as one-time programmable (OTP) devices.

EPROM (Erasable Programmable read Only Memory)

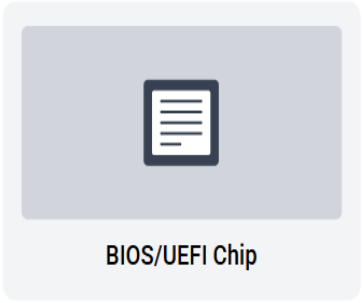
- An EPROM (erasable-and-programmable ROM) is programmed in exactly the same manner as a PROM. However, EPROMs can be erased and reprogrammed repeatedly.
- To erase an EPROM, you simply expose the device to a strong source of ultraviolet light. (A window in the top of the device allows the light to reach the silicon.)
- By doing this, you essentially reset the entire chip to its initial-unprogrammed-state.
- Though more expensive than PROMs, their ability to be reprogrammed makes EPROMs an essential part of the software development and testing process as well rather than only for startup instructions.

EEPROM (Electrically Erasable Programmable Read Only Memory)

- EEPROM is a special type of PROM that can be erased by exposing it to an electric charge.
- Like other ROM it retains its memory even if the power is switched off.
- Data is written or erased one byte at a time
- **EEPROMS** are electrically-erasable-and-programmable. Internally, they are similar to EPROMs, but the erase operation is accomplished electrically, rather than by exposure to ultraviolet light.
- Any **byte within an EEPROM may be erased and rewritten**. Once written, the new data will remain in the device forever-or at least until it is electrically erased.
- The primary tradeoff for this improved functionality is higher cost, though write cycles are also significantly longer than writes to a RAM. So you wouldn't want to use an EEPROM for your main system memory.

ROM: Examples and Applications

- **BIOS/UEFI:** Boot firmware on PCs and servers, enables hardware initialization and OS loading
- **Embedded/MCUs:** Program storage for appliances, IoT devices, and industrial controllers
- **Storage controllers:** SSD/HDD firmware in flash ROM manages data operations and drive health
- **Consumer devices:** Game cartridges, set-top boxes, routers, and smart home devices
- **Automotive:** ECU firmware for engine management, ABS, ADAS, and infotainment systems



BIOS/UEFI Chip



Embedded MCU



Game Cartridge



Automotive ECU

INTERNAL STRUCTURE OF ROM

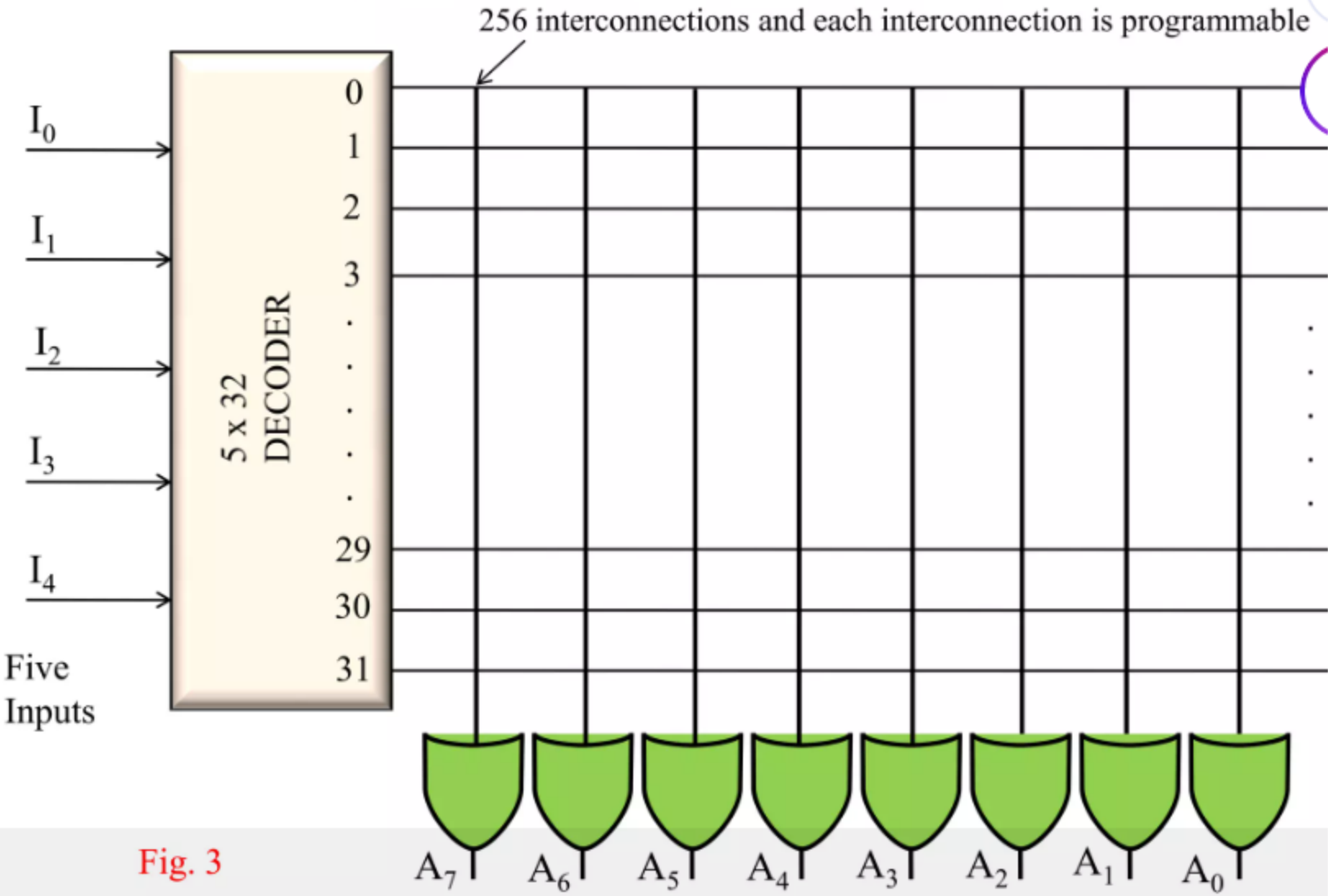


Fig. 3

EXAMPLE: Consider 32 x 8 ROM.

- It consists of 32 words of 8 bits each.
- To access 32 locations (addresses) there are 5 input lines (m =32, $m= 2^5$, k = 5) which forms the binary numbers equivalent to 0 to 31.
- Fig.5 shows the internal logic of a ROM.
- The five inputs are decoded into 32 different outputs with the help of a 5 x 32 decoder and each output of the decoder represent a memory address.
- The 32 outputs of the decoder are connected to each of the eight OR gates. It means that each OR gate having 32 inputs.

Property	RAM	ROM
Volatility	Volatile (loses data when powered off)	Non-volatile (retains data without power)
Mutability	Read/write by CPU at any time	Read-mostly; special methods to update
Speed	Very fast (nanoseconds)	Slower than RAM
Typical Capacity	GBs per module	KBs–MBs on-chip; flash in GBs
Cost per bit	Higher	Lower (especially flash)
Primary Role	Workspace for running code/data	Firmware and immutable data
Examples	DDR4/DDR5 DIMMs, LPDDR, GDDR	BIOS/UEFI, MCU flash, SSD firmware

How RAM and ROM Work Together

Boot flow:

1) **Power on:** CPU begins execution from Boot ROM/UEFI in ROM/flash
2) **Hardware init and POST diagnostics** verify system components
3) **Bootloader loads OS** from storage into RAM
4) **OS and apps run in RAM;** firmware services remain available
5) **Secure boot:** ROM verifies signatures before loading code

Result: ROM provides trusted startup; RAM provides fast execution space

